

Documentación Técnica: Hipatia

Generado automáticamente el 2026-04-11 10:22:18

Índice de Código (completo y verificable)

Resumen rápido (para auditoría en papel)

Métrica	Valor
Archivos <code>.py</code> listados en el índice	442
Incluidos en el cuerpo (tienen bloque en el PDF)	442
Omitidos (reglas de docstrings/otros)	0

Leyenda:

- `pNNNN`: página exacta donde empieza el bloque del archivo en el PDF (placeholder `p0000` si aún no se calculó).
- `Omitido`: el archivo no se incluye en el cuerpo por reglas de docstrings (ver `FRASES_IGNORADAS`).
- `FRASES_IGNORADAS`: Conjunto de textos genéricos (ej. 'Sin descripción disponible') o docstrings vacíos. El script ignora intencionalmente los módulos con estas descripciones porque no aportan información útil.
- `Mypy` `Sí`: tipado estricto aplicado a ese módulo en `mypy.ini`.
- `Mypy` `Parcial`: el proyecto usa una configuración gradual allí; se prioriza estabilidad/coste de esfuerzo.
- `analysis/`
 - `scripts/analysis/analyze_codebase.py` → `p0000` | `clases`: `FileStats`, `DirectorySummary` | `Mypy`: `Parcial` (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
 - `scripts/analysis/analyze_controller.py` → `p0000` | `Mypy`: `Parcial` (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
 - `scripts/analysis/analyze_coverage_risks.py` → `p0000` | `Mypy`: `Parcial` (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
 - `scripts/analysis/analyze_db_usage.py` → `p0000` | `Mypy`: `Parcial` (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
 - `scripts/analysis/analyze_dependencies.py` → `p0000` | `Mypy`: `Parcial` (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
 - `scripts/analysis/analyze_fabrication_dialogs.py` → `p0000` | `Mypy`: `Parcial` (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
 - `scripts/analysis/analyze_loose_mock.py` → `p0000` | `Mypy`: `Parcial` (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
 - `scripts/analysis/analyze_refactoring_impact.py` → `p0000` | `Mypy`: `Parcial` (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)

- `scripts/analysis/analyze_repository_connections.py` → p0000 | clases: FileAnalysisResult, RepoUsageData, ProjectAnalysisResult, RepositoryConnectionAnalyzer | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/analysis/analyze_root_files.py` → p0000 | clases: DefinitionsDict, DefinitionsPayload, ErrorPayload, MissingResult, RootFileAnalysis | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/analysis/analyze_structure.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/analysis/analyze_tracking_impact.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/analysis/analyze_typing_deep.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/analysis/analyze_ui_structure.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/analysis/detect_obsolete_code.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/analysis/verify_naming_conventions.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)

- `camera_manager/`

- `core/camera_manager/__init__.py` → p0000 | Mypy: Sí (`disallow_untyped_defs=True` en `mypy.ini` (fase avanzada/módulo completado).)
- `core/camera_manager/base.py` → p0000 | clases: CameraBackend, CameraInfo | Mypy: Sí (`disallow_untyped_defs=True` en `mypy.ini` (fase avanzada/módulo completado).)
- `core/camera_manager/capture.py` → p0000 | Mypy: Sí (`disallow_untyped_defs=True` en `mypy.ini` (fase avanzada/módulo completado).)
- `core/camera_manager/detector.py` → p0000 | Mypy: Sí (`disallow_untyped_defs=True` en `mypy.ini` (fase avanzada/módulo completado).)
- `core/camera_manager/manager.py` → p0000 | clases: CameraManager | Mypy: Sí (`disallow_untyped_defs=True` en `mypy.ini` (fase avanzada/módulo completado).)
- `core/camera_manager/utils.py` → p0000 | Mypy: Sí (`disallow_untyped_defs=True` en `mypy.ini` (fase avanzada/módulo completado).)

- `dialogs/`

- `effects/`
 - `ui/dialogs/effects/__init__.py` → p0000 | Mypy: Sí (`disallow_untyped_defs=True` en `mypy.ini` (fase avanzada/módulo completado).)
 - `ui/dialogs/effects/golden_glow.py` → p0000 | clases: GoldenGlowEffect | Mypy: Sí (`disallow_untyped_defs=True` en `mypy.ini` (fase avanzada/módulo completado).)
 - `ui/dialogs/effects/green_cycle.py` → p0000 | clases: GreenCycleEffect | Mypy: Sí (`disallow_untyped_defs=True` en `mypy.ini` (fase avanzada/módulo completado).)
 - `ui/dialogs/effects/mixed_gold_green.py` → p0000 | clases: MixedGoldGreenEffect | Mypy: Sí (`disallow_untyped_defs=True` en `mypy.ini` (fase avanzada/módulo completado).)
 - `ui/dialogs/effects/processing_glow.py` → p0000 | clases: ProcessingGlowEffect | Mypy: Sí (`disallow_untyped_defs=True` en `mypy.ini` (fase avanzada/módulo completado).)
 - `ui/dialogs/effects/progress.py` → p0000 | clases: SimulationProgressEffect | Mypy: Sí (`disallow_untyped_defs=True` en `mypy.ini` (fase avanzada/módulo completado).)

○ fabrication/

- ui/dialogs/fabrication/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/fabrication/assignment_dialogs.py → p0000 | clases: AssignPreprocesosDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/fabrication/bitacora_dialog.py → p0000 | clases: BitacoraEntryDTO, FabricacionBitacoraDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/fabrication/create_dialog.py → p0000 | clases: CreateFabricacionDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/fabrication/create_presenter.py → p0000 | clases: CreateFabricacionPresenter | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/fabrication/dialog_dependencies.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/fabrication/input_dialogs.py → p0000 | clases: GetLotelInstanceParametersDialog, GetOptimizationParametersDialog, GetUnitsDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/fabrication/persistence_dialogs.py → p0000 | clases: SavePilaDialog, LoadPilaDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/fabrication/products_dialog.py → p0000 | clases: ProductsSelectionDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/fabrication/selection_dialogs.py → p0000 | clases: PreprocesosSelectionDialog, PreprocesosForCalculationDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/fabrication/ui_dialog_protocols.py → p0000 | clases: OpensFabricacionPreprocesos, ShowsUserMessage | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

○ prep/

- ui/dialogs/prep/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/prep/prep_groups_dialog.py → p0000 | clases: PrepGroupsDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/prep/prep_steps_dialog.py → p0000 | clases: PrepStepsDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/prep/preproceso_dialog.py → p0000 | clases: PreprocesoDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

○ product/

- ui/dialogs/product/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/product/add_iteration_dialog.py → p0000 | clases: AddIterationFormData, AddIterationDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/product/bom_import_preview_dialog.py → p0000 | clases: BOMImportPreviewDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/product/procesos_mecanicos_dialog.py → p0000 | clases: ProcesosMecanicosDialog, AddProcesoMecanicoDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- ui/dialogs/product/product_details_dialog.py → p0000 | clases: ProductDetailsDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/dialogs/product/subfabricaciones_dialog.py → p0000 | clases: SubfabricacionesDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- production_flow/
- ui/dialogs/production_flow/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/dialogs/production_flow/common_dialogs.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/dialogs/production_flow/cycle_end_config_dialog.py → p0000 | clases: CycleEndConfigDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/dialogs/production_flow/define_flow_dialog.py → p0000 | clases: DefineProductionFlowDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/dialogs/production_flow/define_flow_presenter.py → p0000 | clases: DefineFlowPresenter | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/dialogs/production_flow/definir_cantidades_dialog.py → p0000 | clases: DefinirCantidadesDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/dialogs/production_flow/enhanced_flow_dialog.py → p0000 | clases: EnhancedProductionFlowDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/dialogs/production_flow/enhanced_flow_presenter.py → p0000 | clases: EnhancedFlowPresenter | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/dialogs/production_flow/enhanced_flow_state_manager.py → p0000 | clases: EnhancedFlowStateManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/dialogs/production_flow/flow_action_handler.py → p0000 | clases: FlowActionHandler | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/dialogs/production_flow/flow_builder.py → p0000 | clases: FlowBuilder | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/dialogs/production_flow/flow_simulation_handler.py → p0000 | clases: FlowSimulationHandler | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/dialogs/production_flow/machine_resource_manager.py → p0000 | clases: MachineResourceManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/dialogs/production_flow/reassignment_rule_dialog.py → p0000 | clases: ReassignmentRuleDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/backup_restore_dialog.py → p0000 | clases: BackupRestoreDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/connection_dialog.py → p0000 | clases: ConnectionDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/tracking_dialogs.py → p0000 | clases: OrderSetupDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/dialogs/utility_dialogs.py → p0000 | clases: AddBreakDialog, LoginDialog, ChangePasswordDialog, SyncDialog, SeleccionarHojasExcelDialog,

MultiWorkerSelectionDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- facades/

- core/facades/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/facades/planning_facade.py → p0000 | clases: PlanningFacade | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/facades/product_facade.py → p0000 | clases: ProductFacade | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/facades/production_facade.py → p0000 | clases: ProductionFacade | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/facades/reporting_facade.py → p0000 | clases: ReportingFacade | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/facades/system_facade.py → p0000 | clases: SystemFacade | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/facades/workforce_facade.py → p0000 | clases: WorkforceFacade | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- hardware/

- tools/hardware/detect_cameras.py → p0000 | clases: CameraInfo | Mypy: Parcial (configuración gradual: disallow_untyped_defs=False en mypy.ini tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)

- health/

- core/health/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/health/constants.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/health/health_checker.py → p0000 | clases: TableHealth, SystemHealth, TestResults, HealthReport, DatabaseHealthChecker | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/health/health_worker.py → p0000 | clases: HealthCheckWorker | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/health/test_runner.py → p0000 | clases: TestRunner | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- historial/

- controllers/historial/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- controllers/historial/controller.py → p0000 | clases: HistorialController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- controllers/historial/interaction_manager.py → p0000 | clases: HistorialInteractionManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- controllers/historial/protocols.py → p0000 | clases: HistorialControllerProtocol | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- controllers/historial/report_manager.py → p0000 | clases: HistorialReportManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- controllers/historial/view_manager.py → p0000 | clases: HistorialViewManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- import_manager/

- adapters/

- core/import_manager/adapters/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/import_manager/adapters/a3rp_csv_adapter.py → p0000 | clases: A3RPCSVAdapter | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/import_manager/adapters/a3rp_excel_adapter.py → p0000 | clases: A3RPExcelAdapter | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- services/
 - core/import_manager/services/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/import_manager/services/bom_import_service.py → p0000 | clases: BOMImportService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/import_manager/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/import_manager/dto.py → p0000 | clases: BOMImportRole, BOMNodeDTO | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/import_manager/ports.py → p0000 | clases: IBOMImporter | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- interfaces/
 - core/interfaces/controller_interface.py → p0000 | clases: QABCMeta, IController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/interfaces/view_interface.py → p0000 | clases: IView | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/interfaces/worker_view_interface.py → p0000 | clases: IWorkerView | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- label_manager/
 - core/label_manager/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/label_manager/base.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/label_manager/manager.py → p0000 | clases: LabelManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/label_manager/ports.py → p0000 | clases: IDocumentGenerator | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/label_manager/printer.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- maintenance/
 - scripts/maintenance/backup_database.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - scripts/maintenance/reset_admin.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- models/
 - database/models/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - database/models/base.py → p0000 | clases: Base | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - database/models/fabrication.py → p0000 | clases: Fabricacion, FabricacionContador | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - database/models/inventory.py → p0000 | clases: Material, Pila, PasoPila, DiarioBitacora, EntradaDiario, Lote | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- database/models/machine.py → p0000 | clases: Maquina, MachineMaintenanc, GrupoPreparacion, PreparacionPaso | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/models/product.py → p0000 | clases: Producto, Preproceso, Subfabricacion, ProcesoMecanico, ProductIteration | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/models/security.py → p0000 | clases: Configuration, LoginAttempt, AuditLog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/models/tracking.py → p0000 | clases: TrabajoLog, PasoTrazabilidad, IncidenciaLog, IncidenciaAdjunto | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/models/worker.py → p0000 | clases: Trabajador, TrabajadorPilaAnotacion | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- pila/

- controllers/pila/controller.py → p0000 | clases: PilaController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- controllers/pila/lote_manager.py → p0000 | clases: LoteManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- controllers/pila/pila_manager.py → p0000 | clases: PilaManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- controllers/pila/protocols.py → p0000 | clases: IPilaView, IPilaDatabase, IPilaService, IProductService, IFabricacionService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- product/

- controllers/product/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- controllers/product/application_shell.py → p0000 | clases: IApplicationShell | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- controllers/product/fabricacion_manager.py → p0000 | clases: FabricacionManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- controllers/product/fabricacion_products_handler.py → p0000 | clases: IPlanningCalculationProvider, FabricacionProductsHandler | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- controllers/product/material_manager.py → p0000 | clases: MaterialManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- controllers/product/preproceso_manager.py → p0000 | clases: PreprocesoManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- controllers/product/product_manager.py → p0000 | clases: ProductManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- controllers/product/protocols.py → p0000 | clases: IProductView, IProductModel, IFabricacionControllerDelegate, ProductControllerProtocol | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- protocols/

- core/protocols/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/protocols/domain.py → p0000 | clases: IProductService, IFabricacionService, IMaterialService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- qr_scanner/

- core/qr_scanner/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/qr_scanner/base.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/qr_scanner/detector.py → p0000 | clases: QRDetector | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/qr_scanner/scanner.py → p0000 | clases: QrScanner, QrScannerCallback | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/qr_scanner/ui.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- repositories/

- machine/

- database/repositories/machine/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/machine/crud_manager.py → p0000 | clases: MachineCRUDManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/machine/maintenance_manager.py → p0000 | clases: MachineMaintenanceManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/machine/preparation_manager.py → p0000 | clases: MachinePreparationManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/machine/repository.py → p0000 | clases: MachineRepository | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/machine/stats_manager.py → p0000 | clases: MachineStatsManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- pila/

- database/repositories/pila/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/pila/pila_base_manager.py → p0000 | clases: PilaBaseManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/pila/pila_bitacora_manager.py → p0000 | clases: PilaBitacoraManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/pila/pila_crud_manager.py → p0000 | clases: PilaCRUDManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/pila/pila_workflow_manager.py → p0000 | clases: PilaWorkflowManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/pila/repository.py → p0000 | clases: PilaRepository | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- preproceso/

- database/repositories/preproceso/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/preproceso/fabricacion_manager.py → p0000 | clases: FabricacionManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/preproceso/preproceso_manager.py → p0000 | clases: PreprocesoManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/preproceso/repository.py → p0000 | clases: PreprocesoRepository | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase

avanzada/módulo completado).)

○ reports/

- database/repositories/reports/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/reports/reports_incidents_manager.py → p0000 | clases: ReportsIncidentsManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/reports/reports_orders_manager.py → p0000 | clases: ReportsOrdersManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/reports/reports_products_manager.py → p0000 | clases: ReportsProductsManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/reports/reports_search_manager.py → p0000 | clases: ReportsSearchManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/reports/reports_stats_manager.py → p0000 | clases: ReportsStatsManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/reports/repository.py → p0000 | clases: ReportsRepository | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

○ tracking/

- database/repositories/tracking/core_manager.py → p0000 | clases: TrackingCoreManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/tracking/mappers.py → p0000 | clases: TrackingMapper | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/tracking/queries_manager.py → p0000 | clases: TrackingQueriesManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/tracking/steps_manager.py → p0000 | clases: TrackingStepsManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

○ worker/

- database/repositories/worker/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/worker/annotation_manager.py → p0000 | clases: WorkerAnnotationManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/worker/auth_manager.py → p0000 | clases: WorkerAuthManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/worker/repository.py → p0000 | clases: WorkerRepository | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/worker/worker_manager.py → p0000 | clases: WorkerCoreManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

○ database/repositories/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

○ database/repositories/base.py → p0000 | clases: BaseRepository | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- database/repositories/configuration_repository.py → p0000 | clases: ConfigurationRepository | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/incidencia_repository.py → p0000 | clases: IncidenciaRepository | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/iteration_repository.py → p0000 | clases: IterationRepository | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/iteration_repository_helpers.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/label_counter_repository.py → p0000 | clases: LabelCounterRepository | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/lote_repository.py → p0000 | clases: LoteRepository | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/material_repository.py → p0000 | clases: MaterialRepository | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/product_repository.py → p0000 | clases: ProductRepository | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/product_repository_helpers.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/protocols.py → p0000 | clases: RepositoryProtocol, PilaRepositoryProtocol, TrackingRepositoryProtocol | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/tracking_log_repository.py → p0000 | clases: TrackingLogRepository | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/tracking_repository.py → p0000 | clases: TrackingRepository | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- database/repositories/tracking_stats_repository.py → p0000 | clases: TrackingStatsRepository | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- security/
 - core/security/access_control.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/security/password_service.py → p0000 | clases: PasswordService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/security/security_exceptions.py → p0000 | clases: SecurityError, SecurityServiceNotInitializedError, InsufficientPermissionsError, RateLimitExceededError | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/security/security_service.py → p0000 | clases: UserRole, Permission, SecurityService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- services/
 - report_sheets/
 - core/services/report_sheets/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/services/report_sheets/audit.py → p0000 | clases: AuditSheet | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/services/report_sheets/base.py → p0000 | clases: ExcelSheetStrategy | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/services/report_sheets/cronograma.py → p0000 | clases: CronogramaSheet | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/services/report_sheets/cuellos_botella.py → p0000 | clases:

CuellosBotollaSheet | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/report_sheets/graficas.py → p0000 | clases: GraficasSheet | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/report_sheets/resumen.py → p0000 | clases: ResumenEjecutivoSheet | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/report_sheets/trabajadores.py → p0000 | clases: AnalisisTrabajadoresSheet | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/report_sheets/trabajo_paralelo.py → p0000 | clases: TrabajoParaleloSheet | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

○ reporting/

- core/services/reporting/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/reporting/base.py → p0000 | clases: IReporteEstrategia, GeneradorDeInformes | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/reporting/excel_report_strategy.py → p0000 | clases: ReportePilaFabricacionExcelMejorado | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/reporting/pdf_report_sections.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/reporting/pdf_report_strategy.py → p0000 | clases: ReporteHistorialFabricacion, ReporteHistorialIteracion | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/audit_logger.py → p0000 | clases: AuditLogger | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/backup_service.py → p0000 | clases: BackupService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/backup_utils.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/calculation_audit.py → p0000 | clases: DecisionStatus, CalculationDecision | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/calendar_helper.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/data_importer.py → p0000 | clases: Material, IMaterialImporter, ExcelMaterialImporter, MaterialImporterFactory | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/fabricacion_service.py → p0000 | clases: FabricacionService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/flow_builder_service.py → p0000 | clases: FlowBuilderService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/flow_simulation_service.py → p0000 | clases: SimulationSession, FlowSimulationService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/machine_service.py → p0000 | clases: MachineService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/maintenance_service.py → p0000 | clases: MaintenanceWorker, MaintenanceService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/pila_service.py → p0000 | clases: PilaService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- core/services/preparation_service.py → p0000 | clases: PreparationService | Mypy: Sí

- (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/services/product_service.py → p0000 | clases: ProductService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/services/rate_limiter.py → p0000 | clases: RateLimiter | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/services/report_service.py → p0000 | clases: ReportService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/services/report_strategy.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/services/system_integration_service.py → p0000 | clases: SystemIntegrationService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/services/temporal_storage.py → p0000 | clases: RegistroTemporal | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/services/time_calculator.py → p0000 | clases: CalculadorDeTiempos | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/services/tracking_assignment_service.py → p0000 | clases: TrackingAssignmentService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- core/services/worker_service.py → p0000 | clases: WorkerService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- simulation/
 - engine/
 - core/simulation/engine/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/simulation/engine/base.py → p0000 | clases: SimulationState | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/simulation/engine/core_runner.py → p0000 | clases: CoreSimulationRunner | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/simulation/engine/dependency_handler.py → p0000 | clases: DependencyHandler | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/simulation/engine/motor.py → p0000 | clases: MotorDeEventos | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/simulation/engine/results_compiler.py → p0000 | clases: ResultsCompiler | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - simulation_events/
 - core/simulation/simulation_events/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/simulation/simulation_events/base.py → p0000 | clases: EventoDeSimulacion | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/simulation/simulation_events/production.py → p0000 | clases: EventoInicioUnidad, EventoFinUnidad | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/simulation/simulation_events/worker.py → p0000 | clases: EventoReasignacionTrabajador, EventoTiempoInactivo | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - controllers/simulation/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - controllers/simulation/controller.py → p0000 | clases: SimulationController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - controllers/simulation/editor_manager.py → p0000 | clases: SimulationEditorManager |

- controllers/simulation/execution_helpers.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - controllers/simulation/execution_manager.py → p0000 | clases: SimulationExecutionManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - controllers/simulation/optimizer_worker.py → p0000 | clases: OptimizerWorker | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - controllers/simulation/protocols.py → p0000 | clases: SimulationControllerProtocol | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/simulation/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/simulation/event_engine.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/simulation/resource_manager.py → p0000 | clases: IntervaloOcupacion, ReglaReasignacion, GestorDeRecursos | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/simulation/simulation_engine.py → p0000 | clases: SimulationWorker, Optimizer | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/simulation/timeline_task.py → p0000 | clases: LineaTemporalTarea | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/simulation/timeline_task_parallel.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- utils/
 - core/utils/author_loader.py → p0000 | clases: WorkerSignals, AuthorInfoLoader | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/utils/helpers.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/utils/pila_serializer.py → p0000 | clases: PilaJSONEncoder | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/utils/ui_scaler.py → p0000 | clases: UIScaler | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - core/utils/visualization.py → p0000 | clases: VisualizationGenerator | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- validation/
 - core/validation/validator_service.py → p0000 | clases: ValidationResult, ValidatorService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- versions/
 - migrations/versions/a195b5f170d2_add_security_tables.py → p0000 | Mypy: Parcial (configuración gradual: disallow_untyped_defs=False en mypy.ini tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
 - migrations/versions/c1444b2546d3_initial_clean_migration.py → p0000 | Mypy: Parcial (configuración gradual: disallow_untyped_defs=False en mypy.ini tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- widgets/
 - product/
 - ui/widgets/product/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/product/iterations_widget.py → p0000 | clases: ProductIterationsWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- ui/widgets/product/materials_widget.py → p0000 | clases: ProductMaterialsWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- production_flow/
 - ui/widgets/production_flow/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/production_flow/define_control_panel.py → p0000 | clases: DefineControlPanel | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/production_flow/flow_canvas.py → p0000 | clases: _FlowConnectionsLayer, ProductionFlowCanvas | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/production_flow/flow_card_widget.py → p0000 | clases: FlowCardWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/production_flow/flow_connectionPainter.py → p0000 | clases: FlowConnectionPainter | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/production_flow/flow_display_panel.py → p0000 | clases: FlowDisplayPanel | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/production_flow/flow_graph_manager.py → p0000 | clases: FlowGraphManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/production_flow/flow_item_widget.py → p0000 | clases: FlowItemWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/production_flow/flow_toolbar.py → p0000 | clases: FlowToolbarWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/production_flow/inspector_panel.py → p0000 | clases: ProductionTaskInspector | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/production_flow/inspector_presenter.py → p0000 | clases: InspectorPresenter | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/production_flow/inspector_task_loader.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/production_flow/inspector_ui.py → p0000 | clases: InspectorWidgets | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/production_flow/library_panel.py → p0000 | clases: TaskLibraryPanel | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- reports/
 - ui/widgets/reports/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/reports/charts_container.py → p0000 | clases: ReportsChartsWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/reports/charts_renderers.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/reports/order_list.py → p0000 | clases: OrderCard, OrderListWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/reports/smart_search.py → p0000 | clases: SmartSearchWidget |

Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- ui/widgets/reports/stat_card.py → p0000 | clases: StatCard | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- worker/
 - ui/widgets/worker/camera_info_panel.py → p0000 | clases: CameraInfoPanel | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/worker/camera_selector_panel.py → p0000 | clases: CameraSelectorPanel | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/worker/worker_activity_panel.py → p0000 | clases: WorkerActivityPanel | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/worker/worker_details_panel.py → p0000 | clases: WorkerDetailsPanel | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/widgets/worker/worker_incidence_dialog.py → p0000 | clases: WorkerIncidenceDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/base.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/calculate_times_widget.py → p0000 | clases: CalculateTimesWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/dashboard_widget.py → p0000 | clases: DashboardWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/fabrications_widget.py → p0000 | clases: FabricationsWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/gestion_datos_widget.py → p0000 | clases: GestionDatosWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/help_widget.py → p0000 | clases: HelpWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/historial_widget.py → p0000 | clases: HistorialWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/home_widget.py → p0000 | clases: HomeWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/log_terminal_widget.py → p0000 | clases: LogTerminalWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/lotas_widget.py → p0000 | clases: DefinirLoteWidget, LotesWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/machines_widget.py → p0000 | clases: MachinesWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/main_header.py → p0000 | clases: MainHeader | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/main_nav_panel.py → p0000 | clases: MainNavPanel | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/prep_steps_widget.py → p0000 | clases: PrepStepsWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/preprocesos_widget.py → p0000 | clases: PreprocesosWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/products_widget.py → p0000 | clases: ProductsWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/reportes_widget.py → p0000 | clases: ReportesWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/settings_widget.py → p0000 | clases: SettingsWidget | Mypy: Sí

- (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/timeline_widget.py → p0000 | clases: TimelineVisualizationWidget, TaskAnalysisPanel | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- ui/widgets/workers_widget.py → p0000 | clases: WorkersWidget | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- worker/
 - main_window/
 - ui/worker/main_window/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/worker/main_window/ui_manager.py → p0000 | clases: WorkerMainWindowUIManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/worker/main_window/window.py → p0000 | clases: WorkerMainWindow | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - controllers/worker/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - controllers/worker/auth_manager.py → p0000 | clases: WorkerAuthManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - controllers/worker/controller.py → p0000 | clases: WorkerController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - controllers/worker/management_manager.py → p0000 | clases: WorkerManagementManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - controllers/worker/protocols.py → p0000 | clases: IWorkerView, IWorkerService, IWorkerModel, WorkerControllerProtocol | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - controllers/worker/task_manager.py → p0000 | clases: WorkerTaskManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - controllers/worker/worker_camera_config.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/worker/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/worker/camera_config_dialog.py → p0000 | clases: CameraConfigDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
 - ui/worker/camera_config_presenter.py → p0000 | clases: CameraConfigPresenter | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- analyze_ui.py → p0000 | Mypy: Parcial (configuración gradual: disallow_untyped_defs=False en mypy.ini tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- app.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- controllers/__init__.py → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- controllers/app_controller.py → p0000 | clases: AppController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- controllers/backup_controller.py → p0000 | clases: IBackupControllerDatabase, BackupController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- controllers/backup_controller_io_manager.py → p0000 | clases: BackupControllerIOContext, BackupIOManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- `controllers/calculation_controller.py` → p0000 | clases: CalculationController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `controllers/fabricacion_controller.py` → p0000 | clases: FabricacionController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `controllers/file_controller.py` → p0000 | clases: FileController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `controllers/hardware_controller.py` → p0000 | clases: HardwareController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `controllers/lote_controller.py` → p0000 | clases: LoteController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `controllers/machine_controller.py` → p0000 | clases: MachineController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `controllers/navigation_controller.py` → p0000 | clases: NavigationController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `controllers/preproceso_controller.py` → p0000 | clases: PreprocesoController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `controllers/product_controller_v2.py` → p0000 | clases: ProductController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `controllers/report_controller.py` → p0000 | clases: ReportController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `controllers/report_export_helper.py` → p0000 | clases: ReportExportHelper | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `controllers/schedule_controller.py` → p0000 | clases: ScheduleController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `controllers/schedule_helpers.py` → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `controllers/schedule_ui_helper.py` → p0000 | clases: ScheduleUiOpsHelper | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `controllers/session_controller.py` → p0000 | clases: SessionController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `controllers/startup_controller.py` → p0000 | clases: StartupController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `controllers/ui_class_loader.py` → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `controllers/ui_controller.py` → p0000 | clases: UIController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `controllers/ui_signals_controller.py` → p0000 | clases: UISignalsController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `controllers/ui_signals_wiring.py` → p0000 | clases: UISignalsWiring | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/__init__.py` → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- `core/app_model.py` → p0000 | clases: AppModel | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/application_state.py` → p0000 | clases: ApplicationState | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/constants.py` → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/define_flow_form_io.py` → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/define_flow_presenter_io.py` → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/definir_cantidades_dialog_io.py` → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/di_container.py` → p0000 | clases: ServiceLifecycle, ServiceRegistration, DIContainer | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/dtos.py` → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/dtos_catalog.py` → p0000 | clases: ProductDTO, SubfabricacionDTO, ProcesoMecanicoDTO, MaterialDTO, PilaDTO, MaterialStatsDTO, ComponenteDTO, FabricacionProductoDTO, PreprocesoDTO, FabricacionDTO, LoteDTO, ConfigurationDTO (+4) | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/dtos_flow_camera.py` → p0000 | clases: FlowTaskDataDTO, CanvasCyclicConnectionFlags, ProductFlowLibraryProductDTO, FlowTaskConfigDTO, ProductionFlowStepDTO, FlowCanvasTaskDTO, CameraConfigDTO, CameraDetailDTO, FlowItemDTO | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/dtos_models.py` → p0000 | clases: MachineDTO, MachineMaintenanceDTO, PreparationGroupDTO, PreparationStepDTO, WorkerDTO, WorkerAnnotationDTO, WorkerDetailDTO, AuthResponseDTO, BackupInfoDTO, SimulationResultTaskDTO, CalculationSubPartDTO, CalculationProductDTO (+11) | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/enhanced_flow_canvas_state_io.py` → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/enhanced_flow_presenter_io.py` → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/flow_canvas_io.py` → p0000 | clases: CanvasVisualConnection | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/flow_card_labels.py` → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/flow_graph_manager_io.py` → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/flow_inspector_context.py` → p0000 | clases: FlowInspectorTaskContext | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/holidays_config_io.py` → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/inspector_task_payload_io.py` → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- `core/paths.py` → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/planning_session_access.py` → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/production_context.py` → p0000 | clases: ProductionStatus, ProductionContext | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/qr_generator.py` → p0000 | clases: QrGenerator | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/qt_log_handler.py` → p0000 | clases: _SignalEmitter, QtLogHandler | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/quote_service.py` → p0000 | clases: QuoteService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/reassignment_rule_dialog_io.py` → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/reports_dtos.py` → p0000 | clases: ResultadoBusquedaDTO, OrdenFabricacionResumenDTO, OrdenFabricacionDetalleDTO, PromedioTiempoDTO, TiempoTrabajadorDTO, IncidenciaResumenDTO, PuntoEvolucionDTO, UnidadTrabajoDTO, ResumenProductoDTO | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/schedule_config.py` → p0000 | clases: ScheduleConfig | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/subfabricacion_rows.py` → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/sync_service.py` → p0000 | clases: SyncService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/tracking_dtos.py` → p0000 | clases: FabricacionAsignadaDTO, IncidenciaAdjuntoDTO, IncidenciaLogDTO, PasoTrazabilidadDTO, TrabajoLogDTO | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `core/worker_ui_dtos.py` → p0000 | clases: WorkerTaskListRowDTO | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `database/__init__.py` → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `database/config.py` → p0000 | clases: DatabaseConfig | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `database/database_manager.py` → p0000 | clases: DatabaseManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `features/__init__.py` → p0000 | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `features/worker_controller.py` → p0000 | clases: WorkerController | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `features/worker_controller_io_manager.py` → p0000 | clases: WorkerIOManager | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `features/worker_db_sync.py` → p0000 | clases: WorkerDbSync | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)

- `features/worker_incidence_dialog.py` → p0000 | clases: IncidenceDialog | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `features/worker_validation_service.py` → p0000 | clases: QRScannerProtocol, WorkerValidationService | Mypy: Sí (disallow_untyped_defs=True en mypy.ini (fase avanzada/módulo completado).)
- `generate_ui_report.py` → p0000 | Mypy: Parcial (configuración gradual: disallow_untyped_defs=False en mypy.ini tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `migrations/env.py` → p0000 | Mypy: Parcial (configuración gradual: disallow_untyped_defs=False en mypy.ini tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `run_tests.py` → p0000 | clases: Colors | Mypy: Parcial (configuración gradual: disallow_untyped_defs=False en mypy.ini tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `run_tests_safe.py` → p0000 | clases: C | Mypy: Parcial (configuración gradual: disallow_untyped_defs=False en mypy.ini tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/__init__.py` → p0000 | Mypy: Parcial (configuración gradual: disallow_untyped_defs=False en mypy.ini tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/analyze_mixin.py` → p0000 | Mypy: Parcial (configuración gradual: disallow_untyped_defs=False en mypy.ini tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/analyze_pila_controller.py` → p0000 | Mypy: Parcial (configuración gradual: disallow_untyped_defs=False en mypy.ini tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/analyze_product_controller_coverage.py` → p0000 | Mypy: Parcial (configuración gradual: disallow_untyped_defs=False en mypy.ini tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/analyze_ui_state.py` → p0000 | Mypy: Parcial (configuración gradual: disallow_untyped_defs=False en mypy.ini tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/architecture_layer_edges.py` → p0000 | Mypy: Parcial (configuración gradual: disallow_untyped_defs=False en mypy.ini tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/audit_import_graph.py` → p0000 | Mypy: Parcial (configuración gradual: disallow_untyped_defs=False en mypy.ini tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/audit_module_description_quality.py` → p0000 | Mypy: Parcial (configuración gradual: disallow_untyped_defs=False en mypy.ini tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/audit_module_docstrings.py` → p0000 | Mypy: Parcial (configuración gradual: disallow_untyped_defs=False en mypy.ini tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/bootstrap_module_docstrings.py` → p0000 | Mypy: Parcial (configuración gradual: disallow_untyped_defs=False en mypy.ini tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)

- `scripts/build_executable.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/check_documentation_omissions.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/check_typing_coverage.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/codebase_analyzer.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/coverage_focus.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/detect_dead_code.py` → p0000 | clases: `MethodExtractor` | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/doc_audit_common.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/docstrings_queue.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/download_opencv_resources.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/extract_test_quality_in_progress.py` → p0000 | clases: `BacklogItem` | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/generate_comprehensive_report.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/generate_coverage_report.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/generate_daniel_doc.py` → p0000 | clases: `FileIndexInfo`, `DirIndexNode` | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/generate_monolitos_finales.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/generate_quotes_db.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/init_database.py` → p0000 | Mypy: Sí (`disallow_untyped_defs=True` en `mypy.ini` (fase avanzada/módulo completado).)

- `scripts/inject_module_docstrings.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/legacy_analyzer.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/list_mypy_core_services_gaps.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/monolith_analyzer.py` → p0000 | clases: `FileNode`, `GraphStats`, `_ImportCollector` | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/print_summary.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/profile_queries.py` → p0000 | clases: `QueryCounter` | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/refine_module_descriptions.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/reorder_docstring_before_future.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/run_quality_audit.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/security_audit_analyzer.py` → p0000 | clases: `SecurityAuditAnalyzer` | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/seed_data.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/sync_worktree_to_icloud.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/test_quality_analyzer.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/track_docx_dependencias.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/ui_dto_boundary_analyzer.py` → p0000 | clases: `Finding` | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/ui_dto_boundary_decision_report.py` → p0000 | clases: `Decision` | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el

esfuerzo; se prioriza estabilidad.)

- `scripts/ui_dto_findings_catalog.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/update_readme_metrics.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/update_test_imports.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/verify_migration.py` → p0000 | clases: `CodeIssue` | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/verify_qr_optimization.py` → p0000 | clases: `TestQrScannerOptimization` | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/verify_structure.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `scripts/windows_path_audit.py` → p0000 | clases: `Finding` | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `tools/__init__.py` → p0000 | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `tools/analyze_app_controller.py` → p0000 | clases: `AppControllerVisitor` | Mypy: Parcial (configuración gradual: `disallow_untyped_defs=False` en `mypy.ini` tipar todo al 100% no compensa el esfuerzo; se prioriza estabilidad.)
- `ui/__init__.py` → p0000 | Mypy: Sí (`disallow_untyped_defs=True` en `mypy.ini` (fase avanzada/módulo completado).)
- `ui/main_window.py` → p0000 | clases: `MainView` | Mypy: Sí (`disallow_untyped_defs=True` en `mypy.ini` (fase avanzada/módulo completado).)
- `ui/startup_screen.py` → p0000 | clases: `StartupScreen` | Mypy: Sí (`disallow_untyped_defs=True` en `mypy.ini` (fase avanzada/módulo completado).)
- `ui/startup_screen_constants.py` → p0000 | Mypy: Sí (`disallow_untyped_defs=True` en `mypy.ini` (fase avanzada/módulo completado).)
- `ui/startup_screen_report.py` → p0000 | Mypy: Sí (`disallow_untyped_defs=True` en `mypy.ini` (fase avanzada/módulo completado).)
- `ui/startup_screen_ui.py` → p0000 | clases: `StartupSectionWidgets` | Mypy: Sí (`disallow_untyped_defs=True` en `mypy.ini` (fase avanzada/módulo completado).)

Vision General

Hipatia es un sistema industrial para la **Simulación y Optimización de Tiempos de Fabricación**. Permite gestionar flujos de trabajo complejos, planificar cargas de operarios y realizar trazabilidad en tiempo real mediante **códigos QR**.

Funcionalidades principales:

- Cálculo de tiempos de fabricación por procesos mecánicos y manuales
- Trazabilidad completa de componentes y órdenes de fabricación
- Asignación inteligente de trabajadores y máquinas
- Motor de simulación de escenarios de producción
- Gestión de backups, auditoría y seguridad por roles

Mantenimiento industrial, calidad de tests y análisis estático

Estado documentado del repo (actualizado en la generación de esta documentación):

- **scripts/maintenance/** — `backup_database.py` y `reset_admin.py` con **tipado estricto** según `[mypy-scripts.maintenance.*]` en `mypy.ini` (`ignore_errors = False`, `disallow_untyped_defs = True`). El `reset de admin` usa `DatabaseConfig` y raíz del proyecto vía `Path(__file__).resolve().parents[2]` para imports fiables.
- **scripts/detect_dead_code.py** — Analiza el **paquete** `ui/dialogs/` (no el monolito antiguo), genera `Documentacion/Analisis_Codigo_Muerto_ui_dialogs.md` con claves `ruta.py::Clase` y sección de **0 eliminaciones** cuando la heurística no marca métodos muertos (revisión manual obligatoria antes de borrar código).
- **scripts/test_quality_analyzer.py** — Distingue penalizaciones **inevitables** (p. ej. `MagicMock()` en la misma línea que nombres típicos de **widgets Qt**) de penalizaciones **corregibles**; en archivos con `PyQt6`, los **repositorios y servicios del proyecto** siguen pudiendo (y deben) usar `create_autospec` donde aplique (ver `skill_testing_fixtures_y_mocks`).
- **Tests de servicios** — Ejemplos endurecidos: `test_worker_service.py` y `test_machine_service.py` usan `create_autospec(DatabaseManager, instance=True)` y `create_autospec` de los repositorios reales; `test_historial_controller_comprehensive.py` usa `create_autospec` en `IterationRepository` y `ProductRepository` donde corresponde.
- **Historial PDF** — `HistorialReportManager` obtiene el historial de iteraciones vía `db.iteration_repo.get_product_iterations` (alineado con `IterationRepository` y con `interaction_manager`).

Arquitectura del Sistema

Patrón **MVC modernizado** con inyección de dependencias vía `DIContainer`. Cada capa tiene responsabilidad única y se comunica hacia abajo:

Capa	Tecnología	Responsabilidad
UI	PyQt6	Widgets, diálogos, señales/slots
Controllers	Python	Orquestación, delegación, manejo de eventos UI
Services	Python	Lógica de negocio pura, sin dependencia de UI
Core / AppModel	Python	Fachada que expone servicios a los controllers
Database	SQLAlchemy 2.0	Repositorios, modelos ORM, migraciones Alembic

Regla de Arquitectura: Fase 12C (DTO-First)

La Fase 12C define una frontera estricta entre UI y dominio:

- La UI no debe manipular diccionarios crudos de negocio.
- El intercambio entre capas se realiza con DTOs (`*DTO`).
- Los analizadores de frontera verifican que no se reintroduzcan accesos legacy en UI.
- `PrepStepsWidget` lee filas de preproceso/fase con `_ui_record_field` (dict o DTO), evitando `preproceso['id']` en la lista.
- **Vista trabajador:** `WorkerDbSync.get_assigned_fabricaciones` devuelve `WorkerTaskListRowDTO`; `WorkerMainWindow` mantiene la selección tipada y las señales emiten `to_signal_dict()` donde el

receptor aún espera dict.

- Nueva iteración de producto: `AddIterationFormData` en el diálogo; `ProductIterationsWidget` usa `asdict(form)` al llamar al controlador.
- **CI** ejecuta `scripts/ui_dto_boundary_analyzer.py --enforce-zero`: el job **falla** si hay hallazgos en el alcance por defecto de `ui/` (sin `continue-on-error`). En fallo, el workflow sube el artefacto `ui_dto_boundary_report.json` para depuración.

Desacoplamiento UI: widgets hoja frente a MainView

MainView sigue siendo el lugar que recibe `AppController` para navegación, backup y casos especiales (p. ej. settings). Los **widgets hoja** y diálogos reutilizables deben preferir:

- Inyección de **servicios, fachadas** (`ProductFacade`, ...) o **controladores de dominio acotados** (`ProductController`) resueltos desde `DIContainer`.
- **Señales PyQt** o callbacks mínimos (`show_warning`, abrir fichero) en lugar de `controller.view.show_message` desde componentes reutilizables.
- **GestionDatosWidget**: pestañas construidas con dependencias del contenedor donde proceda; **PrepStepsWidget**: validación expuesta vía señales hacia el padre.
- **Tests de cableado**: arranque con `MainView` real sustituye `QChartView` por un `QWidget` hijo válido en `layout`; `WorkersWidget()` sin `controller=` tras registrar `WorkerController` en DI; `PreprocesoDialog` recibe `material_port` (no el hub completo).

```
graph TD
    subgraph UI["📱 Capa UI (PyQt6)"]
        MV[MainView - único orquestador que retiene AppController]
        subgraph REP["Reportes: hijos sin AppController"]
            RW[ReportesWidget]
            SSW[SmartSearchWidget]
            OLW[OrderListWidget]
            RCH[ReportsChartsWidget]
            RW --> SSW
            RW --> OLW
            RW --> RCH
        end
        end
        subgraph LEAF["Widgets hoja / diálogos: dependencias explícitas"]
            PMW[ProductMaterialsWidget]
            PIW[ProductIterationsWidget]
            PDD[ProductDetailsDialog]
            GDW[GestionDatosWidget pestañas vía DI]
            PSW[PrepStepsWidget señales / notificador]
            PMW --> PF[ProductFacade / servicios inyectados]
            PIW --> PF
            PDD --> PMW
            PDD --> PIW
        end
        end
        subgraph WK["Vista trabajador"]
            WMW[WorkerMainWindow filas WorkerTaskListRowDTO]
        end
        end
        WOTH[Otras páginas: Home Historial Fabricación Settings ...]
        DLG[Otros diálogos DefineFlow Bitácora Prep ...]
        MV --> RW
        MV --> WOTH
        MV --> DLG
        MV --> LEAF
        MV --> WK
    end

    subgraph CTRL["⚙️ Capa Controllers"]
        AC[AppController coordinador]
        ST[StartupController]
        SC[SessionController]
        LC[LoteController]
        FC[FabricacionController]
        PC[ProductController]
        WC[WorkerController]
        SIMC[SimulationController]
        RPC[ReportController]
    end
```

```

HRC[HistorialController + HistorialReportManager]
BCIO[BackupControllerIOManager ZIP/TAR import export sync]
AC --> SC
AC --> LC
AC --> FC
AC --> PC
AC --> WC
AC --> SIMC
AC --> RPC
AC --> HRC
AC -.->|copias y fusión BD| BCIO
ST -.->|registro DI + wiring| AC
end

subgraph CORE[" Capa Core: DI + servicios + fachada"]
  DI[DContainer singleton]
  AM[AppModel fachada]
  RS[ReportService]
  WS[WorkerService]
  PS[ProductService]
  FS[FabricacionService]
  PLS[PilaService]
  LM[LabelManager]
  QR[QrGenerator]
  ENG[SimulationEngine]
  AC -->|self.container| DI
  DI -->|resolve| RS
  DI -->|resolve| PF
  AM --> WS
  AM --> PS
  AM --> FS
  AM --> PLS
  AM --> RS
  AM --> LM
  AM --> ENG
  LM --> QR
end

subgraph DB["🗄 Capa Database"]
  DM[DatabaseManager]
  SYN[SyncService compare/apply SQLite]
  DM -.->|compare_with_db apply_sync_changes| SYN
  WR[WorkerRepository]
  PR[ProductRepository]
  IR[IterationRepository]
  TR[TrackingRepository]
  LR[LabelCounterRepo]
  RPR[ReportsRepository]
  DM --> WR
  DM --> PR
  DM --> IR
  DM --> TR
  DM --> LR
  DM --> RPR
end

subgraph SCR["🔧 Scripts mantenimiento y calidad"]
  BK[backup_database.py mypy estricto]
  RA[reset_admin.py mypy estricto]
  DC[detect_dead_code.py paquete ui/dialogs]
  TQA[test_quality_analyzer.py techo vs corregible]
  UDA[ui_dto_boundary_analyzer Fase 12C gate CI]
end

MV -->|set_controller| AC
RW -->|.container → ReportService; si no, .model.report_service| AC
SSW -->|search_reports_data| RS
OLW -->|get_orders_for_product| RS
RCH -->|stats y gráficas| RS
WOTH -->|señales| AC

```

DLG -->|señales| AC
HRC -->|informes PDF iteraciones| IR
FC -.->|delegación| FS
SIMC -.->|motor| ENG
CTRL --> CORE
RS --> RPR
CORE --> DB
SCR -.->|no runtime app| DB

Matriz RBAC (Roles vs Permisos)

Permiso	ADMIN	RESPONSABLE	OPERARIO	INVITADO
MANAGE_USERS	Sí	Sí	No	No
VIEW_PRODUCTS	Sí	Sí	Sí	No
CREATE_PRODUCT	Sí	Sí	No	No
EDIT_PRODUCT	Sí	Sí	No	No
DELETE_PRODUCT	Sí	Sí	No	No
VIEW_FABRICATIONS	Sí	Sí	Sí	No
CREATE_FABRICATION	Sí	Sí	No	No
EDIT_FABRICATION	Sí	Sí	No	No
DELETE_FABRICATION	Sí	Sí	No	No
MANAGE_MACHINES	Sí	Sí	No	No
VIEW_DASHBOARD	Sí	Sí	Sí	No
GENERATE_REPORTS	Sí	Sí	No	No
MANAGE_SETTINGS	Sí	Sí	No	No
VIEW_HISTORY	Sí	Sí	No	No

Defensa en profundidad RBAC (controladores)

Además del filtrado de UI en SessionController, operaciones sensibles usan @require_permission (core/security/access_control.py):

Área	Permiso	Entrypoints
Backup ZIP (import / export / sync) y diálogo restore	MANAGE_SETTINGS	BackupController.on_import_databases, on_export_databases, on_sync_databases, show_backup_restore_dialog
PDF desde historial	GENERATE_REPORTS	HistorialReportManager.on_print_report_clicked
Productos, fabricaciones, máquinas, usuarios, preprocesos	(matriz anterior)	ProductController / managers, MachineController, TaskManager, PreprocesoManager, etc.

Simulación: RegistroTemporal

- SQLite en archivo temporal con **WAL** para reducir pérdida si el proceso termina entre vaciados del buffer.
- `MotorDeEventos.ejecutar_simulacion` **confía en** `consultar_eventos` (vaciado previo del buffer) y en **finally**: `registro_temporal.cleanup()`.
- `RegistroTemporal.cleanup()` **borra el .db y los compañeros** `-wal / -shm`.

Política AppModel y nuevas features

- Registrar servicios en `DIContainer` y resolver dependencias en controladores; evitar nuevos delegadores en `AppModel` salvo señales Qt o compatibilidad documentada.
- Poda de métodos delegadores de `AppModel` solo si **cero usos** en el repo (búsqueda con `rg`), con `pytest/mypy` en módulos tocados.

Acciones Auditables

Acción	Disparo principal	Registro
LOGIN	<code>SessionController.handle_login</code>	<code>AuditLogger.log_login</code>
LOGOUT	<code>SessionController.logout</code>	<code>AuditLogger.log_logout</code>
DELETE	Operaciones sensibles de datos	<code>AuditLogger.log_delete</code>
EXPORT	Exportaciones de reportes/datos	<code>AuditLogger.log_export</code>
IMPORT	Importaciones y restauraciones	<code>AuditLogger.log_import</code>
SETTINGS_CHANGE	Cambios de configuración	<code>AuditLogger.log_settings_change</code>

Arbol de Carpetas

```
graph TD
    ROOT["📁 Hipatia (raíz)"]
    ROOT --> controllers["📁 controllers"]
    controllers --> controllers_historial["historial/"]
    controllers --> controllers_pila["pila/"]
    controllers --> controllers_product["product/"]
    controllers --> controllers_simulation["simulation/"]
    controllers --> controllers_worker["worker/"]
    ROOT --> core["📁 core"]
    core --> core_camera_manager["camera_manager/"]
    core --> core_facades["facades/"]
    core --> core_health["health/"]
    core --> core_import_manager["import_manager/"]
    core --> core_interfaces["interfaces/"]
    core --> core_label_manager["label_manager/"]
    ROOT --> database["📁 database"]
    database --> database_models["models/"]
    database --> database_repositories["repositories/"]
    ROOT --> ui["📁 ui"]
    ui --> ui_dialogs["dialogs/"]
    ui --> ui_widgets["widgets/"]
    ui --> ui_worker["worker/"]
    ROOT --> features["📁 features"]
    ROOT --> scripts["📁 scripts"]
    scripts --> scripts_analysis["analysis/"]
    scripts --> scripts_maintenance["maintenance/"]
    ROOT --> tests["📁 tests"]
    tests --> tests_controllers["controllers/"]
    tests --> tests_db["db/"]
    tests --> tests_debugging["debugging/"]
    tests --> tests_e2e["e2e/"]
    tests --> tests_integration["integration/"]
    tests --> tests_logic["logic/"]
```

```
ROOT --> migrations["📦 migrations"]
migrations --> migrations_versions["versions/"]
```

Carpeta	Contenido
controllers/	Controladores MVC — uno por dominio funcional
core/	AppModel, servicios de negocio, DTOs, simulación, seguridad
database/	Modelos SQLAlchemy, repositorios, DatabaseManager
ui/	Widgets PyQt6, diálogos, ventana principal
features/	Módulos de funcionalidad transversal (worker sync, validación)
scripts/	Generación de docs (generate_daniel_doc), QA (test_quality_analyzer), auditoría frontera UI/DTO Fase 12C (ui_dto_boundary_analyzer, gate en CI con --enforce-zero), detección de código muerto (detect_dead_code), mantenimiento crítico (maintenance/: backup BD, reset admin), init_database.py con mypy estricto en CI
tests/	Suite de tests (unit, integration, e2e)
migrations/	Migraciones Alembic de la base de datos

Modelo de Base de Datos ERD

Esquema completo extraído de los modelos SQLAlchemy en database/models/. Las tablas de enlace Many-to-Many (fabricacion_preproceso_link, trabajador_fabricacion_link, etc.) están implícitas en las relaciones.

```
erDiagram
    Producto {
        string codigo PK
        string descripcion
        string departamento
        int tipo_trabajador
        float tiempo_optimo
    }
    Subfabricacion {
        int id PK
        string producto_codigo FK
        string descripcion
        float tiempo
        int maquina_id FK
    }
    ProcesoMecanico {
        int id PK
        string producto_codigo FK
        string nombre
        float tiempo
    }
    ProductIteration {
        int id PK
        string producto_codigo FK
        datetime fecha_creacion
        string nombre_responsable
    }
    Material {
        int id PK
        string codigo_componente
    }
    Preproceso {
        int id PK
        string nombre
```

```

        float tiempo
        int tipo_trabajador
    }
    Fabricacion {
        int id PK
        string codigo
        string descripcion
    }
    Trabajador {
        int id PK
        string nombre_completo
        string username
        string role
        int tipo_trabajador
        bool activo
    }
    Maquina {
        int id PK
        string nombre
        string departamento
        bool activa
    }
    GrupoPreparacion {
        int id PK
        string nombre
        int maquina_id FK
        string producto_codigo FK
    }
    PreparacionPaso {
        int id PK
        int grupo_id FK
        string nombre
        float tiempo_fase
        bool es_diario
    }
    Pila {
        int id PK
        string nombre
        string producto_origen_codigo FK
    }
    Lote {
        int id PK
        string codigo
    }

    Producto ||--o{ Subfabricacion : "tiene"
    Producto ||--o{ ProcesoMecanico : "tiene"
    Producto ||--o{ ProductIteration : "tiene"
    Producto }o--o{ Material : "requiere"
    Producto }o--o{ Lote : "agrupa"
    Preproceso }o--o{ Material : "consume"
    Preproceso }o--o{ Fabricacion : "vincula"
    Fabricacion }o--o{ Trabajador : "asignados"
    Fabricacion }o--o{ Lote : "agrupa"
    Maquina ||--o{ GrupoPreparacion : "tiene"
    Maquina ||--o{ Subfabricacion : "ejecuta"
    GrupoPreparacion ||--o{ PreparacionPaso : "contiene"
    GrupoPreparacion }o--o| Producto : "especifico_de"
    Pila }o--o| Producto : "origen"

```

Leyenda del Diccionario de Datos

El esquema ERD contiene campos de control lógico cuyo significado literal es:

- **tipo_trabajador** (INT) en Tablas **Producto**, **Subfabricacion**, **ProcesoMecanico** y **Trabajador**:
 - 1: **Operario Básico / Junior** (Capaz de montar y ejecutar tareas estándar).
 - 2: **Especialista / Mid** (Capaz de operar maquinaria pesada, Tornos/Fresas y programación CNC sencilla).

Modelos principales

Modelo	Tabla	Descripción
Producto	productos	Catálogo de productos con tiempos y procesos
Fabricacion	fabricaciones	Orden de fabricación (OF)
Trabajador	trabajadores	Operarios y administradores
Maquina	maquinas	Recursos físicos de planta
Preproceso	preprocesos	Tareas preparatorias reutilizables
Material	materiales	Materias primas y componentes (BOM)
Pila	pilas	Plan de producción para simulación
Lote	lotes	Agrupación logística de fabricaciones
GrupoPreparacion	grupos_preparacion	Pasos de setup de máquina

Flujos Principales

Flujo de Fabricación y Trazabilidad QR

```
sequenceDiagram
    actor U as Operario
    participant UI as UI (Widget)
    participant CTRL as FabricacionController
    participant SVC as FabricacionService
    participant DB as Repository
    participant QR as QR Generator

    U->>UI: Crear nueva Fabricación
    UI->>CTRL: on_create_fabricacion(datos)
    CTRL->>SVC: create_fabricacion(codigo, preprocesos)
    SVC->>DB: fabricacion_repo.add(fabricacion)
    DB-->>SVC: fabricacion_id
    SVC->>QR: generate_labels(fabricacion_id, unidades)
    QR-->>SVC: rutas_etiquetas[]
    SVC-->>CTRL: FabricacionDTO
    CTRL->>UI: refresh_view()
    UI-->>U: Etiquetas QR listas para imprimir

    U->>UI: Escanear QR (inicio tarea)
    UI->>CTRL: on_qr_scan(qr_data)
    CTRL->>SVC: registrar_inicio_trabajo(qr_data)
    SVC->>DB: tracking_repo.log_inicio(trabajador_id, fabricacion_id)
    DB-->>SVC: TrabajoLog creado
    SVC-->>CTRL: ok
    CTRL->>UI: actualizar_estado_operario()
```

Flujo de Importación BOM (A3RP)

```
flowchart TD
    excelFile["Archivo Excel A3RP"] --> excelAdapter["A3RPExcelAdapter.parse_file"]
    excelAdapter --> bomNodeTree["BOMNodeDTO tree"]
    bomNodeTree --> previewDialog["BOMImportPreviewDialog (supervisión)"]
    previewDialog --> bomService["BOMImportService.import_bom_tree"]
    bomService --> productRepo["ProductRepository (crear/actualizar productos)"]
    bomService --> materialRepo["MaterialRepository (crear/vincular materiales)"]
```

```
bomService --> relationLayer["Relaciones BOM producto-material"]
```

Flujo de Simulación y Gantt

```
graph LR
    A[Pila de Trabajo] --> B[SimulationEngine]
    B --> C{Motor de Eventos}
    C --> D[Asignar Trabajador]
    C --> E[Asignar Máquina]
    D --> F[Calcular Tiempo]
    E --> F
    F --> G{¿Conflicto?}
    G -- Sí --> H[Replanificar]
    H --> C
    G -- No --> I[TimelineTask]
    I --> J[ResultsCompiler]
    J --> K[GanttWidget]
    J --> L[SimulationDTO]
```

Sistema de Etiquetado QR (Apli 1861)

Arquitectura desacoplada para la generación de etiquetas de trazabilidad. Soporta plantillas Word estáticas y generación dinámica de cuadrículas A5 (66 etiquetas).

```
graph TD
    subgraph SVC["📦 Core Services"]
        LM[LabelManager]
        QR[QrGenerator]
    end

    subgraph PORT["🔌 Ports (Interfaces)"]
        IDG[IDocumentGenerator]
    end

    subgraph ADAPT["🔌 Adapters (Infraestructure)"]
        DOCX[DocxGeneratorAdapter - Plantillas]
        APLI[Apli1861LabelGenerator - Dinámico A5]
    end

    LM --> IDG
    IDG <|-- DOCX
    IDG <|-- APLI
    LM --> QR
    APLI -->|Genera| DOC[Archivo .docx A5 / 66 etiquetas]
```

Flujo de Sincronización Offline/USB

```
sequenceDiagram
    actor U as Usuario
    participant UI as SyncDialog
    participant SVC as SyncService
    participant FDB as SQLiteExterna
    participant LDB as SQLiteLocal

    U->>UI: Seleccionar archivo .db en USB
    UI->>SVC: compare_databases(foreign_db_path)
    SVC->>FDB: Leer tablas sincronizables
    SVC->>LDB: Leer tablas locales
    SVC-->>UI: DatabaseComparisonDTO
    UI-->>U: Mostrar diferencias por tabla

    U->>UI: Confirmar cambios seleccionados
    UI->>SVC: apply_changes(DatabaseComparisonDTO)
    SVC->>LDB: Upsert por SyncRecordDTO
    LDB-->>SVC: Commit
    SVC-->>UI: Total de cambios aplicados
    UI-->>U: Sincronizacion completada
```


Flujo de Login y Autorización

```
sequenceDiagram
    actor U as Usuario
    participant DLG as LoginDialog
    participant RL as RateLimiter
    participant WS as WorkerService
    participant AL as AuditLogger
    participant SS as SecurityService
    participant UI as MainView

    U->>DLG: Introduce credenciales
    DLG->>RL: is_blocked(username)
    alt Usuario bloqueado
        RL-->>DLG: True
        DLG->>AL: log_login(fallido, bloqueado)
        DLG-->>U: Mensaje de bloqueo
    else Usuario permitido
        RL-->>DLG: False
        DLG->>WS: authenticate_user(username, password)
        alt Credenciales validas
            WS-->>DLG: AuthResponseDTO
            DLG->>RL: check_and_record_attempt(success=True)
            DLG->>AL: log_login(exitoso)
            DLG->>SS: login_user(user_data)
            DLG->>UI: _update_ui_for_role()
            UI-->>U: Acceso habilitado por permisos
        else Credenciales invalidas
            WS-->>DLG: None
            DLG->>RL: check_and_record_attempt(success=False)
            DLG->>AL: log_login(fallido)
            DLG-->>U: Credenciales incorrectas
        end
    end
end
```

Tecnologías

Librería	Versión
Python	3.12+
alembic	N/A
bcrypt	N/A
greenlet	N/A
psycopg2-binary	N/A
PyQt6	6.10.1
PyQt6-Charts	6.10.0
SQLAlchemy	2.0.45
pandas	2.2.3
openpyxl	3.1.5
Pillow	12.0.0
opencv-contrib-python	4.12.0.88
qrcode	8.0

python-docx	1.2.0
reportlab	4.4.7
jinja2	3.1.4
markdown-pdf	1.3.1
requests	2.32.0
concurrent-log-handler	0.9.25
graphviz	0.20.3
wikiquote	0.1.17
wikipedia	1.4.0
pytest	8.4.2
pytest-cov	6.0.0
pytest-html	4.1.1
pytest-qt	4.4.0
pytest-mock	3.14.0
pytest-timeout	2.3.1
pytest-env	1.1.3
coverage	7.6.0
pylint	3.3.0
bandit	1.7.10
flake8	7.1.0
mypy	1.8.0

¿Qué representa cada tecnología? (explicación simple)

- `Python`: el “lenguaje de trabajo” del proyecto; el sistema programa toda la lógica.
- `SQLAlchemy`: traduce objetos Python a “filas/tablas” en la base de datos.
- `PyQt6`: crea la interfaz de escritorio (ventanas, botones, diálogos y señales).
- `Alembic`: gestiona cambios graduales del esquema de la base de datos.
- `pytest`: ejecuta la suite de pruebas para asegurar que todo funciona tras cambios.
- `mypy`: revisa tipos de forma estática para detectar errores antes de ejecutar.
- `Mermaid`: dibuja diagramas (arquitectura, relaciones y flujos) dentro del documento.
- `markdown-pdf`: convierte el Markdown final a PDF listo para leer/compartir.

Instalacion y Despliegue

Requisitos base

- Python 3.11 o superior; CI en 3.11 y 3.12 (`.github/workflows/ci.yml`); referencia de tipado mypy `python_version = 3.12`; `.python-version` recomienda 3.12 para pyenv.
- Entorno virtual (`venv`) recomendado
- Dependencias instaladas desde `requirements.txt`

Instalación local (desarrollo)

1. Crear entorno virtual: `python -m venv .venv`
2. Activar entorno:
 - macOS/Linux: `source .venv/bin/activate`
 - Windows (PowerShell): `.venv\Scripts\Activate.ps1`
3. Instalar dependencias: `pip install -r requirements.txt`
4. Configurar entorno: copiar `.env.example` a `.env` y ajustar variables
5. Ejecutar aplicación: `python app.py`

Configuración de rutas (producción)

- Base de datos: `DB_TYPE` y `DB_PATH` (SQLite por defecto en `data/montaje.db`).
- Logs: `LOG_DIR` (por defecto `logs`).
- Backups: `BACKUP_DIR` (por defecto `backups`).
- Todas las rutas relativas se resuelven desde la raíz del proyecto.

Empaquetado para planta

- Script oficial: `python scripts/build_executable.py`
- Motor: PyInstaller
- Artefacto final: carpeta `dist/` con ejecutable Hipatia
- Incluye recursos críticos de migración (`migrations/` y `alembic.ini`).

Suite de Tests

Sección generada automáticamente desde `test_reports/compliance_data.json`.
Ejecutar `python3 scripts/test_quality_analyzer.py` para actualizar.

Filosofía de Testing

La suite de tests de Hipatia sigue un modelo de **calidad verificable** con tres principios fundamentales:

1. **Mocks estrictos por defecto** — `create_autospec()` o `MagicMock(spec=...)` para cualquier dependencia que tenga stubs de tipo disponibles. Esto garantiza que los tests fallen si la interfaz real cambia.
2. **Excepciones documentadas** — PyQt6 y python-docx no tienen stubs de tipo completos: en **widgets y diálogos Qt** suele usarse `MagicMock()` sin `spec`; el analizador solo trata como **inevitables** los mocks sueltos en líneas con indicios de widget Qt (heurística). Los **repositorios y servicios Python del proyecto** no entran en esa excepción: deben usar `create_autospec(ClaseReal, instance=True)` o `MagicMock(spec=[...])` acotado cuando proceda (ver `.agents/skills/testing_fixtures_y_mock/ SKILL.md`).
3. **Verificación de interacciones explícita** — En controladores y servicios, cada test que verifica una llamada usa `assert x.call_count == N` antes de `assert_called_once_with(...)`. Esto evita el antipatrón de `assert_called_once()` sin argumentos, que no verifica qué se pasó.
4. **Asserts observables por defecto** — `assert True` se considera un **smoke test** y solo se permite como último recurso, siempre documentado como `assert True # smoke_test: ...`. Si existe un observable (estado/retorno/interacción), se prefiere ese `assert`.

El marcador `pytestmark = pytest.mark.unit` se aplica a nivel de módulo en todos los archivos de tests

unitarios, permitiendo ejecutar subconjuntos con `pytest -m unit`.

Sistema de Scoring y Techo Real

El analizador `scripts/test_quality_analyzer.py` asigna un **score absoluto** (0-100) basado en criterios objetivos, y calcula un **score techo** que descuenta las penalizaciones inevitables por dependencias externas sin stubs.

Criterio	Puntos	Notas
Tiene <code>pytestmark 0 @pytest.mark.*</code>	+25	Obligatorio en todos los archivos
Usa <code>create_autospec / spec=</code>	+20	Para dependencias con stubs disponibles
Verifica interacciones (<code>assert_called*</code>)	+15	Obligatorio en controllers/services
Valida DTOs con <code>isinstance(..., XxxDTO)</code>	+15	Para tests de capa de servicio
Todos los <code>@patch</code> tienen <code>autospec=True</code>	+15	Excepto builtins/Qt/OS
Tiene docstrings en clases y métodos	+10	
<code>MagicMock()</code> sin <code>spec</code> (por mock)	-5 (máx -30)	Inevitable si usa PyQt6/docx
<code>@patch</code> sin <code>autospec</code> (por patch)	-3 (máx -20)	Inevitable para builtins/Qt/OS
Test sin ningún <code>assert</code> (por test)	-5 (máx -20)	Siempre corregible
<code>assert True</code> trivial sin justificar	-1 (máx -10)	Solo permitido con <code># smoke_test:</code>
<code>assert_called_once()</code> sin args	-3 (máx -15)	Antipatrón: no verifica argumentos

Cuando un archivo alcanza su **techo real** (`score optimizado = techo`), el analizador lo marca con  y explica la razón (p. ej. importa PyQt6 y el techo solo perdona parte de los `MagicMock()` sueltos en contexto de widgets). El estado del archivo (`Actualizado / En Progreso / Legacy`) se calcula sobre el score techo, no el absoluto, para no penalizar lo ya optimizado.

Resumen Global

Métrica	Valor
Archivos analizados	239
Actualizados (≥80 techo)	213
En Progreso (50-79)	26
Legacy / Pendiente (<50)	0
Score absoluto medio	75.1/100
Score optimizado medio	76.9/100
Archivos en su techo real	239/239

Detalle por Archivo

Columnas: **Score** = score absoluto · **Techo** = score máximo alcanzable · ✓ = en techo real · **Estado** = calculado sobre techo

Archivo	Score	Techo	Estado	Decisión técnica
conftest.py	100	100 ✓	Actualizado	—
__init__.py	100	100 ✓	Actualizado	—
__init__.py	100	100 ✓	Actualizado	—
audit_report_generator.py	100	100 ✓	Actualizado	—
test_iteration_repository.py	100	100 ✓	Actualizado	—
test_app_model.py	100	100 ✓	Actualizado	—
test_dashboard_widget.py	100	100 ✓	Actualizado	DummyChartView(C de QChartView; Da set_controller ni hub desde UIViewController
test_pila_controller_comprehensive.py	100	100 ✓	Actualizado	PilaService/Product FabricacionService create_autospec(); i MagicMock(spec=[n
test_calculate_times_widget.py	100	100 ✓	Actualizado	—
test_product_dialogs_coverage.py	70	100 ✓	Actualizado	Diálogos Qt hereda MagicMock() evita internos; ProductDe ``ProductCont...
test_a3rp_excel_adapter.py	100	100 ✓	Actualizado	—
test_lote_manager_isolated.py	70	100 ✓	Actualizado	create_autospec() c IProductService/IFa para garantizar que respetan las...
test_historial_controller_comprehensive.py	70	100 ✓	Actualizado	Widgets Qt del histo sin spec (inevitable iteration_repo y `create_autos...
test_preproceso_controller_comprehensive.py	100	100 ✓	Actualizado	—
test_machine_repository.py	100	100 ✓	Actualizado	—
test_prep_dialogs_coverage.py	100	100 ✓	Actualizado	—

test_navigation_controller_comprehensive.py	70	100 ✓	Actualizado	Widgets de destino (CalculateTimesWic DefinirLoteWidget, GestionDatosWidge isinstance() pero in.
test_pila_manager_isolated.py	70	100 ✓	Actualizado	create_autospec() c garantizar interfaz; (para isinstance() pe
test_calculation_controller_comprehensive.py	82	100 ✓	Actualizado	CalculateTimesWid MagicMock() con _ para isinstance; @patch('QFileDia
test_worker_controller_comprehensive.py	70	100 ✓	Actualizado	—
__init__.py	100	100 ✓	Actualizado	—
test_camera_manager_full.py	100	100 ✓	Actualizado	—
test_ui_controller_comprehensive.py	70	100 ✓	Actualizado	HomeWidget y widg Qt → MagicMock() ; llamadas asíncrona: assert...
test_lote_controller_comprehensive.py	70	100 ✓	Actualizado	Componentes Qt (Q QSpinBox, PyQtSigr antes de importar L evitar SIGAB...
test_ui_signals_controller_comprehensive.py	100	100 ✓	Actualizado	—
test_worker_main_window.py	70	100 ✓	Actualizado	WorkerMainWindow (PyQt6) → MagicMc widgets internos; us simulado con M...
test_preproceso_repository.py	100	100 ✓	Actualizado	—
test_product_controller_v2_comprehensive.py	100	100 ✓	Actualizado	mock_app con create_autospec(Ap servicios/repos; Pre aserta con material_
test_app_controller_orchestration.py	100	100 ✓	Actualizado	—
test_machine_controller_comprehensive.py	70	100 ✓	Actualizado	MachinesWidget y C son Qt → MagicMoc servicio de seguridad globalmente con au
test_qapp_crash.py	100	100 ✓	Actualizado	—

test_simulation_controller_comprehensive.py	100	100 ✓	Actualizado	SimulationEngine por create_autospec() widgets de resultado MagicMock()
test_define_flow_dialog_edge.py	70	100 ✓	Actualizado	DefineProductionFlow de DefineControlPanel sustituido por FakeControlPanel(C
test_backup_controller_comprehensive.py	100	100 ✓	Actualizado	—
test_product_controller_preprocesos.py	70	100 ✓	Actualizado	ProductController de AppController → Ma QDialog/QMessage patch() para in...
test_di_container_lifecycle.py	100	100 ✓	Actualizado	—
test_charts_container.py	100	100 ✓	Actualizado	—
__init__.py	100	100 ✓	Actualizado	—
__init__.py	100	100 ✓	Actualizado	—
__init__.py	100	100 ✓	Actualizado	—
macos_fix.py	100	100 ✓	Actualizado	—
__init__.py	100	100 ✓	Actualizado	—
test_product_repository_db.py	100	100 ✓	Actualizado	—
__init__.py	100	100 ✓	Actualizado	—
__init__.py	100	100 ✓	Actualizado	—
test_backup_restore_dialog.py	100	100 ✓	Actualizado	Diálogo Qt → Magic para todos los widg
test_flow_item_widget.py	100	100 ✓	Actualizado	—
test_fabrication_dialogs.py	82	94 ✓	Actualizado	—
test_report_controller_comprehensive.py	77	92 ✓	Actualizado	Controlador con mú MagicMock() para v create_autospec p negocio puros
test_cycle_reproduction.py	85	85 ✓	Actualizado	—

test_backup_integration.py	85	85 ✓	Actualizado	—
test_audit_report_generator.py	85	85 ✓	Actualizado	—
test_features_worker_controller.py	85	85 ✓	Actualizado	—
test_preparation_service.py	85	85 ✓	En Progreso	—
test_app_coverage.py	85	85 ✓	Actualizado	Cobertura de app.py @patch('QApplica' (Qt inevitable)
test_label_manager.py	85	85 ✓	Actualizado	python-docx sin stul sys.modules['doc: en módulo; create_autospec(para logger;...
test_scheduler_logic.py	85	85 ✓	Actualizado	Lógica de planificac mocks, solo DTOs r
test_ui_scaler.py	85	85 ✓	Actualizado	—
test_lotes_widget.py	85	85 ✓	Actualizado	—
test_security_improvements.py	85	85 ✓	Actualizado	Módulo de segurida create_autospec f validadores
test_apli_adapter.py	85	85 ✓	Actualizado	—
test_products_widget.py	85	85 ✓	Actualizado	—
test_historial_widget.py	85	85 ✓	Actualizado	—
test_fabrication_dialogs_coverage.py	85	85 ✓	Actualizado	CreateFabricacionD (PyQt6) → MagicMc widgets; objetos Pre simulad...
test_machine_service.py	85	85 ✓	En Progreso	MachineService pur create_autospec(l instance=True) Y `create_autospec(M ...
test_reports_widgets.py	85	85 ✓	Actualizado	StatCard, OrderList SmartSearchWidge ReportsChartsWidg QFrame (PyQt6) → inevitable...
test_backup_controller.py	85	85 ✓	Actualizado	—
test_enhanced_flow_dialog.py	70	85 ✓	Actualizado	—
test_phase5_di_injection.py	85	85 ✓	Actualizado	—
test_product_service_delegation.py	85	85 ✓	Actualizado	—
test_maintenance_service.py	85	85 ✓	En Progreso	—

test_dialog_dependencies.py	85	85 ✓	Actualizado	—
test_reports_infrastructure.py	85	85 ✓	Actualizado	Infraestructura de re create_autospec F @patch('builtins
test_inspector_panel.py	85	85 ✓	Actualizado	—
test_widgets_coverage.py	85	85 ✓	Actualizado	—
test_product_service_core_paths.py	85	85 ✓	Actualizado	—
test_health_checker.py	85	85 ✓	Actualizado	—
test_define_flow_dialog.py	85	85 ✓	Actualizado	—
test_database_manager_full.py	85	85 ✓	Actualizado	—
test_settings_widget.py	85	85 ✓	Actualizado	SettingsWidget Qt - inevitable; create_ ScheduleController
test_enhanced_flow_presenter.py	85	85 ✓	Actualizado	—
test_configuration_repository.py	85	85 ✓	En Progreso	—
test_worker_validation_service.py	85	85 ✓	En Progreso	—
test_common_dialogs.py	85	85 ✓	Actualizado	—
test_preprocesos_widget.py	85	85 ✓	Actualizado	—
test_product_service.py	85	85 ✓	En Progreso	—
test_security_phase2_integration.py	85	85 ✓	Actualizado	Test de integración BD real en memoria de repositorio
test_tracking_repository_stats_export.py	85	85 ✓	En Progreso	—
test_charts_widget.py	85	85 ✓	Actualizado	—
test_session_controller_comprehensive.py	85	85 ✓	Actualizado	SessionController si directas → create_ todos los servicios
test_backup_service.py	85	85 ✓	En Progreso	—
test_startup_controller.py	85	85 ✓	Actualizado	StartupController on MagicMock (spec=[subsistema
test_health_test_runner.py	85	85 ✓	Actualizado	—
test_create_fabricacion_dialog.py	80	85 ✓	Actualizado	—
test_visual_effects.py	85	85 ✓	Actualizado	—

test_worker_service.py	85	85 ✓	En Progreso	WorkerService puro create_autospec (l instance=True) y en WorkerRepositor
test_define_flow_presenter.py	85	85 ✓	Actualizado	DefineFlowPresente create_autospec (l mocks de preparat consultas de...
test_app_model_coverage.py	85	85 ✓	Actualizado	—
test_widgets_dashboard.py	85	85 ✓	Actualizado	—
test_reportes_widget.py	85	85 ✓	Actualizado	ReportesWidget (hu container (DI) y/o model.report_ser create_autospec (l sub...
test_smart_search.py	85	85 ✓	Actualizado	SmartSearch puro F create_autospec f dependencias exter
test_bom_import_service.py	85	85 ✓	En Progreso	—
test_order_list_widget.py	85	85 ✓	Actualizado	—
test_qr_scanner.py	85	85 ✓	Actualizado	QR Scanner con cá @patch ('cv2.Vide autospec (C extensi
test_worker_db_sync.py	85	85 ✓	Actualizado	—
test_machine_controller.py	85	85 ✓	Actualizado	—
test_flow_canvas.py	85	85 ✓	Actualizado	—
test_order_list.py	85	85 ✓	Actualizado	—
test_bitacora_dialog.py	85	85 ✓	Actualizado	FabricacionBitacora controller.model O pila_service; y llamadas ...
test_product_repository.py	85	85 ✓	En Progreso	—
test_hardware_controller.py	85	85 ✓	Actualizado	—
test_historial_report_manager_security.py	85	85 ✓	Actualizado	require_permissi set_security_ser MagicMock (spec=S sin Qt real
test_event_engine_comprehensive.py	85	85 ✓	Actualizado	—
test_timeline_widget.py	85	85 ✓	Actualizado	TimelineWidget Qt p mocks MagicMock (
test_flow_builder_service.py	85	85 ✓	En Progreso	—

test_schedule_controller_comprehensive.py	63	85 ✓	Actualizado	SettingsWidget es C con __class__ forz _make_db/_make_v _make_schedule_m
test_file_controller.py	85	85 ✓	Actualizado	—
test_dialogs.py	85	85 ✓	Actualizado	—
test_workers_widget.py	85	85 ✓	Actualizado	—
test_report_strategy_comprehensive.py	85	85 ✓	Actualizado	Estrategias de repor create_autospec f concreta
test_tracking_exceptions.py	85	85 ✓	Actualizado	Excepciones de dor sin mocks, solo inst
test_worker_camera_config.py	85	85 ✓	Actualizado	—
test_fabricacion_controller_comprehensive.py	85	85 ✓	Actualizado	—
test_simulation_events_comprehensive.py	85	85 ✓	Actualizado	Eventos de simulaci create_autospec f repositorios
test_controller_interface.py	85	85 ✓	Actualizado	—
test_tracking_assignment_service.py	85	85 ✓	En Progreso	TrackingService pur create_autospec f repositorios
test_widgets_integration.py	85	85 ✓	Actualizado	WorkersWidget() tra WorkerController en señales a manager
test_app_model_services_setup.py	85	85 ✓	Actualizado	—
test_reports_ui_integration.py	85	85 ✓	Actualizado	—
test_pila_integration.py	85	85 ✓	Actualizado	—
test_iteration_setup.py	85	85 ✓	Actualizado	—
test_app_services_e2e_setup.py	85	85 ✓	Actualizado	—
test_machine_workflow.py	85	85 ✓	Actualizado	—
test_iteration_workflow.py	85	85 ✓	Actualizado	—
test_security_workflow.py	85	85 ✓	Actualizado	—
test_product_workflow.py	85	85 ✓	Actualizado	—
test_dialogs_e2e.py	85	85 ✓	Actualizado	—
test_backup_audit_e2e.py	85	85 ✓	Actualizado	—
test_fabricacion_manager.py	85	85 ✓	Actualizado	—
test_material_manager.py	85	85 ✓	Actualizado	—
test_product_manager.py	85	85 ✓	Actualizado	—
test_preproceso_manager.py	85	85 ✓	Actualizado	—

test_utility_dialogs_coverage.py	65	65 ✓	Actualizado	—
test_reports_repository.py	65	65 ✓	En Progreso	Repositorio SQLAlc create_autospec (, sesión de BD
test_label_counter_repository.py	65	65 ✓	En Progreso	—
test_pila_repository.py	65	65 ✓	En Progreso	—
test_flow_simulation_service.py	65	65 ✓	En Progreso	—
test_lote_repository.py	65	65 ✓	En Progreso	—
test_password_service.py	65	65 ✓	En Progreso	—
test_worker_repository.py	65	65 ✓	En Progreso	—
test_detect_dead_code.py	65	65 ✓	Actualizado	Script de análisis es MethodExtractor, extract_package_ mockeado con `DIA
test_reports_integration.py	65	65 ✓	Actualizado	—
test_iteration_integration.py	65	65 ✓	Actualizado	—
test_label_counter_integration.py	65	65 ✓	Actualizado	—
test_material_repository.py	65	65 ✓	En Progreso	—
test_tracking_repository_unit.py	65	65 ✓	En Progreso	—
test_pila_service_planning_session.py	60	60 ✓	Actualizado	—
test_flow_toolbar.py	60	60 ✓	Actualizado	—
test_create_dialog.py	50	55 ✓	Actualizado	—
test_audit_infra.py	50	50 ✓	Actualizado	—
test_common_production_dialogs.py	50	50 ✓	Actualizado	—
test_inspector_presenter.py	50	50 ✓	Actualizado	—
test_connection_dialog_comprehensive.py	50	50 ✓	Actualizado	—
test_architecture_layer_edges.py	50	50 ✓	Actualizado	—
test_qt_log_handler.py	50	50 ✓	Actualizado	—
test_help_widget.py	50	50 ✓	Actualizado	—
test_log_terminal_widget.py	50	50 ✓	Actualizado	—

test_dialog_integration_smoke.py	50	50 ✓	Actualizado	CycleEndConfigDialog ReassignmentRuleDialog DefinirCantidadesDialog (PyQt6) → MagicMock
test_ui_opt3_features_no_ui_imports.py	50	50 ✓	Actualizado	—
test_code_quality_config.py	50	50 ✓	Actualizado	—
test_schedule_helpers.py	50	50 ✓	Actualizado	—
test_tracking_repository_coverage_fix.py	50	50 ✓	En Progreso	—
test_prep_steps_widget.py	50	50 ✓	Actualizado	PrepStepsWidget: a con validation_widget qbot.waitSignal en va...
test_report_sheets.py	50	50 ✓	Actualizado	Hojas de reporte co create_autospec (l libro Excel
test_temporal_storage.py	50	50 ✓	Actualizado	RegistroTemporal e real → un evento, c consultar_evento. finally
test_library_panel.py	50	50 ✓	Actualizado	TaskLibraryPanel es → MagicMock() in dependencias visua update_visual_state
test_bom_import_preview_dialog.py	50	50 ✓	Actualizado	—
test_startup_screen_report.py	50	50 ✓	Actualizado	—
test_database_config.py	50	50 ✓	Actualizado	—
test_opt4_ast_guard_no_static_ui_imports.py	50	50 ✓	Actualizado	—
test_protocols_imports.py	50	50 ✓	Actualizado	—
test_startup_screen_constants.py	50	50 ✓	Actualizado	—
test_paths.py	50	50 ✓	Actualizado	—
test_fabrication_module_structure.py	50	50 ✓	Actualizado	—
test_test_quality_analyzer_domain.py	50	50 ✓	Actualizado	—
test_tracking_dialogs.py	50	50 ✓	Actualizado	Diálogos de tracking MagicMock() para v create_autospec p
test_create_presenter.py	50	50 ✓	Actualizado	—
test_ui_opt2_fabrication_dialogs_boundary.py	50	50 ✓	Actualizado	—
test_machine_integration.py	50	50 ✓	Actualizado	—
test_worker_integration.py	50	50 ✓	Actualizado	—
test_configuration_integration.py	50	50 ✓	Actualizado	—

test_product_integration.py	50	50 ✓	Actualizado	—
test_preproceso_integration.py	50	50 ✓	Actualizado	—
test_docx_adapter.py	50	50 ✓	Actualizado	—
test_app_model_integration.py	50	50 ✓	Actualizado	—
test_widgets_setup.py	50	50 ✓	Actualizado	—
test_preproceso_setup.py	50	50 ✓	Actualizado	—
test_product_setup.py	50	50 ✓	Actualizado	—
test_machine_setup.py	50	50 ✓	Actualizado	—
test_pila_setup.py	50	50 ✓	Actualizado	—
test_dialogs_setup.py	50	50 ✓	Actualizado	—
test_worker_setup.py	50	50 ✓	Actualizado	—
test_macos_setup.py	50	50 ✓	Actualizado	—
test_tracking_repository_setup.py	50	50 ✓	En Progreso	—
test_pila_workflow.py	50	50 ✓	Actualizado	—
test_preproceso_workflow.py	50	50 ✓	Actualizado	—
test_worker_workflow.py	50	50 ✓	Actualizado	—
test_label_counter_e2e.py	50	50 ✓	Actualizado	—
test_package_compliance.py	50	50 ✓	Actualizado	—
test_main_header.py	50	50 ✓	Actualizado	—
test_main_nav_panel.py	50	50 ✓	Actualizado	—
test_flow_simulation_handler.py	50	50 ✓	Actualizado	—
test_planning_session_access.py	40	40 ✓	Actualizado	—
test_bom_importer.py	25	25 ✓	Actualizado	—

Referencia de Componentes

Extraído automáticamente de los docstrings del código fuente. Organizado por capa.

Raíz del proyecto

```
graph TD
  X[root] -->|depende| Y[Core]
```


Nombre del Módulo: app

Descripción: Punto de entrada principal para la aplicación Hipatia (cálculo de tiempos de fabricación).
Inicializa Qt, configuración de BD, logging y arranque de controladores.

Crea e instala ``QtLogHandler`` para la terminal visual de logs: tras el login se conecta a ``HomeWidget`` (rol Responsable u otros con vista principal) o a ``WorkerMainWindow`` (rol Trabajador, pestaña Log), una sola vez por sesión.

En ejecutable PyInstaller (Windows), ``\_fix\_qt\_macos`` no aplica; BD, logs y configuración editable se resuelven con ``core.paths`` (directorio del ``.exe``).

En ``main()`` , antes de ``QApplication(sys.argv)`` , se aplica ``QApplication.setHighDpiScaleFactorRoundingPolicy(PassThrough)`` cuando Qt6 lo permite para alinear el escalado fraccional del SO (p. ej. 125 % / 150 % en Windows) con el modo

-  `_check_dependencies`: Verifica e importa dependencias opcionales dinámicamente. En particular, intenta cargar OpenCV (cv2) para funcionalidades de cámara.
 -  `_fix_qt_macos`: Aplica correcciones específicas para macOS. Resuelve problemas conocidos de Qt con espacios en rutas y configuración de plugins.
 -  `setup_logging`: Configura el sistema de registro (logging) en archivo y consola. Implementa rotación de archivos concurrente y salida por consola con diferentes niveles de detalle. El `QtLogHandler` NO se crea aquí: se instala en `main()` después de crear `QApplication`, ya que `QObject` requiere que exista una instancia de `QApplication`.
 -  `main`: Punto de entrada principal que orquesta el arranque de la aplicación. Inicializa la base de datos, el modelo, la vista y el controlador principal, y el flujo de autenticación. Tras el login y la pantalla de salud del sistema: - Rol distinto de Trabajador: conecta `QtLogHandler` al terminal de `HomeWidget` y muestra `MainView`. - Rol Trabajador: abre la interfaz de operario y conecta el mismo handler al `LogTerminalWidget` de la pestaña Log en `WorkerMainWindow`. `QtLogHandler` se crea después de `QApplication` (requiere `QObject`). El buffer interno guarda mensajes hasta la primera `connect_to_widget` y luego los reproduce en la terminal activa.
-

`analyze_ui.py`

Nombre del Módulo: `analyze_ui`

Descripción: Análisis estático (AST) de la capa UI: clases, métodos, complejidad aproximada y señales/conexiones; emite JSON para informes (p. ej. `generate_ui_report`).

-  `analyze_file`: Analiza un archivo Python individual para extraer métricas de estructura y complejidad. Args: `filepath`: Ruta absoluta o relativa al archivo `.py` que se desea analizar. Returns: Un diccionario con el recuento de líneas, ruta del archivo y una lista de clases encontradas con sus métodos y métricas asociadas.
 -  `main`: Punto de entrada principal del script. Escanea los directorios de UI y genera un informe en formato JSON.
-

`generate_ui_report.py`

Nombre del Módulo: `generate_ui_report`

Descripción: Convierte el JSON generado por `analyze_ui` en un informe Markdown de la interfaz Hipatia.

-  `generate_markdown`: Lee un informe JSON de análisis de UI y genera un documento Markdown estructurado. Args: `json_path`: Ruta al archivo JSON generado por `analyze_ui.py`. `output_path`: Ruta donde se guardará el informe Markdown resultante.
-

`run_tests.py`

Nombre del Módulo: `run_tests`

Descripción: Orquestador local de pytest y métricas de calidad por archivo de test (cobertura, resumen visual en consola). Ignora duplicados tipo `test_foo 2.py` de Finder/iCloud.

-  `_is_finder_duplicate_test`: True si es copia accidental tipo `test_foo 2.py` / `test_foo 3.py` (Finder/iCloud).
 -  `_collect_test_files`: Devuelve todos los archivos `test_*.py` bajo `tests/` (sin duplicados `*N.py`).
 -  `_run_file_with_coverage`: Ejecuta un archivo de test con cobertura en subprocesso aislado. Cada archivo genera su propio `.coverage`. que luego se combina. Devuelve (`passed`, `output_summary`).
 -  `_print_worst_files`: Muestra los archivos con peor score corregible para priorizar mejoras.
-

run_tests_safe.py

Nombre del Módulo: run_tests_safe

Descripción: Ejecuta cada archivo de test en un subprocesso aparte para reducir abortos por acumulación de widgets PyQt6 en un solo proceso.

Uso: ``python3 run\_tests\_safe.py`` (todos), ``python3 run\_tests\_safe.py tests/unit/``,
``python3 run\_tests\_safe.py -k "nombre"`` , ``python3 run\_tests\_safe.py --fail-fast``.

-  **collect_test_files**: Devuelve todos los archivos test_*.py bajo los paths indicados.
 -  **run_file**: Ejecuta un archivo de test en subprocesso aislado. Returns: (passed, output, duration_seconds)
-

Capítulo: controllers/

Métrica	Valor
Archivos .py en controllers/	56
Incluidos en el cuerpo	56
Omitidos (docstrings/reglas)	0
Clases detectadas (AST)	60

```
graph TD
    DIC[DIContainer] --> CTRL[Controllers]
    CTRL -->|orquestración| CORE[Core/Services]
```

controllers/ — Referencia

 `controllers/__init__.py`

Nombre del Módulo: controllers Descripción: Centraliza y exporta todos los controladores del sistema Hipatia. Sigue el patrón MVC, donde los controladores actúan como mediadores entre los modelos de datos y las vistas de la interfaz de usuario.

Nombre del Módulo: `app_controller` Descripción: Orquestador central de la aplicación. Gestiona el ciclo de vida de los sub-controladores y coordina la comunicación entre el modelo global y la vista principal.

Clase `AppController`

Controlador Principal de la Aplicación.

Actúa como el 'Hub' central de la lógica de negocio, encargándose de la inicialización de la infraestructura, la inyección de dependencias a través del contenedor DI y la delegación de tareas a controladores especializados.

Métodos Principales:

- `__init__`: Inicializa el orquestador principal y sus dependencias base. Args: `model`: Instancia del modelo de aplicación (`AppModel`). `view`: Instancia de la vista principal (`MainView`). `schedule_manager`: Gestor de configuración de horarios.
 - `initialize_infra`: Inicializa la infraestructura básica y los sub-controladores. Este método prepara los servicios y estados antes de que se conecten las señales de la UI.
 - `connect_all_signals`: Establece todas las conexiones de señales y slots entre controladores y la UI. Debe llamarse una vez que todos los componentes visuales han sido inicializados.
 - `initialize`: Realiza una inicialización completa de la aplicación (infraestructura + señales). Método de conveniencia para arranques estándar.
 - `cleanup`: Limpieza de recursos al cerrar la aplicación.
 - `current_user`: Obtiene el usuario autenticado actualmente a través del controlador de sesión. Returns: Instancia del usuario actual o `None` si no hay sesión activa.
 - `current_user`: Permite sincronizar el usuario actual con `SessionController`.
 - `on_data_changed`: Notifica a los componentes interesados que los datos globales han cambiado. Coordina la actualización de la UI y el refresco de tablas de búsqueda.
 - `config_get_setting`: Compatibilidad para widgets que reciben `AppController` durante arranque y esperan la API de configuración del `ScheduleController`.
 - `config_set_setting`: Compatibilidad para escritura de configuración en arranque temprano.
 - `handle_attach_file`: Delega la adjunción de archivos al `FileController`.
-

`controllers/backup_controller.py`

Nombre del Módulo: `backup_controller` Descripción: Gestiona las operaciones de copia de seguridad (backup), restauración, exportación e importación de la base de datos y logs del sistema.

Clase `IBackupControllerDatabase`

Contrato mínimo de BD para backup/sync (incluye dobles de test ligeros).

Clase `BackupController`

Controlador encargado de la gestión de copias de seguridad.

Centraliza la lógica para crear backups estructurados por fecha, importar datos desde paquetes ZIP, exportar la BD actual y sincronizar cambios entre diferentes archivos de base de datos SQLite.

Métodos Principales:

- `__init__`: Inicializa el controlador de backups. Args: `db`: Instancia del gestor de base de datos. `view`: Referencia a la vista principal para diálogos y mensajes. `logger`: Instancia para el registro de eventos técnicos. `backup_service`: Servicio especializado en lógica de backup (opcional). `audit_logger`: Servicio de auditoría para registrar acciones de usuario (opcional).
 - `show_backup_restore_dialog`: Muestra el diálogo de gestión de backups.
 - `_get_db_path`: Extract SQLite file path from `db_url`. Returns empty string for non-SQLite DBs.
 - `_create_backup_directory_structure`: Crea la estructura de carpetas para backups organizados por fecha y hora.
 - `_backup_and_clean_log`: Realiza backup del log de errores y lo limpia.
 - `create_automatic_backup`: Realiza una copia de seguridad automática completa. Crea una estructura organizada por fecha y hora, copia la base de datos y realiza la rotación/limpieza de los logs de errores. Returns: True si el proceso se completó con éxito, False en caso contrario.
-

controllers/backup_controller_io_manager.py

Nombre del Módulo: backup_controller_io_manager Descripción: Operaciones I/O de importación, exportación y sincronización de la base de datos. BackupController compone BackupIOManager y delega on_import_databases, on_export_databases y on_sync_databases sin herencia múltiple. La sincronización acepta SQLite suelto o copias ZIP/TAR.GZ (extracción temporal), compara con DatabaseComparisonDTO y aplica cambios vía SyncDialog.

Clase BackupControllerIOContext

Contrato mínimo que BackupIOManager exige del controlador (composición, no herencia).

Attributes: view: Vista principal para diálogos y mensajes. db: Gestor de base de datos con compare_with_db, apply_sync_changes, etc. logger: Logger de aplicación. audit_logger: Servicio de auditoría opcional.

Clase BackupIOManager

Colaborador que centraliza importación ZIP completa, exportación ZIP de la BD y sincronización selectiva frente a otra copia SQLite. No sustituye al BackupController; solo ejecuta I/O y UI asociada bajo su contrato BackupControllerIOContext.

Métodos Principales:

- `__init__`: Args: controller: Instancia del controlador de backup que cumple el protocolo de contexto.
 - `on_import_databases`: Restaura datos desde un ZIP de copia de seguridad: extrae junto al directorio de la BD, cierra la conexión actual, sustituye ficheros y reinstancia DatabaseManager. Args: on_success_callback: Opcional; se invoca tras importación exitosa (p. ej. refrescar UI).
 - `on_export_databases`: Ofrece guardar la base de datos actual en un ZIP (un único miembro, nombre basado en db_path). Registra exportación en auditoría si está disponible.
 - `on_sync_databases`: Compara la BD local con otra SQLite (o ZIP/TAR que la contenga), muestra SyncDialog y aplica solo los registros marcados por el usuario mediante apply_sync_changes. Args: on_success_callback: Opcional; se llama tras una sincronización aplicada con éxito.
 -  `_comparison_has_differences`: Indica si hay diferencias reales entre bases según el DTO devuelto por compare_with_db. Args: comparison: Resultado de la comparación local vs extranjero. Returns: True si alguna tabla incluye al menos un registro en differences.
 -  `_find_sqlite_under`: Localiza un fichero SQLite dentro de un directorio (p. ej. tras descomprimir un ZIP). Args: root: Directorio raíz de búsqueda recursiva. Returns: Ruta al .db elegido, o None si no hay ninguno. Se prefiere montaje.db si existe.
 -  `_prepare_foreign_sqlite_path`: Obtiene la ruta absoluta al SQLite que debe usarse para la comparación. Si el usuario eligió .db/.sqlite, se valida que exista. Si eligió .zip o .tar.gz, se extrae en un directorio temporal y se busca un .db; el caller debe borrar ese directorio en un finally cuando el segundo valor no sea None. Args: chosen_path: Ruta seleccionada en el diálogo de ficheros. logger: Logger del controlador para avisos de formato o corrupción. Returns: Tupla (ruta_sqlite, tmpdir_o_None). Si falla la resolución, (None, None) o (None, tmp) tras limpiar el temporal en errores de extracción.
-

Nombre del Módulo: `calculation_controller` Descripción: Gestiona la lógica de cálculo de tiempos de fabricación, incluyendo la interacción con la pila de preprocesos y la exportación de logs de auditoría.

Clase `CalculationController`

Controlador para la lógica de cálculo de tiempos de fabricación.

Responsable de orquestar la página de cálculo, gestionar la conexión de sus señales y procesar las operaciones sobre la pila de preprocesos.

Métodos Principales:

- `__init__`: Inicializa el `CalculationController`. Args: `app_controller`: Referencia al `AppController` principal. `pila_service`: Servicio de pilas (inyectado).
 - `connect_calculate_signals`: Conecta las señales del widget de cálculo. Si la UI no está inicializada, programa la conexión para después.
 - `on_go_home_and_reset_calc`: Limpia el estado de la simulación y retorna a la pantalla principal. Asegura que no queden datos residuales de cálculos anteriores.
 - `on_calc_product_result_selected`: Maneja la selección de un producto en los resultados de cálculo.
 - `on_export_audit_log`: Exporta el contenido del log de auditoría a un archivo HTML.
 - `get_fabricacion_products_for_calculation`: Obtiene y prepara los productos de una fabricación para el motor de cálculo. Args: `fabricacion_id`: Identificador único de la fabricación. Returns: Una lista de `CalculationProductDTO` con los tiempos y cantidades preparados.
 - `add_preprocesos_to_current_pila`: Añade preprocesos a la pila de cálculo actual. Args: `preprocesos`: Lista de `CalculationProductDTO` con la información de los preprocesos. Returns: `int`: Número de preprocesos añadidos exitosamente.
 - `update_lote_content_table`: Refresca la tabla de contenido del lote en la UI.
 - `update_calculate_page_lists`: Actualiza las listas de la página de cálculo. Args: `calc_page`: Instancia opcional del widget.
 - `safe_update_calculate_page`: Actualiza la página de cálculo de forma segura, con manejo de errores. Este método se llama diferido para dar tiempo a Qt a estabilizar los widgets.
-

 `controllers/fabricacion_controller.py`

Nombre del Módulo: `fabricacion_controller` Descripción: Controlador central para la gestión del ciclo de vida de las fabricaciones. Maneja la creación, búsqueda y la integración con preprocesos.

Clase `FabricacionController`

Controlador dedicado a la gestión de fabricaciones.

Actúa como mediador para las operaciones CRUD de fabricaciones, delegando gran parte de la lógica pesada a `ProductControllerV2` para mantener la consistencia.

Métodos Principales:

- `__init__`: Inicializa el controlador de fabricaciones. Args: `db_manager`: Gestor de conexión a la base de datos. `view`: Referencia a la vista principal. `product_controller`: Controlador de productos para delegación. `logger`: Instancia de logging.
 - `connect_signals`: Conecta las señales del widget de gestión de Fabricaciones.
 - `show_create_fabricacion_dialog`: Muestra el diálogo para crear una nueva fabricación.
 - `search_fabricaciones`: Busca fabricaciones por texto. Args: `text`: Texto de búsqueda
 - `show_fabricacion_preprocesos`: Muestra los preprocesos de una fabricación. Args: `fabricacion_id`: ID de la fabricación
 - `refresh_fabricaciones_list`: Actualiza la lista de fabricaciones en la UI.
 - `get_fabricacion_products_for_calculation`: Obtiene todos los productos de una fabricación preparados para cálculo. Args: `fabricacion_id`: ID de la fabricación Returns: Lista de `CalculationProductDTO` con datos para cálculo
-

controllers/file_controller.py

Nombre del Módulo: file_controller Descripción: Gestiona la persistencia de archivos adjuntos, la apertura de documentos del sistema y la importación de datos externos en formato JSON.

Clase FileController

Controlador dedicado a la gestión de archivos y persistencia de datos externos.

Proporciona utilidades para adjuntar imágenes o planos a productos/fabricaciones, visualizar archivos usando aplicaciones del sistema y procesar importaciones JSON de tareas y registros de trabajo.

Métodos Principales:

- `__init__`: Inicializa el controlador de archivos. Args: `db_manager`: Gestor de conexión a la base de datos para operaciones de persistencia. `view`: Referencia a la vista principal para mostrar diálogos de archivo. `logger`: Instancia para el registro de operaciones de sistema de archivos.
 - `handle_attach_file`: Copia un archivo externo al directorio de datos del sistema y genera una ruta relativa. Args: `owner_type`: Tipo de entidad (p. ej. 'producto', 'fabricacion'). `owner_id`: Identificador de la entidad. `source_file_path`: Ruta absoluta del archivo de origen. `file_type`: Categoría del archivo (p. ej. 'imagen', 'plano'). Returns: `FileOperationResultDTO`: Resultado de la operación con estado y ruta o error.
 - `handle_view_file`: Abre un archivo usando el visor por defecto del sistema. Args: `relative_path`: Ruta relativa del archivo
 - `on_import_task_data`: Inicia un diálogo para importar datos de tareas desde un archivo JSON. Fusiona la información importada con la base de datos de tracking local.
 - `on_data_after_import`: Callback auxiliar para actualizar UI después de importar backup.
-

controllers/hardware_controller.py

Nombre del Módulo: hardware_controller Descripción: Gestiona la interacción con dispositivos de hardware, principalmente cámaras de video para el escaneo de códigos QR. La apertura de captura usa `core.camera_manager.capture.open_video_capture` (misma cadena de backends que `CameraManager`).

Clase `HardwareController`

Controlador para la gestión de dispositivos de hardware.

Maneja el ciclo de vida de la conexión con cámaras, la detección de dispositivos compatibles, la configuración de resolución y la integración con el escáner QR.

Métodos Principales:

- `__init__`: Inicializa el controlador de hardware. Args: `db`: Gestor de base de datos para acceder a la configuración de dispositivos. `view`: Referencia a la vista principal de la aplicación. `logger`: Instancia opcional para el registro de eventos de hardware.
 - `initialize_qr_scanner`: Inicializa el escáner QR configurando el dispositivo de captura de video. Resuelve el índice de cámara (configuración guardada, mejor cámara o índice 0), abre la captura con `open_video_capture` (política de backend alineada con `CameraManager`) y pasa el `cv2.VideoCapture` resultante a `QrScanner`. Args: `worker_controller`: Opcional; instancia del controlador de operario para inyectar el scanner.
 - `_get_settings_page_with_camera_combo`: Obtiene la página de ajustes si expone `camera_combo`.
 - `detect_cameras`: Detecta cámaras y actualiza la UI de configuración.
 - `load_hardware_settings`: Carga la configuración de hardware guardada en la UI.
 - `save_hardware_settings`: Guarda la configuración de hardware con validación.
 - `test_camera`: Prueba la cámara seleccionada mostrando un preview.
-

`controllers/lote_controller.py`

Nombre del Módulo: `lote_controller` Descripción: Gestiona la lógica de plantillas de lotes, incluyendo su definición, búsqueda y la actualización de su contenido en la interfaz de cálculo.

Clase `LoteController`

Controlador dedicado a la gestión de lotes y sus plantillas.

Permite el filtrado de lotes existentes, la actualización dinámica de su contenido en tablas editables y la delegación de operaciones de persistencia al controlador de pilas.

Métodos Principales:

- `__init__`: Inicializa el controlador de lotes. Args: `db_manager`: Instancia del gestor de base de datos. `view`: Referencia a la vista principal. `pila_controller`: Controlador de pilas para operaciones delegadas. `logger`: Instancia de logging.
 - `connect_signals`: Conecta las señales del widget de lotes.
 - `connect_definir_lote_signals`: Conecta las señales del widget para definir plantillas de Lote.
 - `update_lotes_view`: Actualiza la vista de lotes.
 - `update_lote_content_table`: Refresca la tabla de contenido del lote en la interfaz de cálculo. Construye filas con controles interactivos (SpinBox) para gestionar cantidades.
 - `on_calc_lote_search_changed`: Maneja cambios en la búsqueda de lotes. Args: `text`: Texto de búsqueda
 - `set_current_lote_content`: Establece el contenido actual del lote. Args: `content`: Lista de items del lote
-

 `controllers/machine_controller.py`

Nombre del Módulo: `machine_controller` Descripción: Controlador encargado de la gestión de maquinaria, mantenimientos y configuración de grupos de preparación de máquinas.

Clase `MachineController`

Controlador para la gestión de máquinas.

Coordina la creación, edición y eliminación de maquinaria, además de supervisar los registros de mantenimiento preventivo/correctivo y los grupos de preparación.

Métodos Principales:

- `__init__`: Inicializa el controlador de máquinas con sus dependencias. Args: `machine_service`: Servicio lógico de gestión de máquinas. `preparation_service`: Grupos y pasos de preparación de máquinas. `product_service`: Catálogo de productos (diálogos de prep). `view`: Interfaz de usuario para interacciones y mensajes. `logger`: Sistema de registro de eventos.
 - `update_machines_view`: Actualiza la vista de máquinas con el listado completo del catálogo.
 - `get_distinct_machine_processes`: Obtiene el conjunto de procesos únicos (ej. 'Inyección', 'Montaje') asignados a las máquinas registradas.
-

controllers/navigation_controller.py

Nombre del Módulo: navigation_controller Descripción: Gestiona la navegación entre las diferentes páginas de la aplicación, controlando la carga de datos específicos y el flujo de transiciones.

Clase NavigationController

Controlador dedicado a la gestión de navegación.

Responsable de orquestar el cambio de vista entre los diferentes widgets funcionales, asegurando que los datos necesarios se refresquen al entrar en cada sección.

Métodos Principales:

- `__init__`: Inicializa el controlador de navegación. Args: `app`: Referencia al controlador principal de la aplicación. `view`: Interfaz de la vista para gestionar el cambio de páginas. `product_service`: Servicio de productos para carga de datos durante la navegación. `logger`: Instancia para el registro de eventos de navegación.
 - `initialize`: Inicializa el controlador y establece las conexiones iniciales.
 - `cleanup`: Limpieza de recursos del controlador.
 - `connect_signals`: Conecta las señales de los botones de navegación.
 - `on_nav_button_clicked`: Maneja el clic en botones de navegación. Args: `name`: Nombre de la página destino
 - `navigate_to`: Navega a una página específica. Args: `page_name`: Nombre de la página Returns: `True` si la navegación fue exitosa
 - `_perform_navigation`: Lógica interna de navegación. Lanza excepciones en caso de error. Args: `name`: Nombre de la página
 - `safe_update_calculate_page`: Actualiza la página de cálculo de forma segura, con manejo de errores. Este método se llama diferido para dar tiempo a Qt a estabilizar los widgets. Args: `calc_page`: Widget de la página de cálculo
 - `on_go_home_and_reset_calc`: Limpia la simulación y vuelve a la pantalla de inicio.
 - `update_page_permissions`: Actualiza los permisos de acceso a páginas según el rol del usuario. Args: `role`: Rol del usuario actual
-

 `controllers/preproceso_controller.py`

Nombre del Módulo: `preproceso_controller` Descripción: Gestiona la lógica de preprocesos, incluyendo su carga desde el modelo, vínculo con componentes y conversión a pasos operativos en la pila.

Clase `PreprocesoController`

Controlador para la gestión de preprocesos de fabricación.

Métodos Principales:

- `__init__`: Inicializa el controlador de preprocesos. Args: `db_manager`: Gestor de conexión a la base de datos. `view`: Referencia a la vista principal. `fabricacion_service`: Servicio lógico de fabricaciones. `logger`: Instancia de logging.
 - `preproceso_repo`: Lazy initialization del repositorio de preprocesos.
 - `connect_signals`: Conecta las señales del widget de preprocesos.
 - `load_preprocesos_data`: Solicita al servicio la carga de todos los preprocesos disponibles y los refleja en la tabla de la interfaz de usuario.
 - `get_all_preprocesos_with_components`: Obtiene todos los preprocesos ya formateados desde el repositorio. Returns: Lista de preprocesos con sus componentes
 - `get_preprocesos_by_fabricacion`: Obtiene los preprocesos asignados a una fabricación. Args: `fabricacion_id`: ID de la fabricación Returns: Lista de diccionarios con información de preprocesos
 - `add_preprocesos_to_current_pila`: Añade preprocesos a la pila de cálculo actual. Args: `preprocesos`: Lista de preprocesos a añadir
 - `convert_preproceso_to_pila_step`: Convierte un preproceso al formato de paso de pila. Args: `preproceso`: Diccionario con datos del preproceso Returns: Diccionario en formato de paso de pila
 - `on_manage_procesos_for_new_product`: Gestiona los procesos para un nuevo producto. Args: `current_procesos`: Procesos actuales del producto
-

controllers/product_controller_v2.py

Nombre del Módulo: product_controller_v2 Descripción: Controlador centralizado para la gestión de productos, fabricaciones y preprocesos. Actúa como fachada (Facade) delegando en gestores especializados.

Clase ProductController

Controlador para la gestión de productos, fabricaciones y preprocesos. Fachada que orquesta la lógica distribuida en Managers especializados.

Métodos Principales:

- `__init__`: Inicializa el controlador de productos V2 con dependencias explícitas. Args: app_shell: Hub (IApplicationShell): adjuntos, sesión, ui_controller. db: DatabaseManager. product_model: Fachada AppModel / protocolo IProductModel (repos y delegación restante). view: Vista principal. product_facade: ProductFacade del dominio. fabricacion_service: Servicio de fabricaciones. planning_facade: PlanningFacade (pila / cálculo). material_service: Servicio de materiales (alias frecuente: product_service). machine_service: Servicio de máquinas. state: ApplicationState compartido.
 - `on_data_changed`: Puente requerido por protocolos: notifica cambios de datos a la UI.
 - `_connect_products_signals`: Delega la conexión de señales al gestor correspondiente.
-

controllers/report_controller.py

Nombre del Módulo: report_controller Descripción: Gestiona la generación y exportación de informes en diversos formatos (Excel, PDF), incluyendo resultados de simulación e historiales.

Clase ReportController

Controlador de informes y exportaciones.

Responsable de orquestar la creación de documentos PDF y Excel a partir de datos de simulación, históricos de piezas o registros de actividad.

Métodos Principales:

- `__init__`: Inicializa el controlador de informes. Args: db: Gestor de base de datos. view: Referencia a la vista principal. worker_service: Servicio de trabajadores. product_service: Servicio de productos. pila_service: Servicio de pilas de fabricación. schedule_manager: Configuración de horarios. logger: Logger opcional.
 - `initialize`: Inicializa el controlador.
 - `cleanup`: Limpieza de recursos.
 - `update_simulation_data`: Actualiza los datos de la última simulación para usar en exportaciones. Args: results: Lista de resultados de simulación audit_log: Log de auditoría de la simulación production_flow: Flujo de producción usado units: Unidades calculadas flexible_workers: Número de trabajadores flexibles necesarios
 - `on_generar_informe_clicked`: Genera el informe seleccionado por el usuario desde la página de Reportes.
 - `on_print_historial_report_clicked`: Genera un informe PDF del historial seleccionado.
-

 `controllers/report_export_helper.py`

Nombre del Módulo: `controllers.report_export_helper`

Descripción: Exportaciones Excel/PDF desde la última simulación (composición sobre ReportController).

 **Clase** `ReportExportHelper`

Delegado sin herencia múltiple; usa el controlador para estado y `handle_error`.

Nombre del Módulo: `schedule_controller` Descripción: Controlador orquestador para la gestión de la planificación de la producción. Gestiona la configuración de horarios laborales, descansos y festivos mediante componentes delegados.

Clase `ScheduleController`

Controlador de horarios y descansos. Utiliza composición para delegar la lógica de UI y API legacy en helpers especializados.

Métodos Principales:

- `__init__`: Inicializa el controlador y sus componentes delegados. Args: `db`: DatabaseManager con acceso a los repositorios. `view`: MainView para mostrar mensajes y acceder a widgets. `schedule_manager`: ScheduleConfig para recargar configuración. `logger`: Logger opcional para registro.
 - `model`: Propiedad puente para compatibilidad con el sistema de widgets antiguos.
 - `save_schedule_settings`: Guarda la configuración del horario laboral persistiendo en la DB.
 - `load_schedule_settings`: Carga la configuración de horarios desde la arquitectura persistente a la UI.
 - `on_add_break_clicked`: Manejador del botón para añadir un nuevo descanso en la configuración.
 - `on_remove_break_clicked`: Manejador para eliminar el descanso seleccionado de la lista de UI.
 - `on_edit_break_clicked`: Manejador para editar un descanso existente.
 - `on_add_break`: Método de compatibilidad para el flujo legacy de añadir descansos.
 - `add_break`: API para añadir descansos programáticamente.
 - `delete_break`: API para eliminar descansos por índice.
 - `save_work_hours`: API directa para guardar horas laborales.
 - `load_schedule_config`: Carga completa de la configuración (Horas, Descansos, Festivos).
 - `load_holidays`: Carga la lista de festivos desde la configuración y la vuelca en la UI.
 - `on_add_holiday`: Añade el día seleccionado en el calendario como festivo.
 - `on_remove_holiday`: Elimina el día seleccionado de la lista de festivos configurada.
 - `config_get_setting`: Obtiene un ajuste de configuración de la persistencia.
 - `config_set_setting`: Establece o actualiza un ajuste de configuración.
 - `reload_config`: Recarga la configuración global de horarios en el sistema.
 -  `get_add_break_dialog_class`: Clase del diálogo de descansos (carga diferida; sin import estático `ui`).
-

`controllers/schedule_helpers.py`

Nombre del Módulo: `controllers.schedule_helpers`

Descripción: Funciones puras de apoyo (sin estado de proceso): `parse_break_text`, `load_breaks_list`, `break_display_lines_from_json`, `normalize_holidays`, `holidays_dates`, `dump_json`. Integración típica con: `__future__`, `json`.

-  `parse_break_text`: Parsea un texto `HH:mm - HH:mm` y devuelve `(start, end)` o `None`.
 -  `load_breaks_list`: Convierte JSON de breaks a lista normalizada de dicts.
 -  `break_display_lines_from_json`: Devuelve textos listos para `QListWidget` (capa de deserialización, sin tocar UI). Evita que el widget acceda con subscripts a dicts crudos del JSON.
 -  `normalize_holidays`: Normaliza holidays legacy (`list[str]` o `list[dict]`) a `list[dict{date,desc}]`.
 -  `holidays_dates`: Extrae `date` de la lista normalizada.
 -  `dump_json`: Serializa JSON de forma consistente.
-

controllers/schedule_ui_helper.py

Nombre del Módulo: schedule_ui_helper Descripción: Helper para operaciones de interfaz de usuario del ScheduleController. Maneja la lógica de interacción con widgets y diálogos de configuración de horarios.

Clase ScheduleUiOpsHelper

Helper encargado de las operaciones que interactúan con la UI. Extraído de ScheduleController para mejorar la cohesión y reducir el tamaño del controlador.

Métodos Principales:

- `__init__`: Inicializa el helper con las dependencias necesarias. Args: db: DatabaseManager para persistencia. view: MainView para acceso a widgets y mensajes. schedule_manager: Gestor de configuración de horarios. logger: Instancia de logger. controller: Referencia al controlador padre para delegación de llamadas mockeables.
 - `save_schedule_settings`: Guarda la configuración completa del horario laboral desde la UI.
 - `load_schedule_settings`: Carga la configuración del horario en los widgets de la UI.
 - `on_add_break_clicked`: Abre el diálogo especializado para añadir un nuevo descanso horaro.
 - `on_remove_break_clicked`: Elimina el descanso seleccionado actualmente en la lista de la UI.
 - `on_edit_break_clicked`: Permite editar un descanso existente abriendo el diálogo con los datos actuales.
 - `on_add_break`: Legacy helper: Abre un diálogo genérico para añadir un descanso.
 - `add_break`: Añade un descanso de forma programática (API legacy / tests).
 - `delete_break`: Elimina un descanso por índice (API legacy / tests).
 - `save_work_hours`: Guarda horas laborales y descansos en configuración (API legacy).
 - `load_schedule_config`: Carga horas y descansos en la UI y delega festivos al controlador.
-

`controllers/session_controller.py`

Nombre del Módulo: `session_controller` Descripción: Orquesta el acceso de cada usuario a la aplicación: ventana de login, comprobación de credenciales, bloqueo temporal tras intentos fallidos, cierre de sesión y apertura de la vista adecuada (responsable o trabajador) según el rol. Registra intentos en auditoría cuando procede.

Clase `SessionController`

Controlador de sesiones y seguridad.

Responsable de validar credenciales, manejar el bloqueo por intentos fallidos (Rate Limiting) y habilitar/deshabilitar funcionalidades de la UI según el rol.

Métodos Principales:

- `__init__`: Inicializa el controlador de sesión. Args: `app_controller`: Referencia al controlador principal de la aplicación. `db`: `DatabaseManager` (misma instancia que expone `AppController`). `worker_service`: Servicio de trabajadores (inyectado).
 - `handle_login`: Muestra el diálogo de login y gestiona la autenticación.
 - `logout`: Cierra la sesión actual.
 - `_update_ui_for_role`: Habilita o deshabilita elementos de la UI según los permisos del usuario.
 - `launch_worker_interface`: Lanza la interfaz simplificada para trabajadores.
-

controllers/startup_controller.py

Nombre del Módulo: startup_controller Descripción: Orquestador del arranque de la aplicación. Se encarga de instanciar servicios, repositorios y todos los controladores del sistema.

Clase StartupController

Controlador responsable de la inicialización de la aplicación. Maneja la configuración de servicios, repositorios y sub-controladores.

Métodos Principales:

- `__init__`: Inicializa el controlador de arranque. Args: `app_controller`: Instancia del controlador principal de la aplicación.
 - `initialize_app`: Orquesta todo el proceso de arranque.
 - `_init_services`: Inicializa y registra los servicios y repositorios core.
 - `_init_scheduler`: Inicializa el planificador de tareas automático.
 - `_check_scheduled_tasks`: Verifica si hay tareas programadas para ejecutar en este momento.
 - `_init_state`: Inicializa el estado global de la aplicación (`ApplicationState`).
 - `_init_controllers`: Inicializa todos los controladores de la aplicación usando el contenedor. Utiliza fábricas (lambdas) para permitir la instanciación diferida y ciclos de vida.
-

`controllers/ui_class_loader.py`

Nombre del Módulo: `controllers.ui_class_loader`

Descripción: Funciones puras de apoyo (sin estado de proceso): `ui_class`. Integración típica con: `__future__`, `importlib`.

-  `ui_class`: Devuelve un atributo (típicamente una clase `QWidget/QDialog`) de un submódulo `ui`. Sin caché inter-test: los tests pueden parchear `ui.dialogs.*` o el nombre reexportado en el módulo controlador antes de cada llamada.
-

`controllers/ui_controller.py`

Nombre del Módulo: `ui_controller`

Descripción: Define protocolos o tipos principales: `UIController`. Controlador de sincronización de la interfaz. Integración típica con: `__future__`, `PyQt6`, `core`.

Clase `UIController`

Controlador de sincronización de la interfaz.

Se encarga de mantener los widgets actualizados frente a cambios en los datos, gestionar barras de progreso y cargar elementos informativos como frases célebres.

Métodos Principales:

- `__init__`: Inicializa el controlador de UI. Args: `view`: Referencia a la interfaz principal.
`machine_service`: Servicio de máquinas. `worker_service`: Servicio de trabajadores. `report_service`: Servicio de informes. `product_service`: Servicio de productos. `worker_controller`: Referencia al controlador de trabajadores. `machine_controller`: Referencia al controlador de máquinas. `quote_service`: Servicio de frases célebres. `thread_pool`: Pool de hilos para tareas asíncronas. `logger`: Instancia de logging.
 - `update_dashboard_view`: Actualiza la vista del dashboard.
 - `update_workers_view`: Actualiza la lista de trabajadores en la vista.
 - `update_machines_view`: Actualiza la lista de máquinas en la vista.
 - `update_simulation_progress`: Actualiza el valor de la barra de progreso en la UI. Args: `value`: Valor de progreso (0-100)
 - `on_data_changed`: Maneja eventos de cambio de datos, actualizando vistas relevantes.
 - `load_quote_for_home`: Anteriormente cargaba una frase de WikiQuote en el `HomeWidget`. El `HomeWidget` ahora muestra el resumen de salud del sistema en su lugar, por lo que este método ya no realiza ninguna acción.
-

 `controllers/ui_signals_controller.py`

Nombre del Módulo: `ui_signals_controller` Descripción: Centralizador de la interconexión mediante señales y slots. Desacopla la lógica de los widgets de los controladores principales.

Clase `UISignalsController`

Controlador de señales y ranuras (signals & slots).

Centraliza la conexión entre los eventos de la interfaz de usuario (clics, cambios de texto) y los métodos de negocio de los diversos controladores.

Métodos Principales:

- `__init__`: Inicializa el controlador de señales. Args: `app_controller`: Referencia al controlador principal.
 - `connect_all_signals`: Conecta todas las señales de la aplicación.
-

 `controllers/ui_signals_wiring.py`

Nombre del Módulo: `controllers.ui_signals_wiring`

Descripción: Cableado de señales Qt entre vista y controladores (composición; sin herencia múltiple).

Clase `UISignalsWiring`

Encapsula la conexión de widgets y slots; recibe app, vista y logger del controlador.

Métodos Principales:

- `run_import_tasks_from_csv_dialog`: Abre diálogo CSV y delega la importación en `AppModel` o `TrackingRepository`.
-

 **controllers/historial/__init__.py**

Nombre del Módulo: controllers.historial

Descripción: Nombre del Paquete: historial Descripción: Gestiona la visualización e interacción con los registros históricos, auditorías e iteraciones de productos.

 `controllers/historial/controller.py`

Nombre del Módulo: `historial.controller` Descripción: Controlador principal del sub-paquete de historial. Utiliza composición para delegar la gestión de UI, interacciones y reportes.

Clase `HistorialController`

Controlador central para el historial.

Orquesta los diferentes gestores (Vista, Interacción, Reportes) para proporcionar una interfaz unificada de consulta de auditoría y bitácoras.

Métodos Principales:

- `__init__`: Inicializa el controlador y compone sus gestores. Args: `db`: Referencia a la base de datos. `pila_service`: Servicio de gestión de pilas de fabricación. `worker_service`: Servicio de gestión de operarios. `view`: Referencia a la vista principal. `logger`: Instancia de logging (opcional).
 - `connect_signals`: Conecta las señales de la vista del historial delegando en los gestores.
-

 `controllers/historial/interaction_manager.py`

Nombre del Módulo: historial.interaction_manager Descripción: Gestiona la lógica de interacción del usuario en la sección de historial, como la selección de elementos y filtros por calendario.

Clase HistorialInteractionManager

Gestor de interacción para el historial.

Se encarga de reaccionar a los eventos de usuario, como la selección de iteraciones o fabricaciones, actualizando los detalles y resaltando fechas.

Métodos Principales:

- `on_item_selected`: Maneja la selección de un ítem en la lista de resultados.
 - `on_calendar_clicked`: Filtra la lista por la fecha seleccionada en el calendario.
-

 **controllers/historial/protocols.py**

Nombre del Módulo: historial.protocols Descripción: Define los protocolos (interfaces estructurales) para garantizar el tipado correcto y la compatibilidad entre el controlador de historial y sus gestores.

 `controllers/historial/report_manager.py`

Nombre del Módulo: historial.report_manager Descripción: Gestor encargado de la generación de informes PDF para el historial de iteraciones y fabricaciones, utilizando estrategias de reporte personalizadas.

Clase HistorialReportManager

Gestor de reportes para el historial.

Se encarga de recolectar los datos necesarios según el modo de visualización y disparar la generación de documentos PDF.

Métodos Principales:

- `__init__`: Inicializa el HistorialReportManager. Args: `db` (Any): Instancia del servicio de base de datos. `pila_service` (Any): Instancia del servicio de pila. `worker_service` (Any): Instancia del servicio de worker. `view` (MainView): Referencia a la vista principal de la aplicación. `controller_ref` (Any, optional): Referencia al controlador, si es necesario. Defaults to None.
 - `on_print_report_clicked`: Generador de informes PDF para historial.
-

 `controllers/historial/view_manager.py`

Nombre del Módulo: `historial.view_manager` Descripción: Gestiona la lógica de presentación de los datos históricos, incluyendo el filtrado de listas, resaltado de calendarios y generación de gráficos QtCharts.

Clase `HistorialViewManager`

Gestor de vista para el historial.

Sincroniza los datos crudos con la interfaz de usuario, manejando la población de listas, el resaltado dinámico de fechas en el calendario y la actualización del gráfico de actividad.

Métodos Principales:

- `update_view`: Actualiza la vista completa de Historial.
 - `populate_list`: Rellena la lista de resultados según el modo y filtros.
 - `update_calendar_highlights`: Actualiza los resaltados del calendario según los ítems listados.
 - `update_activity_chart`: Actualiza el gráfico de actividad (últimos 12 meses).
-

 `controllers/pila/controller.py`

Nombre del Módulo: controller Descripción: Controlador Fachada para la gestión de Pilas y Lotes. Delega la lógica pesada a LoteManager y PilaManager.

Clase PilaController

Controlador Fachada para Pilas y Lotes. Implementa Composición sobre Herencia delegando en Gestores.

Métodos Principales:

- `get_preprocesos_for_fabricacion`: Obtiene los preprocesos asociados a una fabricación específica. Args: `fabricacion_id`: ID de la fabricación. Returns: Lista de diccionarios con id, nombre y descripción de los preprocesos.
 - `_connect_lotes_management_signals`: Conecta las señales de la pestaña de gestión de Lotes.
-

 `controllers/pila/lote_manager.py`

Nombre del Módulo: `lote_manager` Descripción: Gestor especializado en la lógica de plantillas de lote (Templates). Se encarga de la búsqueda de productos, fabricaciones y el guardado de la estructura del lote.

Clase `LoteManager`

Gestor de plantillas de lote. Maneja la interacción entre la vista de definición de lotes y los repositorios.

Métodos Principales:

- `on_calc_lote_search_changed`: Busca plantillas de lote para la pila de cálculo. Con texto vacío se listan todas las plantillas en BD (misma idea que productos en Definir Lote); con texto se filtra por código o descripción.
 - `on_lote_def_product_search_changed`: Busca productos para añadir a una plantilla de lote. Con caja vacía se listan todos (hasta el límite del repositorio) para poder elegir sin escribir; con texto se filtra por código o descripción.
 - `on_lote_def_fab_search_changed`: Busca fabricaciones para añadir a una plantilla de lote. Con caja vacía se listan todas las coincidencias del servicio (misma idea que productos).
 - `update_lotes_view`: Actualiza la lista de gestión de lotes.
 - `save_lote_template`: Guarda una nueva plantilla de lote.
 - `delete_lote_template`: Elimina una plantilla de lote tras confirmación.
-

 `controllers/pila/pila_manager.py`

Nombre del Módulo: `pila_manager` Descripción: Gestor especializado en el ciclo de vida de las Pilas de fabricación. Maneja el cargado, guardado, eliminación y visualización de la bitácora.

Clase `PilaManager`

Gestor de ciclo de vida de Pilas. Coordina la persistencia y recuperación de sesiones de planificación.

Métodos Principales:

- `load_pila`: Muestra el diálogo de carga y procesa la pila seleccionada.
 - `_handle_delete_pila`: Maneja la eliminación de una pila tras confirmación.
 - `_apply_loaded_pila_to_ui`: Actualiza el estado de la aplicación y la UI con los datos cargados.
 - `save_pila`: Muestra el diálogo de guardado y persiste la pila actual.
 - `view_bitacora`: Abre el diálogo de bitácora para la pila actual.
 - `_reparse_dates`: Convierte fechas ISO de resultados de simulación a objetos `datetime`.
-

 `controllers/pila/protocols.py`

Nombre del Módulo: pila.protocols Descripción: Define las interfaces (Protocolos) necesarias para que los gestores de Lotes y Pilas interactúen con la Vista y la Base de Datos de forma desacoplada.

 Clase IPilaView

Interfaz para la vista que maneja Pilas y Lotes.

 Clase IPilaDatabase

Interfaz para el acceso a datos relacionado con Pilas.

 Clase IPilaService

Interfaz para el servicio de Pilas.

 Clase IProductService

Interfaz para el servicio de Productos.

 Clase IFabricacionService

Interfaz para el servicio de Fabricaciones.

 `controllers/product/__init__.py`

Nombre del Módulo: controllers.product

Descripción: Nombre del Paquete: product Descripción: Gestiona la lógica de productos, materiales, sub-fabricaciones y preprocesos asociados mediante gestores especializados (Managers).

 `controllers/product/application_shell.py`

Nombre del Módulo: controllers.product.application_shell

Descripción: Subconjunto tipado de ApplicationController usado por ProductManager / FabricacionManager. Evita depender del tipo completo del hub en la capa de producto.

 **Clase** `IAApplicationShell`

Operaciones del hub requeridas por los gestores de producto/fabricación.

 `controllers/product/fabricacion_manager.py`

Nombre del Módulo: `product.fabricacion_manager` Descripción: Gestor de órdenes de fabricación, encargado de coordinar la creación y edición de producciones junto con sus preprocesos y productos asociados.

Clase `FabricacionManager`

Gestor de fabricaciones y órdenes de trabajo.

Facilita la creación de nuevas fabricaciones mediante diálogos interactivos, gestiona la búsqueda y filtrado de las mismas, y sincroniza sus preprocesos.

Métodos Principales:

- `__init__`: Inicializa el gestor de fabricaciones. Args: `app`: Shell del hub (auditoría, refresco UI). `view`: Referencia a la vista principal (`IProductView`). `fabricacion_service`: Servicio lógico de fabricaciones (`IFabricacionService`). `product_facade`: Fachada de catálogo de productos (`IProductService`). `planning_facade`: Fachada de planificación (datos para motor de cálculo). `state`: Estado compartido de la aplicación (`ApplicationState`). `controller_ref`: Referencia opcional al controlador.
 - `show_fabricacion_products`: Muestra el diálogo para asignar/editar productos de una fabricación.
 - `get_fabricacion_products_for_calculation`: Obtiene productos de la fabricación preparados para el motor de cálculo.
 - `show_create_fabricacion_dialog`: Muestra el diálogo para crear fabricación con preprocesos y productos.
 - `search_fabricaciones`: Busca fabricaciones usando el repositorio de preprocesos.
 - `show_fabricacion_preprocesos`: Muestra el diálogo para asignar/editar preprocesos de una fabricación.
-



`controllers/product/fabricacion_products_handler.py`

Nombre del Módulo: `fabricacion_products_handler` Descripción: Coordinación de productos asociados a fabricaciones (diálogo y datos para cálculo). Extraído en B4.5 desde lógica que antes estaba acoplada al controlador de producto.



Clase `IPPlanningCalculationProvider`

Solo la parte de planificación necesaria para armar DTOs de cálculo.



Clase `FabricacionProductsHandler`

Colaborador con composición: gestión de productos de una fabricación y preparación para el motor de cálculo.

Métodos Principales:

- `show_fabricacion_products`: Muestra el diálogo para asignar/editar productos de una fabricación.
 - `refresh_fabrication_display`: Refresca la visualización de la fabricación en la pestaña. Si `fabricacion_data` ya está cargado (p. ej. tras comprobar que existe), se reutiliza y se evita un segundo `get_fabricacion_by_id`.
 - `get_fabricacion_products_for_calculation`: Obtiene y prepara los productos de una fabricación para el motor de cálculo. Retorna una lista de `CalculationProductDTO`.
-

 `controllers/product/material_manager.py`

Nombre del Módulo: `product.material_manager` Descripción: Gestor encargado de la administración de materiales y componentes del sistema, incluyendo su creación, importación masiva y vinculación con productos.

Clase **MaterialManager**

Gestor de materiales y componentes.

Proporciona funcionalidades para gestionar el catálogo de piezas (componentes), su persistencia y su relación con los productos terminados.

Métodos Principales:

- `__init__`: Inicializa el gestor de materiales. Args: `view`: Referencia a la vista principal (`IProductView`). `material_service`: Servicio lógico de materiales (`IMaterialService`). `controller_ref`: Referencia opcional al controlador (`ProductControllerProtocol`).
 - `handle_import_materials_to_product`: Gestiona la importación de una lista de materiales desde un archivo.
 - `handle_add_material_to_product`: Crea un material y lo vincula al producto.
 - `handle_update_material`: Actualiza los datos de un material.
 - `handle_unlink_material_from_product`: Desvincula un material de un producto.
 - `handle_create_material`: Crea un nuevo material en el sistema.
 - `handle_delete_material`: Elimina un material del sistema.
-

 `controllers/product/preproceso_manager.py`

Nombre del Módulo: `product.preproceso_manager` Descripción: Gestor de rutinas de preproceso, encargado de la definición, edición y eliminación de tareas previas necesarias para la fabricación.

Clase PreprocesoManager

Gestor de rutinas de preproceso.

Administra el ciclo de vida de los preprocesos, permitiendo su creación, modificación y eliminación, así como su visualización en la interfaz de gestión.

Métodos Principales:

- `__init__`: Inicializa el gestor de preprocesos. Args: `view`: Referencia a la vista principal (`IProductView`). `fabricacion_service`: Servicio lógico de fabricaciones (`IFabricacionService`). `material_service`: Servicio lógico de materiales (`IMaterialService`). `controller_ref`: Referencia opcional al controlador (`ProductControllerProtocol`).
 - `get_preprocesos_by_fabricacion`: Obtiene preprocesos vinculados a una fabricación.
 - `_load_preprocesos_data`: Carga datos de preprocesos en la tabla visual.
 - `show_add_preproceso_dialog`: Muestra diálogo para crear preproceso.
 - `show_edit_preproceso_dialog`: Muestra diálogo para editar preproceso.
 - `delete_preproceso`: Solicita confirmación y elimina un preproceso.
-

 `controllers/product/product_manager.py`

Nombre del Módulo: `product.product_manager` Descripción: Gestor central para la administración de productos, incluyendo su creación, edición, eliminación y gestión de iteraciones de diseño.

Clase `ProductManager`

Gestor de productos e iteraciones.

Maneja las operaciones CRUD de productos, la validación de sus datos y la coordinación con los servicios de persistencia e iteraciones.

Métodos Principales:

- `__init__`: Inicializa el gestor de productos. Args: `app`: Shell del hub (adjuntos, sesión, UI). `machine_service`: Servicio de máquinas para listados en diálogos. `view`: Referencia a la vista principal (`IProductView`). `product_facade`: Fachada de catálogo / iteraciones (cumple `IProductService`). `state`: Estado compartido de la aplicación (`ApplicationState`). `controller_ref`: Referencia opcional al controlador de productos.
 - `handle_add_product_iteration`: Gestiona la lógica para añadir una nueva iteración de producto.
 - `handle_add_iteration_image`: Añade una imagen a la galería de la iteración.
 - `_final_product_code_from_tree`: Código del nodo marcado como producto final (recorrido tolerante a ciclos).
 - `_select_product_in_list_and_reload`: Selecciona el producto en la lista de resultados y recarga la ficha derecha.
 - `_log_audit`: Helper para registrar auditoría.
 - `_connect_products_signals`: Conecta las señales del widget de gestión de Productos.
-

 `controllers/product/protocols.py`

Nombre del Módulo: `controllers.product.protocols`

Descripción: Protocolos de la capa producto: vista, modelo de fachada y contrato del controlador.

Clase `IProductView`

Vista principal usada por los gestores de producto (p. ej. `MainView`).

Clase `IProductModel`

Fachada pasada como `product_model` (p. ej. `AppModel`).

Los servicios se anotan con las clases concretas para que mypy acepte `AppModel` sin depender de la subtipificación nominal `QObject+Protocol` en todos los casos.

Clase `IFabricacionControllerDelegate`

Subconjunto de `ProductController` usado por `FabricacionController` (delegación UI).

Clase `ProductControllerProtocol`

Contrato estructural que cumple `ProductController` (vista, servicios, estado, callbacks).

 `controllers/simulation/__init__.py`

Nombre del Módulo: simulation Descripción: Contiene la lógica para la simulación de flujos de producción, optimización de tiempos y gestión del editor visual de tareas.

 `controllers/simulation/controller.py`

Nombre del Módulo: `simulation.controller` Descripción: Controlador principal para el módulo de simulación. Orquesta la ejecución de hilos de cálculo, la optimización de recursos y la persistencia de flujos.

Clase `SimulationController`

Controlador de simulaciones y optimización.

Encargado de coordinar el motor de simulación con la interfaz de usuario, gestionando hilos de ejecución para evitar bloqueos y delegando tareas a los managers.

Métodos Principales:

- `__init__`: Inicializa el controlador de simulaciones. Args: `app_controller`: Referencia al controlador principal de la aplicación. `worker_service`: Servicio de trabajadores (inyectado). `machine_service`: Servicio de máquinas (inyectado). `pila_service`: Servicio de pilas (inyectado).
 - `handle_save_flow_only`: Guarda solo el flujo de producción, reconstruyendo los datos necesarios.
 - `_update_simulation_progress`: Actualiza el valor de la barra de progreso en la UI.
 - `clear_simulation_state`: Limpia el estado de la simulación.
-

 `controllers/simulation/editor_manager.py`

Nombre del Módulo: controllers.simulation.editor_manager

Descripción: Coordinación y señales del subsistema «editor_manager»: enlaza UI, servicios y persistencia para este ámbito de la aplicación Hipatia.

 **Clase** `SimulationEditorManager`

Gestor para la gestión del Editor Visual de Flujo de Producción.

`controllers/simulation/execution_helpers.py`

Nombre del Módulo: `controllers.simulation.execution_helpers`

Descripción: Funciones puras de apoyo (sin estado de proceso): `build_scheduler`, `set_planning_units`, `enable_result_actions`, `start_optimizer_thread`. Integración típica con: `__future__`, `datetime`, `core`.

-  `build_scheduler`: Construye el motor de simulación a partir del estado actual de workers y máquinas.
 -  `set_planning_units`: Aplica unidades de producción a cada ítem (dict o DTO).
 -  `enable_result_actions`: Habilita los botones de acciones disponibles tras resultados.
 -  `start_optimizer_thread`: Crea y arranca el hilo de optimización, registrándolo en `controller_ref`.
-

 `controllers/simulation/execution_manager.py`

Nombre del Módulo: `simulation.execution_manager` Descripción: Gestor encargado de la ejecución física de las simulaciones, manejando hilos de trabajo, optimizadores y comunicación de resultados a la UI.

Clase `SimulationExecutionManager`

Gestor de ejecución de simulaciones.

Encargado de configurar el motor de simulación (Scheduler), gestionar los hilos de ejecución de cálculo manual y disparar el proceso de optimización.

Métodos Principales:

- `__init__`: Inicializa el gestor de ejecución. Args: `app`: Referencia a la aplicación principal. `db`: Gestor de base de datos. `worker_service`: Servicio de trabajadores. `machine_service`: Servicio de máquinas. `pila_service`: Servicio de pilas. `view`: Referencia a la vista principal. `state`: Estado compartido. `schedule_manager`: Gestor de horarios. `controller_ref`: Referencia al controlador de simulación.
 - `_prepare_large_visual_simulation`: Minimiza efectos visuales en simulaciones grandes para reducir carga de UI.
-

 `controllers/simulation/optimizer_worker.py`

Nombre del Módulo: `controllers.simulation.optimizer_worker`

Descripción: Coordinación y señales del subsistema «optimizer_worker»: enlaza UI, servicios y persistencia para este ámbito de la aplicación Hipatia.

Clase `OptimizerWorker`

Worker para ejecutar el Optimizer en un hilo separado.

Métodos Principales:

- `run`: Ejecuta el bucle de optimización usando `MotorDeEventos` **directo**.
 - `_create_scheduler`: Crea y configura una instancia de `MotorDeEventos`.
-

 **controllers/simulation/protocols.py**

Nombre del Módulo: simulation.protocols Descripción: Define los protocolos para asegurar la interoperabilidad entre el SimulationController y sus gestores delegados (Execution y Editor).

 `controllers/worker/__init__.py`

Nombre del Módulo: worker Descripción: Subpaquete del controlador de trabajadores en la vista administración / operario.

Exporta `WorkerController` para registro en el contenedor de dependencias.

 `controllers/worker/auth_manager.py`

Nombre del Módulo: `worker.auth_manager` Descripción: Gestor encargado de la seguridad y autenticación de trabajadores, específicamente el cambio de contraseñas propias y ajenas.

 **Clase `WorkerAuthManager`**

Gestor para el cambio de contraseñas de trabajadores y administración.

 `controllers/worker/controller.py`

Nombre del Módulo: WorkerController Descripción: Controlador principal para la gestión de trabajadores y acceso a la interfaz de operario.

Clase WorkerController

Controlador para la gestión de trabajadores (Admin). Incluye CRUD de trabajadores, asignación de tareas y lanzamiento de interfaz de operario. Implementa WorkerControllerProtocol (Fachada).

Métodos Principales:

- `__init__`: Inicializa el controlador de trabajadores inyectando dependencias. Args: `app_controller`: Controlador principal (sesión, QR, navegación). `view`: Vista principal (protocolo IWorkerView). `worker_service`: Servicio de dominio de trabajadores. `product_service`: Búsqueda de productos en asignación de tareas. `fabricacion_service`: Órdenes de fabricación para autocompletado. `workers_changed_signal`: Señal re-emitida por AppModel al cambiar trabajadores.
 - `update_workers_view`: Actualiza la lista de trabajadores en la vista.
 - `_launch_worker_interface`: Lanza la interfaz simplificada para trabajadores.
-

 `controllers/worker/management_manager.py`

Nombre del Módulo: `management_manager` Descripción: Gestión de personal desde «Gestión de datos»: alta, edición y baja lógica de trabajadores, formulario de ficha y carga de órdenes de fabricación para el autocompletado al asignar tareas.

Clase `WorkerManagementManager`

Gestor para la administración de trabajadores (CRUD).

Métodos Principales:

- `__init__`: Inicializa el gestor de administración de trabajadores. Args: `app`: Instancia del controlador principal. `view`: Interfaz de usuario. `worker_service`: Servicio lógico de gestión de trabajadores. `fabricacion_service`: Servicio opcional para gestión de fabricaciones.
 - `_product_service_for_assignment`: Resuelve `ProductService` desde `AppController` (`product_controller` o `model`).
 - `_load_default_products_for_assignment`: Rellena la lista de productos al abrir un trabajador (asignación de tareas).
 - `update_workers_view`: Actualiza la vista de trabajadores con TODOS los trabajadores.
-

 `controllers/worker/protocols.py`

Nombre del Módulo: `worker.protocols`

Descripción: Contratos `Protocol` del subpaquete `controllers.worker`: vista mínima (`IWorkerView`), servicio de dominio (`IWorkerService`), modelo y fachada del controlador `worker`, alineados con DTOs de `core` y trazabilidad sin acoplar PyQt en la definición del protocolo.

Clase `IWorkerView`

Vista raíz mínima para administración de trabajadores (p. ej. `MainView`). Solo expone lo que usan `management/task/auth` sobre `self.view`.

Clase `IWorkerService`

Contrato alineado con `core.services.worker_service.WorkerService`.

Clase `IWorkerModel`

Interfaz legacy para tests que aún mockean el modelo agregado.

Clase `WorkerControllerProtocol`

Interfaz para el controlador fachada de `Worker`.

 `controllers/worker/task_manager.py`

Nombre del Módulo: worker.task_manager Descripción: Gestor de asignación de tareas desde la vista de administración (Gestión de datos / trabajadores). Búsqueda de productos, creación de fabricación tipo tarea y enlace al trabajador seleccionado, incluyendo orden de fabricación (O.F.) opcional pasada al servicio.

 **Clase WorkerTaskManager**

Gestor para la asignación y cancelación de tareas a trabajadores.

 `controllers/worker/worker_camera_config.py`

Nombre del Módulo: `controllers.worker.worker_camera_config`

Descripción: Diálogo de configuración de cámara para la interfaz operario (controllers + ui; no features).

-  `run_worker_camera_config_dialog`: **Abre** `CameraConfigDialog` y **actualiza** `QrScanner` en el `feature controller`.
-

Capítulo: `core/`

Métrica	Valor
Archivos <code>.py</code> en <code>core/</code>	136
Incluidos en el cuerpo	136
Omitidos (docstrings/reglas)	0
Clases detectadas (AST)	181

```
graph TD
    CTRL[Controllers] -->|invocan| CORE[Core/Services]
    CORE -->|persisten/consultan| DB[Database]
```


 **core/__init__.py**

Nombre del Módulo: core

Descripción: Funciones y datos de apoyo del paquete; conviene enlazar qué controlador o servicio las consume y qué estructuras devuelven (ver firmas al inicio del archivo).

Nombre del Módulo: `app_model.py` Descripción: Fachada principal que centraliza el acceso a todos los servicios de dominio (productos, máquinas, trabajadores, etc.) y la base de datos.

Clase `AppModel`

Modelo unificado de la aplicación (Fachada).

Proporciona un punto de entrada único para los controladores, delegando la lógica de negocio en servicios especializados y emitiendo señales de cambio.

Métodos Principales:

- `__init__`: Inicializa el modelo de la aplicación. Args: `db_manager`: Gestor de conexión a la base de datos.
 - `_connect_service_signals`: Conecta las señales de los servicios a las señales del `AppModel` (Bridge).
 - `get_dashboard_stats`: Obtiene estadísticas consolidadas para el dashboard. Delega a los servicios correspondientes.
 - `get_prep_info_for_product`: Delega en `PreparationService` (grupo y máquina por defecto para el producto).
 - `get_latest_fabricaciones`: Obtiene las últimas órdenes de fabricación creadas.
 - `search_fabricaciones`: Busca órdenes de fabricación por código o descripción.
 - `create_fabricacion`: Crea una nueva orden de fabricación básica.
 - `update_fabricacion_preprocesos`: Vincula un conjunto de preprocesos a una fabricación.
 - `create_preproceso`: Crea un nuevo preproceso en el sistema.
 - `update_preproceso`: Actualiza los datos de un preproceso existente.
 - `delete_preproceso`: Elimina un preproceso del sistema.
 - `get_fabricacion_by_id`: Busca una orden de fabricación por su ID numérico.
 - `get_fabricacion_by_codigo`: Busca una orden de fabricación por su código alfanumérico.
 - `get_products_for_fabricacion`: Obtiene los productos asociados a una orden de fabricación.
 - `create_fabricacion_with_preprocesos`: Crea una fabricación y sus preprocesos asociados en una sola transacción.
 - `set_products_for_fabricacion`: Asigna una lista de productos a una fabricación específica.
 - `delete_product`: Elimina un producto del catálogo por su código.
-

 `core/application_state.py`

Nombre del Módulo: `application_state.py` Descripción: Almacén de estado compartido (State Management) que centraliza las variables globales y temporales de la aplicación.

Clase `ApplicationState`

Estado compartido de la aplicación.

Almacena variables de sesión, resultados de la última simulación, estados de búsqueda y referencias a hilos en segundo plano, permitiendo la comunicación entre componentes desacoplados.

Métodos Principales:

- `__init__`: Inicializa una nueva instancia de `ApplicationState`. Configura el logger y todas las variables de estado iniciales.
-

 `core/constants.py`

Nombre del Módulo: `core.constants`

Descripción: Mapas estáticos compartidos por widgets y controladores: `ICONS` y `DEPARTMENT_COLORS` para la UI, `PILA_STATES` para etiquetas de estado de pilas y `VALIDATION` con límites de búsqueda y longitud de textos de producto.

 **core/define_flow_form_io.py**

Nombre del Módulo: `define_flow_form_io` Descripción: Convierte el dict devuelto por `DefineControlPanel.get_form_data` en `FlowTaskConfigDTO`, fuera de la capa ui (Fase 12C).

-  `define_form_data_to_flow_task_config`: Arma el DTO de configuracion desde el mapa del panel de definicion.
-

`core/define_flow_presenter_io.py`

Nombre del Módulo: `define_flow_presenter_io` Descripción: Lectura de mapas de producto/tarea/paso y conversion legacy a DTOs para `DefineFlowPresenter`, fuera de `ui/dialogs` (Fase 12C).

-  `find_first_positive_duration`: Primera clave con valor float > 0 (misma logica que el `presenter legacy`).
 -  `flow_task_data_from_legacy_step_task`: DTO de tarea desde el dict anidado en un paso de flujo persistido.
 -  `flow_task_config_from_legacy_step`: DTO de config desde un paso de flujo persistido (dict plano).
-

 **core/definir_cantidades_dialog_io.py**

Nombre del Módulo: definir_cantidades_dialog_io Descripción: Etiquetas de filas del plan de produccion para DefinirCantidadesDialog, fuera de ui/dialogs (Fase 12C).

`core/di_container.py`

Nombre del Módulo: `di_container.py` Descripción: Contenedor ligero de Inyección de Dependencias (DI) para gestionar la instanciación y resolución de servicios y controladores con soporte para ciclos de vida (Singleton y Transient).

Clase `ServiceLifecycle`

Define el ciclo de vida de un servicio en el contenedor.

Clase `ServiceRegistration`

Estructura interna para almacenar el registro de un servicio.

Clase `DIContainer`

Contenedor de Inyección de Dependencias (Singleton).

Gestiona el registro de tipos y la resolución de instancias, permitiendo configurar el ciclo de vida de cada componente.

Métodos Principales:

- `get_instance`: Devuelve la instancia única (singleton) del contenedor.
 - `register`: Registra un servicio o componente en el contenedor. Args: `service_type`: El tipo o identificador de la clase. `instance`: Una instancia ya creada (se registrará como SINGLETON). `factory`: Una función que devuelve la instancia (lazy loading). `lifecycle`: El ciclo de vida deseado (SINGLETON por defecto).
 - `resolve`: Resuelve y retorna una instancia del servicio solicitado. Args: `service_type`: El tipo clase a resolver. Returns: La instancia solicitada del servicio. Raises: `KeyError`: Si el servicio no está registrado.
 - `is_registered`: Comprueba si un servicio está registrado en el contenedor.
 - `clear`: Limpia todos los registros. Útil para entornos de test.
-

 `core/dtos.py`

Nombre del Módulo: `core.dtos`

Descripción: Módulo shim que reexporta los DTOs definidos en `core.dtos_models` bajo el import estable `core.dtos`, para que el resto del código no acople nombres internos del paquete de modelos.

 **core/dtos_catalog.py**

Nombre del Módulo: core.dtos_catalog

Descripción: Define protocolos o tipos principales: ProductDTO, SubfabricacionDTO, ProcesoMecanicoDTO, MaterialDTO, PilaDTO. Integración típica con: datetime.

 `core/dtos_flow_camera.py`

Nombre del Módulo: `core.dtos_flow_camera`

Descripción: Define protocolos o tipos principales: `FlowTaskDataDTO`, `CanvasCyclicConnectionFlags`, `ProductFlowLibraryProductDTO`, `FlowTaskConfigDTO`, `ProductionFlowStepDTO`. Metadatos de pintado para aristas cíclicas en el canvas de flujo (UI). Integración típica con: `__future__`, `datetime`.

Clase `FlowTaskDataDTO`

Métodos Principales:

- `from_legacy_mapping`: Construye un DTO a partir de un dict legado (canvas / tests). Centraliza la conversión para que la UI no use `.get` sobre mapas crudos.

Clase `CanvasCyclicConnectionFlags`

Metadatos de pintado para aristas cíclicas en el canvas de flujo (UI).

Métodos Principales:

- `from_connection_mapping`: Interpreta flags desde el dict de conexión legado del canvas.

Clase `ProductFlowLibraryProductDTO`

Agrupación de descripción de producto y tareas (`FlowTaskDataDTO`) para biblioteca / panel de definición.

Clase `FlowItemDTO`

DTO para representar un ítem del flujo en la vista (Fase 12C).

 `core/dtos_models.py`

Nombre del Módulo: `core.dtos_models`

Descripción: Funciones y datos de apoyo del paquete; conviene enlazar qué controlador o servicio las consume y qué estructuras devuelven (ver firmas al inicio del archivo). Integración típica con: `datetime`, `core`.

 **Clase `FileOperationResultDTO`**

Resultado de una operación de adjuntar o mover archivos.

 **Clase `ProductDetailsDTO`**

Detalles completos de un producto y sus subcomponentes.

 **Clase `QuotedTO`**

Frase célebre.

 **Clase `AuthorInfoDTO`**

Información enriquecida de un autor de Wikipedia.

 **Clase `SyncRecordPayloadDTO`**

Contenedor de datos dinámicos para un registro de sincronización.

 **Clase `SyncRecordDTO`**

Un registro individual para sincronización.

 **Clase `SyncTableDifferencesDTO`**

Diferencias detectadas en una tabla específica.

 **Clase `DatabaseComparisonDTO`**

Resultado completo de la comparación de dos bases de datos.

 **Clase `WorkerFormDataDTO`**

Datos extraídos del formulario de un trabajador.

 **Clase `LoteInstanceParametersDTO`**

Parámetros para instanciar un lote desde UI.

 **Clase `CalculationStepDTO`**

Representa un paso o ítem en la sesión de planificación/cálculo.

`core/enhanced_flow_canvas_state_io.py`

Nombre del Módulo: `enhanced_flow_canvas_state_io` Descripción: Mutaciones y consultas sobre entradas `canvas_tasks` del flujo enhanced (data/config/position), fuera de ui/dialogs (Fase 12C).

-  `canvas_state_all_logical_connections`: Aristas globales: dependencias, ciclos y orden por defecto en el lienzo. Las tareas con inicio por fecha (sin dependencia explícita) se enlazan en cadena $(i-1) \rightarrow i$ para mostrar el orden de planificación al colocar tarjetas. Si una tarea define `start_condition` tipo `dependency`, no se añade ese eslabón secuencial hacia ella (el orden lo marca solo el predecesor elegido).
-

 `core/enhanced_flow_presenter_io.py`

Nombre del Módulo: `enhanced_flow_presenter_io` Descripción: Carga de flujo, biblioteca de productos y exportación a dicts para FlowBuilder (`ui/dialogs/production_flow/flow_builder.py`), fuera de `ui/dialogs` (Fase 12C).

-  `_main_task_as_mapping`: Acepta dict (legado) o `CalculationProductDTO` (sesión desde `PilaService`).
-

Nombre del Módulo: `flow_canvas_io` Descripción: Lectura tipada de mapas del grafo de flujo (entradas `canvas_tasks` del `presenter`) desde capa no-UI, para que los widgets no usen `.get/[]` en bucles de pintado. Expone `canvas_task_body`, `flow_task_entry_config`, `flow_task_entry_widget`, normalización de aristas (`CanvasVisualConnection`) y flags de ciclo en `config`.

Clase `CanvasVisualConnection`

Arista visual entre dos widgets del canvas de flujo de producción (`ProductionFlowCanvas`).

-  `canvas_visual_connection_from_mapping`: Construye una conexión visual desde el dict histórico `start/end/type`.
 -  `normalize_canvas_visual_connections`: Normaliza entradas dict o DTO a lista homogénea.
 -  `flow_task_entry_widget`: Widget PyQt asociado a una entrada `canvas_tasks[i]` (clave `widget`).
 -  `flow_task_entry_config`: Subdict `config` de una entrada de tarea en el canvas de flujo.
 -  `flow_task_entry_is_cycle_start`: True si la entrada marca inicio de ciclo en `config`.
 -  `canvas_task_body`: Cuerpo data de una entrada `presenter.canvas_tasks[i]`.
 -  `canvas_task_display_name`: Nombre de tarea para listas de diálogo (data dict o DTO con `.name`).
 -  `flow_task_config_is_cycle_end_flag`: True si `config` marca fin de ciclo.
 -  `flow_task_config_is_cycle_start_flag`: True si `config` marca inicio de ciclo.
 -  `flow_task_config_cycle_return_to_index`: Índice de tarea a la que regresa el ciclo (clave `cycle_return_to_index` en `config`).
 -  `cycle_end_dialog_configuration_values`: Par (`is_cycle_end`, `return_to_index`) del dict de `CycleEndConfigDialog.get_configuration`.
 -  `worker_line_config_display_name`: Nombre visible de una línea de trabajador en el canvas (clave `name`).
 -  `worker_line_config_reassignment_rule`: Regla de reasignación asociada a la línea de trabajador, si existe.
 -  `worker_line_config_set_reassignment_rule`: Persiste la regla de reasignación en la `config` mutable de la línea.
 -  `connection_widgets_pair`: Devuelve (`start`, `end`) tal como los guarda el modelo de conexiones del canvas.
 -  `connection_link_type`: Tipo de arista: 'normal' o 'cyclic'.
 -  `connection_cyclic_paint_flags`: Flags de pintado para aristas cíclicas (mapeo serializable o `CanvasVisualConnection`).
-

`core/flow_card_labels.py`

Nombre del Módulo: `flow_card_labels` Descripción: Textos para tarjetas del canvas de flujo; lectura de mapas de tarea fuera de ui/ (Fase 12C).

-  `flow_card_primary_html`: HTML principal nombre + duracion.
 -  `flow_card_task_id_str`: Identificador logico de la tarea para señales.
 -  `flow_card_with_workers_html`: Returns: (texto QLabel, tooltip)
-

`core/flow_graph_manager_io.py`

Nombre del Módulo: `flow_graph_manager_io` Descripción: Lectura y mutacion del estado canvas/presenter usada por `FlowGraphManager`, fuera de `subscripts` y `.get` en la capa `ui` (Fase 12C).

-  `logical_connection_indices`: Indices from/to desde `get_logical_connections`.
 -  `logical_connection_highlights`: `highlight_parent`, `highlight_child`, `highlight_destination`, `highlight_origin`.
 -  `apply_loaded_flow_step_to_presenter_config`: Copia campos persistidos del paso al subdict config del presenter.
-

 `core/flow_inspector_context.py`

Nombre del Módulo: `flow_inspector_context` Descripción: DTO de vista para enlazar el grafo de flujo con el inspector de tarea sin construir mapas intermedios con subíndices en el diálogo.

Clase `FlowInspectorTaskContext`

Datos listos para `InspectorPanel.set_task` y listas asociadas.

Métodos Principales:

- `inspector_step_payload`: Formato esperado por el inspector (id + task + config).
-

`core/holidays_config_io.py`

Nombre del Módulo: `holidays_config_io` Descripción: Normalización y consultas sobre la lista de festivos en configuración, fuera de la capa ui (Fase 12C).

-  `normalize_holidays_json`: Convierte JSON/str/lista heterogenea en lista deduplicada de dicts `date/desc`.
 -  `holiday_dates_set`: Conjunto de fechas ISO ya normalizadas.
 -  `holidays_without_date`: Copia sin la entrada con la fecha dada.
 -  `iter_holiday_dates_iso`: Lista ordenada de fechas para resaltar en calendario.
-

 `core/inspector_task_payload_io.py`

Nombre del Módulo: `inspector_task_payload_io` Descripción: Lectura de filas task/config del inspector de flujo, fuera de ui/ (Fase 12C).

Nombre del Módulo: paths Descripción: Rutas de aplicación: desarrollo frente a ejecutable PyInstaller (`sys.frozen`).

- Solo lectura embebida: usar `core.utils.helpers.resource_path(_MEIPASS)`.
 - Escritura (SQLite, logs, backups, copia de usuario de `config.ini`): directorio del ejecutable en frozen; raíz del repositorio en desarrollo.
 - Evitar situar esa carpeta de datos escribibles bajo sincronización en la nube (iCloud, OneDrive, etc.): para estabilidad del SQLite y trazabilidad, preferir backup/restauración explícitos en lugar de sync transparente del directorio.
 - 🐞 `get_writable_app_root`: Directorio donde la app puede crear `data/`, `logs/`, etc. En binario PyInstaller (`onedir/onefile`) coincide con la carpeta del `.exe`. En desarrollo, la raíz del repositorio (padre de `core/`).
 - 🐞 `resolve_user_config_ini`: Ruta efectiva de `config/config.ini`. En desarrollo lee el fichero del repo. En frozen copia una vez desde el bundle a `<exe_dir>/config/config.ini` para que la conexión recordada sea escribible.
-

 `core/planning_session_access.py`

Nombre del Módulo: `core.planning_session_access`

Descripción: Funciones puras de apoyo (sin estado de proceso): `planning_unidades`,
`planning_identificador`, `planning_deadline`, `planning_lote_codigo`, `deadline_to_date`.

Integración típica con: `__future__`, `datetime`, `core`.

-  `planning_unidades`: Unidades de fabricación asociadas al ítem de sesión.
-

 `core/production_context.py`

Nombre del Módulo: `core.production_context`

Descripción: Define protocolos o tipos principales: `ProductionStatus`, `ProductionContext`. Data class to hold the status of the current production session.

Clase `ProductionStatus`

Data class to hold the status of the current production session.

Clase `ProductionContext`

Manages the context of the current production session for a worker. Keeps track of the current Order (OF), progress (1 of X), and current process layer.

Métodos Principales:

- `start_session`: Starts a new production session.
 - `increment_unit`: Increments the completed units counter.
 - `is_complete`: Checks if the target number of units has been reached.
 - `get_progress_label`: Returns a formatted string like 'Unit 5 of 100'.
 - `reset`: Clears the current session.
-

Nombre del Módulo: `core.qr_generator`

Descripción:

=====

GENERADOR DE CÓDIGOS QR - SISTEMA DE TRAZABILIDAD

=====

===== Genera
códigos QR únicos para trazabilidad de unidades individuales y proporciona funciones para convertirlos a
diferente...

Clase `qrGenerator`

Generador de códigos QR para trazabilidad.

Attributes: `logger`: Logger para registro de operaciones `qr_version`: Versión del código QR (tamaño)
`error_correction`: Nivel de corrección de errores `box_size`: Tamaño de cada "caja" del QR en píxeles `border`:
Tamaño del borde en cajas

Métodos Principales:

- `__init__`: Inicializa el generador de QR. Args: `qr_version`: Versión del QR (1-40, None para auto)
`error_correction`: Nivel de corrección (L, M, Q, H) `box_size`: Tamaño de cada caja en píxeles `border`:
Tamaño del borde en cajas
 - `generate_unique_id`: Genera un identificador único para una unidad. El formato es: FAB{id}-{producto}-UNIT{num}-{timestamp}-{hash} Ejemplo: FAB123-PROD001-UNIT1-20250131143022-A3F9
Args: `fabricacion_id`: ID de la fabricación `producto_codigo`: Código del producto `unit_number`:
Número de unidad `timestamp`: Timestamp específico (opcional, usa `datetime.now()` por defecto)
Returns: String con el identificador único
 - `generate_qr_code`: Genera un código QR a partir de datos. Args: `data`: Datos a codificar en el QR
`size`: Tamaño final de la imagen (ancho, alto) en píxeles (opcional) Returns: PIL Image con el código
QR o None si hay error
 - `generate_qr_pixmap`: Genera un QPixmap de PyQt6 con el código QR. Útil para mostrar el QR
directamente en la UI de PyQt6. Args: `data`: Datos a codificar `size`: Tamaño del QPixmap (ancho,
alto) Returns: QPixmap con el código QR o None si hay error
 - `save_qr_to_file`: Guarda un código QR en un archivo. Args: `data`: Datos a codificar `filepath`: Ruta
donde guardar el archivo `size`: Tamaño opcional de la imagen Returns: True si se guardó
correctamente, False si hubo error
 - `generate_batch_qr_codes`: Genera múltiples códigos QR en lote. Args: `base_data`: Datos base
para el QR `count`: Número de QRs a generar `size`: Tamaño opcional de las imágenes Returns: Lista
de tuplas (data, imagen)
 -  `generate_simple_qr`: Función de utilidad para generar rápidamente un QR. Args: `data`: Datos a
codificar `size`: Tamaño del QR Returns: QPixmap con el QR o None si hay error
 -  `generate_production_qr_id`: Función de utilidad para generar un ID de producción estándar.
Args: `fabricacion_id`: ID de la fabricación `producto_codigo`: Código del producto `unit_number`:
Número de unidad Returns: ID único para la unidad
-

Nombre del Módulo: `qt_log_handler` Descripción: Handler de logging de Python que integra el registro estándar con el hilo de la interfaz Qt. Por defecto el nivel del handler es `INFO`, por lo que los mensajes `INFO` y superiores se reenvían a la UI mediante señales Qt (apto para uso desde otros hilos).

Diseño:

- `QtLogHandler` hereda de `logging.Handler` (no puede ser `QObject` simultáneamente, por eso se delega la señal a `_SignalEmitter`).
- `_SignalEmitter` es un `QObject` mínimo que expone la señal `log_emitted(str)`. Al conectar esa señal a un slot del hilo principal, Qt garantiza que la ejecución del slot ocurre en el event-loop correcto aunque `emit()` se invoque desde otro hilo.
- Almacena en un buffer interno los mensajes que llegan antes de que la UI esté lista. Cuando `connect_to_widget()` se invoca, reproduce el buffer completo para que el usuario vea también los warnings de arranque generados antes del login.

Clase `_SignalEmitter`

Objeto Qt auxiliar que alberga la señal de log.

Se separa en su propia clase porque `logging.Handler` no puede heredar de `QObject` (herencia múltiple incompatible con la metaclassa de Qt).

Signals: `log_emitted`: emitida por cada registro de log procesado. Transporta el mensaje ya formateado como cadena.

Clase `QtLogHandler`

Handler que reenvía a la UI de Qt los registros desde el nivel del handler (por defecto `INFO`) en adelante.

Conexión thread-safe vía señal `log_emitted`. Buffer interno hasta la primera llamada a `connect_to_widget()` (arranque y login sin terminal).

En la aplicación, `app.py` delega en `HomeWidget.connect_log_handler` o `WorkerMainWindow.connect_log_handler`, que a su vez llaman a este método con el `append_log` del `LogTerminalWidget` correspondiente.

Attributes: `emitter: _SignalEmitter` con la señal `log_emitted(str)`.

Métodos Principales:

- `__init__`: Inicializa el handler con nivel `INFO`, formatter estándar y buffer vacío. El formatter incluye hora, nivel y nombre del logger para facilitar la identificación del origen del mensaje en la terminal visual.
 - `emit`: Procesa un registro de log y emite la señal Qt con el mensaje formateado. Si el widget de destino aún no está conectado, el mensaje se almacena en el buffer interno para ser reproducido posteriormente. Llamado automáticamente por el sistema de logging cada vez que se genera un mensaje cuyo nivel supera el mínimo del handler. Args: `record`: Registro de log generado por el framework estándar de Python.
 - `connect_to_widget`: Conecta el handler al slot del widget y reproduce el buffer acumulado. Debe llamarse **una sola vez por sesión**, cuando ya existe el widget de destino: terminal de `HomeWidget` (vista principal) o de `WorkerMainWindow` (pestaña Log). A partir de aquí los mensajes van en tiempo real al slot y dejan de acumularse solo en memoria. Args: `slot`: Callable que recibe el mensaje ya formateado; habitualmente `LogTerminalWidget.append_log`.
-

`core/quote_service.py`

Nombre del Módulo: `quote_service` Descripción: Frases inspiradoras para la pantalla de inicio y textos secundarios de la UI.

Carga `resources/quotes.json`; si no existe o es inválido, usa una lista integrada. Opcionalmente enriquece con resumen o imagen del autor desde Wikipedia en español (con caché en memoria por sesión; sin red, solo se muestra la cita).

Clase `QuoteService`

Expone frases aleatorias y búsqueda de resumen/imagen de autor con caché en memoria por sesión.

Las llamadas a Wikipedia son perezosas y pueden fallar sin red; en ese caso `get_author_info` devuelve `None` y la UI puede mostrar solo la cita.

Métodos Principales:

- `__init__`: Args: `quotes_json_path`: Ruta absoluta opcional al JSON; por defecto `resource_path('resources/quotes.json')`.
 - `_load_quotes`: Carga las frases del JSON local (`resources/quotes.json`); si no existe, usa frases integradas.
 - `get_random_quote`: Devuelve una frase aleatoria. Returns: Instancia de `QuoteDTO` con la frase y el autor.
 - `get_author_info`: Busca información del autor en Wikipedia (Bio + Imagen). Args: `author_name`: Nombre del autor a buscar. Returns: `AuthorInfoDTO` con el resumen e imagen, o `None` si no se encuentra.
-

 `core/reassignment_rule_dialog_io.py`

Nombre del Módulo: reassignment_rule_dialog_io Descripción: Lectura de tareas canvas y reglas de reasignación para el diálogo, fuera de ui/dialogs (Fase 12C).

 `core/reports_dtos.py`

Nombre del Módulo: `core.reports_dtos`

Descripción: Define protocolos o tipos principales: `ResultadoBusquedaDTO`, `OrdenFabricacionResumenDTO`, `OrdenFabricacionDetalleDTO`, `PromedioTiempoDTO`, `TiempoTrabajadorDTO`. DTO para resultados de búsqueda inteligente. Integración típica con: `datetime`.

Clase `ResultadoBusquedaDTO`

DTO para resultados de búsqueda inteligente. Representa un producto, fabricación u orden encontrada.

Clase `OrdenFabricacionResumenDTO`

DTO para resumen de una Orden de Fabricación. Muestra información agregada sin detalles individuales.

Clase `OrdenFabricacionDetalleDTO`

DTO para detalle completo de una Orden de Fabricación. Incluye información extendida para vista de detalle.

Clase `PromedioTiempoDTO`

DTO para estadísticas de tiempo promedio de un producto. Incluye métricas de dispersión para análisis.

Clase `TiempoTrabajadorDTO`

DTO para tiempos promedio por trabajador en un producto. Permite comparar rendimiento entre operarios.

Clase `IncidenciaResumenDTO`

DTO para resumen de incidencias agrupadas por tipo. Usado en gráficas de patrón de incidencias.

Clase `PuntoEvolucionDTO`

DTO para un punto en la gráfica de evolución temporal. Representa el tiempo promedio en un período específico.

Clase `UnidadTrabajoDTO`

DTO para detalle de una unidad individual de trabajo. Usado en la vista expandida de una orden.

Clase `ResumenProductoDTO`

DTO para resumen estadístico de un producto. Información general mostrada al seleccionar un producto.

 `core/schedule_config.py`

Nombre del Módulo: `core.schedule_config`

Descripción: Define protocolos o tipos principales: `ScheduleConfig`. Integración típica con: `datetime`, `json`.

Clase `ScheduleConfig`

Métodos Principales:

- `_load_from_database`: Carga la configuración desde la base de datos.
 - `__getstate__`: Prepara el estado del objeto para ser 'pickled' (guardado). Excluimos los atributos que no se pueden guardar, como el logger y el gestor de BD.
 - `__setstate__`: Restaura el estado del objeto al ser 'unpickled' (cargado).
 - `reload_config`: Recarga la configuración desde la base de datos. Args: `db_manager`: Gestor de base de datos (opcional, usa `self.db_manager` si no se proporciona)
 - `_parse_time`: Convierte string de tiempo a objeto `time`.
 - `_process_holidays`: Procesa los datos de festivos desde la base de datos y los convierte a objetos `date` para compatibilidad con el sistema de calendario.
-

`core/subfabricacion_rows.py`

Nombre del Módulo: `subfabricacion_rows` Descripción: Normaliza filas de subfabricación (dict legado u objetos con atributos) a `SubfabricacionDTO`. Vive en `core/` para que la UI no use `.get/` claves sueltas en la frontera Fase 12C (analizador `ui_dto_boundary`).

-  `coerce_subfabricaciones_rows`: El widget de productos puede guardar subfabricaciones como dict; el diálogo opera con DTOs. Args: `rows`: Filas heterogéneas (DTO, dict compatible u objetos con los mismos campos). Returns: Lista de `SubfabricacionDTO` listos para pintar o persistir.
-

Nombre del Módulo: `sync_service` Descripción: Comparación y fusión selectiva entre la base SQLite local y un fichero SQLite externo (`sneakernet` / USB / copia exportada). Construye `DatabaseComparisonDTO` para la UI y aplica cambios elegidos respetando orden de claves foráneas e incluyendo tablas de asociación (BOM producto-material).

Clase `SyncService`

Servicio que compara y fusiona registros entre la sesión local y una segunda base SQLite.

La lista `SYNCABLE_TABLES` define el orden de comparación y de aplicación (productos y máquinas antes que subfabricaciones y procesos mecánicos; materiales antes que `producto_material_link`). Las tablas en `ASSOCIATION_TABLES` solo generan filas `new` cuando el vínculo existe en extranjero y no en local.

Métodos Principales:

- `__init__`: Args: `local_session_factory`: Fábrica que devuelve sesiones SQLAlchemy sobre la BD local (típicamente `sessionmaker` ligado al motor de la app).
 - `compare_databases`: Compara todas las tablas configuradas entre local y el fichero SQLite externo. Args: `foreign_db_path`: Ruta absoluta al `.db` ya resuelto (no comprimido). Returns: DTO con una entrada por tabla que tenga diferencias (`new` o `updated` en extranjero).
 - `_compare_table`: Compara una tabla ORM columna a columna (sin relaciones cargadas). Args: `local_session`: Sesión sobre la BD local. `foreign_session`: Sesión sobre la BD extranjera. `model_class`: Modelo declarativo SQLAlchemy. `primary_key`: Nombre del atributo que actúa como clave primaria. Returns: Lista de `SyncRecordDTO` con `action` `new` o `updated` respecto a local. Los `datetime` se consideran distintos si difieren en más de un segundo.
 - `_association_row_set`: Lee todas las filas de una tabla de enlace como conjunto de tuplas de claves. Args: `session`: Sesión activa. `table`: Objeto `Table` SQLAlchemy (p. ej. `producto_material_link`). `key_columns`: Columnas que forman la identidad de la fila. Returns: Conjunto de tuplas hashables para comparación de presencia.
 - `_compare_association_table`: Detecta vínculos que existen en extranjero y faltan en local (solo inserciones). Args: `local_session`: Sesión local. `foreign_session`: Sesión extranjera. `table`: Tabla de asociación. `key_columns`: Columnas de la clave compuesta. Returns: Lista de `SyncRecordDTO` con `action='new'` por cada fila a insertar.
 - `_sort_table_diffs_for_apply`: Reordena los bloques seleccionados por el usuario según `SYNCABLE_TABLES + ASSOCIATION_TABLES`. Args: `tables`: Fragmentos del DTO tal como devuelve el diálogo (orden de pestañas). Returns: Misma información ordenada para minimizar errores de FK al aplicar.
 - `apply_changes`: Aplica en la BD local los registros incluidos en `comparison` (una transacción con `commit`). Args: `comparison`: Subconjunto devuelto por `SyncDialog.get_selected_changes`. Returns: Número de filas aplicadas con éxito (ORM + asociaciones).
 - `_apply_single_record`: Inserta o actualiza una fila ORM según `record_dto.action`. Args: `session`: Sesión local (sin `commit` aún). `model_class`: Modelo destino. `primary_key`: Nombre del campo PK en el modelo. `record_dto`: Carga útil con campos serializables. Returns: `False` si `updated` y no existe la fila local; `True` en caso contrario.
 - `_apply_association_row`: Ejecuta `INSERT` en una tabla `Table` de metadatos compartidos. Args: `session`: Sesión local. `table`: Tabla de enlace. `record_dto`: Debe traer `action=='new'` y campos alineados con las columnas. Returns: `True` si se insertó; `False` si la acción no es de alta.
-

 `core/tracking_dtos.py`

Nombre del Módulo: `core.tracking_dtos`

Descripción: Define protocolos o tipos principales: `FabricacionAsignadaDTO`, `IncidenciaAdjuntoDTO`, `IncidenciaLogDTO`, `PasoTrazabilidadDTO`, `TrabajoLogDTO`. DTO para fabricaciones asignadas a un trabajador. Integración típica con: `datetime`, `core`.

 **Clase `FabricacionAsignadaDTO`**

DTO para fabricaciones asignadas a un trabajador.

 **Clase `IncidenciaAdjuntoDTO`**

DTO para adjuntos de una incidencia.

 **Clase `IncidenciaLogDTO`**

DTO para incidencias registradas.

 **Clase `PasoTrazabilidadDTO`**

DTO para pasos de trazabilidad (sellos).

 **Clase `TrabajoLogDTO`**

DTO para el log principal de trabajo (Pasaporte).

 `core/worker_ui_dtos.py`

Nombre del Módulo: `core.worker_ui_dtos`

Descripción: Nombre del Modulo: `worker_ui_dtos` Descripcion: DTOs tipados para la vista trabajador (lista de fabricaciones asignadas).

 **Clase `WorkerTaskListRowDTO`**

Fila plana para la lista de tareas/fabricaciones en `WorkerMainWindow`.

Métodos Principales:

- `to_signal_dict`: Payload para señales y controladores que aún esperan dict plano.
 - `from_flat_mapping`: Construye desde el dict histórico de `WorkerDbSync` (tests y migración).
-

 `core/camera_manager/__init__.py`

Nombre del Módulo: `core.camera_manager`

Descripción: Concentra datos de configuración o catálogos estáticos: `__all__`, consumidos por la UI y controladores. Integración típica con: `base`, `manager`.

-  `main`: Función principal para pruebas manuales.
-

 `core/camera_manager/base.py`

Nombre del Módulo: `camera_manager.base`

Descripción: Define protocolos o tipos principales: `CameraBackend`, `CameraInfo`.

Nombre del Módulo: `camera_manager.capture`

Descripción: Apertura unificada de `cv2.VideoCapture` con el mismo criterio que `CameraManager` (vía `get_system_backend`). En Windows el primario es `DirectShow` y se prueban a continuación `MSMF` y `AUTO` como respaldo si una webcam no abre con un solo API.

-  `default_capture_backend_chain`: Construye la lista ordenada de backends a probar al abrir una cámara. Returns: Cadena sin duplicados: primero el backend del SO (p. ej. `DSHOW` en Windows, `AVFoundation` en macOS), luego en Windows `MSMF` y `AUTO`; en el resto, `AUTO` si faltaba.
 -  `merge_backend_priority`: Prioriza un backend explícito y añade el resto según `default_capture_backend_chain`. Args: `primary`: Backend solicitado por el llamador (p. ej. validación con backend concreto). Returns: Lista de backends sin duplicados, con `primary` en primer lugar.
 -  `open_video_capture_with_backends`: Intenta `VideoCapture(index, backend)` en orden hasta que `isOpened()` sea verdadero. Libera cada captura que no abre antes de probar la siguiente. Si `cv2` no está disponible, devuelve `None` sin lanzar excepción. Args: `index`: Índice de dispositivo de vídeo. `backends`: Secuencia de backends OpenCV a probar (valores del enum `CameraBackend`). Returns: Instancia de `cv2.VideoCapture` abierta, o `None` si ningún backend abre el índice.
 -  `open_video_capture`: Abre la cámara en `index` usando `default_capture_backend_chain`. Args: `index`: Índice de dispositivo de vídeo. Returns: `VideoCapture` listo para usar, o `None` si no hay `cv2` o no abre con ningún backend.
-

Nombre del Módulo: `camera_manager.detector`

Descripción: Detección y prueba de cámaras vía OpenCV: nombres heurísticos por SO, validación de hardware con lectura de frames y vista previa. La apertura de `VideoCapture` se delega en `capture.open_video_capture_with_backends` para mantener la misma cadena de backends que el resto del gestor de cámara.

-  `get_camera_name`: Devuelve un nombre legible y si la cámara se considera externa (heurística por índice y SO). Args: `index`: Índice del dispositivo de vídeo. `backend`: Backend OpenCV asociado (se usa sobre todo para trazas; la heurística es por SO). Returns: Tupla (`nombre`, `es_externa`).
 -  `validate_hardware`: Abre la cámara con la cadena de backends fusionada y valida lectura de frames. Args: `index`: Índice del dispositivo de vídeo. `backend`: Backend preferido; se combina con `default_capture_backend_chain` vía `merge_backend_priority`. `frames`: Número de intentos de lectura para decidir si la cámara `is_working`. Returns: `CameraInfo` con dimensiones y estado, o `None` si no hay `cv2` o no abre la cámara.
 -  `test_preview`: Muestra una ventana de previsualización durante `duration` segundos (o hasta ESC). Args: `index`: Índice del dispositivo de vídeo. `backend`: Backend preferido para construir la cadena de apertura. `duration`: Segundos máximos de bucle de captura. Returns: `True` si se mostró al menos un frame; `False` si no hay OpenCV, no abre la cámara o no hay frames.
-

 `core/camera_manager/manager.py`

Nombre del Módulo: `manager.py` (CameraManager) Descripción: Gestor de hardware de cámara. Controla el acceso, la captura de frames y la liberación de recursos de video.

Clase `CameraManager`

Gestiona la detección, validación y acceso a cámaras de video conectadas al sistema. Proporciona funcionalidades para listar cámaras disponibles, obtener información detallada y seleccionar la mejor cámara para una aplicación.

 **core/camera_manager/Utils.py**

Nombre del Módulo: camera_manager.Utils

Descripción: Funciones puras de apoyo (sin estado de proceso): `get_system_backend`. Integración típica con: `platform`, `base`.

 `core/facades/__init__.py`

Nombre del Módulo: `core.facades`

Descripción: Expone `Facade` como API estable de aplicación sobre servicios ya inyectados; no contiene reglas de persistencia directa. Integración típica con: `__future__`, `core`.

 `core/facades/planning_facade.py`

Nombre del Módulo: `core.facades.planning_facade`

Descripción: Expone `PlanningFacade` como API estable de aplicación sobre servicios ya inyectados; no contiene reglas de persistencia directa. Integración típica con: `__future__`, `datetime`, `core`.

 **Clase `PlanningFacade`**

Punto estable para planificación; delega en `PilaService`.

 `core/facades/product_facade.py`

Nombre del Módulo: `core.facades.product_facade`

Descripción: Expone `ProductFacade` como API estable de aplicación sobre servicios ya inyectados; no contiene reglas de persistencia directa. Integración típica con: `__future__`, `core`.

Clase `ProductFacade`

Punto estable para el dominio producto; delega en `ProductService`.

Métodos Principales:

- `service`: Acceso al servicio durante la migración (señales Qt, tests).
-

 `core/facades/production_facade.py`

Nombre del Módulo: `core.facades.production_facade`

Descripción: Expone `ProductionFacade` como API estable de aplicación sobre servicios ya inyectados; no contiene reglas de persistencia directa. Integración típica con: `__future__`.

 **Clase `ProductionFacade`**

Agrupación de `FabricacionService` y operaciones de repositorio pendientes de migrar.

 `core/facades/reporting_facade.py`

Nombre del Módulo: `core.facades.reporting_facade`

Descripción: Expone `ReportingFacade` como API estable de aplicación sobre servicios ya inyectados; no contiene reglas de persistencia directa. Integración típica con: `__future__`.

 **Clase** `ReportingFacade`

Agrupación de operaciones de `ReportService`.

 `core/facades/system_facade.py`

Nombre del Módulo: `core.facades.system_facade`

Descripción: Expone `SystemFacade` como API estable de aplicación sobre servicios ya inyectados; no contiene reglas de persistencia directa. Integración típica con: `__future__`.

 **Clase `systemFacade`**

Agrupación: `MachineService`, `PreparationService` y utilidades de configuración/lotes.

 `core/facades/workforce_facade.py`

Nombre del Módulo: `core.facades.workforce_facade`

Descripción: Expone `WorkforceFacade` como API estable de aplicación sobre servicios ya inyectados; no contiene reglas de persistencia directa. Integración típica con: `__future__`.

 **Clase `WorkforceFacade`**

Agrupación de operaciones de `WorkerService` y `TrackingAssignmentService`.

 `core/health/__init__.py`

Nombre del Módulo: `core.health`

Descripción: Funciones y datos de apoyo del paquete; conviene enlazar qué controlador o servicio las consume y qué estructuras devuelven (ver firmas al inicio del archivo).

 `core/health/constants.py`

Nombre del Módulo: `core.health.constants`

Descripción: Concentra datos de configuración o catálogos estáticos: `CRITICAL_TABLES`, `THRESHOLDS`, consumidos por la UI y controladores.

 `core/health/health_checker.py`

Nombre del Módulo: `health_checker` Descripción: Verifica el estado de la base de datos y del sistema al arranque. Sin dependencia de UI — solo lógica pura.

Clase `TableHealth`

Estado de una tabla de la base de datos.

Clase `SystemHealth`

Información de salud general del sistema.

Clase `TestResults`

Resultados de la ejecución de tests unitarios.

Clase `HealthReport`

Informe completo de salud del sistema.

Clase `DatabaseHealthChecker`

Verifica el estado de la base de datos y del sistema.

Métodos Principales:

- `check`: Ejecuta todas las verificaciones y devuelve un `HealthReport`. Args: `db_manager`: Instancia de `DatabaseManager`. Returns: `HealthReport` con el estado completo del sistema.
 - `_check_tables`: Verifica el estado de cada tabla conocida.
 - `_check_system`: Recopila información de salud del sistema.
 - `_compute_status`: Calcula el estado general basado en el informe.
-

 `core/health/health_worker.py`

Nombre del Módulo: `health_worker` Descripción: `QThread` que orquesta `DatabaseHealthChecker` y `TestRunner` emitiendo señales de progreso a la UI.

Clase `HealthCheckWorker`

Hilo de verificación de salud del sistema. Emite señales de progreso para que la UI las consuma sin bloquearse.

Métodos Principales:

- `__init__`: Inicializa el worker con el gestor de base de datos. Args: `db_manager`: Instancia de `DatabaseManager`. `run_tests`: Si `True`, ejecuta los tests unitarios tras verificar la BD.
 - `run`: Ejecuta las verificaciones en el hilo secundario.
-

 `core/health/test_runner.py`

Nombre del Módulo: `test_runner` Descripción: Ejecuta `pytest -m unit` en un subprocess y parsea el progreso en tiempo real.

Clase `TestRunner`

Ejecuta `pytest -m unit` en un subprocess y emite progreso línea a línea.

Métodos Principales:

- `run`: Lanza `pytest -m unit` y llama a los callbacks con el progreso. Args: `progress_callback: (test_name, current, total)` `finished_callback: (TestResults)`
 - `_read_coverage`: Lee la cobertura del último run guardado en `test_reports/compliance_data.json`.
-

 `core/import_manager/__init__.py`

Nombre del Módulo: `import_manager` Descripción: Paquete de importación de listas de materiales (BOM) desde formatos A3RP.

Incluye adaptadores de fichero, DTOs de árbol y el servicio que persiste en la base de datos tras la revisión en pantalla.

 `core/import_manager/dto.py`

Nombre del Módulo: `import_manager.dto` Descripción: Estructuras de datos para un listado de materiales (BOM) leído desde Excel A3RP.

Cada nodo puede marcarse en pantalla para importar o no, y recibir un rol (producto final, subfabricación, proceso mecánico o componente) antes de persistir en base de datos.

Clase `BOMImportRole`

Clasificación explícita de cada fila marcada en la supervisión de importación A3RP.

Clase `BOMNodeDTO`

Nodo del árbol BOM leído desde A3RP y enriquecido en la supervisión de importación.

Attributes: `nivel`: Profundidad en la lista de materiales. `codigo_componente`: Identificador de fila (código de pieza o operación según contexto). `capitulo`: Metadato de capítulo del listado, si existe. `denominacion`: Texto descriptivo mostrado al usuario. `es_subfabricacion`: Hint del adaptador Excel (*Compuesto*); la verdad operativa es `import_role`. `cantidad`: Cantidad de la línea. `hijos`: Subárbol recursivo. `import_selected`: Si `True`, el usuario marcó la fila para persistir. `import_role`: Rol asignado en el diálogo (obligatorio si está marcada).

 `core/import_manager/ports.py`

Nombre del Módulo: `import_manager.ports` Descripción: Contratos mínimos para lectores de BOM (interfaz común entre adaptadores y tests).

 **Clase** `IBOMImporter`

Lector de fichero que devuelve la raíz del árbol de materiales como `BOMNodeDTO`.

Métodos Principales:

- `parse_file`: Lee la ruta indicada y devuelve el nodo raíz del BOM.
-

 `core/import_manager/adapters/__init__.py`

Nombre del Módulo: `import_manager.adapters` Descripción: Adaptadores que leen ficheros A3RP (Excel, CSV) y producen un `BOMNodeDTO`.

 `core/import_manager/adapters/a3rp_csv_adapter.py`

Nombre del Módulo: a3rp_csv_adapter Descripción: Lee exportaciones CSV de A3RP (con cabecera Nivel/Componente) y arma el mismo árbol `BOMNodeDTO` que el adaptador Excel, para entornos sin .xlsx.

 `core/import_manager/adapters/a3rp_excel_adapter.py`

Nombre del Módulo: `a3rp_excel_adapter` Descripción: Lee hojas Excel (.xlsx) exportadas por A3RP y construye el árbol BOM en memoria.

Implementa `IBOMImporter` con `pandas`: interpreta columnas como Nivel, Componente y Tipo (Compuesto/Simple) para enlazar padres e hijos antes de la supervisión en UI.

Clase `A3RPExcelAdapter`

Adaptador concreto para leer estructuras de producto desde archivos Excel (.xlsx).

Analiza la estructura jerárquica exportada por A3RP, reconstruyendo el árbol BOM (Bill of Materials) basándose en la columna 'Nivel'.

Atributos de columna esperados: Nivel: Profundidad (0=Raíz). Componente: Código único. Denominación: Descripción. Tipo: 'Compuesto' o 'Simple'. Cantidad: Unidades para el padre.

Métodos Principales:

- `parse_file`: Lee el archivo Excel y devuelve el nodo raíz del árbol BOM. Args: `file_path`: Ruta absoluta al archivo .xlsx. `**kwargs`: Argumentos adicionales (por ejemplo, `sheet_name`). Returns: `BOMNodeDTO`: Estructura jerárquica con el nodo raíz y sus hijos anidados. Raises: `ValueError`: Si no se encuentra un nodo raíz (Nivel 0) o si el formato es inválido. `FileNotFoundError`: Si el archivo no existe.
-

 `core/import_manager/services/__init__.py`

Nombre del Módulo: `import_manager.services` Descripción: Servicios de dominio relacionados con la importación de BOM (p. ej. escritura en BD).

 `core/import_manager/services/bom_import_service.py`

Nombre del Módulo: `bom_import_service` Descripción: Servicio de dominio que persiste un BOM A3RP tras la supervisión en UI. Lee `BOMNodeDTO` con `import_selected` e `import_role`, exige un único producto final, actualiza el registro en `productos` y crea/vincula subfabricaciones, procesos mecánicos y materiales al código final.

Clase `BOMImportService`

Orquesta la escritura en BD a partir del árbol supervisado en `BOMImportPreviewDialog`.

No crea productos de catálogo para subfabricaciones: estas van a la tabla `subfabricaciones` del producto final. El tiempo óptimo del producto se incrementa con la suma de tiempos de procesos mecánicos realmente nuevos respecto al estado previo.

Métodos Principales:

- `__init__`: Args: `product_service`: Fachada/servicio con CRUD de productos y, si aplica, `add_material / link_material_to_product / get_materials_for_product`.
 - `import_bom_tree`: Importa únicamente nodos con `import_selected=True` y `import_role` definido. Requiere exactamente un nodo `BOMImportRole.FINAL_PRODUCT` entre los seleccionados. Args: `root_node`: Raíz del árbol (se recorre en profundidad). Returns: Diccionario de contadores: `creados`, `actualizados`, `errores`, `subfabricaciones_vinculadas`, `procesos_mecanicos`, `componentes`.
 - `_collect_selected`: Recorre el árbol y devuelve nodos marcados con código y rol válidos (sin duplicar por `id`). Args: `node`: Nodo raíz desde el que iniciar el DFS. Returns: Lista plana de nodos elegibles para importación.
 - `_ensure_main_product`: Garantiza que exista la fila de producto final antes de fusionar detalles. Args: `node`: Nodo producto final. `has_subfabs`: Si hay subfabricaciones seleccionadas (afecta departamento y flag). `stats`: Contadores mutables (creación vs error).
 - `_sub_row_from_node`: Construye el dict de subfabricación esperado por `update_product` desde un nodo.
 - `_proceso_dict_from_node`: Mapea un nodo de proceso mecánico al payload de proceso en producto.
 - `_merge_subfabricaciones`: Une subfabricaciones ya persistidas con filas nuevas sin duplicar por código lógico.
 - `_merge_procesos`: Concatena procesos existentes y propuestos, evitando duplicados por nombre (casefold).
 - `_link_component`: Crea material si hace falta y lo enlaza al producto final vía `link_material_to_product`. Args: `final_code`: Código del producto final. `node`: Nodo componente seleccionado. `stats`: Contador `componentes / errores`.
-

 `core/interfaces/controller_interface.py`

Nombre del Módulo: `core.interfaces.controller_interface`

Descripción: Define protocolos o tipos principales: `QABCMeta`, `IController`. Interface base para todos los controladores de la aplicación. Integración típica con: `PyQt6`.

Clase `IController`

Interface base para todos los controladores de la aplicación. Hereda de `QObject` para permitir el uso de señales y slots. Establece un contrato estándar para inicialización, limpieza y manejo de errores.

Métodos Principales:

- `initialize`: Configura los recursos necesarios, conecta señales y prepara el controlador para su uso. Debe ser llamado explícitamente después de la instanciación si es necesario, o como parte del `init` si no hay dependencias circulares.
 - `cleanup`: Libera recursos, desconecta señales y realiza tareas de limpieza antes de destruir el controlador. Critical para prevenir memory leaks en aplicaciones PyQt.
 - `handle_error`: Manejo estándar de errores. Puede ser sobrescrito por subclases. Args: `error`: La excepción capturada. `context`: Descripción opcional del contexto donde ocurrió el error.
-

 `core/interfaces/view_interface.py`

Nombre del Módulo: `core.interfaces.view_interface`

Descripción: Define protocolos o tipos principales: `IView`. Interfaz abstracta para la vista principal.

Clase `IView`

Interfaz abstracta para la vista principal. Define los métodos que el controlador puede invocar sin depender de la implementación concreta de PyQt.

Métodos Principales:

- `show_message`: Muestra un mensaje al usuario.
 - `show_confirmation_dialog`: Muestra un diálogo de confirmación.
 - `switch_page`: Cambia la página visible.
 - `get_page`: Obtiene una página (widget) específica por nombre.
 - `get_products_tab`: Retorna el widget de gestión de productos.
 - `get_fabrications_tab`: Retorna el widget de gestión de fabricaciones.
 - `pages`: Diccionario de páginas registradas.
-

 `core/interfaces/worker_view_interface.py`

Nombre del Módulo: `core.interfaces.worker_view_interface`

Descripción: Define protocolos o tipos principales: `IWorkerView`. Interfaz abstracta para la vista del trabajador. Integración típica con: `core`.

Clase `IWorkerView`

Interfaz abstracta para la vista del trabajador.

Métodos Principales:

- `update_tasks_list`: Actualiza la lista de tareas asignadas.
 - `update_task_state`: Actualiza el estado visual de la tarea actual.
 - `show_message`: Muestra un mensaje al trabajador.
 - `show_confirmation_dialog`: Muestra un diálogo de confirmación.
 - `enable_action_buttons`: Habilita o deshabilita los botones de acción.
-

 `core/label_manager/__init__.py`

Nombre del Módulo: `core.label_manager`

Descripción: Concentra datos de configuración o catálogos estáticos: `__all__`, consumidos por la UI y controladores. Integración típica con: `pathlib`, `base`, `manager`.

 **core/label_manager/base.py**

Nombre del Módulo: core.label_manager.base

Descripción: Concentra datos de configuración o catálogos estáticos: LABEL_FORMATS, consumidos por la UI y controladores. Integración típica con: pathlib.

 `core/label_manager/manager.py`

Nombre del Módulo: `core.label_manager.manager`

Descripción: Paquete: `core.label_manager` Descripción: Sistema de gestión y generación de etiquetas de trazabilidad.

Clase `LabelManager`

Gestor central de plantillas y generación de documentos de etiquetas.

Esta clase actúa como fachada para la generación de documentos, coordinando la búsqueda de plantillas, el recuento de placeholders y la invocación de los generadores adecuados (físicos o dinámicos).

Attributes: `LABEL_FORMATS` (dict): Diccionario de formatos de etiquetas soportados.

Métodos Principales:

- `__init__`: Inicializa el gestor de etiquetas. Args: `templates_dir`: Directorio raíz donde residen las plantillas .docx. `qr_generator`: Instancia del generador de códigos QR únicos. `doc_generator`: Adaptador por defecto para generación de documentos.
 - `_get_generator_and_path`: Determina el generador y la ruta de la plantilla (física o virtual). Soporta 'plantillas virtuales' como `apli_1861_qr.docx` que no requieren un archivo físico en el disco y usan generadores especializados de bajo nivel. Args: `plantilla`: Nombre del archivo de plantilla o identificador virtual. `formato`: Formato de la hoja (A5, A4, etc.). Returns: Tupla (Generador, Ruta/DummyPath).
 - `count_qr_placeholders`: Cuenta los espacios disponibles para QRs en la plantilla. Si la plantilla es virtual, delega en el generador especializado. Si es física, escanea el documento Word buscando el placeholder `{{qr}}`. Args: `plantilla`: Nombre de la plantilla. `formato`: Formato de la hoja. Returns: Número total de huecos para códigos QR.
 - `generate_labels`: Genera el documento de etiquetas.
-

 `core/label_manager/ports.py`

Nombre del Módulo: `core.label_manager.ports`

Descripción: Define protocolos o tipos principales: `IDocumentGenerator`. Protocolo que define cómo debe comportarse un generador de documentos. Integración típica con: `pathlib`.

Clase `IDocumentGenerator`

Protocolo que define cómo debe comportarse un generador de documentos sin acoplarse a librerías específicas (como `python-docx`).

Métodos Principales:

- `count_qr_placeholders`: Cuenta los placeholders `{{qr}}` en el documento indicado.
 - `generate_labels`: Genera un documento reemplazando placeholders por los datos provistos.
 - `create_sample`: Crea un documento de ejemplo para un formato especificado ('A4', 'A5', etc.).
-

`core/label_manager/printer.py`

Nombre del Módulo: printer Descripción: Envío a impresora y copia de respaldo de documentos de etiquetas (macOS/Linux con CUPS `lp/lpr`, Windows).

Si no hay impresora predeterminada o el envío falla, copia el fichero a `Documentos/Etiquetas` y abre esa carpeta para que el usuario pueda imprimir manualmente.

-  `_lpstat_says_no_default`: Detecta ausencia de impresora predeterminada (inglés / español en macOS).
 -  `is_printer_available`: Comprueba si hay una impresora predeterminada configurada.
 -  `save_to_documents`: Guarda el documento en la carpeta de Documentos del usuario.
 -  `open_file_location`: Abre la ubicación del archivo en el explorador de archivos.
 -  `print_document`: Envía `doc_path` a la impresora predeterminada o a la indicada. Si no hay cola predeterminada o `lp/lpr` fallan, intenta copiar el archivo a `Documentos/Etiquetas` y devuelve `(True, ruta_copiada)`. Returns: Tupla (éxito, `ruta_opcional`): en impresión directa exitosa la ruta suele ser `None`; si solo se guardó copia, la ruta del archivo guardado.
-

 `core/protocols/__init__.py`

Nombre del Módulo: `core.protocols`

Descripción: Protocolos de dominio compartidos (servicios). Implementados nominalmente en `core.services`.

 **core/protocols/domain.py**

Nombre del Módulo: core.protocols.domain

Descripción: Define protocolos o tipos principales: `IProductService`, `IFabricacionService`, `IMaterialService`. Contrato del servicio de catálogo, iteraciones e imágenes de producto. Integración típica con: `__future__`, `core`.

Clase `IProductService`

Contrato del servicio de catálogo, iteraciones e imágenes de producto.

Clase `IFabricacionService`

Contrato del servicio de fabricaciones y preprocesos.

Clase `IMaterialService`

Subconjunto de operaciones de materiales (vía `ProductService`), incluida la lectura por producto.

 **core/qr_scanner/__init__.py**

Nombre del Módulo: core.qr_scanner

Descripción: Concentra datos de configuración o catálogos estáticos: __all__, consumidos por la UI y controladores. Integración típica con: scanner, detector, base.

 `core/qr_scanner/base.py`

Nombre del Módulo: `core.qr_scanner.base`

Descripción: Concentra datos de configuración o catálogos estáticos: `logger`, consumidos por la UI y controladores. Integración típica con: `re`, `datetime`.

-  `validate_qr`: Valida que un QR tenga el formato correcto de trazabilidad.
 -  `get_qr_info`: Obtiene información de un QR de trazabilidad.
-

 `core/qr_scanner/detector.py`

Nombre del Módulo: `qr_scanner.detector`

Descripción: Funciones y datos de apoyo del paquete; conviene enlazar qué controlador o servicio las consume y qué estructuras devuelven (ver firmas al inicio del archivo). Integración típica con: `os`.

 `core/qr_scanner/scanner.py`

Paquete: `core.qr_scanner` Nombre del Módulo: `scanner.py` (QrScanner) Descripción: Sistema de detección y decodificación de códigos QR en tiempo real. Implementa la lógica de escaneo mediante visión artificial para la lectura de etiquetas de trazabilidad.

 `core/qr_scanner/ui.py`

Nombre del Módulo: `core.qr_scanner.ui`

Descripción: Dibujo de overlays QR sobre frames OpenCV (`core.qr_scanner`, no el paquete de interfaz `ui/`).

-  `draw_qr_detection`: Dibuja indicadores visuales en el frame.
-

`core/security/access_control.py`

Nombre del Módulo: `access_control.py` Descripción: Proporciona decoradores y utilidades para el control de acceso basado en funciones (RBAC) en toda la aplicación.

-  `set_security_service`: Inicializa la instancia global del servicio de seguridad.
 -  `get_security_service`: Obtiene la instancia global del servicio de seguridad.
 -  `require_permission`: Decorador para restringir el acceso basado en permisos. Si el usuario no tiene el permiso, la función no se ejecuta.
 -  `require_role`: Decorador para restringir el acceso a un rol específico.
-

 `core/security/password_service.py`

Nombre del Módulo: `core.security.password_service`

Descripción: Define protocolos o tipos principales: `PasswordService`. Servicio para el manejo seguro de contraseñas utilizando `bcrypt`. Integración típica con: `bcrypt`.

Clase `PasswordService`

Servicio para el manejo seguro de contraseñas utilizando `bcrypt`. Reemplaza el uso de SHA-256 simple.

Métodos Principales:

- `hash_password`: Genera un hash seguro para la contraseña proporcionada usando `bcrypt`. Args: `plain_password`: La contraseña en texto plano. Returns: El hash de la contraseña como string (utf-8).
 - `verify_password`: Verifica si la contraseña coincide con el hash almacenado. Args: `plain_password`: La contraseña en texto plano a verificar. `hashed_password`: El hash almacenado en la base de datos. Returns: True si coinciden, False en caso contrario.
 - `validate_password`: Valida que la contraseña cumpla con los requisitos de complejidad. Requisitos:
 - Mínimo 8 caracteres
 - Al menos un número
 - Al menos una letraReturns: `Tuple[bool, str]`: (Es válida, Mensaje de error si no lo es)
-

 `core/security/security_exceptions.py`

Nombre del Módulo: `core.security.security_exceptions`

Descripción: Define protocolos o tipos principales: `SecurityError`,
`SecurityServiceNotInitializedError`, `InsufficientPermissionsError`, `RateLimitExceededError`.
Excepción base para errores de seguridad.

Clase `SecurityError`

Excepción base para errores de seguridad.

Clase `SecurityServiceNotInitializedError`

El servicio de seguridad no está inicializado.

Clase `InsufficientPermissionsError`

El usuario no tiene los permisos necesarios.

Clase `RateLimitExceededError`

Se excedió el límite de intentos permitidos.

 `core/security/security_service.py`

Nombre del Módulo: `security_service.py` Descripción: Servicio central de seguridad para la gestión de roles (RBAC), autenticación de usuarios y verificación de permisos.

Clase `SecurityService`

Servicio central de seguridad para autenticación y autorización (RBAC).

Métodos Principales:

- `login_user`: Registra al usuario actual en el servicio de seguridad.
 - `logout`: Cierra la sesión del usuario actual.
 - `get_current_role`: Obtiene el rol del usuario actual como Enum.
 - `has_permission`: Verifica si el usuario actual tiene un permiso específico.
 - `check_access`: Alias de `has_permission` para uso en decoradores.
-

 `core/services/audit_logger.py`

Nombre del Módulo: `audit_logger` Descripción: Persistencia de acciones sensibles (login, importación, sincronización, etc.) en la tabla de auditoría de base de datos para trazabilidad y cumplimiento.

Clase `AuditLogger`

Registra acciones sensibles en la base de datos.

Métodos Principales:

- `log`: Registra una acción en el log de auditoría. Args: `username`: Nombre del usuario que realiza la acción `action`: Tipo de acción (LOGIN, DELETE, EXPORT, etc.) `entity_type`: Tipo de entidad afectada (opcional) `entity_id`: ID de la entidad afectada (opcional) `description`: Descripción adicional (opcional) `user_id`: ID del usuario en la BD (opcional) `success`: Si la acción fue exitosa `error_message`: Mensaje de error si falló `ip_address`: Dirección IP (opcional)
 - `log_login`: Registra un intento de login.
 - `log_logout`: Registra un logout.
 - `log_delete`: Registra eliminación de entidad.
 - `log_export`: Registra exportación de datos.
 - `log_import`: Registra importación de datos.
 - `log_settings_change`: Registra cambio de configuración.
 - `cleanup_old_logs`: Elimina registros de auditoría más antiguos que el periodo de retención. Args: `retention_days`: Número de días a retener los logs. Returns: Número de registros eliminados.
-

 `core/services/backup_service.py`

Nombre del Módulo: `backup_service` Descripción: Copias de seguridad automáticas del directorio de datos (empaquetado, rotación y comprobaciones).

Incluye retención por días y verificación de integridad de los archivos generados.

Clase BackupService

Gestiona backups automatizados con rotación y verificación.

Métodos Principales:

- `__init__`: Inicializa el servicio de backup. Args: `data_dir`: Directorio a respaldar (ej: 'data/')
`backup_dir`: Directorio donde guardar backups (default: `data/backups/`)
 - `create_backup`: Crea un backup comprimido del directorio de datos. Returns: `Tuple[bool, str]`: (éxito, mensaje/path)
 - `cleanup_old_backups`: Elimina backups antiguos según política de retención. Returns: `int`: Número de backups eliminados
 - `list_available_backups`: Lista todos los backups disponibles ordenados por fecha (más reciente primero). Returns: `List[BackupInfoDTO]`: Lista de backups con metadata
 - `restore_backup`: Restaura un backup a un directorio temporal para revisión. Args: `backup_name`: Nombre del archivo de backup `target_dir`: Directorio destino (default: `data/restore_staging/`) Returns: `Tuple[bool, str]`: (éxito, mensaje/path)
-

 **core/services/backup_utils.py**

Nombre del Módulo: backup_utils

Descripción: Funciones puras de apoyo (sin estado de proceso): `check_disk_space`, `verify_tar_backup`, `create_checksum`, `verify_checksum`. Integración típica con: `__future__`, `hashlib`, `shutil`, `tarfile`, `pathlib`.

-  `check_disk_space`: Verifica que exista espacio libre suficiente en el disco.
 -  `verify_tar_backup`: Verifica que el archivo de backup se pueda abrir y tenga contenido.
 -  `create_checksum`: Crea archivo SHA256 para el backup.
 -  `verify_checksum`: Valida checksum de backup si existe; si no existe, lo considera válido.
-

 `core/services/calculation_audit.py`

Nombre del Módulo: `calculation_audit` Descripción: Tipos para auditar decisiones del motor de cálculo (estado visual y motivo legible).

`CalculationDecision` enriquece cada paso con contexto de producto/tarea para informes Excel/PDF y depuración sin leer logs crudos.

 **Clase** `DecisionStatus`

Define los estados visuales para las decisiones en la interfaz.

 **Clase** `CalculationDecision`

Representa una decisión o evento único dentro del motor de cálculo, enriquecida con contexto para una mejor experiencia de usuario (UX).

`core/services/calendar_helper.py`

Nombre del Módulo: `calendar_helper` Descripción: Acceso global opcional a la configuración de calendario y jornada (compatibilidad).

Algunos módulos históricos leen el horario vía `get_schedule_config` tras un `set_schedule_config` en el arranque de la aplicación.

-  `set_schedule_config`: Establece la configuración de horario global para compatibilidad.
 -  `get_schedule_config`: Obtiene la configuración de horario global.
-

 `core/services/data_importer.py`

Nombre del Módulo: `data_importer` Descripción: Importación de materiales desde fichero (patrón factoría: interfaz común, implementación Excel con `pandas`). Devuelve objetos `Material` listos para validación o persistencia según el flujo que invoque el módulo.

Clase `Material`

Representa un material o componente importado.

Clase `IMaterialImporter`

Interfaz abstracta para los importadores de materiales. Define el contrato que todas las implementaciones deben seguir.

Métodos Principales:

- `import_materials`: Lee un archivo y retorna una lista de objetos `Material`.

Clase `ExcelMaterialImporter`

Importador concreto para archivos Excel (.xlsx). Utiliza la librería `pandas` para un manejo eficiente de los datos.

Métodos Principales:

- `import_materials`: Lee los materiales desde un archivo Excel. Se espera que el archivo contenga las columnas 'codigo' y 'descripcion'.

Clase `MaterialImporterFactory`

Clase factoría que decide qué importador crear basado en la extensión del archivo.

Métodos Principales:

- `create_importer`: Retorna una instancia del importador apropiado para la extensión dada.
-

 `core/services/fabricacion_service.py`

Nombre del Módulo: `fabricacion_service` Descripción: Órdenes de fabricación, preprocesos vinculados y datos para seguimiento.

Orquesta validaciones y persistencia usando DTOs (`FabricacionDTO`, `PreprocesoDTO`) como frontera estable hacia controladores y vistas.

Clase `FabricacionService`

Gestión centralizada de fabricaciones y preprocesos asociados.

Valida reglas de negocio antes de persistir, coordina creaciones con varios productos o preprocesos y expone solo DTOs hacia la capa de interfaz.

Métodos Principales:

- `__init__`: Inicializa el servicio de fabricación con su gestor de base de datos. Args: `db_manager`: Instancia central de gestión de persistencia.
 - `preproceso_repo`: Acceso directo al repositorio de preprocesos.
 - `tracking_repo`: Acceso directo al repositorio de seguimiento.
 - `get_latest_fabricaciones`: Obtiene las fabricaciones más recientes. Args: `limit`: Número máximo de fabricaciones a retornar. Returns: Lista de objetos de fabricación.
 - `search_fabricaciones`: Busca fabricaciones por código o descripción. Args: `query`: Texto de búsqueda. Returns: Lista de fabricaciones coincidentes.
 - `create_fabricacion_with_preprocesos`: Orquesta la creación de una fabricación con sus preprocesos asociados. Recibe un `FabricacionDTO` completo y delega la persistencia al repositorio. Este método es el punto de entrada principal para flujos que requieren integridad referencial entre la cabecera de fabricación y su checklist técnica.
 - `get_preprocesos_by_fabricacion`: Obtiene la lista de preprocesos asociados a una fabricación. Args: `fabricacion_id`: ID único de la fabricación. Returns: Lista de preprocesos.
 - `get_all_ordenes_fabricacion`: Obtiene la lista de todas las órdenes de fabricación.
-

 `core/services/flow_builder_service.py`

Nombre del Módulo: `flow_builder_service` Descripción: Construcción y ajuste de listas de pasos de flujo de producción (dict en memoria).

Aplica unidades de disparo, copia profunda de overrides del editor visual y asignación heurística de trabajadores por nivel de habilidad requerido en la tarea.

Clase `FlowBuilderService`

Construye y refina flujos de producción antes de simulación o persistencia.

Métodos Principales:

- `build_flow_from_override`: Clona el flujo definido por el usuario (p. ej. editor) y fija `trigger_units` en cada paso. Args: `production_flow_override`: Secuencia de pasos con tareas y metadatos. `units`: Unidades de disparo a aplicar a cada paso. Returns: Lista nueva de pasos o lista vacía si no hay override.
 - `resolve_worker_assignments`: Rellena `workers` en pasos vacíos eligiendo el primer operario que cumple el nivel requerido. Args: `production_flow`: Pasos del flujo; se mutan entradas sin trabajadores asignados. `available_workers_sorted`: Candidatos ordenados (p. ej. de mayor a menor habilidad). Returns: El mismo flujo con listas `workers` rellenas o vacías si no hay candidato.
-

 `core/services/flow_simulation_service.py`

Nombre del Módulo: `flow_simulation_service` Descripción: Simulación paso a paso de flujos de producción y orden de ejecución de tareas.

Incluye `SimulationSession` para avanzar tarea a tarea y `FlowSimulationService` para preparar el orden y datos auxiliares del motor de simulación.

Clase `SimulationSession`

Gestiona el estado de una sesión de simulación paso a paso.

Métodos Principales:

- `next_step`: Avanza al siguiente paso de la simulación. Returns: int or None: El índice de la tarea actual, -1 para indicador visual, o None si la simulación ha terminado.

Clase `FlowSimulationService`

Servicio encargado de la lógica de simulación y cálculo del orden de ejecución para flujos de producción.

Métodos Principales:

- `calculate_preview_order`: Calcula el orden de ejecución teórico basándose en: 1. Tareas de inicio de ciclo o sin dependencias. 2. Dependencias directas. 3. Saltos cíclicos (se indican pero no se siguen recursivamente en preview). Args: `canvas_tasks` (list): Lista de diccionarios con la configuración de las tareas (formato del canvas/diálogo). Returns: list: Lista de índices en el orden de ejecución teórico. Puede incluir -1 para indicar un salto cíclico visual.
 - `start_simulation`: Inicia una sesión de simulación. Args: `canvas_tasks` (list): Lista de tareas del canvas. Returns: `SimulationSession`: Objeto de sesión para controlar la simulación.
 - `identify_last_tasks_in_cycles`: Identifica las últimas tareas de cada cadena de ciclo. Una tarea es "última" si tiene `next_cyclic_task_index` pero ninguna otra tarea tiene a esta como `next_cyclic_task_index` (es decir, nadie apunta a ella en el ciclo). Args: `canvas_tasks` (list): Lista de tareas del canvas. Returns: set: Conjunto de índices de tareas que son el último paso de un ciclo.
-

 `core/services/machine_service.py`

Nombre del Módulo: `machine_service` Descripción: Servicio de dominio especializado en la gestión de máquinas, mantenimientos y procesos.

Clase `MachineService`

Catálogo y mantenimiento lógico de máquinas de planta (consultas y cambios vía repositorio).

Métodos Principales:

- `get_all_machines`: Obtiene todas las máquinas.
 - `get_latest_machines`: Obtiene las últimas máquinas añadidas.
 - `get_machines_by_process_type`: Obtiene máquinas filtradas por tipo de proceso.
 - `add_machine`: Añade una nueva máquina.
 - `update_machine`: Actualiza la información de una máquina.
 - `delete_machine`: Elimina una máquina.
 - `get_machine_history`: Obtiene el historial de una máquina.
 - `add_machine_maintenance`: Añade un registro de mantenimiento a una máquina.
 - `get_distinct_machine_processes`: Obtiene la lista de procesos únicos definidos en las máquinas.
 - `get_machine_usage_stats`: Obtiene las estadísticas de uso de las máquinas.
-

 `core/services/maintenance_service.py`

Nombre del Módulo: `maintenance_service` Descripción: Mantenimiento programado en segundo plano: limpieza de intentos de login, retención de auditoría, copias de seguridad y rotación de ficheros antiguos.

`MaintenanceService` encola un `MaintenanceWorker` en el `QThreadPool` global.

Clase `MaintenanceWorker`

Worker para ejecutar tareas de mantenimiento en segundo plano.

Clase `MaintenanceService`

Servicio que orquesta tareas de mantenimiento:

- Limpieza de intentos de login antiguos.
- Limpieza de logs de auditoría antiguos.
- Creación de backups automatizados.
- Rotación de backups antiguos.

Métodos Principales:

- `run_background_maintenance`: Inicia el mantenimiento en un hilo separado.
 - `perform_maintenance`: Ejecuta las tareas de mantenimiento secuencialmente.
-

 `core/services/pila_service.py`

Nombre del Módulo: `pila_service` Descripción: Pilas de trabajo, simulación y ensamblado de DTOs para el motor de cálculo.

Delega persistencia en `DatabaseManager` y `PilaRepository`; emite señales cuando cambian pilas o terminan simulaciones.

Clase `PilaService`

Servicio de dominio para gestionar Pilas de fabricación y Simulaciones.

Métodos Principales:

- `add_diario_evento`: Alias de compatibilidad para el nombre histórico del método. La UI usa `add_diario_evento`; el método canónico del servicio es `add_diario_entry`.
 - `get_data_for_calculation`: Obtiene datos de un producto estructurados en DTOs para el motor de cálculo.
 - `get_data_for_calculation_from_session`: Aplana la sesión de planificación en una lista de `CalculationProductDTO` listos para calcular. Acepta por elemento: un `CalculationProductDTO` (se incluye tal cual); un `CalculationStepDTO` (se resuelve plantilla o pila directa embebida); o un dict compatible con el formato histórico de paso de lote. Para plantilla de lote usa `lote_repo.get_lote_details`; enriquece cada DTO con `deadline`, `fabricacion_id`, `units_for_this_instance` y, en kits, `cantidad_en_kit`.
-

 `core/services/preparation_service.py`

Nombre del Módulo: `preparation_service` Descripción: Servicio de dominio especializado en la gestión de grupos y pasos de preparación de máquinas.

Clase `PreparationService`

Grupos y pasos de preparación de máquinas (tiempos y secuencias usados antes de fabricar).

Métodos Principales:

- `get_groups_for_machine`: Obtiene los grupos de preparación asociados a una máquina.
 - `get_prep_info_for_product`: IDs de grupo y máquina del primer grupo de preparación asociado al producto.
 - `add_prep_group`: Añade un nuevo grupo de preparación a una máquina.
 - `update_prep_group`: Actualiza un grupo de preparación existente.
 - `delete_prep_group`: Elimina un grupo de preparación.
 - `get_steps_for_group`: Obtiene los pasos de preparación de un grupo.
 - `add_prep_step`: Añade un paso de preparación a un grupo.
 - `update_prep_step`: Actualiza un paso de preparación existente.
 - `delete_prep_step`: Elimina un paso de preparación.
 - `get_prep_step_details`: Obtiene los detalles de un paso de preparación específico.
 - `get_group_details`: Obtiene los detalles de un grupo de preparación.
 - `get_prep_step_details_by_ids`: Obtiene detalles de múltiples pasos por sus IDs.
-

 `core/services/product_service.py`

Nombre del Módulo: `product_service` Descripción: Catálogo de productos: búsqueda, detalle, iteraciones, materiales y altas o bajas lógicas. Expone señales PyQt6 para que la interfaz refresque listas tras cambios.

Clase `ProductService`

Servicio de dominio para gestionar la lógica relacionada con productos. Maneja:

- Búsqueda y recuperación de detalles de producto.
- Gestión de iteraciones y materiales.
- Operaciones CRUD básicas (delegadas al repositorio).

Métodos Principales:

- `update_product_iteration`: Alias para compatibilidad con protocolos y controladores.
 - `add_product`: Valida y añade un producto usando el repositorio.
-

 `core/services/rate_limiter.py`

Nombre del Módulo: `rate_limiter` Descripción: Limita intentos de inicio de sesión fallidos (bloqueo temporal y limpieza de históricos).

Registra cada intento en base de datos y bloquea el usuario unos minutos tras superar el umbral.

Clase `RateLimiter`

Gestiona el rate limiting para intentos de login.

Métodos Principales:

- `check_and_record_attempt`: Verifica si el usuario puede intentar login y registra el intento. Args: `username`: Nombre de usuario `success`: Si el intento fue exitoso `ip_address`: Dirección IP del intento (opcional) Returns: True si el intento está permitido, False si está bloqueado
 - `is_blocked`: Verifica si un usuario está bloqueado sin registrar un intento. Args: `username`: Nombre de usuario Returns: True si el usuario está bloqueado
 - `cleanup_old_attempts`: Elimina registros antiguos para mantener la tabla limpia.
-

 **core/services/report_service.py**

Nombre del Módulo: report_service Descripción: Informes y estadísticas de producción (órdenes, tiempos, incidencias, evolución).

Interfaz delgada entre controladores de informes y `ReportsRepository`; los DTOs de salida viven en `core.reports_dtos`.

Clase ReportService

Servicio de dominio para gestionar Reportes y Estadísticas. Actúa como interfaz entre la UI/Controladores y el repositorio de reportes.

Métodos Principales:

- `get_product_dashboard`: Obtiene el bundle de dashboard para un producto en un contrato único.
-

 **core/services/report_strategy.py**

Nombre del Módulo: report_strategy Descripción: Punto de entrada estable para estrategias de informe (Excel y PDF).

Reexporta IReporteEstrategia, GeneradorDeInformes y las clases concretas del subpaquete core.services.reporting para imports simples desde controladores.

 `core/services/system_integration_service.py`

Nombre del Módulo: `system_integration_service` Descripción: Operaciones transversales de sistema: lotes, configuración persistente y tracking.

Fachada delgada sobre los repositorios expuestos por `DatabaseManager` para evitar acoplar widgets a la capa de datos.

Clase `SystemIntegrationService`

Fachada delgada sobre repos de lotes, config y tracking.

Métodos Principales:

- `lote_repo`: Acceso puntual al repo (p. ej. APIs aún no envueltas).
-

 `core/services/temporal_storage.py`

Nombre del Módulo: `temporal_storage` Descripción: Registro incremental de eventos de simulación en SQLite (disco o memoria).

`RegistroTemporal` es seguro entre hilos: buffer con volcado periódico y conexión por hilo para no bloquear el motor de eventos al escribir trazas largas.

Clase `RegistroTemporal`

Almacenamiento incremental de eventos de simulación en disco (o `:memory:`) con buffer.

El acceso concurrente se serializa con un candado y conexiones SQLite locales al hilo.

Métodos Principales:

- `__init__`: Prepara el almacenamiento de eventos. La conexión a la BD se creará en el hilo que la necesite.
 - `_get_conn`: Crea y devuelve una conexión a la base de datos específica para el hilo actual. Si ya existe una para este hilo, la reutiliza. Tras `close()`, `conn` queda en `None` y se vuelve a crear al consultar.
 - `_default_serializer`: Serializador JSON para objetos `datetime` y `Enum`.
 - `guardar_evento`: Añade un evento al buffer y lo vuelca a disco si está lleno.
 - `_flush_buffer_to_disk`: Escribe el contenido del buffer en la base de datos SQLite y lo limpia.
 - `close`: Asegura que el buffer se guarde y cierra la conexión del hilo actual.
 - `cleanup`: Cierra la conexión y elimina el archivo de BD del disco (solo si no es `:memory:`).
 - `consultar_eventos`: Lee eventos de la base de datos SQLite.
-

Nombre del Módulo: `time_calculator` Descripción: Cálculo de fechas y duraciones respetando jornada laboral, descansos y festivos.

`CalculadorDeTiempos` avanza por bloques de trabajo continuos (no minuto a minuto) para sumar o restar tiempo de forma eficiente y coherente con la configuración de planta.

Clase `CalculadorDeTiempos`

Clase robusta para realizar cálculos de tiempo respetando los horarios laborales, descansos y días festivos definidos en la configuración.

Matemáticas y Heurísticas (Forward-Iteration): El algoritmo evita calcular minuto a minuto ($O(N)$ ineficiente). En su lugar, itera a través del tiempo segmentándolo en "bloques ininterrumpidos de trabajo" definidos por:

- Inicio de jornada (`WORK_START_TIME`).
- Descansos predefinidos (`parsed_breaks`).
- Fin de jornada (`WORK_END_TIME`).

Para añadir tiempo, el algoritmo salta al inicio del siguiente bloque continuo. Calcula escalarmente el tamaño del bloque frente al saldo de tiempo requerido; si el bloque acomoda el saldo entero, se suma algebraicamente, logrando una altísima precisión $O(1)$ por salto sobre noches, fines de semana y festivos.

Métodos Principales:

- `is_workday`: Verifica si un día es laborable (no es fin de semana ni festivo).
 - `find_next_workday`: Encuentra el siguiente día laborable a partir de una fecha dada.
 - `_move_to_next_valid_work_moment`: Ajusta un datetime al siguiente momento laborable disponible. Salta fines de semana, festivos, horarios no laborales y descansos.
 - `add_work_minutes`: Calcula una nueva fecha proyectando Carga de Trabajo (Workload) hacia el futuro. Matemáticas Puras - Modelo Geométrico de Proyección: Dada una carga de trabajo constante `req_mins` y un calendario de bloques de tiempo `B_i`: $T_{actual} = \{\max(\text{start_datetime}, S_{0_start})\}$
 $\text{while } req_mins > 0: S_i = \{\text{bloque válido horario actual}\} dT_i = \text{abs}(S_{i_end} - T_{actual})$ if $dT_i \geq req_mins$: $\text{return } T_{actual} + req_mins$ ($O(1)$ Aritmética) else: $req_mins = req_mins - dT_i$ $T_{actual} = \{\text{saltar_noche_y_descanso_hacia}(S_{i+1_start})\}$ Esta heurística evita calcular `for minute in req_mins: check_is_workday()`. Salta segmentos masivos reduciendo la complejidad de $O(\text{Minutos})$ a $O(\text{Bloques})$.
 - `count_workdays`: Cuenta los días laborables entre dos fechas.
 - `calculate_work_minutes_between`: Calcula los minutos REALES de trabajo (W_{total}) entre dos instantes de tiempo espaciados (t_0, t_1). Cálculo Integral Analítico - Segmentación Temporal: El lapso total $\Delta(t) = t_1 - t_0$ se intersecta con la función bool `is_working(T)`. Para evitar iteración continua, se aplica el Teorema de Superposición de Intervalos: $B = \text{Conjunto de bloques ininterrumpidos laborales en el calendario}$ $W_{total} = \text{Sumatoria}(\text{para cada } i \text{ en } B) \text{ de } [\min(t_1, B_{i_end}) - \max(t_0, B_{i_start})]$ La sumatoria ignora matemáticamente aquellos subconjuntos donde $\max > \min$ (es decir, saltos completos de domingos o madrugadas sin colisión métrica), permitiendo resolver lapsos de meses en milisegundos $O(\text{días_afectados})$. Args: `start_datetime`: Fecha y hora de inicio (t_0). `end_datetime`: Fecha y hora de fin (t_1). Returns: float: Sumatoria escalar (W_{total}) de los minutos de intersección válidos.
-

 `core/services/tracking_assignment_service.py`

Nombre del Módulo: `tracking_assignment_service` Descripción: Enlaza y desenlaza trabajadores con órdenes de fabricación en base de datos.

Escribe en la tabla `trabajador_fabricacion_link` (fecha de asignación en UTC y estado, por ejemplo `activo` o `cancelado`) para que la vista de operario y los informes de trazabilidad reflejen siempre las asignaciones vigentes.

Clase `TrackingAssignmentService`

Operaciones de dominio sobre el vínculo trabajador–fabricación (asignar, actualizar estado, desasignar).

Métodos Principales:

- `get_fabricaciones_por_trabajador`: Recupera las fabricaciones asignadas a un trabajador desde el repositorio. Args: `trabajador_id`: ID del trabajador. Returns: Lista de fabricaciones asignadas.
 - `asignar_trabajador_a_fabricacion`: Enlaza un trabajador con una fabricación si ambos existen y el enlace no existe aún. Inserta una fila en `trabajador_fabricacion_link` con `fecha_asignacion` en UTC y estado `activo`. Si el par ya estaba enlazado, devuelve `True` sin duplicar.
 - `desasignar_trabajador_de_fabricacion`: Elimina la fila de `trabajador_fabricacion_link` para el par dado. Returns: `True` si se borró al menos una fila; `False` si no existía el enlace.
-

 `core/services/worker_service.py`

Nombre del Módulo: `worker_service` Descripción: Servicio de dominio especializado en la gestión de trabajadores, historial y carga de trabajo.

Clase `WorkerService`

Operaciones de dominio sobre trabajadores: listados, detalle, historial y asignación de tareas.

Métodos Principales:

- `get_all_workers`: Obtiene todos los trabajadores.
 - `get_latest_workers`: Obtiene los últimos trabajadores añadidos.
 - `get_worker_details`: Obtiene detalles de un trabajador por ID.
 - `add_worker`: Añade un nuevo trabajador.
 - `update_worker`: Actualiza la información de un trabajador.
 - `delete_worker`: Elimina un trabajador.
 - `assign_task_to_worker`: Crea una nueva 'Fabricación' (Fase 12C) y la asigna a un trabajador para su seguimiento. Este método simplifica el flujo para tareas directas, encapsulando: 1. Generación de un código único basado en el nombre del trabajador y timestamp. 2. Creación de la cabecera de fabricación normalizada mediante `FabricacionDTO`. 3. Asociación del producto requerido a través del repositorio. 4. Registro de la asignación en el `TrackingAssignmentService`.
 - `get_worker_history`: Obtiene el historial de fabricaciones y anotaciones de un trabajador.
 - `get_worker_activity_log`: Obtiene el log de actividad detallado de un trabajador.
 - `actualizar_estado_asignacion`: Actualiza el estado de una fabricación asignada a un trabajador (seguimiento).
 - `get_worker_load_stats`: Calcula la carga de trabajo (duración total de tareas) por trabajador basándose en los resultados de simulación de todas las pilas.
 - `authenticate_user`: Autentica a un usuario.
 - `update_user_password`: Actualiza la contraseña de un usuario.
-

 **core/services/report_sheets/__init__.py**

Nombre del Módulo: report_sheets Descripción: Hojas Excel reutilizables del informe de pila (resumen, cronograma, gráficas, etc.).

Cada módulo define una subclase de `ExcelSheetStrategy` que añade una pestaña al libro.

 **core/services/report_sheets/audit.py**

Nombre del Módulo: report_sheets.audit

Descripción: Define protocolos o tipos principales: AuditSheet. Integración típica con: datetime, openpyxl, base, core.

 `core/services/report_sheets/base.py`

Nombre del Módulo: `report_sheets.base` Descripción: Clase base abstracta para cada hoja Excel del informe de pila de fabricación.

 **Clase** `ExcelSheetStrategy`

Métodos Principales:

- `create_sheet`: Añade una hoja al libro `wb` según el análisis recibido en `kwargs`.
-

 **core/services/report_sheets/cronograma.py**

Nombre del Módulo: report_sheets.cronograma

Descripción: Define protocolos o tipos principales: CronogramaSheet. Integración típica con: datetime, openpyxl, base.

 **core/services/report_sheets/cuellos_botella.py**

Nombre del Módulo: report_sheets.cuellos_botella

Descripción: Define protocolos o tipos principales: CuellosBotollaSheet. Integración típica con: re, datetime, openpyxl, base, core.

 `core/services/report_sheets/graficas.py`

Nombre del Módulo: report_sheets.graficas Descripción: Hoja Excel de gráficos (barras, circular) a partir de resultados de simulación.

 **Clase GraficasSheet**

Genera la hoja de gráficos y visualizaciones para el reporte Excel.

 **core/services/report_sheets/resumen.py**

Nombre del Módulo: report_sheets.resumen

Descripción: Define protocolos o tipos principales: ResumenEjecutivoSheet. Integración típica con: datetime, openpyxl, base.

 **core/services/report_sheets/trabajadores.py**

Nombre del Módulo: report_sheets.trabajadores

Descripción: Define protocolos o tipos principales: AnalisisTrabajadoresSheet. Integración típica con: datetime, openpyxl, base.

 `core/services/report_sheets/trabajo_paralelo.py`

Nombre del Módulo: `report_sheets.trabajo_paralelo`

Descripción: Define protocolos o tipos principales: `TrabajoParaleloSheet`. Genera la hoja de análisis de trabajo paralelo por instancia para el reporte Excel. Integración típica con: `datetime`, `openpyxl`, `base`.

 **Clase** `TrabajoParaleloSheet`

Genera la hoja de análisis de trabajo paralelo por instancia para el reporte Excel.

 **core/services/reporting/__init__.py**

Nombre del Módulo: reporting Descripción: Subpaquete de generación de informes (estrategias Excel/PDF y piezas compartidas).

Ver `report_strategy` en el nivel superior del paquete `core.services` para imports cómodos.

 `core/services/reporting/base.py`

Nombre del Módulo: `reporting.base` Descripción: Contrato común (`IReporteEstrategia`) y contexto `GeneradorDeInformes` para exportar informes.

Desacopla la recogida de datos del formato de salida (Excel, PDF, etc.) para que la UI no dependa de librerías de ofimática concretas.

 **Clase `IReporteEstrategia`**

Métodos Principales:

- `generar_reporte`: Genera el informe y lo deja en `output_path` (o delega en submétodos según la estrategia).
-

 `core/services/reporting/excel_report_strategy.py`

Nombre del Módulo: `excel_report_strategy` Descripción: Generación de informes Excel de pilas de fabricación (varias hojas: resumen, cronograma, trabajadores, auditoría, gráficas, etc.).

Implementa `IReporteEstrategia` usando `openpyxl` y las clases de `report_sheets`.

Clase `ReportePilaFabricacionExcelMejorado`

Generador mejorado de reportes Excel con lectura correcta del `audit_log` y presentación clara de grupos secuenciales.

Métodos Principales:

- `generar_reporte`: Orquesta la creación de todas las hojas del informe en memoria.
 - `_crear_hoja_grupos_secuenciales`: Crea una hoja con el detalle de los grupos secuenciales planificados.
-

 `core/services/reporting/pdf_report_sections.py`

Nombre del Módulo: pdf_report_sections Descripción: Bloques reutilizables de contenido ReportLab (tablas de auditoría, diagnósticos).

Ensambla párrafos y tablas que consume pdf_report_strategy sin duplicar maquetación.

-  add_diagnostics_section: Añade diagnósticos de recursos e inactividad al PDF.
 -  add_sequential_group_diagnostics_section: Añade diagnóstico de grupos secuenciales.
 -  add_audit_log_table_section: Añade tabla detallada de auditoría.
-

 `core/services/reporting/pdf_report_strategy.py`

Nombre del Módulo: pdf_report_strategy Descripción: Informes PDF de historial de fabricación e iteraciones (reportlab).

Incluye resumen ejecutivo, diagnósticos de secuencia y tablas de auditoría vía secciones reutilizables.

 **Clase** ReporteHistorialFabricacion

Estrategia para generar un informe PDF de optimización, incluyendo resumen ejecutivo, diagnóstico de cuellos de botella y log detallado.

 **core/simulation/__init__.py**

Nombre del Módulo: core.simulation

Descripción: Funciones y datos de apoyo del paquete; conviene enlazar qué controlador o servicio las consume y qué estructuras devuelven (ver firmas al inicio del archivo).

 `core/simulation/event_engine.py`

Nombre del Módulo: `core.simulation.event_engine`

Descripción:

===== MOTOR DE
EVENTOS — PUNTO DE ENTRADA DEL PAQUETE

===== Fachada de
compatibilidad (Fase 2.2): reexporta `MotorDeEventos` desde `core.simulation.engine` para orquestar el bucle
de even...

 `core/simulation/resource_manager.py`

Nombre del Módulo: `resource_manager.py` Descripción: Gestiona la disponibilidad y asignación de recursos (operarios y máquinas) durante la simulación, controlando solapamientos.

Clase IntervaloOcupacion

Representa un bloque de tiempo en el que un recurso está ocupado.

Clase ReglaReasignacion

Define y almacena las condiciones lógicas del negocio bajo las cuales se reasignan operarios físicos en la cadena de montaje de producción.

Reglas de Negocio soportadas y modeladas:

- 'AFTER_UNITS' (Balanceo de carga in-situ / Desplazamientos Secuenciales): Implica especificar la regla con `condicion_tipo="AFTER_UNITS"` y un número N en `condicion_valor`. El sistema asume que el operario ensamblará forzosamente N iteraciones íntegras del proceso (`tarea_origen_id`), finalizando el cupo tras el cual, sin mediación de su supervisor, transicionará automáticamente al inicio de registro de un nodo alternativo (`tarea_destino_id`) equilibrando cargas.

Clase GestorDeRecursos

Gestiona la disponibilidad y asignación de trabajadores y máquinas.

Métodos Principales:

- `registrar_recurso`: Inicializa el calendario para un nuevo trabajador o máquina.
 - `programar_reasignacion`: Registra una nueva regla de reasignación para ser evaluada.
 - `encontrar_siguiete_momento_disponible`: Encuentra la primera fecha/hora en que un recurso está libre a partir de '`desde_fecha`', respetando tanto los bloques de trabajo ya asignados como el horario laboral. THREAD-SAFE: Protegido con lock para prevenir lecturas inconsistentes.
 - `asignar_recurso`: Añade un nuevo intervalo de ocupación al calendario de un recurso. THREAD-SAFE: Protegido con lock para prevenir modificaciones concurrentes.
 - `notificar_unidades_completadas`: Evalúa si la finalización de unidades en una tarea dispara alguna regla de reasignación.
-

Nombre del Módulo: `simulation_engine.py` Descripción: Motor de simulación de producción. Incluye el trabajador de hilo (Worker) y el optimizador de recursos para cumplir plazos.

Clase `SimulationWorker`

Trabajador de hilo (Worker) para ejecutar la simulación en segundo plano.

Permite que la interfaz de usuario permanezca sensible mientras se realizan los cálculos intensivos del motor de simulación.

Signals: `finished (list, list)`: Emitida al completar, envía (resultados, logs_auditoria). `progress_update (int)`: Emitida durante el proceso para actualizar barras de progreso.

Clase `Optimizer`

Motor de optimización algorítmica iterativa para asignación de recursos.

Algoritmo de Optimización (Constraint Satisfaction Heuristic): Determina matemáticamente el número mínimo y estrictamente necesario de recursos complementarios (trabajadores extra) para cumplir un plazo.

Estrategia de resolución iterativa:

1. Efectúa una simulación "Forward-Pass" logística asumiendo 0 extras.
2. Compara el array de resultables de hitos (Fin) contra los 'deadlines'.
3. Si satisface todos los plazos (`_verify_deadlines`), retorna matriz óptima.
4. Si se viola algún plazo, ajusta `trabajadores_flexibles += 1` y reinicia ciclo.
5. Protege el cálculo contra ciclos con `MAX_FLEXIBLE_WORKERS = 20` para evitar bucles O(infinito) en condiciones objetivamente imposibles por agenda horaria.

Métodos Principales:

- `_load_workers`: Loads active workers and their skills from the model.
 - `_prepare_and_prioritize_tasks`: Recopila, expande y aplanas todas las tareas. Si se proporciona un 'production_flow_override', lo usa directamente. De lo contrario, construye las tareas desde la sesión de planificación.
 - `run_simulation`: Ejecuta el bloque iterativo del solver de Satisfacción de Restricciones (CSP).
Heurística Matemática del Motor (Pseudo-código analítico):
Variable Objetivo: `w_flexibles` (enteros, recursos dinámicos).
Constraints (Restricciones): para todo `f_i` en Tareas: $\text{Fin}(f_i) \leq \text{Deadline}(f_i)$.
Iteración 0: `P_result = SimuladorDiscreto(Tareas_Base, Trabajadores=W_fijos + w_flexibles=0)`
`Holguras H_i = Deadline(f_i) - Fin(f_i)` de `P_result` if `eval(min(H_i)) >= 0`: return `P_result`, log #
Optimidad Alcanzada else: if `w_flexibles < 20`: `w_flexibles ++ 1` Goto Iteración 0 else: return `P_result(inviable_cortado)`, log_warning #
Restricción Dura Coordina intrínsecamente la preparación del dataset, el event-loop del scheduler y la verificación vectorizada de plazos contra los `T_actuales` devueltos.
 - `_verify_deadlines`: Solver evaluador de restricciones (Deadline Constraints). Matemáticas Puras -
Análisis de Holgura (Slack Time): Sean `f_1, f_2... f_n` el arreglo de sub-ítems pertenecientes a una `Fab_A`.
Calcula el Maximo Absoluto temporal: $T_{\max}(\text{Fab_A}) = \text{Max}(\text{Fin}(f_1), \text{Fin}(f_2) \dots)$.
Compara la inecuación: $T_{\max}(\text{Fab_A}).\text{date}() \leq \text{Deadline}(\text{Fab_A}).\text{date}()$
Si la inecuación se viola, calcula la penalización métrica: $\text{Retraso Delta_D}(\text{Fab_A}) = \text{Abs}(T_{\max}(\text{Fab_A}).\text{date}() - \text{Deadline_A}(\text{Fab_A}).\text{date}())$ en días.
Si para todo `i`, $T_{\max}(i) \leq \text{Deadline}(i)$ -> Constraint Satisfecha (return True).
-

Nombre del Módulo: `timeline_task.py` Descripción: Representa el ciclo de vida de una tarea individual en la simulación, gestionando sus unidades, tiempos y transiciones de estado.

Clase `LineaTemporalTarea`

Representa el estado y la progresión de una única tarea a lo largo del tiempo. Gestiona sus propios recursos, dependencias y genera sus propios eventos de simulación.

Métodos Principales:

- `iniciar_instancia_inicial`: Crea la primera instancia de trabajo para esta tarea. Llamado desde:
- `generar_eventos_de_produccion()` - `EventoInicioUnidad.procesar()` (para la primera unidad) Args:
`trabajadores`: Lista de IDs de trabajadores `fecha_inicio`: Momento de inicio de la instancia Returns:
`id_instancia`: UUID de la instancia creada
 - `agregar_instancia_paralela`: Añade un trabajador en una nueva instancia paralela. Llamado desde:
- `EventoReasignacionTrabajador.procesar()` cuando `action='UNIRSE_PARALELO'` Args:
`trabajador_id`: ID del trabajador que se une `fecha_inicio`: Momento en que se une `motor_eventos`: Referencia al motor para generar eventos Returns: `id_instancia` si se creó exitosamente, `None` si no hay trabajo disponible
 - `completar_unidad_instancia`: Marca una unidad como completada para una instancia específica. Actualiza contadores y ELIMINA la instancia, devolviendo sus trabajadores para que el motor de eventos decida su próximo paso. Llamado desde:
- `EventoFinUnidad.procesar()` Args: `id_instancia`: UUID de la instancia que completó su unidad Returns: Dict con información de la finalización: {
'`instancia_completada`': `True`, # Siempre es `True` si se encontró '`tarea_completada`': `bool`, # `True` si la tarea entera ha terminado '`siguiente_unidad`': `None`, # El motor decidirá esto '`trabajadores_liberados`': `List[str]` # Trabajadores a liberar }
 - `obtener_instancia`: Obtiene los datos de una instancia específica. Args: `id_instancia`: UUID de la instancia Returns: Dict con datos de la instancia o `None` si no existe
 - `generar_eventos_de_produccion`: Genera el evento de inicio para la primera unidad, creando la instancia inicial. CAMBIO: Ya no genera evento de fin, solo de inicio.
 - `agregar_trabajador`: Añade un trabajador a la tarea y dispara un recálculo.
 - `recalcular_eventos_futuros`: Cancela todos los eventos futuros de esta tarea y genera nuevos eventos basados en el estado actual. Este es el núcleo del recálculo dinámico.
 - `info_instancias`: Devuelve string con información de todas las instancias activas.
 - `esta_completada`: Propiedad que devuelve `True` si la tarea ha completado todas sus unidades.
-

 `core/simulation/timeline_task_parallel.py`

Nombre del Módulo: `core.simulation.timeline_task_parallel`

Descripción: Funciones puras de apoyo (sin estado de proceso): `agregar_instancia_paralela_ops`, `completar_unidad_instancia_ops`. Integración típica con: `__future__`, `uuid`, `datetime`.

-  `agregar_instancia_paralela_ops`: Crea una instancia paralela y programa su evento de inicio.
 -  `completar_unidad_instancia_ops`: Completa unidad de una instancia y retorna su estado agregado.
-

 `core/simulation/engine/__init__.py`

Nombre del Módulo: `core.simulation.engine`

Descripción: Funciones y datos de apoyo del paquete; conviene enlazar qué controlador o servicio las consume y qué estructuras devuelven (ver firmas al inicio del archivo).

 `core/simulation/engine/base.py`

Nombre del Módulo: `core.simulation.engine.base`

Descripción: Define protocolos o tipos principales: `SimulationState`. Mantiene el estado volátil y reactivo de una simulación en curso. Integración típica con: `datetime`, `timeline_task`.

Clase `SimulationState`

Mantiene el estado volátil y reactivo de una simulación en curso.

Almacena la cola de eventos, las líneas temporales de las tareas y el estado actual de los recursos asignados.

 `core/simulation/engine/core_runner.py`

Nombre del Módulo: `core.simulation.engine.core_runner`

Descripción: Define protocolos o tipos principales: `CoreSimulationRunner`. Gestiona el bucle principal de la simulación y la cola de eventos. Integración típica con: `heapq`, `time`, `pickle`, `os`, `datetime`, `threading`.

Clase `CoreSimulationRunner`

Gestiona el bucle principal de la simulación y la cola de eventos.

Métodos Principales:

- `programar_eventos`: Añade una lista de eventos al heap de forma segura para hilos.
 - `cancelar_eventos`: Marca eventos para cancelación.
 - `tiene_evento_futuro`: Verifica si ya existe un evento programado para una unidad/instancia.
 - `save_checkpoint`: Guarda el estado actual en un archivo.
 - `load_checkpoint`: Carga el estado desde un archivo.
-

 `core/simulation/engine/dependency_handler.py`

Nombre del Módulo: `core.simulation.engine.dependency_handler`

Descripción: Define protocolos o tipos principales: `DependencyHandler`. Gestiona la validación y propagación de dependencias entre tareas. Integración típica con: `datetime`, `simulation_events`.

Clase `DependencyHandler`

Gestiona la validación y propagación de dependencias entre tareas.

Métodos Principales:

- `encontrar_tareas_dependientes`: Encuentra todas las tareas que dependen de la tarea especificada.
 - `verificar_dependencias_cumplidas`: Verifica si se desbloquean nuevas unidades tras completar una tarea. (Versión mejorada con propagación recursion).
-

 `core/simulation/engine/motor.py`

Nombre del Módulo: `core.simulation.engine.motor`

Descripción: Define protocolos o tipos principales: `MotorDeEventos`. Orquestador principal del motor de simulación basado en eventos. Integración típica con: `__future__`, `heapq`, `time`, `os`, `tempfile`, `datetime`.

Clase `MotorDeEventos`

Orquestador principal del motor de simulación basado en eventos.

Esta clase es el "corazón" del sistema de cálculo de tiempos. Utiliza un bucle de eventos (Event Loop) con una cola de prioridad (heapq) para avanzar el tiempo virtual y procesar hitos de producción.

Responsabilidades: - Gestionar el tiempo virtual de la simulación. - Coordinar el Gestor de Recursos (trabajadores y máquinas). - Manejar dependencias entre tareas (DependencyHandler). - Compilar resultados y auditorías (ResultsCompiler). - Ejecutar la lógica de eventos (CoreSimulationRunner).

Métodos Principales:

- `_inicializar_estado_inicial`: Prepara las líneas temporales y recursos.
 - `_generar_eventos_iniciales`: Identifica tareas raíz y genera sus eventos iniciales.
 - `ejecutar_simulacion`: Bucle principal de simulación.
 - `find_task_index_by_id`: Devuelve el índice de flujo para un `tarea_id` usando el mapeo interno del motor.
-

 `core/simulation/engine/results_compiler.py`

Nombre del Módulo: `core.simulation.engine.results_compiler`

Descripción: Define protocolos o tipos principales: `ResultsCompiler`. Compila los resultados de la simulación y genera el log de auditoría. Integración típica con: `datetime`, `core`.

Clase `ResultsCompiler`

Compila los resultados de la simulación y genera el log de auditoría.

Métodos Principales:

- `compilar_resultados`: Crea una entrada de resultado por cada unidad individual completada.
 - `compilar_audit_log`: Convierte los eventos en un audit log legible.
 - `_generar_descripcion`: Genera descripciones específicas para cada tipo de evento.
-

 `core/simulation/simulation_events/__init__.py`

Nombre del Módulo: `core.simulation.simulation_events`

Descripción: Concentra datos de configuración o catálogos estáticos: `__all__`, consumidos por la UI y controladores. Integración típica con: `base`, `production`, `worker`.

 `core/simulation/simulation_events/base.py`

Nombre del Módulo: `core.simulation.simulation_events.base`

Descripción: Define protocolos o tipos principales: `EventoDeSimulacion`. Clase base para todos los eventos de la simulación. Integración típica con: `datetime`.

Clase `EventoDeSimulacion`

Clase base para todos los eventos de la simulación.

Define la estructura mínima de un evento, incluyendo su marca de tiempo, prioridad y la lógica para ser procesado por el motor de eventos.

Métodos Principales:

- `procesar`: Ejecuta la lógica asociada al evento y devuelve nuevos eventos generados. Args: `motor_eventos`: Instancia del motor que está procesando la cola. Returns: Lista de nuevos eventos a programar en la cola.
-

 `core/simulation/simulation_events/production.py`

Nombre del Módulo: `core.simulation.simulation_events.production`

Descripción: Define protocolos o tipos principales: `EventoInicioUnidad`, `EventoFinUnidad`. Evento que marca el inicio del trabajo en una unidad de una tarea. Integración típica con: `datetime`, `base`, `worker`.

Clase `EventoInicioUnidad`

Evento que marca el inicio del trabajo en una unidad de una tarea.

Métodos Principales:

- `procesar`: Planifica una unidad para una INSTANCIA específica.

Clase `EventoFinUnidad`

Evento que marca la finalización de una unidad, liberando recursos.

 `core/simulation/simulation_events/worker.py`

Nombre del Módulo: `core.simulation.simulation_events.worker`

Descripción: Define protocolos o tipos principales: `EventoReasignacionTrabajador`, `EventoTiempoInactivo`. Evento que reasigna un trabajador de una tarea a otra. Integración típica con: `base`.

Clase `EventoReasignacionTrabajador`

Evento que reasigna un trabajador de una tarea a otra.

Métodos Principales:

- `procesar`: Procesa la reasignación de un trabajador con soporte para modo paralelo.

Clase `EventoTiempoInactivo`

Evento que registra cuando un trabajador queda inactivo esperando trabajo. No genera nuevos eventos, solo registra la situación en el audit log.

Métodos Principales:

- `procesar`: Registra el tiempo de inactividad en el audit log.
-

 `core/utils/author_loader.py`

Nombre del Módulo: `author_loader.py` Descripción: Utilidad para cargar dinámicamente información sobre los autores y colaboradores del proyecto.

 **Clase** `WorkerSignals`

Señales para el worker de carga de info de autor.

 **Clase** `AuthorInfoLoader`

Worker para cargar información de Wikipedia en segundo plano sin bloquear la UI.

 **core/Utils/helpers.py**

Nombre del Módulo: core.utils.helpers

Descripción: Funciones puras de apoyo (sin estado de proceso): `resource_path`. Integración típica con: `sys`, `os`.

-  `resource_path`: Get absolute path to resource, works for dev and for PyInstaller
-

 `core/utils/pila_serializer.py`

Nombre del Módulo: `core.utils.pila_serializer`

Descripción: ♥ Serializador robusto para pilas de cálculo. Maneja correctamente todos los tipos de datos y previene pérdida de información.

Clase `PilaJSONEncoder`

Encoder personalizado para serializar pilas con todos sus tipos de datos.

-  `decode_pila_json`: Decoder personalizado para restaurar objetos complejos desde JSON. Se usa con `json.loads(data, object_hook=decode_pila_json)`
 -  `serialize_production_flow`: ✓ Serializa un flujo de producción con validación completa. Retorna una tupla (`json_string`, `validation_summary`)
 -  `deserialize_production_flow`: ✓ Deserializa un flujo de producción con validación completa. Retorna una tupla (`production_flow`, `validation_summary`)
-

Nombre del Módulo: ui_scaler.py Descripción: Proporciona la lógica matemática y de generación de estilos para el escalado dinámico de la interfaz en función de la geometría de pantalla.

Usa `QScreen.availableGeometry()` (píxeles lógicos Qt6), alineado con el escalado alto DPI del sistema: no multiplicar de nuevo por `devicePixelRatio` salvo un estudio explícito en Windows (evitar doble escalado con `PassThrough` en `app`).

Clase **UIScaler**

Motor encargado de calcular factores de escala para la interfaz de usuario y generar hojas de estilo dinámicas (QSS) para mejorar la visualización en diferentes resoluciones de pantalla.

Métodos Principales:

- `calculate_scale_factor`: Calcula el factor de escala basado en la altura de la pantalla disponible. Args: `screen_height`: Altura disponible de la pantalla en píxeles. Returns: Un multiplicador (float) entre `MIN_SCALE` y `MAX_SCALE`.
 - `generate_dynamic_qss`: Genera un bloque global de QSS (hoja de estilos Qt) con tamaños ajustados en función del factor de escala. Args: `scale_factor`: El factor de escala previamente calculado. Returns: Una cadena de texto con el CSS/QSS dinámico listo para inyectar en `QApplication`.
 - `get_current_screen_height`: Intenta obtener la altura útil (available resolution) de la pantalla donde actualmente reside el widget proporcionado. Args: `active_widget`: Instancia de un `QWidget` visible (como `MainView`). Returns: Altura de la pantalla en píxeles o `cls.BASE_HEIGHT` si falla.
-

 `core/utils/visualization.py`

Nombre del Módulo: visualization.py Descripción: Generador de diagramas de flujo y organigramas de producción utilizando Graphviz para representar los resultados de la simulación.

Clase VisualizationGenerator

Clase para generar visualizaciones del flujo de producción y organigramas basados en los resultados de una simulación. Utiliza Graphviz para crear diagramas detallados que muestran tareas, dependencias, recursos asignados y posibles cuellos de botella (tiempos de espera).

Métodos Principales:

- `generate_organigram_image`: Crea el gráfico completo y lo renderiza a un archivo de imagen.
-

 `core/validation/validator_service.py`

Nombre del Módulo: `validator_service.py` Descripción: Servicio de validación de datos. Centraliza las reglas de negocio para asegurar la integridad de productos, códigos y cantidades.

 **Clase `ValidatorService`**

Servicio centralizado para validación de entradas de usuario. Previene datos corruptos y mejora la experiencia de usuario.

Capítulo: database/

Métrica	Valor
Archivos .py en database/	59
Incluidos en el cuerpo	59
Omitidos (docstrings/reglas)	0
Clases detectadas (AST)	70

```
graph TD
  CORE[Core/Services] -->|repositorios| DB[Database/Repos]
```


 **database/___init___**.py

Nombre del Módulo: database

Descripción: Funciones y datos de apoyo del paquete; conviene enlazar qué controlador o servicio las consume y qué estructuras devuelven (ver firmas al inicio del archivo).

`database/config.py`

Nombre del Módulo: `database.config`

Descripción: Define protocolos o tipos principales: `DatabaseConfig`. Integración típica con: `os`, `pathlib`, `dotenv`, `core`.

Clase `DatabaseConfig`

Métodos Principales:

- `set_db_url`: Allows runtime override of the database URL.
 - `get_db_url`: Returns the database connection URL. Priority: Runtime override > Environment variables > Default SQLite.
 - `get_echo_sql`: Returns True if SQL queries should be logged.
 - `get_log_dir`: Returns the directory for log files.
 - `get_backup_dir`: Returns the directory for backup files.
-

database/database_manager.py

Nombre del Módulo: database.database_manager

Descripción: Define protocolos o tipos principales: `DatabaseManager`. Gestiona todas las operaciones de la base de datos para la aplicación. Integración típica con: `__future__`, `types`, `sqlalchemy`, `models`, `config`, `repositories`.

Clase `DatabaseManager`

Gestiona todas las operaciones de la base de datos para la aplicación utilizando SQLAlchemy.

Métodos Principales:

- `__init__`: Inicializa el gestor y configura el motor de SQLAlchemy. Args: `db_url` (str, optional): URL de conexión. `engine` (Engine, optional): Motor SQLAlchemy pre-configurado (útil para tests).
 - `_create_tables_if_not_exist`: Crea las tablas definidas en los modelos si no existen.
 - `_init_repositories`: Inicializa los repositorios con la fábrica de sesiones.
 - `close`: Cierra todas las conexiones a la base de datos.
 - `get_session`: Devuelve una nueva sesión de SQLAlchemy.
 - `db_path`: Devuelve la ruta al archivo de base de datos (solo para SQLite). Extrado de `db_url`.
 - `compare_with_db`: Compare local database with a foreign SQLite database file. Args: `foreign_db_path`: Path to the foreign .db file (from USB) Returns: `DatabaseComparisonDTO` containing differences per table
 - `apply_sync_changes`: Apply selected changes from a sync operation. Args: `comparison`: `DatabaseComparisonDTO` with changes to apply Returns: Number of records successfully applied
-

 **database/models/__init__.py**

Nombre del Módulo: database.models

Descripción: Concentra datos de configuración o catálogos estáticos: `__all__`, consumidos por la UI y controladores. Integración típica con: `base`, `product`, `fabrication`, `worker`, `tracking`, `machine`.

 `database/models/base.py`

Nombre del Módulo: `database.models.base`

Descripción: Modelos ORM base (SQLAlchemy): DeclarativeBase, metadatos compartidos y tablas de enlace many-to-many entre productos, materiales y preprocesos.

 `database/models/fabrication.py`

Nombre del Módulo: `database.models.fabrication`

Descripción: Define protocolos o tipos principales: `Fabricacion`, `FabricacionContador`. Entidad principal que representa una Orden de Fabricación (OF). Integración típica con: `sqlalchemy`, `base`.

Clase `Fabricacion`

Entidad principal que representa una Orden de Fabricación (OF).

Vincula un conjunto de productos y preprocesos para su ejecución en planta, gestionando la trazabilidad y los operarios asignados.

Atributos (Columnas): - `id` (int): Identificador único autoincremental. - `codigo` (str): Código único de la orden (ej: OF-2024-001). - `descripcion` (str, optional): Breve descripción o notas de la orden.

Relaciones: - `preprocesos`: Lista de preprocesos asociados (Many-to-Many). - `trabajadores_asignados`: Operarios vinculados a esta OF (Many-to-Many). - `trabajo_logs`: Registro cronológico de actividades en planta (One-to-Many).

Clase `FabricacionContador`

Contador para numeración correlativa de etiquetas de unidad en una fabricación.

Permite garantizar que cada unidad física de un lote tenga un ID único incremental.

Atributos (Columnas): - `fabricacion_id` (int): FK hacia 'fabricaciones'. Parte de la PK. - `ultimo_numero_unidad` (int): Último correlativo generado (ej: 42 para la unidad 42).

Relaciones: - `fabricacion`: Acceso al objeto Fabricación padre.

 `database/models/inventory.py`

Nombre del Módulo: `database.models.inventory`

Descripción: Define protocolos o tipos principales: `Material`, `Pila`, `PasoPila`, `DiarioBitacora`, `EntradaDiario`. Representa un componente físico o materia prima. Integración típica con: `sqlalchemy`, `base`, `datetime`.

Clase Material

Representa un componente físico o materia prima.

Se vincula a productos, preprocesos e iteraciones para gestionar la lista de materiales (BOM) necesaria en cada fabricación.

Clase Pila

Contenedor lógico para planes de producción complejos.

Agrupar múltiples fabricaciones y lotes para realizar simulaciones de carga de trabajo y seguimiento de hitos diarios.

Clase DiarioBitacora

Registro diario de actividad vinculado a una Pila.

Almacena las entradas de lo planificado vs lo realizado cada día de la producción activa.

Clase Lote

Agrupación logística de productos o fabricaciones.

Permite gestionar unidades que deben viajar juntas o que comparten una misma prioridad de entrega/procesamiento.

 `database/models/machine.py`

Nombre del Módulo: `database.models.machine`

Descripción: Define protocolos o tipos principales: `Maquina`, `MachineMaintenanc`, `GrupoPreparacion`, `PreparacionPaso`. Representa un recurso físico (fresa, torno, etc.) en planta. Integración típica con: `sqlalchemy`, `base`.

Clase `Maquina`

Representa un recurso físico (fresa, torno, etc.) en planta.

Gestiona su estado de disponibilidad, departamento y los grupos de preparación asociados para el cálculo de tiempos.

Clase `GrupoPreparacion`

Conjunto de pasos de preparación necesarios para una máquina.

Puede ser genérico para la máquina o específico para un producto.

Clase `PreparacionPaso`

Tarea individual dentro de un grupo de preparación.

Incluye el tiempo estimado, si es una tarea diaria o de verificación de primera pieza.

 `database/models/product.py`

Nombre del Módulo: `database.models.product`

Descripción: Define protocolos o tipos principales: `Producto`, `Preproceso`, `Subfabricacion`, `ProcesoMecanico`, `ProductIteration`. Modelo que representa un Producto en el catálogo. Integración típica con: `sqlalchemy`, `base`, `datetime`.

Clase Producto

Modelo que representa un Producto en el catálogo.

Almacena la configuración base, tiempos de fabricación estimados y relaciones con subfabricaciones, materiales e iteraciones.

Atributos (Columnas): - `codigo` (str): PK. Identificador único del producto. - `descripcion` (str): Nombre descriptivo del producto. - `departamento` (str): Área productiva responsable (ej: Mecanizado). - `tipo_trabajador` (int): Mínimo nivel de habilidad requerido. [Diccionario de Datos] 1 = Operario Básico/Junior (tareas rutinarias). 2 = Especialista/Mid (manejo de maquinaria compleja). 3 = Experto/Senior (calidad, configuración pesada o supervisión). - `donde` (str, optional): Ubicación física o referencia de almacenamiento. - `tiene_subfabricaciones` (bool): Indica si depende de otros componentes. - `tiempo_optimo` (float, optional): Tiempo de fabricación estimado por unidad.

Relaciones: - `subfabricaciones`: Lista de componentes dependientes. - `materiales`: Materias primas asociadas. - `procesos_mecanicos`: Pasos de máquina específicos. - `iteraciones`: Historial de cambios y control de calidad.

Clase Preproceso

Modelo para tareas preparatorias reutilizables.

Define trabajos que no son procesos mecánicos de máquina pero consumen tiempo y recursos de operario (ej. limpieza, rebabado).

Atributos (Columnas): - `id` (int): PK autoincremental. - `nombre` (str): Nombre único del proceso. - `descripcion` (str, optional): Texto detallado del trabajo. - `tiempo` (float): Tiempo estimado de ejecución. - `tipo_trabajador` (int): Nivel de habilidad requerido.

Relaciones: - `materiales`: Consumibles necesarios para el preproceso. - `fabricaciones`: Órdenes de fabricación que incluyen este paso.

Métodos Principales:

- `componentes`: Setter para mantener compatibilidad.

Clase Subfabricacion

Define un componente que forma parte de un producto pero que tiene su propio flujo de procesos o es una pieza independiente.

Atributos (Columnas): - `id` (int): PK autoincremental. - `producto_codigo` (str): FK hacia 'productos'. - `descripcion` (str): Nombre del componente. - `tiempo` (float): Tiempo de fabricación. - `tipo_trabajador` (int): Nivel de habilidad requerido. - `maquina_id` (int, optional): FK hacia 'maquinas' (si aplica).

Relaciones: - `producto`: Referencia al Producto padre. - `maquina`: Máquina específica asignada.

Clase ProcesoMecanico

Representa una operación de máquina específica (fresado, torneado, etc.) vinculada a un producto con un tiempo de ejecución calculado.

Atributos (Columnas): - id (int): PK autoincremental. - producto_codigo (str): FK hacia 'productos'. - nombre (str): Nombre de la operación. - descripcion (str): Detalles técnicos del proceso. - tiempo (float): Tiempo de máquina por unidad. - tipo_trabajador (int): Nivel de habilidad de operario.

Relaciones: - producto: Producto al que pertenece esta operación.

Clase ProductIteration

Registro histórico de cambios en un producto.

Almacena revisiones de diseño, responsables, planos y fotos de piezas reales fabricadas para control de calidad.

Atributos (Columnas): - id (int): PK autoincremental. - producto_codigo (str): FK hacia 'productos'. - fecha_creacion (datetime): Fecha de la revisión. - nombre_responsable (str): Quién realizó el cambio. - descripcion_cambio (str): Motivo o detalle de la iteración. - ruta_imagen (str): Enlace a foto de la pieza fabricada. - tipo_fallo (str): Categorización de error si aplica. - ruta_plano (str): Enlace al dibujo técnico.

Relaciones: - producto: Producto objeto de la iteración. - materiales: Versión específica de materiales en esta iteración.

 `database/models/security.py`

Nombre del Módulo: `database.models.security`

Descripción: Define protocolos o tipos principales: `Configuration`, `LoginAttempt`, `AuditLog`. Par clave/valor para configuraciones persistentes del sistema. Integración típica con: `sqlalchemy`, `base`, `datetime`.

Clase `Configuration`

Par clave/valor para configuraciones persistentes del sistema.

Clase `LoginAttempt`

Intento de autentificacion utilizado por la politica de rate limiting.

Clase `AuditLog`

Registro auditable de acciones de seguridad y administracion.

 **database/models/tracking.py**

Nombre del Módulo: database.models.tracking

Descripción: Define protocolos o tipos principales: TrabajoLog, PasoTrazabilidad, IncidenciaLog, IncidenciaAdjunto. Registro principal de trabajo ejecutado para una fabricacion. Integración típica con: sqlalchemy, base, datetime.

Clase TrabajoLog

Registro principal de trabajo ejecutado para una fabricacion.

Clase PasoTrazabilidad

Evento de trazabilidad de un paso concreto dentro de un trabajo.

Clase IncidenciaLog

Incidencia reportada en un trabajo, con estado y resolucion.

Clase IncidenciaAdjunto

Adjunto asociado a una incidencia (archivo, tipo y metadatos).

 `database/models/worker.py`

Nombre del Módulo: `database.models.worker`

Descripción: Define protocolos o tipos principales: `Trabajador`, `TrabajadorPilaAnotacion`. Modelo que representa a un operario o administrador del sistema. Integración típica con: `sqlalchemy`, `base`, `datetime`.

Clase `Trabajador`

Modelo que representa a un operario o administrador del sistema.

Gestiona la autenticación, roles por departamento y la vinculación con los registros de trabajo e incidencias en planta.

Diccionario de Datos: - `tipo_trabajador` (int): Nivel de capacidad técnica para asignaciones automáticas. 1 = Operario Básico (operaciones estándar). 2 = Especialista (maquinaria específica). 3 = Experto (resolver cuellos de botella y supervisión global). El optimizador equipará este nivel con el mínimo exigido por Producto.

`database/repositories/__init__.py`

Nombre del Módulo: `repositories` Descripción: Paquete de acceso a datos: repositorios por dominio (producto, máquina, pila, trabajador, informes, tracking, etc.) y exportaciones para `DatabaseManager`.

Las clases públicas se listan en `__all__` para imports explícitos desde `database.repositories`.

-  `__getattr__`: Carga perezosa de `ReportsRepository` para no exigir el subpaquete `reports` en imports parciales.
-

 `database/repositories/base.py`

Nombre del Módulo: base Descripción: Clase base de repositorios: fábrica de sesiones SQLAlchemy, `safe_execute` y registro de errores comunes a toda la capa de acceso a datos.

Clase BaseRepository

Clase base para todos los repositorios. Proporciona funcionalidades comunes como manejo de sesiones, logging y operaciones CRUD básicas.

Métodos Principales:

- `__init__`: Inicializa el repositorio base. Args: `session_factory`: Factory de sesiones de SQLAlchemy (SessionLocal)
 - `get_session`: Obtiene una nueva sesión de SQLAlchemy. Returns: Session de SQLAlchemy o None si hay error
 - `safe_execute`: Ejecuta una operación de base de datos de forma segura con manejo de errores. **VERSIÓN MEJORADA** con mejor logging para debugging.
 - `_get_default_error_value`: Valor por defecto a devolver en caso de error. Cada repositorio puede sobrescribir este método.
-

 `database/repositories/configuration_repository.py`

Nombre del Módulo: `configuration_repository` Descripción: Pares clave–valor de configuración persistente (JSON o tipos simples) en base de datos.

Lectura y escritura sobre el modelo `Configuration` con valores por defecto opcionales.

Clase `ConfigurationRepository`

Repositorio para gestión de configuración de la aplicación. Almacena pares clave-valor de configuración.

Métodos Principales:

- `_get_default_error_value`: Valor por defecto en caso de error.
 - `get_setting`: Obtiene un valor de configuración por su clave. Args: `key`: Clave de configuración `default_value`: Valor por defecto si no existe Returns: Valor de configuración o `default_value`
 - `set_setting`: Guarda o actualiza un valor de configuración. Args: `key`: Clave de configuración `value`: Valor a guardar (se convertirá a string) Returns: `True` si se guardó correctamente, `False` en caso contrario
 - `get_holidays`: Obtiene la lista de días festivos. Returns: Lista de objetos `date` con los festivos
 - `add_holiday`: Añade un día festivo. Args: `holiday_date`: Fecha del festivo `description`: Descripción opcional del festivo Returns: `True` si se añadió correctamente, `False` en caso contrario
 - `remove_holiday`: Elimina un día festivo. Args: `holiday_date`: Fecha del festivo a eliminar Returns: `True` si se eliminó correctamente
-

 **database/repositories/incidencia_repository.py**

Nombre del Módulo: incidencia_repository Descripción: Registro de incidencias en planta, adjuntos en disco y vínculo con TrabajoLog.

Persistencia de IncidenciaLog e IncidenciaAdjunto con rutas de fichero bajo el directorio de datos.

Clase IncidenciaRepository

Repositorio para gestión de incidencias.

Métodos Principales:

- registrar_incidencia: Registra una nueva incidencia.
 - _crear_adjunto: Crea un adjunto fotográfico (uso interno).
 - añadir_foto_a_incidencia: Añade una foto a una incidencia existente.
 - resolver_incidencia: Marca una incidencia como resuelta.
 - obtener_incidencias_abiertas: Obtiene todas las incidencias abiertas.
 - _map_to_incidencia_log_dto: Map an IncidenciaLog ORM object to IncidenciaLogDTO.
 - _map_to_incidencia_adjunto_dto: Map IncidenciaAdjunto ORM to DTO.
-

database/repositories/iteration_repository.py

Nombre del Módulo: iteration_repository Descripción: Historial de iteraciones de producto, imágenes asociadas y consultas para la UI.

Persistencia y mapeo a DTOs sobre los modelos `ProductIteration` y `Material`.

Clase `IterationRepository`

Repositorio para gestión de iteraciones de productos. Maneja el historial de cambios, mejoras y gestión de imágenes.

Métodos Principales:

- `get_all_iterations_with_dates`: Obtiene todas las iteraciones de todos los productos para la vista de historial. Returns: Lista de `ProductIterationDTO`
 - `get_product_iterations`: Obtiene todas las iteraciones de un producto con sus materiales. Args: `producto_codigo`: Código del producto Returns: Lista de `ProductIterationDTO` con materiales
 - `get_product_iterations_by_id_or_similar`: Devuelve una iteración por ID.
 - `add_product_iteration`: Añade una nueva iteración de producto con sus materiales.
 - `update_product_iteration`: Actualiza los campos de una iteración de producto.
 - `delete_product_iteration`: Elimina una iteración de producto.
 - `update_iteration_image_path`: Actualiza la ruta de la imagen para una iteración.
 - `update_iteration_file_path`: Actualiza la ruta de un archivo (imagen/plano) para una iteración.
 - `add_image`: Añade una imagen a una iteración.
 - `get_images`: Obtiene todas las imágenes de una iteración.
 - `delete_image`: Elimina una imagen de la base de datos.
 - `_get_default_error_value`: Valor por defecto en caso de error.
-

 **database/repositories/iteration_repository_helpers.py**

Nombre del Módulo: `iteration_repository_helpers` Descripción: Funciones puras de mapeo entre filas ORM de iteración y `ProductIterationDTO`.

Mantiene `IterationRepository` enfocado en SQL; sin dependencia de sesión aquí.

-  `material_to_dto`: Convierte un material ORM a DTO.
 -  `iteration_to_dto`: Convierte una iteración ORM a DTO.
-

 `database/repositories/label_counter_repository.py`

Nombre del Módulo: `label_counter_repository` Descripción: Reserva atómica de rangos de numeración de unidad para etiquetas por fabricación.

Usa el modelo `FabricacionContador` en la base principal para evitar colisiones al imprimir lotes.

Clase `LabelCounterRepository`

Numeración secuencial de unidades por orden de fabricación (reserva en transacción).

Métodos Principales:

- `get_next_unit_range`: Obtiene y reserva un rango de números de unidad únicos para una fabricación. Operación atómica. Args: `fabricacion_id` (int): El ID de la fabricación. `cantidad` (int): Cuántos números se necesitan. Returns: `LabelRangeDTO` con el rango asignado (`start`, `end`, `count`) o `None` si hay error.
 - `_get_next_unit_range_logic`: Lógica interna transaccional para obtener el rango.
 - `close`: Método de compatibilidad. No hace nada porque la sesión se maneja por request/operación.
-

 `database/repositories/lote_repository.py`

Nombre del Módulo: `lote_repository` Descripción: Plantillas de lote: CRUD, vínculos con productos y fabricaciones, consultas para cálculo.

Opera sobre los modelos `Lote`, `Producto` y `Fabricacion` vía SQLAlchemy.

Clase `LoteRepository`

Gestiona las operaciones CRUD para el modelo `Lote` utilizando SQLAlchemy.

Métodos Principales:

- `create_lote`: Crea una nueva plantilla de Lote y asocia sus componentes.
 - `get_lote_details`: Obtiene los detalles de una plantilla de Lote, incluyendo sus componentes.
 - `search_lotes`: Busca plantillas de Lote por código o descripción.
 - `update_lote`: Actualiza una plantilla de Lote existente.
 - `delete_lote`: Elimina una plantilla de Lote.
-

 `database/repositories/material_repository.py`

Nombre del Módulo: `material_repository` Descripción: Materiales de catálogo y tabla de enlace producto-material (BOM parcial).

Expone consultas y altas/bajas usadas por `ProductService` y la importación BOM.

Clase `MaterialRepository`

Repositorio para gestión de materiales. Maneja la persistencia y relaciones de componentes industriales.

Métodos Principales:

- `get_all_materials`: Obtiene todos los materiales registrados como DTOs.
 - `get_material_by_code`: Busca un material por su código único.
 - `search_materials`: Busca materiales por código o descripción. Si el término es `None` o vacío, devuelve todos. Si el término tiene menos de 2 caracteres (y no es vacío), devuelve vacío.
 - `add_material`: Añade un material o actualiza su descripción si ya existe.
 - `update_material`: Actualiza el código y descripción de un material.
 - `delete_material`: Elimina un material por su ID.
 - `get_problematic_components_stats`: Obtiene estadísticas de componentes más frecuentes en iteraciones de fallo.
 - `link_material_to_product`: Vincula un material a un producto.
 - `unlink_material_from_product`: Desvincula un material de un producto.
 - `link_material_to_iteration`: Vincula un material a una iteración específica.
 - `delete_material_link_from_iteration`: Desvincula un material de una iteración (alias para `unlink_material_from_iteration`).
 - `unlink_material_from_iteration`: Desvincula un material de una iteración.
-

 `database/repositories/product_repository.py`

Nombre del Módulo: `product_repository` Descripción: Persistencia y consultas del catálogo de productos (subfabricaciones, procesos mecánicos, materiales y vínculos con fabricaciones).

Convierte filas SQLAlchemy a DTOs de dominio para servicios y controladores.

Clase `ProductRepository`

Repositorio para gestión de productos. Maneja la persistencia de artículos, escandallos y relaciones de fabricación.

Métodos Principales:

- `add_product`: Añade un producto, subfabricaciones y procesos mecánicos.
 - `update_product`: Actualiza un producto, subfabricaciones y procesos mecánicos.
 - `delete_product`: Elimina un producto por su código.
 - `get_all_products`: Obtiene la lista completa de productos registrados.
 - `get_product_by_code`: Busca un producto por su código único.
 - `get_latest_products`: Obtiene los últimos productos (orden descendente por código).
 - `get_product_details`: Obtiene detalles, subfabricaciones y procesos de un producto.
 - `search_products`: Busca productos por código o descripción (ilike, cualquier longitud). Si el término es `None` o vacío, devuelve todos (hasta el límite).
 - `get_materials_for_product`: Obtiene la lista de materiales vinculados a un producto.
 - `get_products_by_fabricacion`: Obtiene los productos asociados a una fabricación. Mantiene compatibilidad con `DatabaseManager`.
 -  `_subfabricacion_from_row`: Crea un modelo `Subfabricacion` desde dict (UI/diálogo) o DTO/objeto con atributos. Ignora `id/producto_codigo` en dicts para evitar choque de kwargs y PKs obsoletas.
-

 `database/repositories/product_repository_helpers.py`

Nombre del Módulo: `product_repository_helpers` Descripción: Mapeo de modelos ORM de producto a DTOs y normalización de identificadores de máquina.

Funciones puras usadas por `ProductRepository` para no duplicar conversiones en cada consulta.

-  `to_product_dto`: Convierte el modelo `Producto` a `ProductDTO`.
 -  `to_subfabricacion_dto`: Convierte el modelo `Subfabricacion` a `SubfabricacionDTO`.
 -  `to_proceso_mecanico_dto`: Convierte el modelo `ProcesoMecanico` a `ProcesoMecanicoDTO`.
 -  `to_material_dto`: Convierte el modelo `Material` a `MaterialDTO`.
 -  `normalize_machine_id`: Normaliza `maquina_id` a `int` | `None`.
-

 `database/repositories/protocols.py`

Nombre del Módulo: protocols Descripción: Protocolos tipados compartidos por repositorios y servicios de acceso a datos.

 **Clase RepositoryProtocol**

Contrato común de repositorios basados en BaseRepository (sesión y ejecución segura).

 **Clase PilaRepositoryProtocol**

Protocolo específico para el repositorio de Pila.

 **Clase TrackingRepositoryProtocol**

Protocolo específico para el repositorio de Tracking.

 `database/repositories/tracking_log_repository.py`

Nombre del Módulo: `tracking_log_repository` Descripción: Logs de trabajo, pasos de trazabilidad y consultas pesadas vía subgestores.

Compone `TrackingCoreManager`, `TrackingStepsManager`, `TrackingQueriesManager` y `TrackingMapper` para no concentrar todo el SQL en una sola clase.

Clase `TrackingLogRepository`

Repositorio para gestión de logs de trabajo y pasos de trazabilidad. Implementa el patrón Fachada delegando en managers especializados.

Métodos Principales:

- `_map_to_trabajo_log_dto`: Wrapper de compatibilidad. Algunos tests pasan `logger` como argumento posicional o keyword; se ignora y se usa siempre `self.logger` para evitar duplicidad en la llamada al mapper.
 - `_map_to_incidencia_log_dto`: Wrapper de compatibilidad; ver `_map_to_trabajo_log_dto`.
 - `_map_to_paso_trazabilidad_dto`: Wrapper de compatibilidad; ver `_map_to_trabajo_log_dto`.
 - `get_fabricaciones_por_trabajador`: Delega la obtención de fabricaciones asignadas al gestor de consultas. Args: `trabajador_id`: ID del trabajador. Returns: Lista de DTOs de fabricaciones asignadas.
-

 `database/repositories/tracking_repository.py`

Nombre del Módulo: `tracking_repository` Descripción: Fachada de trazabilidad: delega en repositorios de logs, incidencias y estadísticas.

Unifica el acceso que la aplicación usa para seguimiento operario—fabricación sin acoplar la capa de servicios a cada subrepositorio por separado.

Clase `TrackingRepository`

Repositorio FACADE para operaciones de tracking y trazabilidad. Delega la lógica a repositorios especializados.

Métodos Principales:

- `__init__`: Inicializa el repositorio y sus sub-repositorios.
 - `get_fabricaciones_por_trabajador`: Obtiene las fabricaciones asignadas a un trabajador (vía `log_repo`). Args: `trabajador_id`: ID del trabajador. Returns: Lista de fabricaciones con sus productos asociados en formato DTO.
 - `import_tasks_from_csv`: Importa tareas desde un archivo CSV. (Stub para compatibilidad UI)
-

 `database/repositories/tracking_stats_repository.py`

Nombre del Módulo: `tracking_stats_repository` Descripción: Agregados y estadísticas de seguimiento (tiempos por trabajador, incidencias, etc.).

Consultas de solo lectura sobre `TrabajoLog`, `IncidenciaLog` y tablas relacionadas.

Clase `TrackingStatsRepository`

Repositorio para consultas estadísticas de seguimiento.

Métodos Principales:

- `obtener_estadisticas_trabajador`: Obtiene estadísticas de un trabajador.
 - `obtener_estadisticas_fabricacion`: Obtiene estadísticas de una fabricación.
 - `obtener_trabajadores_de_fabricacion`: Obtiene todos los trabajadores asignados a una fabricación.
-

 **database/repositories/machine/__init__.py**

Nombre del Módulo: machine Descripción: Acceso a datos de máquinas (CRUD, mantenimiento, preparación y estadísticas).

 `database/repositories/machine/crud_manager.py`

Nombre del Módulo: machine.crud_manager Descripción: Acceso a datos de máquinas (CRUD, mantenimiento, preparación y estadísticas).

 **Clase MachineCRUDManager**

Gestor DAO para operaciones CRUD básicas de máquinas.

Proporciona métodos para listar, añadir, actualizar y eliminar registros de maquinaria en la base de datos, convirtiéndolos automáticamente a DTOs.

 `database/repositories/machine/maintenance_manager.py`

Nombre del Módulo: machine.maintenance_manager Descripción: Acceso a datos de máquinas (CRUD, mantenimiento, preparación y estadísticas).

 **Clase MachineMaintenanceManager**

Gestor DAO para el historial de mantenimiento de máquinas.

 `database/repositories/machine/preparation_manager.py`

Nombre del Módulo: machine.preparation_manager Descripción: Acceso a datos de máquinas (CRUD, mantenimiento, preparación y estadísticas).

 **Clase MachinePreparationManager**

Gestor DAO para la configuración de preparación de máquinas.

Métodos Principales:

- `get_prep_info_for_product`: Primer grupo de preparación vinculado al código de producto (si existe). Returns: (grupo_id, maquina_id) o (None, None) si no hay coincidencia.
-

 `database/repositories/machine/repository.py`

Nombre del Módulo: `machine.repository` Descripción: Acceso a datos de máquinas (CRUD, mantenimiento, preparación y estadísticas).

Clase `MachineRepository`

Repositorio para la gestión de máquinas. Implementa el patrón Fachada delegando en DAO Managers especializados.

Métodos Principales:

- `_sync_managers`: Sincroniza la configuración del repositorio con sus gestores internos.
 - `__setattr__`: Propaga cambios en `session_factory` o `safe_execute` a los managers.
 - `get_machine_history`: Wrapper de compatibilidad para `DatabaseManager.get_machine_history()`. Históricamente este método devolvía un diccionario con el historial de la máquina. En la implementación actual, el historial se obtiene desde el manager de mantenimiento.
-

 `database/repositories/machine/stats_manager.py`

Nombre del Módulo: machine.stats_manager Descripción: Acceso a datos de máquinas (CRUD, mantenimiento, preparación y estadísticas).

 **Clase MachineStatsManager**

Gestor DAO para estadísticas relacionadas con máquinas.

 **database/repositories/pila/__init__.py**

Nombre del Módulo: pila Descripción: Persistencia y consultas de pilas, lotes, bitácora y flujo de trabajo de fabricación.

 `database/repositories/pila/pila_base_manager.py`

Nombre del Módulo: `pila.pila_base_manager` Descripción: Persistencia y consultas de pilas, lotes, bitácora y flujo de trabajo de fabricación.

Clase PilaBaseManager

Gestor de utilidades base para el dominio de Pilas (serialización de flujos).

Métodos Principales:

- `convert_indices_to_ids`: Convierte índices relativos en IDs únicos persistentes para el flujo.
 - `convert_ids_to_indices`: Reconvierte IDs persistentes en índices relativos para uso en memoria/UI.
-

 `database/repositories/pila/pila_bitacora_manager.py`

Nombre del Módulo: pila.pila_bitacora_manager Descripción: Persistencia y consultas de pilas, lotes, bitácora y flujo de trabajo de fabricación.

Clase PilaBitacoraManager

Gestor DAO para la bitácora diaria de seguimiento de pilas.

Métodos Principales:

- `add_diario_evento`: Añade o sobrescribe la entrada de un día en la bitácora de la pila.
-

 `database/repositories/pila/pila_crud_manager.py`

Nombre del Módulo: `pila.pila_crud_manager` Descripción: Persistencia y consultas de pilas, lotes, bitácora y flujo de trabajo de fabricación.

 **Clase `PilaCRUDManager`**

Gestor DAO para operaciones CRUD básicas de pilas.

 `database/repositories/pila/pila_workflow_manager.py`

Nombre del Módulo: pila.pila_workflow_manager Descripción: Persistencia y consultas de pilas, lotes, bitácora y flujo de trabajo de fabricación.

 **Clase PilaWorkflowManager**

Gestor DAO para la lógica de negocio y persistencia de flujos de trabajo (Pilas).

 `database/repositories/pila/repository.py`

Nombre del Módulo: pila.repository Descripción: Persistencia y consultas de pilas, lotes, bitácora y flujo de trabajo de fabricación.

 **Clase PilaRepository**

Repositorio para la gestión de pilas de producción. Implementa el patrón Fachada delegando en DAO Managers especializados.

 **database/repositories/preproceso/__init__.py**

Nombre del Módulo: preproceso

Descripción: Concentra datos de configuración o catálogos estáticos: `__all__`, consumidos por la UI y controladores. Integración típica con: `repository`.

Nombre del Módulo: `preproceso.fabricacion_manager`

Descripción: Define protocolos o tipos principales: `FabricacionManager`. Gestor DAO para la entidad `Fabricacion`. Integración típica con: `sqlalchemy, models, core, base`.

Clase `FabricacionManager`

Gestor DAO para la entidad `Fabricacion`. Hereda de `BaseRepository` para aprovechar el patrón de ejecución segura `safe_execute`.

Esta clase implementa la Fase 12C (DTO-First), eliminando el paso de diccionarios o tuplas crudas entre la base de datos y las capas superiores. Todas las operaciones de lectura y escritura se normalizan mediante `FabricacionDTO`.

Métodos Principales:

- `get_all_fabricaciones`: Obtiene todas las fabricaciones registradas en formato DTO. Realiza una consulta ordenada por ID descendente para mostrar primero las más recientes. Cada registro de SQLAlchemy se mapea a un `FabricacionDTO` para garantizar el aislamiento de la capa de persistencia.
 - `get_products_for_fabricacion`: Recupera la lista de productos asociados a una fabricación específica. Utiliza SQL directo a la tabla puente `fabricacion_productos` para optimizar la recuperación. Cada fila se encapsula en un `FabricacionProductoDTO` que incluye el código del producto y la cantidad asignada.
 - `add_product_to_fabricacion`: Añade un producto a una fabricación o actualiza su cantidad si ya existe.
 - `set_products_for_fabricacion`: Sobrescribe completamente la lista de productos de una fabricación. Implementa una operación atómica de 'limpiar y reemplazar': 1. Elimina todas las asociaciones actuales de la fabricación mediante SQL DELETE. 2. Inserta los nuevos productos contenidos en la lista de `FabricacionProductoDTO`. Si ocurre algún error, se realiza un rollback automático para mantener la integridad.
 - `get_fabricacion_by_codigo`: Busca una única Orden de Fabricación por su código exacto.
 - `search_fabricaciones`: Busca fabricaciones por código o descripción.
 - `get_fabricacion_by_id`: Obtiene una fabricación con sus preprocesos.
 - `create_fabricacion_with_preprocesos`: Crea una fabricación y le asigna sus preprocesos.
 - `update_fabricacion_and_preprocesos`: Actualiza los datos de una fabricación y su lista de preprocesos.
 - `delete_fabricacion`: Elimina una fabricación de la base de datos.
 - `get_latest_fabricaciones`: Obtiene las últimas fabricaciones añadidas.
 - `get_preprocesos_by_fabricacion`: Obtiene los preprocesos de una fabricación.
 - `update_fabricacion_preprocesos`: Actualiza solamente la lista de preprocesos de una fabricación.
-

 `database/repositories/preproceso/preproceso_manager.py`

Nombre del Módulo: `preproceso.preproceso_manager`

Descripción: Define protocolos o tipos principales: `PreprocesoManager`. Gestor DAO para la entidad Preproceso. Integración típica con: `sqlalchemy, models, core, base`.

Clase `PreprocesoManager`

Gestor DAO para la entidad Preproceso. Hereda de `BaseRepository` para utilizar `safe_execute`.

Métodos Principales:

- `get_all_preprocesos`: Obtiene todos los preprocesos con sus materiales como DTOs.
 - `get_preproceso_components`: Obtiene los componentes (materiales) asociados a un preproceso específico.
 - `create_preproceso`: Crea un nuevo preproceso y lo asocia con sus materiales.
 - `update_preproceso`: Actualiza un preproceso existente.
 - `delete_preproceso`: Elimina un preproceso y sus relaciones.
-

 **database/repositories/preproceso/repository.py**

Nombre del Módulo: preproceso.repository

Descripción: Define protocolos o tipos principales: `PreprocesoRepository`. Gestiona las operaciones CRUD para los modelos Preproceso y Fabricacion. Integración típica con: `sqlalchemy`, `core`, `base`, `preproceso_manager`, `fabricacion_manager`.

Clase `PreprocesoRepository`

Gestiona las operaciones CRUD para los modelos Preproceso y Fabricacion utilizando exclusivamente SQLAlchemy. Implementa el patrón Fachada delegando en managers especializados.

Métodos Principales:

- `get_all_preprocesos_with_components`: Devuelve preprocesos junto con sus componentes. Se usa como método de compatibilidad para `DatabaseManager.get_all_preprocesos_with_components()`.
 - `create_fabricacion`: Crea una nueva fabricación simple (wrapper para compatibilidad).
-

 **database/repositories/reports/__init__.py**

Nombre del Módulo: reports Descripción: Consultas SQL para informes: órdenes, productos, incidencias, búsqueda y estadísticas.

 `database/repositories/reports/reports_incidences_manager.py`

Nombre del Módulo: reports.reports_incidences_manager Descripción: Consultas SQL para informes: órdenes, productos, incidencias, búsqueda y estadísticas.

 **Clase ReportsIncidencesManager**

Gestor DAO para análisis de incidencias en reportes.

 `database/repositories/reports/reports_orders_manager.py`

Nombre del Módulo: reports.reports_orders_manager Descripción: Consultas SQL para informes: órdenes, productos, incidencias, búsqueda y estadísticas.

 **Clase ReportsOrdersManager**

Gestor DAO para consultas sobre órdenes de fabricación en reportes.

 `database/repositories/reports/reports_products_manager.py`

Nombre del Módulo: reports.reports_products_manager Descripción: Consultas SQL para informes: órdenes, productos, incidencias, búsqueda y estadísticas.

 **Clase ReportsProductsManager**

Gestor DAO para resúmenes de productos en reportes.

 `database/repositories/reports/reports_search_manager.py`

Nombre del Módulo: reports.reports_search_manager Descripción: Consultas SQL para informes: órdenes, productos, incidencias, búsqueda y estadísticas.

 **Clase** ReportsSearchManager

Gestor DAO para búsquedas transversales orientadas a reportes.

 `database/repositories/reports/reports_stats_manager.py`

Nombre del Módulo: reports.reports_stats_manager Descripción: Consultas SQL para informes: órdenes, productos, incidencias, búsqueda y estadísticas.

 **Clase ReportsStatsManager**

Gestor DAO para cálculos estadísticos complejos en reportes.

 `database/repositories/reports/repository.py`

Nombre del Módulo: `reports.repository` Descripción: Consultas SQL para informes: órdenes, productos, incidencias, búsqueda y estadísticas.

Clase `ReportsRepository`

Repositorio especializado en consultas de agregación y análisis para reportes. Implementa el patrón Fachada delegando en DAO Managers especializados.

Métodos Principales:

- `obtener_dashboard_producto`: Obtiene en una sola llamada lógica todos los datos del dashboard de un producto. Centraliza el contrato consumido por UI para reducir round-trips en capas superiores.
-

 `database/repositories/tracking/core_manager.py`

Nombre del Módulo: `tracking.core_manager`

Descripción: Define protocolos o tipos principales: `TrackingCoreManager`. Gestor DAO para la gestión central de trabajos (obtención, creación, finalización). Integración típica con: `datetime`, `sqlalchemy`, `database`, `core`, `base`, `mappers`.

 **Clase `TrackingCoreManager`**

Gestor DAO para la gestión central de trabajos (obtención, creación, finalización).

 `database/repositories/tracking/mappers.py`

Nombre del Módulo: tracking.mappers

Descripción: Define protocolos o tipos principales: `TrackingMapper`. Utilidad para mapear modelos de Trazabilidad a DTOs. Integración típica con: `datetime`, `database`, `core`.

 **Clase `TrackingMapper`**

Utilidad para mapear modelos de Trazabilidad a DTOs.

 `database/repositories/tracking/queries_manager.py`

Nombre del Módulo: `tracking.queries_manager` Descripción: Gestor central de consultas complejas para el sistema de tracking. Incluye exportación de datos y recuperación de fabricaciones asignadas.

Clase `TrackingQueriesManager`

Gestor DAO para consultas complejas y exportación de datos de tracking.

Centraliza la lógica de consultas de solo lectura pesadas y transformaciones a DTOs para la interfaz de trabajador y exportaciones.

Métodos Principales:

- `get_fabricaciones_por_trabajador`: Obtiene las fabricaciones asignadas a un trabajador incluyendo sus productos. Realiza un JOIN entre la tabla de enlace de asignaciones y las fabricaciones, trayendo además los productos vinculados a cada una mediante un outer join. Agrupa los resultados para construir objetos `FabricacionAsignadaDTO`. Args: `trabajador_id`: ID del trabajador cuyas asignaciones se desean recuperar. Returns: Lista de DTOs con la información de las fabricaciones y sus productos.
-

 `database/repositories/tracking/steps_manager.py`

Nombre del Módulo: `tracking.steps_manager`

Descripción: Define protocolos o tipos principales: `TrackingStepsManager`. Gestor DAO para la gestión de pasos de trazabilidad. Integración típica con: `datetime`, `sqlalchemy`, `database`, `core`, `base`, `mappers`.

 **Clase `TrackingStepsManager`**

Gestor DAO para la gestión de pasos de trazabilidad.

 `database/repositories/worker/__init__.py`

Nombre del Módulo: worker Descripción: Datos de trabajadores, anotaciones y repositorio compuesto del subpaquete worker.

 `database/repositories/worker/annotation_manager.py`

Nombre del Módulo: `worker.annotation_manager` Descripción: Datos de trabajadores, anotaciones y repositorio compuesto del subpaquete `worker`.

Clase `WorkerAnnotationManager`

Gestor DAO para la gestión de anotaciones de trabajadores.

Métodos Principales:

- `get_worker_annotations`: Obtiene todas las anotaciones para un trabajador específico.
 - `add_worker_annotation`: Añade una nueva anotación para un trabajador asociada a una pila específica.
-

 `database/repositories/worker/auth_manager.py`

Nombre del Módulo: `auth_manager` Descripción: Comprueba usuario y contraseña de operarios frente a la tabla `trabajadores`.

El nombre de usuario se normaliza (minúsculas, sin espacios al inicio o al final) y se compara con `func.lower(Trabajador.username)` para que el inicio de sesión no dependa de mayúsculas.

Clase `WorkerAuthManager`

Gestor DAO para la gestión de autenticación y credenciales de trabajadores.

Métodos Principales:

- `authenticate_user`: Verifica usuario y contraseña; devuelve `AuthResponseDTO` si el trabajador está activo. El nombre de usuario se normaliza (minúsculas, sin espacios laterales) antes de consultar la base de datos.
 - `update_user_credentials`: Actualiza los datos de login de un trabajador.
 - `update_user_password`: Actualiza únicamente la contraseña de un trabajador.
 -  `_normalize_username`: Usuario de login: sin espacios laterales y en minúsculas para comparar con BD.
-

 `database/repositories/worker/repository.py`

Nombre del Módulo: worker.repository Descripción: Datos de trabajadores, anotaciones y repositorio compuesto del subpaquete worker.

 **Clase** `WorkerRepository`

Repositorio para la gestión de trabajadores. Implementa el patrón Fachada delegando en DAO Managers especializados.

 `database/repositories/worker/worker_manager.py`

Nombre del Módulo: `worker_manager` Descripción: Acceso a datos de trabajadores (listados, detalle, altas y modificaciones).

`WorkerCoreManager` encapsula consultas SQLAlchemy sobre `Trabajador` y devuelve DTOs listos para la capa de servicios o la interfaz de administración.

Clase `WorkerCoreManager`

Gestor DAO para la gestión de datos básicos de trabajadores.

Métodos Principales:

- `get_all_workers`: Obtiene una lista de todos los trabajadores.
 - `get_latest_workers`: Obtiene los últimos trabajadores añadidos.
 - `get_worker_details`: Obtiene los detalles de un trabajador específico por su ID.
 - `add_worker`: Añade un nuevo trabajador o actualiza uno existente.
 - `update_worker`: Actualiza los datos de un trabajador existente.
 - `delete_worker`: Elimina un trabajador de la base de datos.
 -  `_persist_username`: Unifica el nombre de usuario en BD (minúsculas, sin espacios). None = sin cambiar en callers.
-

Capítulo: *features/*

Métrica	Valor
Archivos <code>.py</code> en <code>features/</code>	6
Incluidos en el cuerpo	6
Omitidos (docstrings/reglas)	0
Clases detectadas (AST)	6

```
graph TD
    CTRL[Controllers] -->|usa módulos| FEAT[Features]
    FEAT -->|apoya| CORE[Core/Services]
```


 **features/___init___**.py

Nombre del Módulo: features

Descripción: Capa de funcionalidad vertical del rol trabajador: controlador, sincronización con BD, validaciones, E/S de UI y diálogo de incidencias (consumidos desde `app` y la vista).

 **features/worker_controller.py**

Nombre del Módulo: worker_controller

Descripción: Cerebro de la ventana del rol trabajador: lista de tareas asignadas, inicio y fin de trabajo con QR, incidencias, etiquetas e importación/exportación.

La persistencia y las reglas de datos van a `WorkerDbSync` y al repositorio de trazabilidad; la vista solo emite señales que este controlador atiende.

Clase WorkerController

Controlador PyQt de la ventana de trabajador; enlaza vista, BD y hardware opcional (QR).

Métodos Principales:

- `initialize`: Inicializa los datos y conecta señales.
 - `_load_assigned_fabricaciones`: Consulta fabricaciones asignadas al usuario logueado y actualiza la lista en la vista. Registra en log el `trabajador_id` y el número de filas para diagnóstico.
 - `refresh_data`: Recarga todos los datos.
 - `_handle_task_selected`: Actualiza el estado de la UI al seleccionar una tarea.
 - `_handle_consult_qr`: Maneja la consulta de un código QR.
 - `_handle_start_task`: Maneja el inicio de un paso.
 - `_handle_end_task`: Maneja la finalización de un paso.
 - `_handle_register_incidence`: Maneja incidencias.
-

 `features/worker_controller_io_manager.py`

Nombre del Módulo: `worker_controller_io_manager`

Descripción: Colaborador de `WorkerController` para diálogos Qt, ficheros y flujos de E/S (etiquetas QR, exportaciones, mensajes y apertura de configuración de cámara).

 **Clase `WorkerIOManager`**

Colaborador de composición para operaciones I/O del `WorkerController`.

 `features/worker_db_sync.py`

Nombre del Módulo: `worker_db_sync`

Descripción: Lectura y escritura de datos del rol trabajador frente al repositorio de trazabilidad. Las fabricaciones asignadas se exponen como `WorkerTaskListRowDTO` en `get_assigned_fabricaciones` (no como dicts opacos).

Clase `WorkerDbSync`

Maneja las operaciones de lectura/escritura en base de datos para el trabajador.

Métodos Principales:

- `get_assigned_fabricaciones`: Obtiene y formatea las fabricaciones asignadas a un trabajador para la UI. Solicita al repositorio las asignaciones (como DTOs) y devuelve filas tipadas (`WorkerTaskListRowDTO`) para la lista del trabajador. Args: `trabajador_id`: ID del trabajador logueado. Returns: Lista de DTOs con `id`, `codigo`, `producto_codigo`, etc.
 - `get_active_trabajos`: Obtiene los trabajos actualmente en proceso para el trabajador.
 - `get_paso_activo`: Obtiene el paso actual en proceso del trabajador.
 - `get_trabajo_por_qr`: Busca el historial (`TrabajoLog`) de una unidad por su QR.
 - `get_trabajo_por_id`: Obtiene un `TrabajoLog` por su ID.
 - `iniciar_o_recuperar_trabajo`: Inicia un nuevo registro de unidad o recupera uno existente.
 - `iniciar_paso`: Registra el inicio de un nuevo paso de trabajo.
 - `finalizar_paso`: Registra la finalización de un paso de trabajo.
 - `registrar_incidencia`: Registra una incidencia asociada a un trabajo.
 - `get_estadisticas`: Obtiene estadísticas de rendimiento del trabajador.
 - `get_data_for_export`: Obtiene datos nuevos para exportación.
-

 **features/worker_incidence_dialog.py**

Nombre del Módulo: worker_incidence_dialog

Descripción: Diálogo PyQt para que el operario registre una incidencia (tipo, descripción y fotos).

Clase IncidenceDialog

Diálogo modal para que el trabajador registre una incidencia, incluyendo título, descripción y la posibilidad de adjuntar fotos.

 `features/worker_validation_service.py`

Nombre del Módulo: `worker_validation_service`

Descripción: Validaciones de negocio para el flujo trabajador: parseo de QR, coincidencia de producto y coherencia de datos antes de persistir (sin depender de widgets).

Clase `WorkerValidationService`

Gestiona la lógica de validación para procesos de trabajadores. Desacopla la lógica de decisión del controlador UI.

Métodos Principales:

- `validate_qr_data`: Valida el formato de un código QR. Returns: Tuple (`is_valid`, `parsed_data`, `error_message`)
 - `validate_product_match`: Verifica si el producto del QR coincide con el de la tarea seleccionada.
 - `is_step_duplicated`: Comprueba si un paso ya ha sido completado para una unidad específica.
-

Capítulo: `ui/`

Métrica	Valor
Archivos <code>.py</code> en <code>ui/</code>	107
Incluidos en el cuerpo	107
Omitidos (docstrings/reglas)	0
Clases detectadas (AST)	104

```
graph TD
    UI[UI (PyQt6)] -->|señales/slots| CTRL[Controllers]
    CTRL -->|delegación| CORE[Core/Services]
```


 `ui/__init__.py`

Nombre del Módulo: ui

Descripción: Funciones y datos de apoyo del paquete; conviene enlazar qué controlador o servicio las consume y qué estructuras devuelven (ver firmas al inicio del archivo).

 `ui/main_window.py`

Nombre del Módulo: `ui.main_window`

Descripción: Define protocolos o tipos principales: `MainView`. Vista principal de la aplicación (la ventana).
Integración típica con: `sys, os, PyQt6, ui, core`.

Clase `MainView`

Vista principal de la aplicación (la ventana).

Métodos Principales:

- `__init__`: Inicializa la ventana principal y sus componentes de UI.
 - `init_ui`: Inicializa todos los componentes de la interfaz.
 - `_init_pages`: Instancia y registra todas las páginas de la aplicación.
 - `set_controller`: Asigna el controlador a esta vista y a sus widgets hijos.
 - `_create_main_layout`: Configura el layout principal con `NavPanel`, `Header` y `StackedWidget`.
 - `switch_page`: Cambia la página visible en el widget apilado.
 - `get_page`: Obtiene una página específica por nombre.
 - `get_products_tab`: Retorna el widget de gestión de productos.
 - `get_fabrications_tab`: Retorna el widget de gestión de fabricaciones.
 - `pages`: Interfaz de compatibilidad para el reporte de páginas.
 - `buttons`: Compatibilidad legacy: expone los botones del panel de navegación.
 - `_forzar_auto_ajuste`: Fuerza un recalcular dinámico del factor de escala y repinta.
 - `show_message`: Muestra un diálogo de mensaje al usuario.
 - `show_confirmation_dialog`: Muestra un diálogo de confirmación (Sí/No).
 - `run_simulation_and_display`: Legacy helper para mostrar resultados de simulación.
 - `display_simulation_results`: Envía resultados al widget de cálculo.
 - `closeEvent`: Maneja el cierre de la aplicación con backup automático.
-

 `ui/startup_screen.py`

Nombre del Módulo: `startup_screen` Descripción: Ventana de arranque que verifica el estado del sistema antes de mostrar la aplicación principal. Diseñada para usuarios no técnicos con mensajes contextuales claros y opción de exportar informe.

Clase `StartupScreen`

Diálogo modal de arranque que verifica BD y ejecuta tests unitarios. Diseñado para usuarios no técnicos con mensajes contextuales.

Métodos Principales:

- `_start_auto_advance`: Inicia cuenta regresiva para entrar automáticamente.
 - `_tick_auto`: Tick de la cuenta regresiva.
 - `_generate_report_text`: Genera el texto completo del informe para exportación.
 - `closeEvent`: Limpia recursos al cerrar.
-

 `ui/startup_screen_constants.py`

Nombre del Módulo: `ui.startup_screen_constants`

Descripción: Constantes usadas por la ventana de arranque (StartupScreen). Extraídas para reducir LOC del monolito y facilitar tests.

 `ui/startup_screen_report.py`

Nombre del Módulo: `ui.startup_screen_report`

Descripción: Generación de texto del informe de verificación del sistema (StartupScreen). Lógica pura sin Qt; testeable con HealthReport mock o real.

-  `generate_startup_report_text`: Genera el texto completo del informe para exportación. Args: `report`: Informe de salud. Si es None, devuelve "Sin datos disponibles". `log_path`: Ruta al archivo de log. Si None, usa `<writable_root>/logs/EvolucionTiempos.log`. Returns: Texto formateado del informe.
-

 `ui/startup_screen_ui.py`

Nombre del Módulo: `ui.startup_screen_ui`

Descripción: Define protocolos o tipos principales: `StartupSectionWidgets`. Referencias a frame y layouts de una seccion del StartupScreen. Integración típica con: `__future__`, `PyQt6`.

Clase `StartupSectionWidgets`

Referencias a frame y layouts de una seccion del StartupScreen.

-  `make_section`: Crea una sección con título, descripción y un layout de contenido.
 -  `clear_layout`: Elimina widgets hijos de un layout.
 -  `build_startup_ui`: Construye la UI del StartupScreen y asigna atributos esperados.
 -  `render_db_report`: Rellena la sección de BD a partir de un `HealthReport`.
-

`ui/dialogs/__init__.py`

Nombre del Módulo: `ui.dialogs` Descripción: Paquete de diálogos PyQt6 (fabricación, flujo, productos, preparación y utilidades).

Reexporta las clases públicas más usadas para simplificar `from ui.dialogs import ...`

El grafo visual de flujo de producción vive en `ui.widgets.production_flow` (`ProductionFlowCanvas`, `FlowCardWidget`); no se reexportan widgets de canvas aquí.

 `ui/dialogs/backup_restore_dialog.py`

Nombre del Módulo: `backup_restore_dialog`

Descripción: Define protocolos o tipos principales: `BackupRestoreDialog`. Diálogo para gestionar la restauración de backups. Integración típica con: `pathlib`, `datetime`, `PyQt6`, `core`.

Clase `BackupRestoreDialog`

Diálogo para gestionar la restauración de backups.

Métodos Principales:

- `init_ui`: Inicializa la interfaz del diálogo.
 - `load_backups`: Carga la lista de backups disponibles.
-

 `ui/dialogs/connection_dialog.py`

Nombre del Módulo: `connection_dialog` Descripción: Diálogo de arranque para elegir modo de base de datos local (SQLite) o servidor (PostgreSQL), antes de abrir la aplicación principal.

 **Clase** `ConnectionDialog`

Ventana modal que captura la preferencia de conexión a datos (fichero local vs motor servidor).

Métodos Principales:

- `get_selection`: Returns a tuple: (mode_string, remember_bool) mode_string: 'sqlite' or 'postgresql'
-

 `ui/dialogs/tracking_dialogs.py`

Nombre del Módulo: `tracking_dialogs` Descripción: Diálogos auxiliares de trazabilidad y arranque de sesión de producción (orden y cantidad).

Formularios simples reutilizados desde controladores de fabricación o seguimiento.

 **Clase** `OrderSetupDialog`

Solicita número de orden de fabricación (O.F.) y cantidad total al iniciar una serie de producción.

`ui/dialogs/utility_dialogs.py`

Nombre del Módulo: `utility_dialogs` Descripción: Diálogos PyQt6 reutilizables (descansos, login, contraseña, sincronización de BD, selección de hojas Excel, etc.). Incluye `SyncDialog` para la fusión selectiva contra una copia SQLite y mapeo de títulos de pestaña legibles.

Clase `AddBreakDialog`

Diálogo simple para añadir un nuevo descanso.

Métodos Principales:

- `get_times`: Devuelve las horas seleccionadas en formato de texto.

Clase `LoginDialog`

Diálogo para la autenticación de usuarios.

Métodos Principales:

- `get_credentials`: Devuelve el usuario y la contraseña introducidos. El nombre de usuario se normaliza con `strip` (espacios accidentales). La contraseña no se recorta: debe coincidir byte a byte con la guardada al crear el usuario.

Clase `ChangePasswordDialog`

Diálogo para cambiar la contraseña de un usuario.

Métodos Principales:

- `get_passwords`: Devuelve las contraseñas introducidas.

Clase `SyncDialog`

Presenta las diferencias detectadas por `SyncService.compare_databases` en pestañas por tabla.

Por defecto todas las filas quedan marcadas para importar; el usuario puede desmarcar filas o pestañas enteras vía «Desmarcar todo». Al aceptar, `get_selected_changes` construye el DTO parcial para `apply_sync_changes`.

Métodos Principales:

- `__init__`: Args: `comparison`: Resultado completo de la comparación local vs copia. `parent`: Ventana padre opcional.
- `_populate_tabs`: Genera una pestaña con `QTableWidget` por cada `SyncTableDifferencesDTO` no vacío.
- `_iter_sync_tables`: Recorre cada pestaña y su `QTableWidget` (búsqueda recursiva por si el layout oculta el hijo directo).
- `_check_all_rows`: Marca la columna «Importar» en todas las filas de todas las pestañas.
- `_uncheck_all_rows`: Desmarca todas las filas importables (útil para seleccionar solo un subconjunto manual).
- `get_selected_changes`: Recopila filas con checkbox marcado y devuelve un DTO listo para `apply_sync_changes`. Returns: `DatabaseComparisonDTO` que puede estar vacío si no hubo selección.

Clase `SeleccionarHojasExcelDialog`

Diálogo para que el usuario elija qué hojas incluir en el informe Excel.

Métodos Principales:

- `get_opciones`: Devuelve un diccionario con las opciones seleccionadas.

Clase `MultiWorkerSelectionDialog`

Diálogo para seleccionar múltiples trabajadores de una lista.

Métodos Principales:

- `get_selected_workers`: Devuelve una lista con los nombres de los trabajadores seleccionados.
 -  `_sync_dialog_tab_title`: Convierte el nombre técnico de tabla (p. ej. `producto_material_link`) en título de pestaña. Args: `table_name`: Identificador igual al usado en `SyncTableDifferencesDTO.table_name`. Returns: Cadena para `QTabWidget.addTab`; el nombre canónico se guarda aparte en `sync_table_name`.
-

 `ui/dialogs/effects/__init__.py`

Nombre del Módulo: `ui.dialogs.effects` Descripción: Efectos visuales Qt (resplandores y progreso) usados en canvas y simulación.

 `ui/dialogs/effects/golden_glow.py`

Nombre del Módulo: `ui.dialogs.effects.golden_glow` Descripción: Efecto visual o animación para el canvas de flujo o simulación (pintado con `QTimer`).

Clase `GoldenGlowEffect`

Widget que dibuja un círculo dorado giratorio alrededor de una tarjeta para indicar que es una tarea de inicio de ciclo.

Rendimiento Visual (UI y Concurrencia): Dado que este widget pinta gradientes cónicos (`QConicalGradient`) constantemente para simular un hilo de luz (efecto neón girando a 60 FPS), su arquitectura aísla el dibujo delegando la iteración al `EventLoop` de PyQt6. En lugar de un `while` bloqueante, se apoya en un `QTimer` que dispara señales intermitentes de `update()` motivando a `paintEvent` sólo a demanda, minimizando la huella de CPU. Usa `EventFilters` en sus padres para recálculos morfológicos "Lazy" optimizados.

Métodos Principales:

- `eventFilter`: Filtra eventos de la tarjeta padre y del canvas para actualizar la geometría cuando sea necesario.
 - `_update_geometry`: Actualiza posición y tamaño para rodear la tarjeta. CORREGIDO: Usa `mapTo()` para obtener las coordenadas correctas relativas al canvas.
 - `paintEvent`: Dibuja un efecto neón con luz circulante continua, sin puntos discretos.
 - `stop_animation`: Detiene la animación y limpia recursos.
-

 `ui/dialogs/effects/green_cycle.py`

Nombre del Módulo: `ui.dialogs.effects.green_cycle` Descripción: Efecto visual o animación para el canvas de flujo o simulación (pintado con QTimer).

 **Clase** `GreenCycleEffect`

Widget que dibuja un aro verde con efecto neón para tareas intermedias del ciclo.

Métodos Principales:

- `paintEvent`: Efecto neón verde ESTÁTICO (sin animación).
-

 `ui/dialogs/effects/mixed_gold_green.py`

Nombre del Módulo: `ui.dialogs.effects.mixed_gold_green` Descripción: Efecto visual o animación para el canvas de flujo o simulación (pintado con QTimer).

 **Clase `MixedGoldGreenEffect`**

Widget que dibuja un aro con efecto mixto dorado-verde para tareas finales de ciclo.

Métodos Principales:

- `paintEvent`: Efecto neón mixto ESTÁTICO (sin animación).
-

 `ui/dialogs/effects/processing_glow.py`

Nombre del Módulo: `ui.dialogs.effects.processing_glow` Descripción: Efecto visual o animación para el canvas de flujo o simulación (pintado con `QTimer`).

Clase `ProcessingGlowEffect`

Widget que dibuja un círculo naranja pulsante alrededor de una tarjeta para indicar que está siendo procesada por la simulación.

Rendimiento Visual y Optimización Matemática: El efecto de respiración (pulso) se basa en la interpolación lineal algorítmica del canal alfa de colores directos sobre capas progresivamente concéntricas. Para salvaguardar el Frame-Rate durante simulaciones pesadas (en threads remotos), este componente permanece aislado en el Thread Principal, siendo inerte a clicks, gestionando la opacidad en una variable `pulse_value` que repinta (`drawEllipse`) a golpe de latidos guiados por el `QEventLoop` del sistema, evitando atascos.

Métodos Principales:

- `_update_geometry`: Actualiza posición y tamaño para rodear la tarjeta.
 - `paintEvent`: Dibuja el círculo naranja pulsante con efecto neón.
 - `stop_animation`: Detiene la animación del pulso.
-

 `ui/dialogs/effects/progress.py`

Nombre del Módulo: `ui.dialogs.effects.progress` Descripción: Efecto visual o animación para el canvas de flujo o simulación (pintado con `QTimer`).

Clase `SimulationProgressEffect`

Widget que dibuja un aro azulado grisáceo giratorio con efecto neón para indicar que una tarjeta está siendo procesada por la simulación.

Métodos Principales:

- `eventFilter`: Filtra eventos para actualizar geometría cuando sea necesario.
 - `_update_geometry`: Actualiza posición y tamaño para rodear la tarjeta.
 - `paintEvent`: Dibuja un efecto neón azulado con luz circulante continua.
-

 `ui/dialogs/fabrication/__init__.py`

Nombre del Módulo: `ui.dialogs.fabrication` Descripción: Diálogos de fabricación: crear orden, seleccionar preprocesos/productos, bitácora y asignaciones.

Reexporta las clases más usadas para imports desde `ui.dialogs` o controladores.

 `ui/dialogs/fabrication/assignment_dialogs.py`

Nombre del Módulo: `ui.dialogs.fabrication.assignment_dialogs` Descripción: Diálogo o presentador de fabricación: órdenes, preprocesos, productos y persistencia de pilas.

Clase `AssignPreprocesosDialog`

Diálogo para asignar preprocesos a fabricaciones desde el menú de Preprocesos.

Métodos Principales:

- `load_fabricaciones`: Carga todas las fabricaciones disponibles.
 - `on_fabricacion_selected`: Maneja la selección de una fabricación.
 - `load_current_preprocesos`: Carga los preprocesos actuales de la fabricación.
 - `modify_selected_fabricacion`: Abre el diálogo para modificar preprocesos de la fabricación seleccionada.
-

 `ui/dialogs/fabrication/bitacora_dialog.py`

Nombre del Módulo: `ui.dialogs.fabrication.bitacora_dialog`

Descripción: Diálogo de bitácora de pilas (`FabricacionBitacoraDialog`). Resolución de datos (orden): `pila_service` inyectado (p. ej. desde `pila_manager`) → `resolve_pila_service` (DI → `pila_controller.pila_service` → `model.pila_service`) → `model.planning_facade` (misma API: `get_diario_bitacora`, `add_diario_evento`). No se usan delegadores eliminados de `AppModel` para bitácora.

Clase `BitacoraEntryDTO`

DTO de entrada del diario de bitácora (plan/realizado/notas).

Clase `FabricacionBitacoraDialog`

Diario de bitácora por pila (calendario + entradas plan/realizado/notas).

Persistencia vía `_bitacora_backend` (`PilaService` o `PlanningFacade`), no vía fachada `AppModel`.

Métodos Principales:

- `_load_and_process_data`: Carga los datos iniciales, formatea el calendario y selecciona el día actual.
 - `_highlight_work_days`: Resalta en el calendario los días con trabajo planificado.
 - `_update_history_table`: Rellena la tabla del historial con las entradas guardadas.
 - `_get_planned_work_for_day`: Genera un resumen del trabajo planificado para una fecha específica.
 - `_add_diario_evento`: Guarda o actualiza la entrada para la fecha seleccionada.
-

 `ui/dialogs/fabrication/create_dialog.py`

Nombre del Módulo: `ui.dialogs.fabrication.create_dialog` Descripción: Diálogo o presentador de fabricación: órdenes, preprocesos, productos y persistencia de pilas.

Clase `CreateFabricacionDialog`

Diálogo para crear fabricaciones: preprocesos, productos con cantidades y validación del código.

La recogida de datos y la validación se delegan en `CreateFabricacionPresenter`; al aceptar, el resultado se expresa como `FabricacionDTO`.

Métodos Principales:

- `_setup_preprocesos_tab`: Configura la pestaña de Preprocesos.
 - `_setup_productos_tab`: Configura la pestaña de Productos.
 - `load_initial_data`: Carga los datos iniciales en las listas.
 - `get_fabricacion_data`: Consolida y retorna el estado actual del formulario como un objeto DTO. Este método delega en el Presenter la creación de los DTOs de productos y la cabecera de fabricación, garantizando que los datos estén tipados y normalizados para su envío al servicio de dominio.
-

 `ui/dialogs/fabrication/create_presenter.py`

Nombre del Módulo: `ui.dialogs.fabrication.create_presenter` Descripción: Diálogo o presentador de fabricación: órdenes, preprocesos, productos y persistencia de pilas.

Clase `CreateFabricacionPresenter`

Presenter para la creación de Fabricaciones, encargado de la gestión de estado.

Responsabilidades:

- Filtrar y ordenar listas de preprocesos y productos disponibles.
- Mantener el estado de las asignaciones temporales (memoria de sesión).
- Realizar la validación cruzada de datos (ej: código no vacío).
- Mapear el estado interno a objetos `FabricacionDTO` y `FabricacionProductoDTO`.

Métodos Principales:

- `get_products_data`: Retorna la lista de productos configurada como DTOs.
-

 `ui/dialogs/fabrication/dialog_dependencies.py`

Nombre del Módulo: `ui.dialogs.fabrication.dialog_dependencies`

Descripción: Resolución centralizada de servicios para diálogos de fabricación. Prioridad fija (testeable vía `resolve_fabricacion_service / resolve_pila_service`):

- **FabricacionService:** DI registrado → `product_controller.fabricacion_service` → `model.fabricacion_service`.
- **PilaService:** DI registrado → `pila_controller.pila_service` → `model.pila_service`.

La bitácora y `FlowActionHandler` reutilizan `resolve_pila_service`; si sigue siendo `None`, la UI puede usar `model.planning_facade` (no métodos de bitácora en `AppModel`).

-  `resolve_fabricacion_service`: Resuelve `FabricacionService` para UI sin duplicar reglas en cada diálogo.
 -  `resolve_pila_service`: Resuelve `PilaService` para UI (bitácora, flujo de producción, etc.).
-

 `ui/dialogs/fabrication/input_dialogs.py`

Nombre del Módulo: `ui.dialogs.fabrication.input_dialogs` Descripción: Diálogo o presentador de fabricación: órdenes, preprocesos, productos y persistencia de pilas.

Clase `GetLoteInstanceParametersDialog`

Diálogo para solicitar los parámetros de una instancia de Lote al añadirla a la Pila.

Métodos Principales:

- `get_data`: Devuelve un objeto `LoteInstanceParametersDTO` con los parámetros introducidos por el usuario.

Clase `GetOptimizationParametersDialog`

Diálogo para solicitar fecha de inicio, fecha de fin y unidades para la optimización.

Clase `GetUnitsDialog`

Diálogo simple para solicitar el número de unidades a producir.

 `ui/dialogs/fabrication/persistence_dialogs.py`

Nombre del Módulo: `ui.dialogs.fabrication.persistence_dialogs` Descripción: Diálogo o presentador de fabricación: órdenes, preprocesos, productos y persistencia de pilas.

Clase `SavePilaDialog`

Diálogo para pedir nombre y descripción al guardar una pila.

Métodos Principales:

- `get_data`: Retorna (nombre, descripcion).

Clase `LoadPilaDialog`

Diálogo para mostrar y seleccionar pilas guardadas.

Métodos Principales:

- `get_selected_id`: Devuelve el ID seleccionado, ya sea para cargar o eliminar.
-

 `ui/dialogs/fabrication/products_dialog.py`

Nombre del Módulo: `ui.dialogs.fabrication.products_dialog` Descripción: Diálogo o presentador de fabricación: órdenes, preprocesos, productos y persistencia de pilas.

Clase `ProductsSelectionDialog`

Diálogo para asignar/editar productos de una fabricación existente. Permite añadir, quitar y modificar cantidades.

Métodos Principales:

- `get_products_data`: Retorna la lista de productos configurada como DTOs.
-

 `ui/dialogs/fabrication/selection_dialogs.py`

Nombre del Módulo: `ui.dialogs.fabrication.selection_dialogs` Descripción: Diálogo o presentador de fabricación: órdenes, preprocesos, productos y persistencia de pilas.

Clase `PreprocesosSelectionDialog`

Diálogo para seleccionar qué preprocesos asignar a una fabricación.

Métodos Principales:

- `get_selected_preprocesos`: Retorna lista de IDs de preprocesos seleccionados.

Clase `PreprocesosForCalculationDialog`

Diálogo para mostrar y seleccionar preprocesos disponibles para añadir al cálculo de tiempos de una fabricación.

Métodos Principales:

- `select_all`: Selecciona todos los preprocesos.
 - `clear_selection`: Limpia la selección.
 - `get_selected_preprocesos`: Retorna lista de preprocesos seleccionados. Returns: list: Lista de DTOs con datos de preprocesos
-

 `ui/dialogs/fabrication/ui_dialog_protocols.py`

Nombre del Módulo: `ui.dialogs.fabrication.ui_dialog_protocols`

Descripción: Protocolos mínimos para comandos de aplicación usados desde diálogos de fabricación.

Clase `OpensFabricacionPreprocesos`

Abre la gestión de preprocesos para una fabricación (p. ej. `AppController`).

Clase `ShowsUserMessage`

Muestra mensajes al usuario (alineado con `IView.show_message / MainView`).

 `ui/dialogs/prep/__init__.py`

Nombre del Módulo: ui.dialogs.prep Descripción: Diálogos de grupos y pasos de preparación de máquinas y preprocesos asociados.

 `ui/dialogs/prep/prep_groups_dialog.py`

Nombre del Módulo: `ui.dialogs.prep.prep_groups_dialog`

Descripción: Define protocolos o tipos principales: `PrepGroupsDialog`. Diálogo para gestionar los Grupos de Preparación de una máquina. Integración típica con: `__future__`, `os`, `datetime`, `core`, `PyQt6`, `prep_steps_dialog`.

Clase `PrepGroupsDialog`

Diálogo para gestionar los Grupos de Preparación de una máquina. Permite organizar fases de preparación en grupos lógicos.

Métodos Principales:

- `__init__`: Inicializa el diálogo de grupos de preparación. Args: `machine_id`: ID de la máquina. `machine_name`: Nombre de la máquina. `preparation_service`: Servicio de grupos y pasos de preparación. `product_service`: Catálogo de productos para el combo. `view`: Vista para mensajes y confirmaciones. `parent`: Widget padre.
 - `_toggle_form`: Habilita o deshabilita los campos del formulario.
 - `_load_groups`: Carga los grupos de preparación de la máquina en la lista.
 - `_add_group`: Prepara el formulario para añadir un nuevo grupo.
 - `_save_group`: Guarda o actualiza el grupo actual.
 - `_delete_group`: Elimina el grupo seleccionado.
 - `_manage_steps`: Abre el diálogo de pasos para el grupo seleccionado.
-

 `ui/dialogs/prep/prep_steps_dialog.py`

Nombre del Módulo: `ui.dialogs.prep.prep_steps_dialog`

Descripción: Define protocolos o tipos principales: `PrepStepsDialog`. Diálogo para gestionar los pasos individuales de un grupo de preparación. Integración típica con: `__future__`, `os`, `datetime`, `core`, `PyQt6`.

Clase `PrepStepsDialog`

Diálogo para gestionar los pasos individuales de un grupo de preparación. Permite visualizar, añadir, actualizar y eliminar pasos.

Métodos Principales:

- `__init__`: Inicializa el diálogo de pasos de preparación. Args: `group_id`: ID del grupo de preparación. `group_name`: Nombre del grupo para el título. `preparation_service`: Servicio de grupos y pasos de preparación. `view`: Vista para mensajes y confirmaciones. `parent`: Widget padre.
 - `_load_steps`: Carga los pasos del grupo y los muestra en la tabla.
 - `_clear_form`: Limpia el formulario para añadir un nuevo paso.
 - `_add_or_update_step`: Añade un nuevo paso o actualiza el seleccionado en el grupo.
 - `_delete_step`: Elimina el paso seleccionado.
-

 `ui/dialogs/prep/preproceso_dialog.py`

Nombre del Módulo: `ui.dialogs.prep.preproceso_dialog`

Descripción: Define protocolos o tipos principales: `PreprocesoDialog`. Diálogo para crear o editar un Preproceso, permitiendo la asignación. Integración típica con: `__future__`, `os`, `datetime`, `core`, `PyQt6`.

Clase `PreprocesoDialog`

Diálogo para crear o editar un Preproceso, permitiendo la asignación de materiales (componentes).

Métodos Principales:

- `__init__`: Inicializa el diálogo de preproceso. Args: `preproceso_existente`: Datos del preproceso a editar (opcional). `all_materials`: Lista de todos los materiales disponibles. `material_port`: Controlador de producto / materiales (p. ej. `ProductController`). `parent`: Widget padre.
 - `setup_ui`: Configura la interfaz gráfica del diálogo.
 - `_populate_materials_list`: Rellena la lista con los materiales disponibles. Marca como seleccionados aquellos que ya pertenecen al preproceso.
 - `_refresh_data`: Recarga los materiales desde el modelo a través del controlador y actualiza la visualización de la lista.
 - `_update_assigned_ids_from_selection`: Sincroniza el conjunto interno de IDs asignados con los elementos actualmente seleccionados en el widget de lista.
 - `get_data`: Recolecta los datos del formulario y los devuelve como un diccionario. Returns: Diccionario con datos del preproceso o `None` si la validación falla.
-

 `ui/dialogs/product/__init__.py`

Nombre del Módulo: `ui.dialogs.product` Descripción: Diálogos del catálogo de productos (detalle, iteraciones, subfabricaciones, procesos mecánicos).

 `ui/dialogs/product/add_iteration_dialog.py`

Nombre del Módulo: `ui.dialogs.product.add_iteration_dialog`

Descripción: Diálogo para añadir iteración de producto. `AddIterationFormData` concentra los campos del formulario; el widget de iteraciones pasa `asdict(form)` al controlador para mantener la firma histórica basada en dict.

 **Clase `AddIterationFormData`**

Valores del formulario de nueva iteración (frontera tipada frente a dict opaco).

 **Clase `AddIterationDialog`**

Diálogo para añadir una nueva iteración con todos los campos requeridos.

 `ui/dialogs/product/bom_import_preview_dialog.py`

Nombre del Módulo: `bom_import_preview_dialog` Descripción: Diálogo PyQt6 de supervisión previa a importar un BOM A3RP. Muestra un árbol (`QTreeWidget`) con columna «Importar» (desmarcada por defecto) y desplegable de `BOMImportRole` habilitado solo si la fila está marcada. Valida que exista exactamente un producto final antes de `accept`.

Clase `BOMImportPreviewDialog`

Permite al usuario decidir qué ramas del BOM se persisten y con qué rol semántico.

Tras `accept`, `get_supervised_tree` devuelve el mismo `BOMNodeDTO` raíz con `import_selected` e `import_role` rellenos para el servicio de importación.

Métodos Principales:

- `validate_row_selections_from_states`: Valida la selección sin depender de Qt (testable en unit puro). Args: `states`: Lista de (`marcado`, `rol`) por fila. `rol` es `None` si no aplica o equivale al placeholder. Returns: Mensaje de error o `None` si la selección es válida.
 - `__init__`: Args: `root_node`: Árbol BOM (se muta al confirmar con `get_supervised_tree`). `parent`: Ventana padre opcional de Qt.
 - `_make_role_combo`: Crea el `QComboBox` de rol con ítem placeholder y estilo de error vía señal.
 - `_item_for_combo`: Localiza la fila del árbol que contiene el combo dado (recorrido en profundidad).
 - `_populate_tree`: Crea un `QTreeWidgetItem` por nodo DTO, checkbox desmarcado y combo asociado.
 - `_iter_items`: Recorre en preorden el subárbol colgando de `item` (generador).
 - `_validation_error_message`: Compone la lista de estados de fila y delega en `validate_row_selections_from_states`.
 - `get_supervised_tree`: Recorre el árbol de la UI y escribe `import_selected` / `import_role` en cada DTO. Returns: La misma instancia `root_node` mutada para pasar a `BOMImportService.import_bom_tree`.
 - `_sync_node_from_item`: Propaga checkbox y combo del ítem al `BOMNodeDTO` almacenado en `UserRole`.
-

 `ui/dialogs/product/procesos_mecanicos_dialog.py`

Nombre del Módulo: `ui.dialogs.product.procesos_mecanicos_dialog` Descripción: Formulario PyQt6 del catálogo de productos (detalle, iteraciones, subfabricaciones o procesos).

Clase `ProcesosMecanicosDialog`

Diálogo para gestionar los procesos mecánicos de un producto. Similar a `SubfabricacionesDialog` pero sin máquinas.

Métodos Principales:

- `_normalize_procesos`: Normaliza `current_procesos` a DTOs para evitar dicts en la UI (Fase 12C). Acepta listas de dicts legacy por compatibilidad, pero internamente usa `ProcesoMecanicoDTO` para que el widget no dependa de claves mágicas.

Clase `AddProcesoMecanicoDialog`

Diálogo para añadir un nuevo proceso mecánico.

 `ui/dialogs/product/product_details_dialog.py`

Nombre del Módulo: `ui.dialogs.product.product_details_dialog`

Descripción: Define protocolos o tipos principales: `ProductDetailsDialog`. Diálogo que utiliza sub-widgets para gestionar componentes e iteraciones. Integración típica con: `__future__`, `PyQt6`, `ui`.

Clase `ProductDetailsDialog`

Diálogo que utiliza sub-widgets para gestionar componentes e iteraciones.

Recibe `ProductController` (no `AppController`): materiales e iteraciones delegan en ese controlador y en la vista principal como padre para diálogos Qt.

Métodos Principales:

- `load_all_data`: Carga los datos en ambos sub-widgets.
-

 `ui/dialogs/product/subfabricaciones_dialog.py`

Nombre del Módulo: `ui.dialogs.product.subfabricaciones_dialog` Descripción: Formulario PyQt6 del catálogo de productos (detalle, iteraciones, subfabricaciones o procesos).

Clase `SubfabricacionesDialog`

Diálogo para gestionar (CRUD) la lista de sub-fabricaciones de un producto.

Métodos Principales:

- `accept`: Sobrescribe el método `accept` para avisar si hay datos en el formulario sin guardar.
 -  `_make_combo_readable`: Ensancha el combo y la lista desplegable para textos largos (máquinas, etc.).
-

 `ui/dialogs/production_flow/__init__.py`

Nombre del Módulo: `ui.dialogs.production_flow` Descripción: Diálogos de definición y simulación de flujo de producción (canvas, reglas, cantidades).

Punto de entrada para `DefineProductionFlowDialog`, `EnhancedProductionFlowDialog` y auxiliares.

 `ui/dialogs/production_flow/common_dialogs.py`

Nombre del Módulo: `ui.dialogs.production_flow.common_dialogs`

Descripción: Re-exporta diálogos comunes del flujo de producción y tipos Qt usados conjuntamente.

 `ui/dialogs/production_flow/cycle_end_config_dialog.py`

Nombre del Módulo: `ui.dialogs.production_flow.cycle_end_config_dialog`

Descripción: Diálogo modal para marcar el fin de ciclo de una tarea y elegir a qué tarea de inicio de ciclo debe volver el flujo. Lee el estado vía `flow_task_entry_*` y `canvas_task_display_name` sobre la lista `canvas_tasks` del flujo mejorado.

 **Clase** `CycleEndConfigDialog`

Diálogo para configurar el fin de ciclo de una tarea. Permite seleccionar a qué tarea de inicio de ciclo regresar.

 `ui/dialogs/production_flow/define_flow_dialog.py`

Nombre del Módulo: `ui.dialogs.production_flow.define_flow_dialog`

Descripción: Diálogo «Definir / editar pila de producción» (árbol de tareas, flujo y guardado). Construye `DefineFlowPresenter` solo con servicios de dominio (`MachineService`, `PreparationService`, `FabricacionService`) resueltos por DI o extraídos de `hub.model`; el presenter no mantiene referencia a `AppModel`.

Clase `DefineProductionFlowDialog`

Diálogo orquestador para definir la secuencia de tareas, dependencias y trabajadores.

Métodos Principales:

- `_init_components`: Inicializa los sub-componentes y managers.
 - `_initial_load`: Realiza la carga inicial de datos.
 - `_add_or_update_step`: Añade o actualiza un paso en la pila.
 - `_update_flow_display`: Actualiza el panel de visualización del flujo.
 - `_reset_form`: Limpia el formulario y resincroniza.
 - `_edit_step`: Prepara el formulario para editar un paso.
 - `_toggle_start_condition`: Coordina la habilitación de condiciones de inicio.
 - `_update_previous_task_menu`: Pueba el menú de dependencias.
 - `_delete_step`: Elimina un paso tras confirmación.
 - `_assign_worker_to_group`: Asigna trabajadores a un grupo secuencial.
 - `_group_selected_steps`: Agrupa los pasos seleccionados.
 - `flow_item_widgets`: Capa de compatibilidad para la suite de tests.
-

 `ui/dialogs/production_flow/define_flow_presenter.py`

Nombre del Módulo: `ui.dialogs.production_flow.define_flow_presenter`

Descripción: Presenter del diálogo «Definir pila de producción» (lógica pura, sin Qt). Consultas de dominio solo a través de `machine_service`, `preparation_service` y `fabricacion_service` inyectados o resueltos en `DefineProductionFlowDialog` (DI o atributos en `AppModel`). No recibe ni usa `AppModel` como fachada.

Clase `DefineFlowPresenter`

Presenter/Lógica para aislar el ensamblado de datos y configuraciones de la vista (`DefineProductionFlowDialog`).

Métodos Principales:

- `prepare_task_data`: Organiza la lista plana de tareas primarias en DTOs agrupados por producto.
 - `set_production_flow`: Inicializa el flujo de producción convirtiéndolo a DTOs si es necesario.
 - `get_production_flow`: Retorna el flujo de producción actual.
 - `add_step`: Añade un nuevo paso al flujo.
 - `update_step`: Actualiza un paso existente.
 - `delete_step`: Elimina un paso y limpia dependencias rotas.
 - `get_step`: Obtiene un paso por su índice.
 - `get_machines_for_task`: Obtiene las máquinas compatibles con el tipo de proceso de la tarea.
 - `get_prep_info`: Obtiene información de preparación por defecto para un producto.
 - `get_prep_steps_for_machine`: Obtiene todas las fases de preparación asociadas a una máquina.
 - `get_default_step_ids`: Obtiene los IDs de los pasos pertenecientes a un grupo.
 - `get_step_view_model`: Genera un `FlowItemDTO` listo para la vista con strings formateados (Fase 12C).
 - `group_tasks`: Crea un grupo secuencial a partir de las tareas seleccionadas (Fase 12C).
 -  `_normalize_prep_info_response`: Convierte respuesta legada (tupla o lista de al menos 2 elementos) a par (grupo, máquina).
-

 `ui/dialogs/production_flow/definir_cantidades_dialog.py`

Nombre del Módulo: `ui.dialogs.production_flow.definir_cantidades_dialog` Descripción: Definición o simulación del flujo de producción (estado, presentadores, reglas y diálogos auxiliares).

 **Clase** `DefinirCantidadesDialog`

Diálogo para definir la cantidad a producir para cada tarea/grupo.

 `ui/dialogs/production_flow/enhanced_flow_dialog.py`

Nombre del Módulo: `ui.dialogs.production_flow.enhanced_flow_dialog` Descripción: Definición o simulación del flujo de producción (estado, presentadores, reglas y diálogos auxiliares).

Clase `EnhancedProductionFlowDialog`

Diálogo para la planificación visual del flujo de producción. Delegado en `FlowGraphManager` (UI Canvas) y `EnhancedFlowPresenter` (Lógica).

Métodos Principales:

- `closeEvent`: Asegura la limpieza de recursos antes de cerrar.
 - `cleanup`: Detiene timers y libera referencias circulares para evitar `SegFaults`.
 - `_preview_execution_order`: Lanza la previsualización delegando en el handler.
-

 `ui/dialogs/production_flow/enhanced_flow_presenter.py`

Nombre del Módulo: `ui.dialogs.production_flow.enhanced_flow_presenter` Descripción: Definición o simulación del flujo de producción (estado, presentadores, reglas y diálogos auxiliares).

 **Clase** `EnhancedFlowPresenter`

Presenter/Lógica para aislar el ensamblado de datos y configuraciones de la vista.

 `ui/dialogs/production_flow/enhanced_flow_state_manager.py`

Nombre del Módulo: `ui.dialogs.production_flow.enhanced_flow_state_manager` Descripción: Definición o simulación del flujo de producción (estado, presentadores, reglas y diálogos auxiliares).

 **Clase `EnhancedFlowStateManager`**

Colaborador de composición para estado del canvas y preview de simulación.

 `ui/dialogs/production_flow/flow_action_handler.py`

Nombre del Módulo: `ui.dialogs.production_flow.flow_action_handler`

Descripción: Acciones del diálogo de flujo visual (ciclos, guardar/cargar pila, biblioteca). `load_saved_pila` obtiene API de pilas vía `resolve_pila_service`; si no hay servicio, usa `model.planning_facade` y en último término `model` (`get_all_pilas` / `load_pila`).

Clase `FlowActionHandler`

Gestiona las acciones de configuración (ciclos, reasignaciones) y persistencia (guardar/cargar) del diálogo visual.

Métodos Principales:

- `_pila_list_load_api`: `PilaService` resuelto; si no, `planning_facade` o modelo completo como último recurso.
 - `initialize_library`: Prepara y carga los datos en el panel de la biblioteca.
 - `setup_floating_widgets`: Crea y configura el botón de previsualización y la etiqueta de estado.
-

 `ui/dialogs/production_flow/flow_builder.py`

Nombre del Módulo: `ui.dialogs.production_flow.flow_builder`

Descripción: Construcción y serialización de flujos de producción (composición con `EnhancedFlowPresenter`).

Clase `FlowBuilder`

Carga/reconstrucción de estado y construcción/preparación de flujos (delegado por el presenter).

Métodos Principales:

- `load_flow`: Inicializa el estado del Presenter desde datos externos. Retorna lista de tareas procesadas con posiciones para que la vista cree los widgets.
 - `prepare_task_data`: Organiza la lista plana de tareas primarias en DTOs agrupados por producto.
 - `build_production_flow`: Construye el flujo final extraído del estado lógico o de la lista proporcionada.
-

 `ui/dialogs/production_flow/flow_simulation_handler.py`

Nombre del Módulo: `ui.dialogs.production_flow.flow_simulation_handler` Descripción: Definición o simulación del flujo de producción (estado, presentadores, reglas y diálogos auxiliares).

Clase `FlowSimulationHandler`

Gestiona la lógica de previsualización de simulación en el editor visual. Controla el timer, las actualizaciones de la etiqueta de progreso y la interacción con el presenter y canvas.

Métodos Principales:

- `start`: Inicia el proceso de previsualización.
 - `stop`: Detiene la previsualización y limpia efectos.
 - `_position_label`: Posiciona la etiqueta de simulación centrada en el canvas.
-

 `ui/dialogs/production_flow/machine_resource_manager.py`

Nombre del Módulo: `ui.dialogs.production_flow.machine_resource_manager` Descripción: Definición o simulación del flujo de producción (estado, presentadores, reglas y diálogos auxiliares).

Clase `MachineResourceManager`

Gestiona la lógica de recursos de máquina y fases de preparación para `DefineProductionFlowDialog`. Desacopla la carga dinámica de componentes de la UI del diálogo principal.

Métodos Principales:

- `update_machines_for_task`: Configura el menú de máquinas basado en el tipo de tarea y carga valores por defecto.
 - `load_prep_steps`: Carga dinámicamente los checkboxes de fases de preparación para la máquina seleccionada.
-

 `ui/dialogs/production_flow/reassignment_rule_dialog.py`

Nombre del Módulo: `ui.dialogs.production_flow.reassignment_rule_dialog` Descripción: Definición o simulación del flujo de producción (estado, presentadores, reglas y diálogos auxiliares).

 **Clase** `ReassignmentRuleDialog`

Diálogo para definir la regla de reasignación de un trabajador para una tarea.

 **ui/widgets/__init__.py**

Nombre del Módulo: widgets

Descripción: Reexporta los widgets principales de la vista de escritorio y los subpaquetes `product`, `production_flow` y `reports`.

 **ui/widgets/base.py**

Nombre del Módulo: base

Descripción: Imports comunes de PyQt6 y utilidades compartidas por los widgets de escritorio (patrón `from .base import *` en pantallas legacy). Incluye intento opcional de `QtCharts`.

 `ui/widgets/calculate_times_widget.py`

Nombre del Módulo: `calculate_times_widget`

Descripción: Pantalla de simulación y cálculo de tiempos: búsqueda de fabricación/producto, línea de tiempo, auditoría y acciones de exportación o limpieza vía señales Qt.

 **Clase** `CalculateTimesWidget`

Widget para la pantalla de cálculo de tiempos de fabricación.

Métodos Principales:

- `apply_empty_plan_results_state`: Sin simulación reciente coherente con la pila: oculta cronograma/log y limpia tablas.
 - `_plan_table_row_values`: Textos de fila: (#, tipo, detalle, unidades, fecha).
 - `clear_lote_search_results`: Limpia la tabla de resultados de búsqueda de lotes.
 - `set_lote_search_results`: Reemplaza los resultados de búsqueda de lotes (id, código, descripción).
 - `get_selected_lote_search_result`: Devuelve (id, código) del lote seleccionado en la tabla de búsqueda.
 - `add_step_to_pila`: Añade un paso (tarea/preproceso) a la pila manualmente.
-

 `ui/widgets/dashboard_widget.py`

Nombre del Módulo: `dashboard_widget`

Descripción: Cuadrícula de gráficos de producción (máquinas, operarios, componentes, actividad). Los datos se actualizan desde el controlador de UI; no mantiene `AppController`.

Clase `DashboardWidget`

Widget para mostrar gráficos y estadísticas de producción.

Los datos llegan vía `update_*` desde el controlador de UI; no mantiene `AppController`.

Métodos Principales:

- `setup_ui`: Configura la interfaz del dashboard.
 - `_create_chart_view`: Función auxiliar para crear un `QChartView` con un título.
 - `update_machine_usage`: Actualiza el gráfico de uso de máquinas.
 - `update_worker_load`: Actualiza el gráfico de carga de trabajo.
 - `update_problematic_components`: Actualiza el gráfico de componentes problemáticos.
 - `update_monthly_activity`: Actualiza el nuevo gráfico de actividad mensual.
-

 `ui/widgets/fabrications_widget.py`

Nombre del Módulo: FabricationsWidget Descripción: Componente de interfaz para la gestión (CRUD) de órdenes de fabricación y preprocesos.

 **Clase** `FabricationsWidget`

Widget específico para la gestión de Fabricaciones (CRUD).

Métodos Principales:

- `__init__`: Inicializa el widget de fabricaciones. `_app_controller` se ignora (compat `MainView`). La lógica vive en controladores conectados por señales; este widget solo emite señales Qt.
 - `update_fabrications_table`: Bridge method para compatibilidad con `FabricacionManager`.
-

 `ui/widgets/gestion_datos_widget.py`

Nombre del Módulo: `gestion_datos_widget`

Descripción: Pestañas unificadas para productos, fabricaciones, máquinas, trabajadores y lotes. Cada pestaña resuelve su controlador vía DI.

 **Clase** `GestionDatosWidget`

Widget unificado que contiene pestañas para gestionar los datos principales de la aplicación.

Las pestañas resuelven controladores de dominio vía DI; no reciben `AppController` desde esta vista ni se mantiene referencia al hub.

 `ui/widgets/help_widget.py`

Nombre del Módulo: `help_widget`

Descripción: Ayuda embebida en HTML («Cómo funciona») dentro de un `QTextEdit` de solo lectura.

 **Clase `HelpWidget`**

Widget para mostrar la página de ayuda 'Cómo Funciona'.

 `ui/widgets/historial_widget.py`

Nombre del Módulo: historial_widget

Descripción: Historial de iteraciones de producto y de fabricaciones con filtros, calendario e impresión de informes (señales hacia el controlador de historial).

 **Clase** `HistorialWidget`

Widget para la nueva sección de historial de iteraciones y fabricaciones.

 `ui/widgets/home_widget.py`

Nombre del Módulo: `home_widget` Descripción: Pantalla de inicio de la aplicación Hipatia. Muestra el resumen del último arranque del sistema (estado de BD, integridad, datos) y alberga la terminal interna de advertencias y errores en tiempo real para que el usuario pueda revisar la salud del programa en cualquier momento sin necesidad de acceder a archivos de log.

```
El mismo tipo de terminal (LogTerminalWidget) existe en la vista de trabajador (pestaña Log); el QtLogHandler solo se conecta a uno u otro según el rol tras el login.
```

Clase `HomeWidget`

Widget de inicio: resumen esquemático del último arranque y terminal interna.

Integra dos paneles verticales:

- Panel de salud del sistema: estado de BD, tablas y último backup.
- Terminal de log: mensajes desde el nivel del `QtLogHandler` (INFO por defecto) en tiempo real, con resaltado para WARNING/ERROR/CRITICAL y exportación a archivo.

Métodos Principales:

- `connect_log_handler`: Conecta el handler de logging Qt a la terminal interna del widget. Invoca `connect_to_widget()` del handler, que además de conectar la señal reproduce el buffer de mensajes acumulados durante el arranque (antes de que la UI estuviera lista). Debe llamarse una vez desde `app.py` en la rama de vista principal (no Trabajador), tras registrar el handler en el logger root. Args: handler: Instancia de `QtLogHandler` ya añadida al logger root mediante `logging.getLogger().addHandler(handler)`.
 - `update_health_report`: Actualiza el panel con el `HealthReport` de forma esquemática y descriptiva. Args: report: instancia de `HealthReport`.
-

 `ui/widgets/log_terminal_widget.py`

Nombre del Módulo: `log_terminal_widget` Descripción: Terminal interna reutilizable en inicio (responsable) y en la vista trabajador (pestaña Log). Recibe líneas desde `QtLogHandler` (nivel por defecto `INFO` y superior); colorea de forma destacada `WARNING`, `ERROR` y `CRITICAL`, y deja otros niveles en tono neutro. Incluye limpieza de la vista y exportación a `.txt` para soporte, sin borrar el archivo de log en disco.

Clase `LogTerminalWidget`

Panel de texto enriquecido conectado al `QtLogHandler` (vía `connect_to_widget` sobre `append_log`, o `connect_handler` si solo se enlaza la señal).

Colores: `WARNING` / `ERROR` / `CRITICAL` resaltados; resto de niveles en gris claro. Botones Limpiar (solo vista) y Exportar `.txt`.

Métodos Principales:

- `__init__`: Inicializa el widget de terminal y construye su interfaz. Args: `parent`: Widget padre de Qt, o `None` si es raíz.
 - `_build_ui`: Construye todos los elementos visuales del widget.
 - `_build_header`: Construye la cabecera con título y botones de acción. Returns: Widget que contiene el encabezado completo.
 - `_button_style`: Genera el estilo CSS de un botón de la cabecera. Args: `bg`: Color de fondo normal en formato hexadecimal. `hover`: Color de fondo al pasar el cursor. Returns: Cadena de estilo QSS.
 - `append_log`: Añade una línea de log formateada al panel de la terminal. Detecta el nivel de log en el texto recibido y aplica la coloración correspondiente mediante HTML incrustado en el `QTextEdit`. Desplaza automáticamente el panel al final del contenido. Args: `text`: Mensaje de log ya formateado, tal como lo entrega `QtLogHandler` tras pasar por su `Formatter`.
 - `connect_handler`: Conecta la señal del handler de logging al slot de visualización. Args: `handler`: Instancia de `QtLogHandler` ya registrada en el logger root. A partir de esta llamada, cada mensaje `WARNING/ERROR/ CRITICAL` generado aparecerá automáticamente en el panel.
-

 `ui/widgets/lotes_widget.py`

Nombre del Módulo: `lotes_widget`

Descripción: Listado y edición de plantillas de lote (`DefinirLoteWidget`) y vista de gestión asociada; dependencias de dominio vía DI.

Clase `DefinirLoteWidget`

Widget para crear y editar plantillas de Lote.

Métodos Principales:

- `__init__`: `_app_controller` se ignora en ctor; opcionalmente se usa en `set_controller`.
- `set_controller`: `Compat MainView`; opcionalmente mejora `FabricacionService` vía hub y repuebla listas.
- `populate_fabrications_list`: Obtiene todas las fabricaciones y llena la lista, excluyendo las tareas generadas automáticamente.
- `filter_fabrications`: Filtra la lista de fabricaciones según el texto ingresado.
- `populate_products_list`: Obtiene todos los productos y llena la lista.
- `filter_products`: Filtra la lista de productos según el texto ingresado.

Clase `LotesWidget`

Widget específico para editar y visualizar las plantillas de Lote.

Métodos Principales:

- `__init__`: `_app_controller` se ignora (`compat MainView`).
 - `set_controller`: `Compat MainView`; listas y CRUD usan `LoteController` del DI.
-

 `ui/widgets/machines_widget.py`

Nombre del Módulo: `machines_widget`

Descripción: CRUD de máquinas con lista filtrable, formulario de detalle y señales para grupos y mantenimiento.

 **Clase `MachinesWidget`**

Widget para gestionar la base de datos de máquinas (CRUD).

Métodos Principales:

- `__init__: _app_controller` se ignora (compat `MainView`); dependencias vía DI.
-

 `ui/widgets/main_header.py`

Nombre del Módulo: `main_header`

Descripción: Define protocolos o tipos principales: `MainHeader`. Widget de cabecera que contiene el botón de auto-ajuste de escala. Integración típica con: `PyQt6`.

Clase `MainHeader`

Widget de cabecera que contiene el botón de auto-ajuste de escala.

Métodos Principales:

- `__init__`: Inicializa la cabecera y sus componentes.
 - `_init_ui`: Crea el layout de la cabecera y el botón de auto-ajuste.
-

 `ui/widgets/main_nav_panel.py`

Nombre del Módulo: `main_nav_panel` Descripción: Widget lateral de navegación para la ventana principal. Gestiona los botones de acceso a las diferentes secciones y el menú de planificación.

Clase `MainNavPanel`

Panel lateral de navegación con botones categorizados y menú de operaciones.

Métodos Principales:

- `__init__`: Inicializa el panel de navegación y sus estilos.
 - `_setup_style`: Establece la apariencia visual del panel lateral.
 - `_init_ui`: Crea los botones y categorías del panel, dentro de un área con scroll.
 - `_create_category_label`: Crea una etiqueta de categoría estilizada.
 - `_create_nav_button`: Crea un botón de navegación que emite una señal al ser pulsado.
 - `_show_planificacion_menu`: Abre el menú de planificación bajo el botón (sin acoplar menú al `QPushButton`).
 - `planificacion_menu`: Menú contextual de planificación (tests y depuración).
 - `update_active_button`: Actualiza visualmente qué botón aparece marcado como activo. Args: `active_page`: nombre interno de la página activa.
-

 `ui/widgets/prep_steps_widget.py`

Nombre del Módulo: `prep_steps_widget`

Descripción: CRUD de fases de preparación (pasos) con validación y señales hacia el controlador.

Clase `PrepStepsWidget`

Widget para gestionar la base de datos de fases de preparación (CRUD).

Métodos Principales:

- `load_preprocesos_data`: Carga los datos de los preprocesos en la lista.
 - `clear_details_area`: Limpia el panel de detalles.
 - `_create_form_widgets`: Crea la estructura del formulario de detalles.
 -  `_ui_record_field`: Lee un campo de un dict o de un objeto/DTO (la UI acepta ambas formas).
-

 `ui/widgets/preprocesos_widget.py`

Nombre del Módulo: `preprocesos_widget`

Descripción: Gestión de preprocesos: lista a la izquierda, detalle y acciones a la derecha.

Clase `PreprocesosWidget`

Widget rediseñado para la gestión de Preprocesos. Muestra una lista a la izquierda y los detalles del seleccionado a la derecha.

Métodos Principales:

- `set_controller`: `Compat MainView.set_controller`; la vista usa DI y `ProductController`.
-

 `ui/widgets/products_widget.py`

Nombre del Módulo: `products_widget` Descripción: Widget de catálogo de productos (búsqueda, formulario, subfabricaciones y procesos). Emite señales hacia `ProductController` e incluye entrada para importar BOM A3RP.

Clase `ProductsWidget`

Vista principal de la pestaña Productos: lista, detalle editable y accesos a diálogos relacionados.

Resuelve `ProductController` vía `DIContainer` en `__init__` (patrón sin `AppController` en el widget).

Métodos Principales:

- `__init__: _app_controller` se ignora (compat `MainView`); dependencias vía DI.
 - `display_product_form`: Muestra el formulario para editar un producto o crear uno nuevo. Args: `data`: DTO del producto o código (si es nuevo). `sub_data`: Lista de subfabricaciones existentes. `is_new`: Si es `True`, configura el formulario para creación.
 -  `_subfabricacion_row_from_domain`: Serializa una subfabricación de dominio a dict para el formulario y persistencia. Args: `sub`: `SubfabricacionDTO` o objeto con atributos homólogos. Returns: Diccionario con claves `id`, `descripcion`, `tiempo`, `tipo_trabajador`, `maquina_id`.
-

 `ui/widgets/reportes_widget.py`

Nombre del Módulo: `reportes_widget`

Descripción: Vista principal de reportes: búsqueda inteligente, lista de órdenes y gráficas. `ReportService` se resuelve desde el contenedor del hub o desde `model.report_service`; los sub-widgets consumen solo ese servicio.

 **Clase `ReportesWidget`**

Módulo de reportes: búsqueda, órdenes por producto y gráficas.

Tras `set_controller(hub)` los hijos reciben únicamente `ReportService` resuelto del hub.

Métodos Principales:

- `set_controller`: Enlaza hub: actualiza `ReportService` en sub-widgets.
-

 `ui/widgets/settings_widget.py`

Nombre del Módulo: `settings_widget.py` Descripción: Widget de configuración general para la aplicación Hipatia. Maneja la lógica de horarios laborales, descansos, festivos y copias de seguridad.

Clase `SettingsWidget`

Panel de configuración de la aplicación. Orquesta la vista de horarios, descansos y parámetros del sistema.

La vista depende de `ScheduleController` (`set_schedule_controller`) y, si hace falta carga antes de existir ese controlador, de `DatabaseManager` vía `set_config_db_fallback`; no mantiene referencia a `AppController`.

Métodos Principales:

- `__init__`: Inicializa el widget de configuración. Args: `schedule_controller`: Controlador para gestión de horarios. `parent`: Widget padre opcional.
 - `set_schedule_controller`: Asigna el controlador de horarios (sin pasar por `AppController`).
 - `set_config_db_fallback`: Base de datos con `config_repo` para carga temprana si aún no hay `ScheduleController`.
 - `_init_ui`: Configura la estructura visual del panel.
 - `load_schedule_settings`: Solicita al controlador que cargue los ajustes en los widgets.
 - `on_add_break_clicked`: Evento para añadir un descanso mediante el diálogo del controlador.
 - `on_edit_break_clicked`: Evento para editar el descanso seleccionado.
 - `on_remove_break_clicked`: Evento para eliminar el descanso seleccionado.
 - `on_add_holiday_clicked`: Marca el día del calendario como festivo.
 - `on_remove_holiday_clicked`: Elimina el carácter festivo del día seleccionado.
 - `on_save_all_clicked`: Guarda la configuración completa incluyendo la hora de backup.
 - `_update_break_buttons_state`: Habilita o deshabilita botones según selección en la lista.
 - `_highlight_holidays`: Pinta en el calendario los días definidos como festivos (visual).
-

 `ui/widgets/timeline_widget.py`

Nombre del Módulo: `timeline_widget`

Descripción: Diagrama de Gantt interactivo (`TimelineVisualizationWidget`) y panel de análisis de tareas (`TaskAnalysisPanel`) para resultados de simulación.

 **Clase `TimelineVisualizationWidget`**

Widget que dibuja un diagrama de Gantt interactivo y detallado.

 **Clase `TaskAnalysisPanel`**

Widget que muestra el detalle de una tarea seleccionada.

 `ui/widgets/workers_widget.py`

Nombre del Módulo: `workers_widget` Descripción: Pestaña de trabajadores en «Gestión de datos»: lista, ficha, asignación de tareas e historial, coordinada con `WorkerController` mediante señales PyQt6.

Clase `WorkersWidget`

Widget principal para la gestión de trabajadores (Orquestador).

Centraliza la vista de lista de trabajadores y los paneles de detalle. Gestiona la comunicación entre sub-widgets y el `WorkerController`. Incluye un área de scroll para asegurar la visibilidad de los botones de acción.

Métodos Principales:

- `__init__: _app_controller` se ignora (compat `MainView`); dependencias vía DI.
 - `populate_list`: Pone en la lista lateral de trabajadores.
 - `clear_details_area`: Oculta los paneles y muestra el placeholder.
 - `show_worker_details`: Muestra la información de un trabajador en los paneles correspondientes.
 - `show_add_new_form`: Configura el panel de detalles para un nuevo trabajador.
 - `get_form_data`: Delega la extracción de datos de formulario al panel de detalles.
 - `get_assignment_data`: Delega la extracción de datos de asignación al panel de detalles.
 - `update_product_search_results`: Actualiza los resultados de búsqueda en el panel de detalles.
 - `setup_of_completer`: Configura el autocompletado de O.F. en el panel de detalles.
 - `show_incidents_dialog`: Abre el diálogo modal de incidencias.
 - `populate_history_tables`: Método de compatibilidad para el controlador.
 - `clear_assignment_form`: Limpia los campos de asignación de tareas en el panel de detalles.
 - `form_widgets`: Propiedad de compatibilidad para acceder a los widgets del formulario.
-

 `ui/widgets/product/__init__.py`

Nombre del Módulo: product

Descripción: Subpaquete de widgets de detalle de producto (materiales e iteraciones).

 `ui/widgets/product/iterations_widget.py`

Nombre del Módulo: `iterations_widget`

Descripción: Widget para visualizar y gestionar el historial de iteraciones de un producto. Incluye la gestión de materiales asociados y galería de imágenes.

Al añadir una iteración, el diálogo devuelve `AddIterationFormData`; aquí se convierte a dict con `asdict` para `handle_add_product_iteration` del controlador.

Clase `ProductIterationsWidget`

Panel de visualización de iteraciones y mejoras de productos. Gestiona el listado de cambios y la galería de imágenes asociada.

Métodos Principales:

- `__init__`: Inicializa el widget de iteraciones. Args: `product_code`: Código del producto actual. `product_controller`: Controlador de productos (servicios, adjuntos vía `app`). `parent`: Widget padre opcional.
 - `_init_ui`: Configura la estructura visual del panel mediante un splitter horizontal.
 - `load_data`: Carga las iteraciones para el producto especificado o el actual.
 - `_refresh_list`: Actualiza el listado visual de iteraciones.
 - `_clear_details_panel`: Limpia los campos de detalle (alias para compatibilidad).
 - `_show_details_panel`: Mantiene compatibilidad con tests de visualización.
 - `_reselect_current_iteration`: Busca y selecciona de nuevo la iteración actual en la lista.
 - `_add_image_to_gallery`: Añade una imagen a la galería visual (alias para compatibilidad con tests).
 - `on_new_iteration_clicked`: Evento para crear una nueva iteración del producto.
 - `on_edit_iteration_clicked`: Evento para editar la iteración seleccionada.
 - `on_delete_iteration_clicked`: Evento para eliminar la iteración seleccionada.
 - `on_view_plano_clicked`: Abre el plano adjunto de la iteración seleccionada.
 - `refresh_gallery`: Actualiza la vista de miniaturas de la galería.
 - `on_add_image_clicked`: Solicita la subida de una nueva imagen de iteración.
 - `on_delete_image_clicked`: Elimina la imagen seleccionada en la galería.
-

 `ui/widgets/product/materials_widget.py`

Nombre del Módulo: `materials_widget`

Descripción: Lista y edición de materiales (componentes) de un producto dentro de un `QTableWidget`.

Clase `ProductMaterialsWidget`

Gestión de la lista de materiales (componentes) de un producto.

Usa `ProductController` para servicios y comandos; `view` es la ventana principal para `show_message` / `show_confirmation_dialog` (protocolo `IView`).

Métodos Principales:

- `load_data`: Carga la lista de materiales del producto en la tabla.
-

 `ui/widgets/production_flow/__init__.py`

Nombre del Módulo: `production_flow`

Descripción: Subpaquete de widgets del planificador y del flujo de producción (canvas, tarjetas, inspector).

 `ui/widgets/production_flow/define_control_panel.py`

Nombre del Módulo: `define_control_panel`

Descripción: Panel lateral para añadir o editar pasos del flujo (tareas, condiciones de inicio y recursos).

Clase `DefineControlPanel`

Panel de control lateral para añadir y editar pasos en el flujo de producción. Encapsula la interfaz de configuración de tareas, condiciones de inicio y recursos.

Métodos Principales:

- `get_form_data`: Recoge todos los datos configurados en el panel.
 - `populate_form`: Pueba el formulario con datos de un paso existente (Fase 12C).
-

 `ui/widgets/production_flow/flow_canvas.py`

Nombre del Módulo: `flow_canvas`

Descripción: Canvas PyQt6 del flujo de producción: tarjetas `FlowCardWidget` arrastrables, rejilla de fondo y conexiones en un hijo `_FlowConnectionsLayer` encima de las tarjetas (sin capa, las flechas quedarían tapadas). Ajusta geometría en `resizeEvent`, `set_connections` y `add_task_widget`; clic en fondo emite `backgroundClicked`. Las aristas se delegan a `FlowConnectionPainter`.

Clase `_FlowConnectionsLayer`

Hijo a pantalla completa del canvas: solo pinta conexiones, sin capturar raton (`WA_TransparentForMouseEvents`), para que el trazo quede visible sobre las tarjetas.

Clase `ProductionFlowCanvas`

Area de trabajo del grafo de tareas: drop desde biblioteca, seleccion y movimiento de tarjetas, lista `connections` normalizada con `CanvasVisualConnection` y capa superior para flechas. Senales: `taskDropped`, `cardSelected` (UID canvas o id logico), `cardMoved`, `backgroundClicked`.

Métodos Principales:

- `set_connections`: Actualiza la lista de conexiones (dict o `CanvasVisualConnection`) y redibuja.
 - `add_task_widget`: Registra un widget de tarea en el canvas y conecta sus señales.
 - `clear_widgets`: Limpia todos los widgets de tareas y conexiones.
 - `mousePressEvent`: Detecta clics en el fondo del canvas.
 - `paintEvent`: Rejilla de fondo (las flechas van en `_FlowConnectionsLayer`, encima de las tarjetas).
 -  `_drag_source_item_user_data: UserRole` en el ítem: `QListWidget` usa `data(role)`; `QTreeWidget/QTableWidget`, `data(0, role)`.
-

 `ui/widgets/production_flow/flow_card_widget.py`

Nombre del Módulo: `flow_card_widget` Descripción: Tarjeta de tarea en el canvas de flujo; textos delegados en `core.flow_card_labels`.

Clase `FlowCardWidget`

Una tarjeta visual y MOVIBLE que representa una tarea en el canvas. Emite 'clicked' al ser seleccionada y 'moved' al ser movida.

Métodos Principales:

- `_apply_base_style`: Aplica el estilo CSS base.
 - `mousePressEvent`: Se activa al hacer clic en la tarjeta.
 - `mouseMoveEvent`: Se activa al mover el ratón mientras se mantiene presionado.
 - `mouseReleaseEvent`: Se activa al soltar el botón del ratón.
 - `_snap_to_grid`: Ajusta la posición de la tarjeta al punto más cercano de la cuadrícula.
 - `set_selected`: Marca visualmente la tarjeta como seleccionada.
 - `set_highlighted`: Resalta la tarjeta con un color específico.
 - `update_workers`: Actualiza la visualización de los trabajadores asignados.
-

Nombre del Módulo: `flow_connectionPainter`

Descripción: Pintado y enrutado de conectores entre tarjetas del canvas de flujo de producción.

- Enrutado (`calculateSmartPath`): polilínea ortogonal Manhattan que no cruza ninguna tarjeta visible, incluidos origen y destino. Margen de exclusión `CONNECTOR_OBSTACLE_PAD`; los puntos de ruta usan `CONNECTOR_EDGE_STUB` más allá del borde para quedar fuera de ese rectángulo. Si el tramo directo es libre, se prefieren codos en L o un «jog» para evitar líneas rectas entre celdas alineadas. Respaldo: barrido de corredores y desvíos en U.
- Dibujo: terminales visuales en el borde de la celda se anteponen/sufijan a la polilínea de ruta para que línea y flecha encajen con la tarjeta (sin hueco flotante). Trazo con `QPainterPath` y esquinas redondeadas (`quadTo`). Conexiones cíclicas: anclajes verticalmente (abajo/arriba), gradiente y flecha según tangente real del path.
- API de aristas: tipos y flags cíclicos vía `CanvasCyclicConnectionFlags` (sin dict en la firma de pintado).

Clase `FlowConnectionPainter`

Utilidad de pintado sobre un `QPainter`: calcula polilínea de conexión (evitando tarjetas) y la dibuja con trazo redondeado, flecha alineada a la tangente del path y variante cíclica con resplandor. Los métodos públicos de interés para tests o reutilización son `drawConnection`, `calculateSmartPath` y `drawGrid`.

Métodos Principales:

- `drawConnection`: Dibuja la arista entre dos tarjetas según `conn_type` (normal o cíclica) y flags de ciclo. Args: `all_widgets`: Lista de tarjetas del canvas para obstáculos y orden de pintado.
- `_drawNormalConnection`: Conexión estándar: anclajes laterales y ruta ortogonal.
- `_drawCyclicConnection`: Conexión cíclica con efectos de brillo.
- `_strokeRoundedPathGradient`: Trazado suavizado con gradiente aproximado (eje inicio→fin del conector).
- `calculateSmartPath`: Calcula la polilínea de enrutamiento entre `start` y `end` (puntos ya desplazados con `stub`). Returns: Lista de `QPointF` sin los terminales visuales en borde de celda; el llamador que dibuja suele anteponer/sufijar `_visualTerminals_*`.
- `_obstacleRects`: Todas las tarjetas visibles son obstáculos; origen y destino también (los anclajes quedan fuera con `EDGE_STUB`).
- `_preferOrthogonalElbow`: Aunque el segmento A–B sea libre, propone un trazado en L o con «jog» para que siempre haya codos visibles (las tarjetas alineadas no dejan línea recta única).
- `drawGrid`: Dibuja la cuadrícula de fondo.
-  `_anchorsByVector`: Puntos de enrutamiento fuera del borde (`stub`), con Y o X orientada hacia la otra tarjeta. Deben usar `stub >= CONNECTOR_OBSTACLE_PAD` para quedar fuera del rectángulo de colisión.
-  `_cyclicAnchors`: Anclajes de ruta bajo el borde inferior del origen y sobre el superior del destino (con `stub`).
-  `_visualTerminalsLateral`: Bordes visibles de las tarjetas (sin hueco); la ruta geométrica usa `EDGE_STUB` más allá.
-  `_visualTerminalsCyclic`: Borde inferior de origen y borde superior de destino (flecha llega a la celda).
-  `_dedupeConsecutivePoints`: Elimina vértices consecutivos casi coincidentes al unir terminales visuales con la ruta.

-  `_orthogonal_polyline_to_rounded_path`: Convierte una polilínea ortogonal en un `QPainterPath` con esquinas redondeadas (`quadTo`), estilo conector tipo diagrama profesional.
 -  `_tangent_near_path_end`: Punto previo y final del trazado para orientar la flecha según la curva real.
-

 `ui/widgets/production_flow/flow_display_panel.py`

Nombre del Módulo: `flow_display_panel`

Descripción: Panel derecho del diálogo de definición de pila: lista ordenada de tareas del flujo y botones de edición, borrado, asignación de operarios y guardado.

Clase `FlowDisplayPanel`

Panel derecho de `DefineProductionFlowDialog`. Gestiona la visualización de la secuencia de tareas y las acciones sobre el flujo.

Métodos Principales:

- `update_display`: Refresca la visualización de la lista de pasos.
 - `get_selected_indices`: Retorna los índices de los pasos seleccionados mediante checkbox.
-

 `ui/widgets/production_flow/flow_graph_manager.py`

Nombre del Módulo: `flow_graph_manager`

Descripción: Coordina el presenter del flujo mejorado con un `ProductionFlowCanvas`: creación de tarjetas, sincronización de índices, efectos de ciclo/simulación y conexiones lógicas. Escucha `cardMoved` y `cardSelected` del canvas; `update_connections` obtiene las aristas con `canvas_state_all_logical_connections` y las pinta en bloque.

Clase `FlowGraphManager`

Puente entre estado del presenter (`canvas_tasks`) y widgets en `ProductionFlowCanvas`.

Registra movimientos y seleccion de tarjetas, reconstruye el grafo desde datos de flujo, aplica efectos (madre de ciclo, simulacion) y delega el dibujo de flechas en el canvas via `canvas_state_all_logical_connections + canvas.set_connections`.

Métodos Principales:

- `cleanup`: Libera recursos y rompe referencias circulares para evitar `SegFaults`.
 - `add_task_widget`: Crea un widget para una tarea y lo sincroniza con el presenter.
 - `update_task_config`: Actualiza la configuración de una tarea y reaplica efectos visuales.
 - `load_from_flow`: Reconstruye el canvas y el estado lógico desde datos de flujo.
 - `remove_task_widget`: Elimina el widget y actualiza el estado lógico.
 - `clear`: Limpia todo el canvas y el estado.
 - `select_task`: Marca visualmente una tarea como seleccionada y actualiza relaciones.
 - `update_connections`: Dibuja todas las flechas del flujo; si hay selección, resalta aristas relacionadas.
 - `apply_mother_effect`: Aplica o quita el efecto de `GoldenGlowEffect`.
 - `update_all_cycle_effects`: Sincroniza todos los efectos de ciclo intermedios y finales.
 - `highlight_processing_task`: Aplica el efecto azul de simulación.
 - `clear_simulation_effects`: Limpia todos los efectos de resaltado de procesamiento.
 - `synchronize_positions`: Sincroniza las posiciones de los widgets con el estado del presenter.
 - `get_task_inspector_context`: Prepara el contexto tipado para el inspector de tareas.
-

 `ui/widgets/production_flow/flow_item_widget.py`

Nombre del Módulo: `flow_item_widget`

Descripción: Fila o grupo en la lista textual de la pila de producción (vista `FlowItemDTO`).

Clase `FlowItemWidget`

Widget especializado para representar un paso individual o un grupo en la lista de la pila de producción.

Métodos Principales:

- `is_selected`: Retorna si el checkbox está marcado (solo para pasos individuales).
-

 `ui/widgets/production_flow/flow_toolbar.py`

Nombre del Módulo: `flow_toolbar`

Descripción: Barra inferior del planificador visual: limpiar, cargar, guardar y calcular (señales Qt).

Clase `FlowToolbarWidget`

Barra de herramientas inferior para el Planificador Visual de Producción. Gestiona las acciones principales (limpiar, cargar, guardar, calcular).

Métodos Principales:

- `set_buttons_enabled`: Habilita o deshabilita los botones de la barra.
-

 `ui/widgets/production_flow/inspector_panel.py`

Nombre del Módulo: `inspector_panel`

Descripción: `ProductionTaskInspector`: panel lateral para ver y editar la configuración de la tarea seleccionada en el canvas (presenter + widgets generados en `inspector_ui`).

Clase `ProductionTaskInspector`

Panel lateral para inspeccionar y editar las propiedades de una tarea seleccionada en el flujo de producción.

Métodos Principales:

- `_init_ui`: Inicializa la interfaz gráfica del inspector.
 - `_toggle_start_widgets`: Habilita/deshabilita widgets según el modo seleccionado.
 - `_emit_change`: Emite la señal de cambio si hay una tarea activa.
 - `get_selected_assigned_worker`: Devuelve el nombre del trabajador asignado seleccionado, o `None`.
 - `set_task`: Carga una tarea en el inspector. Args: `task_data` (dict): Datos de la tarea (configuración).
`all_tasks` (list): Lista de todas las tareas para llenar dependencias. `machines` (list): Lista de máquinas disponibles. `available_workers` (list): Lista de nombres de todos los trabajadores.
 - `clear`: Limpia el inspector ocultando el formulario y mostrando el placeholder.
-

 `ui/widgets/production_flow/inspector_presenter.py`

Nombre del Módulo: `inspector_presenter`

Descripción: Lógica pura del inspector de tareas: estado de la tarea actual, trabajadores posibles y mutaciones sobre el payload de configuración.

 **Clase** `InspectorPresenter`

Métodos Principales:

- `set_task`: Almacena la tarea actual y los trabajadores posibles.
 - `get_workers_lists`: Devuelve (nombres asignados, nombres disponibles).
 - `assign_workers`: Añade trabajadores por nombre; devuelve la lista completa de asignados.
 - `unassign_workers`: Quita trabajadores por nombre; devuelve la lista de asignados resultante.
 - `build_dependency_list`: Lista (texto para combo, índice) para dependencias; omite la tarea actual.
-

 `ui/widgets/production_flow/inspector_task_loader.py`

Nombre del Módulo: `inspector_task_loader`

Descripción: Aplica el payload de una tarea a los controles del inspector (`apply_task_to_widgets`).

-  `apply_task_to_widgets`: Aplica `task_data` a los widgets del inspector y sincroniza presenter. Retorna (`current_task_id`, `current_task_data`).
-

 `ui/widgets/production_flow/inspector_ui.py`

Nombre del Módulo: `inspector_ui`

Descripción: Construcción de la UI del inspector de tareas: factoría `build_inspector_ui` y dataclass `InspectorWidgets` con referencias a los controles Qt.

Clase `InspectorWidgets`

Referencias tipadas a los controles del inspector de tarea.

-  `build_inspector_ui`: Construye la UI del inspector y devuelve `InspectorWidgets`, `content_scroll`, `placeholder`.
-

 `ui/widgets/production_flow/library_panel.py`

Nombre del Módulo: `library_panel`

Descripción: Biblioteca de tareas por producto (árbol) para arrastrar plantillas al canvas del flujo.

Clase `TaskLibraryPanel`

Panel lateral que muestra la biblioteca de tareas disponibles agrupadas por producto. Permite arrastrar tareas al canvas.

Métodos Principales:

- `populate_tasks`: Rellena el árbol con los datos de tareas agrupados por producto.
 - `set_canvas_tasks`: Actualiza la lista de IDs de tareas que están en el canvas para dar feedback visual.
 - `update_visual_state`: Colorea las tareas que ya están en el canvas.
-

 `ui/widgets/reports/__init__.py`

Nombre del Módulo: reports

Descripción: Reexporta widgets de reportes usados desde `reportes_widget` (p. ej. búsqueda).

 `ui/widgets/reports/charts_container.py`

Nombre del Módulo: charts_container

Descripción: Pestañas de gráficas y tarjetas de resumen para un producto seleccionado en reportes. Los datos llegan solo vía `ReportService` inyectado en el widget.

 **Clase** `ReportsChartsWidget`

Widget contenedor para las gráficas de análisis. Muestra estadísticas y gráficas para un producto seleccionado.

Métodos Principales:

- `_create_placeholder_tabs`: Crea tabs con placeholders.
 - `update_charts`: Actualiza todas las gráficas para un producto. Args: `producto_codigo`: Código del producto
 - `_update_stats_cards`: Actualiza las tarjetas de estadísticas.
 - `_update_evolution_chart`: Actualiza la gráfica de evolución temporal.
 - `_update_workers_chart`: Actualiza la gráfica de tiempos por trabajador.
 - `_update_incidents_chart`: Actualiza la gráfica de incidencias (pie chart).
 - `clear`: Limpia el widget.
-

 `ui/widgets/reports/charts_renderers.py`

Nombre del Módulo: charts_renderers

Descripción: Funciones auxiliares para poblar tarjetas de estadísticas del contenedor de gráficas.

-  `clear_stats_layout`: Limpia el layout de estadísticas del contenedor.
-

 `ui/widgets/reports/order_list.py`

Nombre del Módulo: `order_list`

Descripción: Lista de órdenes de fabricación de un producto con tarjetas expandibles; datos vía `ReportService`.

Clase `OrderCard`

Tarjeta individual para mostrar resumen de una orden de fabricación.

Métodos Principales:

- `__init__`: Args: `order_data`: `OrdenFabricacionResumenDTO`
- `set_selected`: Actualiza estilo visual para reflejar selección.
- `mousePressEvent`: Emite señal al hacer clic.

Clase `OrderListWidget`

Widget que muestra lista de órdenes de fabricación.

Signals: `order_selected(str)`: Emitido cuando se selecciona una orden.

Métodos Principales:

- `load_orders_for_product`: Carga las órdenes de fabricación de un producto. Args: `producto_codigo`: Código del producto
 - `_display_orders`: Muestra las órdenes en tarjetas.
 - `_clear_cards`: Elimina todas las tarjetas.
 - `select_order`: Marca visualmente una orden como seleccionada en la lista actual.
 - `clear`: Limpia el widget.
-

 `ui/widgets/reports/smart_search.py`

Nombre del Módulo: `smart_search`

Descripción: Búsqueda con autocompletado y resultados en tiempo real para el módulo de reportes, usando exclusivamente `ReportService`.

Clase `SmartSearchWidget`

Widget de búsqueda inteligente que ofrece autocompletado y filtrado en tiempo real para el módulo de reportes.

Métodos Principales:

- `_perform_search`: Ejecuta la búsqueda contra `ReportService`.
 - `_update_results_list`: Actualiza la lista visual de resultados.
 - `set_report_service`: Actualiza el servicio de reportes (p. ej. tras `set_controller` en el padre).
 - `clear_search`: Limpia el campo de búsqueda y resultados.
 - `set_controller`: Resuelve `ReportService` desde el hub (container DI o `model.report_service`).
-

 `ui/widgets/reports/stat_card.py`

Nombre del Módulo: `stat_card`

Descripción: Tarjeta compacta de métrica (título, valor y subtítulo) para el panel de reportes.

 **Clase `StatCard`**

Tarjeta de estadística individual.

 `ui/widgets/worker/camera_info_panel.py`

Nombre del Módulo: camera_info_panel

Descripción: Muestra detalle y estado de validación de la cámara seleccionada (trabajador / hardware).

 **Clase CameraInfoPanel**

Panel para mostrar información detallada y estados de validación.

Métodos Principales:

- `update_info`: Actualiza el texto y estilo del panel.
-

 `ui/widgets/worker/camera_selector_panel.py`

Nombre del Módulo: camera_selector_panel

Descripción: Selector de cámara con sondeo rápido y señales hacia la vista de trabajador.

Clase CameraSelectorPanel

Panel para la selección y detección de cámaras.

Métodos Principales:

- `set_loading`: Muestra estado de carga en el combo.
 - `update_cameras`: Pueba el combo con la lista de cámaras y selecciona la actual.
 - `get_selected_camera`: Devuelve el DTO seleccionado actualmente.
-

 `ui/widgets/worker/worker_activity_panel.py`

Nombre del Módulo: `worker_activity_panel`

Descripción: Tabla de tareas asignadas, registro de trabajo e incidencias del trabajador seleccionado.

Clase `WorkerActivityPanel`

Panel que muestra el historial de tareas y logs de actividad de un trabajador.

Métodos Principales:

- `_configure_history_table_header`: Distribuye el espacio: fechas y códigos legibles; producto absorbe el resto.
 - `_configure_activity_log_header`: Misma idea: columnas de tiempo compactas; descripción amplia.
 - `populate_history`: Pueba la tabla de historial de tareas.
 - `populate_activity_log`: Pueba la tabla de logs de actividad.
 - `clear`: Limpia las tablas.
-

 `ui/widgets/worker/worker_details_panel.py`

Nombre del Módulo: `worker_details_panel` Descripción: Formulario y zona de asignación rápida de un trabajador (datos personales, acceso al sistema, búsqueda de producto y orden de fabricación, botón asignar).

Emite señales hacia `WorkersWidget / WorkerController`; no contiene reglas de negocio.

Clase `WorkerDetailsPanel`

Panel que contiene el formulario de detalles y asignación de un trabajador.

Métodos Principales:

- `set_worker_data`: Pone el formulario con los datos de un trabajador.
 - `get_form_data`: Extrae los datos del formulario en un `WorkerFormDataDTO`.
 - `get_assignment_data`: Extrae los datos de la nueva tarea a asignar.
 - `update_product_results`: Actualiza la lista de resultados de búsqueda de productos.
 - `clear_assignment_search_fields`: Vacía el buscador de producto y reinicia la cantidad en el bloque de asignación.
 - `set_of_completer`: Configura autocompletado para el campo de orden de fabricación.
-

 `ui/widgets/worker/worker_incidence_dialog.py`

Nombre del Módulo: `worker_incidence_dialog`

Descripción: Define protocolos o tipos principales: `WorkerIncidenceDialog`. Diálogo para mostrar el detalle de las incidencias de un trabajador. Integración típica con: `PyQt6`, `datetime`.

Clase `WorkerIncidenceDialog`

Diálogo para mostrar el detalle de las incidencias de un trabajador.

Métodos Principales:

- `_populate_incidences`: Pueba la lista con los datos de las incidencias.
-

 `ui/worker/__init__.py`

Nombre del Módulo: worker

Descripción: Paquete de la ventana de trabajador (login de operario): diálogo de cámara y `main_window` con `WorkerMainWindow`.

 `ui/worker/camera_config_dialog.py`

Nombre del Módulo: camera_config_dialog

Descripción: Diálogo modal de configuración de cámara QR para la vista trabajador: selector, panel de detalle y CameraConfigPresenter sobre CameraManager.

 **Clase** CameraConfigDialog

Configuración de cámara con presenter y paneles reutilizables.

 `ui/worker/camera_config_presenter.py`

Nombre del Módulo: `camera_config_presenter`

Descripción: Lógica de sondeo, detalle y validación de cámaras para `CameraConfigDialog`, usando `CameraManager` y DTOs de configuración.

 **Clase `CameraConfigPresenter`**

Presentador que desacopla la lógica de `CameraManager` de la UI de configuración.

Métodos Principales:

- `detect_cameras_light`: Realiza un sondeo rápido de cámaras y devuelve DTOs.
 - `get_camera_detail`: Obtiene información detallada de una cámara (puede ser info previa o nueva).
 - `test_camera`: Realiza una validación pesada con preview.
 - `validate_before_save`: Valida la cámara antes de permitir el guardado si no está validada.
 - `_map_to_detail_dto`: Convierte `CameraInfo` de core a `CameraDetailDTO`.
-

 `ui/worker/main_window/__init__.py`

Nombre del Módulo: `main_window`

Descripción: Concentra datos de configuración o catálogos estáticos: `__all__`, consumidos por la UI y controladores. Integración típica con: `window`.

 `ui/worker/main_window/ui_manager.py`

Nombre del Módulo: `ui_manager` Descripción: Montaje visual de la ventana del trabajador (cabecera, pie, pestañas Tareas y Log).

`WorkerMainWindowUIManager` construye el layout sobre `WorkerMainWindow`: lista de tareas, detalle, botones de acción y la terminal de log enlazada a `QtLogHandler` como en la vista del responsable.

Clase `WorkerMainWindowUIManager`

Gestor de layout y widgets iniciales de `:class:WorkerMainWindow`.

Responsable de cabecera, pie, `stacked_widget` y la pantalla `dashboard` con pestañas Tareas / Log.

Métodos Principales:

- `setup_main_window`: Configura la interfaz de usuario principal sobre la ventana.
 - `_create_initial_screens`: Registra la pantalla `dashboard` (pestañas Tareas y Log) en el stack.
 - `_create_dashboard_screen`: Pantalla principal: `QTabWidget` con pestaña Tareas y pestaña Log. Asigna `w.log_terminal` al `:class:~ui.widgets.log_terminal_widget.LogTerminalWidget` de la segunda pestaña (para `connect_log_handler` en `app.py`).
 - `_create_tasks_tab_content`: Contenido del panel de tareas (splitter lista + detalles).
 - `_create_task_actions_widget`: Panel derecho: estado de la tarea y botones (etiquetas, QR, incidencia, fin).
-

 `ui/worker/main_window/window.py`

Nombre del Módulo: window Descripción: Ventana principal del operario: tareas asignadas, acciones (QR, incidencias, etiquetas) y pestaña Log con la misma terminal que ve el responsable en inicio.

Las señales hacia `WorkerController` llevan datos de tarea como diccionario plano (`WorkerTaskListRowDTO.to_signal_dict()`). La conexión del log se hace desde `app.py`.

Clase `WorkerMainWindow`

Ventana principal para el rol de trabajador.

Atributos relevantes: `current_selected_task`: `WorkerTaskListRowDTO` cuando hay tarea seleccionada en la pestaña Tareas; `None` si no. `log_terminal`:

:class:~ui.widgets.log_terminal_widget.LogTerminalWidget de la pestaña Log (conexión con `QtLogHandler` desde `app.py`).

Métodos Principales:

- `connect_log_handler`: Conecta el handler de logging Qt a la terminal de la pestaña Log. Equivalente a `HomeWidget.connect_log_handler`: llama a `handler.connect_to_widget(self.log_terminal.append_log)`, reproduce el buffer acumulado hasta el login y a partir de ahí muestra en tiempo real los mensajes según el nivel del handler (por defecto INFO y superior).
 - `enable_action_buttons`: Habilita o deshabilita los botones de control de tareas.
 - `_forzar_auto_ajuste`: Fuerza un recalcu­lo dinámico del factor de escala y repinta toda la aplicación iterando sobre sus hijos.
 - `add_screen`: Añade una nueva vista al contenedor de pantallas de la ventana.
 - `show_confirmation_dialog`: Muestra un diálogo de confirmación Sí/No.
-

Capítulo: `scripts/`

Métrica	Valor
Archivos <code>.py</code> en <code>scripts/</code>	67
Incluidos en el cuerpo	67
Omitidos (docstrings/reglas)	0
Clases detectadas (AST)	25

```
graph TD
  SCRIPTS[Scripts/Tools de análisis] --> QA[Calidad/Docs/Test]
```


 **scripts/___init___**.py

Nombre del Módulo: scripts

Descripción: Script ejecutable (___init___): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

 **scripts/analyze_mixin.py**

Nombre del Módulo: scripts.analyze_mixin

Descripción: Analiza un archivo Python (clase grande o módulo acoplado): lista atributos y llamadas vía self.* Útil para planificar extracción a composición o a gestores independientes.

 `scripts/analyze_pila_controller.py`

Nombre del Módulo: `scripts.analyze_pila_controller`

Descripción: Script ejecutable (`analyze_pila_controller`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

 `scripts/analyze_product_controller_coverage.py`

Nombre del Módulo: `scripts.analyze_product_controller_coverage`

Descripción: Script ejecutable (`analyze_product_controller_coverage`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

 **scripts/analyze_ui_state.py**

Nombre del Módulo: scripts.analyze_ui_state

Descripción: Script ejecutable (`analyze_ui_state`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

`scripts/architecture_layer_edges.py`

Nombre del Módulo: `scripts.architecture_layer_edges`

Descripción: Grafo de imports entre capas de primer nivel (ui, controllers, core, database, features).

-  `collect_import_targets`: Nombres de módulo completos importados (sin relativos).
 -  `scan_layers`: `module_name` -> conjunto de strings importados (módulos).
 -  `build_layer_edge_list`: (`from_layer`, `to_layer`) -> [(`source_module`, `imported_module`), ...].
 -  `find_simple_cycles`: 2-ciclos y 3-ciclos entre capas (suficiente para N pequeño).
-

 `scripts/audit_import_graph.py`

Nombre del Módulo: `scripts.audit_import_graph`

Descripción: Concentra datos de configuración o catálogos estáticos: `PREFIXES`, consumidos por la UI y controladores. Integración típica con: `__future__`, `argparse`, `ast`, `json`, `pathlib`.

 `scripts/audit_module_description_quality.py`

Nombre del Módulo: `audit_module_description_quality`

Descripción: Detecta Descripción de módulo débiles (genéricas, cortas o con frases prohibidas) en el alcance Daniel y escribe un informe Markdown bajo `reports/`.

 **scripts/audit_module_docstrings.py**

Nombre del Módulo: scripts.audit_module_docstrings

Descripción: Auditoría de docstrings de módulo: lista archivos .py sin descripción útil al nivel de módulo.

 `scripts/bootstrap_module_docstrings.py`

Nombre del Módulo: `bootstrap_module_docstrings`

Descripción: Inserta o sustituye el docstring inicial de módulo con `Nombre del Módulo` y `Descripción`, reutilizando el primer párrafo del docstring existente cuando sea útil.

-  `_leading_special_lines`: Devuelve n líneas iniciales (shebang, coding, noqa de archivo) a conservar.
-

 `scripts/build_executable.py`

Nombre del Módulo: `scripts.build_executable`

Descripción: Script ejecutable (`build_executable`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

-  `clean_build_environment`: Limpia compilaciones anteriores para evitar basura o conflictos.
 -  `build_hipatia`: Ejecuta PyInstaller de forma programática con toda la configuración de Hipatia.
-

`scripts/check_documentation_omissions.py`

Nombre del Módulo: `check_documentation_omissions` Descripción: Verifica automáticamente que la documentación técnica generada no tenga archivos omitidos en el índice de código.

-  `regenerate_docs`: Regenera la documentación técnica usando el script oficial.
 -  `read_omitted_count`: Extrae el valor de "Omitidos (reglas de docstrings/otros)" del markdown.
 -  `main`: Punto de entrada del chequeo de regresión documental.
-

 **`scripts/check_typing_coverage.py`**

Nombre del Módulo: `scripts.check_typing_coverage`

Descripción: Script ejecutable (`check_typing_coverage`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

 **scripts/codebase_analyzer.py**

Nombre del Módulo: scripts.codebase_analyzer

Descripción: Script ejecutable (codebase_analyzer): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

 **scripts/coverage_focus.py**

Nombre del Módulo: scripts.coverage_focus

Descripción: Concentra datos de configuración o catálogos estáticos: `REPO_ROOT`, consumidos por la UI y controladores. Integración típica con: `__future__`, `argparse`, `json`, `os`, `subprocess`, `sys`.

 `scripts/detect_dead_code.py`

Nombre del Módulo: `scripts.detect_dead_code`

Descripción: Detección de código muerto en el paquete `ui/dialogs/`.

===== Recorre cada `*.py` bajo `ui/dialogs/`, extrae métodos por clase y busca referencias en `app.py`, `ui/`, `controllers/`, `core/`, `tests/`.

Clase `MethodExtractor`

Extrae todos los métodos de cada clase.

-  `extract_package_classes`: Parsea todos los `.py` bajo `package_dir` y fusiona clases con clave `rel/path.py::ClassName`.
 -  `find_references_in_file`: Busca referencias a métodos y clases en un archivo. Retorna un diccionario con las referencias encontradas.
 -  `find_all_references`: Busca referencias en todo el proyecto.
 -  `_class_reference_key`: Nombre corto de clase para coincidir con `find_all_references`.
 -  `analyze_dead_code`: Analiza y clasifica métodos según su uso.
 -  `generate_report`: Genera el reporte en formato Markdown.
 -  `main`: Función principal.
-

 `scripts/doc_audit_common.py`

Nombre del Módulo: `scripts.doc_audit_common`

Descripción: Criterios compartidos para auditoría de docstrings de módulo (Daniel doc + `audit_module_docstrings`).

-  `module_docstring_raw`: Texto del docstring de módulo. Incluye el caso frecuente `from __future__ ...` seguido de un literal `"""..."""`, que **no** expone `ast.get_docstring` pero es doc de módulo válido en tiempo de ejecución.
 -  `module_docstring_is_acceptable`: True si el módulo tiene docstring de módulo no trivial según `FRASES_IGNORADAS`. Equivale a `'doc_valid'` en `generate_daniel_doc` para el nodo raíz.
 -  `parse_module`: Parsea un archivo UTF-8; devuelve `(tree, None)` o `(None, mensaje de error)`.
 -  `summarize_module_for_audit`: Resumen para informes JSON (clases/funcs top-level sin depender del docstring).
-

 `scripts/docstrings_queue.py`

Nombre del Módulo: `docstrings_queue`

Descripción: Genera la cola ordenada de módulos sin Nombre del Módulo (oleadas A–F) y permite verificar cuántos faltan; debe mantenerse alineado con `generate_daniel_doc`.

-  `collect_missing_ordered`: Lista (`wave_id`, `path`) en orden A→F, luego alfabético dentro de cada oleada.
 -  `verify`: Devuelve (`total_scope`, `missing_count`, `missing_rel_paths`).
-

 **scripts/download_opencv_resources.py**

Nombre del Módulo: scripts.download_opencv_resources

Descripción: Descarga recursos auxiliares para usar OpenCV (modelos y documentación).

-  `download_file`: Descarga un fichero y lo guarda en `dest_path`.
 -  `main`: Punto de entrada del script de descargas.
-

`scripts/extract_test_quality_in_progress.py`

Nombre del Módulo: `extract_test_quality_in_progress` Descripción: Genera un backlog detallado de archivos de test en estado "En Progreso" según `test_reports/compliance_data.json`, incluyendo penalizaciones corregibles y recomendaciones conservadoras para elevar el score.

Uso: `python3 scripts/extract_test_quality_in_progress.py`

Salidas: - Documentacion/Mejora_Calidad/backlog_tests_en_progreso.md - `.agents/skills/backlog_tests_en_progreso/SKILL.md`

-  `_recommendations`: Devuelve recomendaciones conservadoras basadas en penalizaciones.
Nota: No intenta "forzar 100/100" si el techo real está limitado por Qt/docx/builtins.
-

 `scripts/generate_comprehensive_report.py`

Nombre del Módulo: `scripts.generate_comprehensive_report`

Descripción: Script ejecutable (`generate_comprehensive_report`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

`scripts/generate_coverage_report.py`

Nombre del Módulo: `scripts.generate_coverage_report`

Descripción: Script ejecutable (`generate_coverage_report`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

-  `run_coverage`: Runs pytest with coverage json report.
 -  `load_coverage_data`: Loads coverage.json.
 -  `get_category`: Determines category based on filename.
 -  `format_missing_lines`: Formats a list of missing lines into ranges (e.g. '1-5, 8, 10-12').
 -  `print_report`: Prints the categorized report.
-

Nombre del Módulo: `generate_daniel_doc` Descripción: Genera documentación técnica completa de Hipatia en Markdown con diagramas Mermaid automáticos (ERD, arquitectura, árbol de carpetas, flujo de fabricación) extraídos del código real del proyecto. Incluye sección de Suite de Tests generada desde `compliance_data.json`.

La narrativa embebida (p. ej. Fase 12C y CI) debe mantenerse alineada con ``github/workflows/ci.yml`` y con las skills de Fase 12C al cambiar gates.

-  `_build_folder_tree_mermaid`: Genera un diagrama Mermaid del árbol de carpetas principales.
 -  `_path_to_module`: Convierte una ruta tipo `core/services/foo.py` a módulo Python: `core.services.foo`. Si es `__init__.py`, se interpreta como paquete.
 -  `_load_mypy_disallow_untyped_defs`: Devuelve un mapa `{pattern: disallow_untyped_defs}`. `pattern` es el nombre del módulo/patrón que sigue a `mypy-` en `mypy.ini`.
 -  `_pattern_matches_module`: Soporte simple de patrones tipo `core.*` y `controllers.foo`.
 -  `_mypy_strict_for_file`: Etiqueta + motivo (breve) de por qué el archivo no está al 100%.
 -  `_parse_all_classes`: Extrae solo nombres de clases (sin depender de docstrings). Esto permite que el índice no "omite" clases aunque luego no aparezcan en el cuerpo.
 -  `_parse_all_symbols`: Extrae nombres de clases y funciones top-level sin depender de docstrings.
 -  `_has_ast_content`: Determina si un archivo Python contiene contenido parseable por AST.
 -  `_is_package_init`: Indica si el archivo es un `__init__.py` de paquete.
 -  `_collect_index_tree`: Recorre todas las rutas `.py` bajo los directorios del alcance del índice y construye el árbol de carpetas/subcarpetas.
 -  `_render_index_tree_md`: Renderiza el árbol como lista jerárquica (texto), incluyendo archivos y clases.
 -  `_folder_connections_mermaid`: Diagrama Mermaid compacto de conexiones por capa. Basado en la arquitectura fija UI -> Controllers -> Core -> Database.
 -  `_index_stats`: Calcula métricas rápidas para auditoría en papel: - `total_py`: total de `.py` considerados en el índice - `included`: incluidos en cuerpo - `omitted`: omitidos (docstrings/reglas)
 -  `_folder_stats`: Resumen por carpeta para portada de capítulo (modo papel).
 -  `_split_markdown_into_sections`: Particiona el markdown en secciones independientes para impresión: - `prefix`: portada + índice + secciones intro hasta antes de la referencia por carpetas/archivos - `sections`: lista ordenada de (kind:key, text) kind: - `folder`: - `file`:<rel_path>
 -  `_measure_page_starts_for_file_sections`: Renderiza el markdown en secciones discretas para obtener: `{rel_path: page_start}`.
 -  `_render_pdf_from_markdown`: Convierte markdown -> PDF usando partición por archivo para estabilidad.
 -  `_collect_files`: Devuelve (`root_files`, `{dirname: [rel_paths]}`).
 -  `_load_compliance_data`: Carga los datos de compliance desde `test_reports/compliance_data.json`. Si el archivo no existe, devuelve lista vacía (la sección se omite).
 -  `_write_testing_section`: Escribe la sección 'Suite de Tests' en el documento Markdown. Incluye: - Filosofía general de testing del proyecto - Explicación del sistema de scoring y techo real - Tabla resumen global (score absoluto, techo, estado) - Tabla por archivo con decisión técnica de mocking
-

 `scripts/generate_monolitos_finales.py`

Nombre del Módulo: `scripts.generate_monolitos_finales`

Descripción: Script ejecutable (`generate_monolitos_finales`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

-  `main`: Genera bajo demanda `Documentacion/Monolitos_finales.md` (no versionado por defecto).
-

 **scripts/generate_quotes_db.py**

Nombre del Módulo: scripts.generate_quotes_db

Descripción: Script ejecutable (generate_quotes_db): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

-  fetch_quotes: Descarga frases de múltiples fuentes JSON.
 -  generate_database: Genera el fichero JSON final con las frases.
-

 **scripts/init_database.py**

Nombre del Módulo: scripts.init_database

Descripción: Script ejecutable (`init_database`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

-  `init_database`: Inicializa la base de datos creando todas las tablas.
-

 **scripts/inject_module_docstrings.py**

Nombre del Módulo: scripts.inject_module_docstrings

Descripción: Inyecta docstrings de módulo donde faltan (criterio doc_audit_common), sin tocar archivos que ya tienen docstring aceptable.

-  `_describe_module`: Una o dos frases en español; suficientemente específicas para no caer en frases genéricas prohibidas.
-

Nombre del Módulo: `scripts.legacy_analyzer`

Descripción: Analizador de Código Legacy — Proyecto Hipatia

===== Fase 4 del Plan de Mejora de Calidad: detecta patrones legacy para su eliminación o sustitución (print → logger, marcadores deprecated, docstrings obsoletos, delegaciones/shim y código muerto candidato).

-  `get_production_python_files`: Lista todos los archivos `.py` en directorios de producción.
 -  `get_all_python_files`: Lista todos los `.py` del proyecto (excepto `venv/git`).
 -  `find_print_statements`: Detecta llamadas a `print()` en archivos de producción.
 -  `find_bare_except`: Detecta `except`: sin tipo (`bare except`).
 -  `find_deprecated_markers`: Busca líneas con marcadores `@deprecated`, `TODO: Remove`, `DEPRECATED`.
 -  `find_legacy_in_docstrings`: Busca docstrings que mencionan `obsoleto/legacy/deprecated`.
 -  `_is_simple_delegation`: Comprueba si la función solo delega en otra (`return other()` o `self.other()`). Devuelve (`True`, `nombre_delegado`) o (`False`, `None`).
 -  `find_simple_delegations`: Detecta funciones que solo delegan en otra (posibles shims/alias legacy).
 -  `_count_symbol_mentions`: Cuenta menciones literales de un símbolo en una lista de archivos Python. Nota: Heurística basada en regex. Es suficiente para detectar shims no usados sin introducir dependencias externas.
 -  `filter_unused_delegations`: Filtra delegaciones simples dejando solo las que no tienen uso externo detectable. Criterio: si el símbolo aparece 1 vez en todo el proyecto (su propia definición), lo tratamos como candidato a eliminación.
 -  `find_legacy_re_exports`: Busca comentarios que indican re-exports o métodos legacy (ej. `app_controller`).
 -  `build_report`: Construye el informe completo de código legacy.
 -  `generate_md`: Genera el informe en Markdown.
 -  `main`: Punto de entrada.
-

 **scripts/list_mypy_core_services_gaps.py**

Nombre del Módulo: `scripts.list_mypy_core_services_gaps`

Descripción: Lista módulos bajo `core/services` que aún no aparecen en `mypy.ini` dentro de un bloque `[mypy-...]` con `disallow_untyped_defs = True`.

 `scripts/monolith_analyzer.py`

Nombre del Módulo: `scripts.monolith_analyzer`

Descripción: Funciones y datos de apoyo del paquete; conviene enlazar qué controlador o servicio las consume y qué estructuras devuelven (ver firmas al inicio del archivo). Integración típica con: `argparse`, `ast`, `json`, `datetime`, `pathlib`.

 **Clase `_ImportCollector`**

Métodos Principales:

- `visit_If`: Ignora imports dentro de: `if TYPE_CHECKING`: ya que no forman parte del grafo de dependencias en runtime.
 -  `_resolve_internal_target`: Resuelve un import a un archivo del repo si coincide con un módulo interno. Heurística: intenta el módulo completo y prefijos.
 -  `_scc_kosaraju`: SCC por Kosaraju (sin dependencias externas).
-

 **scripts/print_summary.py**

Nombre del Módulo: scripts.print_summary

Descripción: Script ejecutable (`print_summary`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

 **scripts/profile_queries.py**

Nombre del Módulo: scripts.profile_queries

Descripción: Script ejecutable (profile_queries): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

-  profile_method: Executes a method and counts SQL queries.
-

 `scripts/refine_module_descriptions.py`

Nombre del Módulo: `refine_module_descriptions`

Descripción: Sustituye el párrafo `Descripción:` del docstring de módulo usando heurísticas sobre imports, clases y constantes de nivel superior; conserva `Nombre del Módulo`.

-  `_mostly_reexports_from_single_module`: True si el módulo solo importa y reexporta (p. ej. `shim core.dtos`).
-

 `scripts/reorder_docstring_before_future.py`

Nombre del Módulo: `scripts.reorder_docstring_before_future`

Descripción: Reordena el docstring de módulo inmediatamente **antes** del bloque `from __future__`.

-  `reorder_source`: Devuelve texto reordenado o None si no aplica.
-

 **`scripts/run_quality_audit.py`**

Nombre del Módulo: `scripts.run_quality_audit`

Descripción: Script ejecutable (`run_quality_audit`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

 `scripts/security_audit_analyzer.py`

Nombre del Módulo: `scripts.security_audit_analyzer`

Descripción: Security Audit Analyzer Script Analyzes the codebase for security issues identified in the technical audit.

 **Clase `SecurityAuditAnalyzer`**

Analyzes code for security vulnerabilities.

Métodos Principales:

- `analyze_credentials`: Find hardcoded credentials.
 - `analyze_fail_open_policy`: Find fail-open security patterns.
 - `analyze_hashlib_usage`: Find hashlib usage for password hashing.
 - `analyze_rate_limiting`: Check for rate limiting in authentication.
 - `analyze_audit_logging`: Check for audit logging.
 - `analyze_database_issues`: Check for database-related security issues.
 - `generate_report`: Generate a formatted report of findings.
 - `run_analysis`: Run all analysis methods.
-

 **scripts/seed_data.py**

Nombre del Módulo: scripts.seed_data

Descripción: Script ejecutable (`seed_data`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

-  `seed_data`: Inserta datos de prueba en la base de datos.
-

 **`scripts/sync_worktree_to_icloud.py`**

Nombre del Módulo: `scripts.sync_worktree_to_icloud`

Descripción: Copia archivos modificados o sin seguimiento desde `SOURCE_ROOT` a `HIPATIA_ICLOUD`.

Nombre del Módulo: `scripts.test_quality_analyzer`

Descripción: Concentra datos de configuración o catálogos estáticos: `_RE_STRICT MOCK`, `_RE_LOOSE MOCK`, `_RE_FINDER_DUP_TEST`, `_RE_FINDER_DUP_CONFTEST`, `_RE_PATCH_NO_AUTOSPEC`, `_RE_PATCH_WITH_AUTOSPEC`, `_RE_ASSERT_CALLED`, `_RE_ASSERT_CALLED_NO_ARGS`, consumidos por la UI y controladores. Integración típica con: `os`, `re`, `json`, `pathlib`, `ast`.

-  `_is_whitelisted_patch`: Devuelve True si el target del patch está en la whitelist de inevitables.
 -  `_count_inevitable_patches`: Cuenta @patch sin autospec que son inevitables (builtins, Qt, OS).
 -  `_count_inevitable_loose_magicmocks`: Cuenta MagicMock() sueltos que son inevitables por dependencias externas sin stubs. - Qt: widgets PyQt6 no tienen stubs de tipo → MagicMock() es la única opción - docx: python-docx no tiene stubs → MagicMock() para Document, Run, etc.
 -  `_calculate_ceiling`: Calcula el techo real de score para un archivo de test. Separa penalizaciones inevitables (dependencias externas sin stubs) de penalizaciones corregibles (antipatrones reales). Retorna dict con: `ceiling_score`, `ceiling_penalties`, `actionable_penalties`, `at_ceiling`, `ceiling_explanation`. Nota: el «Actualizado» global no se deduce solo de aquí; `resolve_analyzer_status()` aplica reglas distintas para la cohorte `strict_domain` (score absoluto 100).
 -  `_count_patches`: Devuelve (total_patches, patches_con_autospec).
 -  `_count_loose_magicmocks_ast`: Cuenta llamadas reales a MagicMock()/Mock() sin args/kwargs. Importante: no contar ocurrencias en docstrings/comentarios (los regex dan falsos positivos).
 -  `_count_tests_without_assert`: Cuenta tests que no tienen ningún assert significativo. Regla: `assert True` se considera **trivial** y no cuenta como verificación real. Un test se considera "sin assert" si NO contiene: - asserts no triviales (`assert x == ...`, `assert mock.call_count == ...`, etc.), o - verificaciones de interacción (`.assert_called_*`, `.call_count`, etc.) Solo se considera fin de test al ver "def test..." o "async def test..." al mismo indent o menor; así se evita truncar el cuerpo por líneas "def"/"class" dentro de strings multilínea (p. ej. código de ejemplo en el test).
 -  `_is_controller_or_service_file`: Detecta si el archivo testea controladores o servicios.
 -  `_is_tests_db_path`: True si el archivo está bajo tests/db/ (convención de tests de persistencia).
 -  `classify_test_tier`: Asigna la cohorte de política de estado (independiente del score numérico). Returns: `infra`: conftest y utilidades de test (lista fija por nombre de archivo). `strict_domain`: ruta `.../tests/db/...` o nombre `test_*service.py / testrepository.py` (salvo `_STRICT_DOMAIN_EXCLUDED_NAMES`). Objetivo: score absoluto 100 para «Actualizado». `ui_qt`: resto de tests de producto; aplica techo real y umbral `ceiling_score >= 80`.
 -  `resolve_analyzer_status`: Deriva status (Actualizado / En Progreso / Legacy), texto de detalle y `domain_status`. `strict_domain`: «Actualizado» solo con `score >= 100`; `domain_status` Listo/ Pendiente dominio. Con `score < 100` y sin penalizaciones corregibles (`at_ceiling`), sigue En Progreso: el techo no sustituye al 100 absoluto en esta cohorte. `infra` y `ui_qt`: sin `domain_status` (None). Misma lógica que antes: `at_ceiling` ⇒ Actualizado aunque el techo sea bajo (p. ej. PyQt); si no, `ceiling_score >= 80 / >= 50 / resto`. Returns: Tupla (status, status_detail, domain_status). `domain_status` es None fuera de `strict_domain`.
 -  `_count_weak_any_only_interaction_lines`: Cuenta aserciones cuyos argumentos son solo ANY; ignora líneas con `# noqa: weak_any`.
 -  `analyze_test_file`: Analiza un archivo de test: métricas, score, techo y estado según cohorte. Añade respecto al histórico: `test_tier`, `strict_domain` (bool), `domain_status` (str|None).
 -  `run_analysis`: Ejecuta el análisis en toda la carpeta de tests.
-

 **`scripts/track_docx_dependencies.py`**

Nombre del Módulo: `scripts.track_docx_dependencies`

Descripción: Script para rastrear dependencias directas de 'docx' (python-docx) y generar un informe de archivos afectados.

scripts/ui_dto_boundary_analyzer.py

Nombre del Módulo: ui_dto_boundary_analyzer Descripción: Audita la frontera UI/DTO para detectar accesos tipo diccionario dentro de ui/ (p.ej. `obj["campo"]`, `obj.get("campo")`) que suelen indicar datos sin tipar o mezclas DTO vs dict. Ignora `os.environ.get` (variables de entorno, no DTO). Genera informes para la Fase 12C.

Las variables en `INTERNAL_UI_DICT_VARS` (p. ej. ``data``) se ignoran salvo que la clave literal esté en `DOMAIN_DICT_KEYS_FORCE_REPORT` (campos típicos de entidad/BD), para reducir falsos negativos en ``data["id"]`` sin reabrir todo el ruido de estado UI.

Uso: `python3 scripts/ui_dto_boundary_analyzer.py` `python3 scripts/ui_dto_boundary_analyzer.py --json-only`
`python3 scripts/ui_dto_boundary_analyzer.py --md-only` `python3 scripts/ui_dto_boundary_analyzer.py --enforce-zero` # CI: falla si hay hallazgos `python3 scripts/ui_dto_boundary_analyzer.py --max-findings 5`
`python3 scripts/ui_dto_boundary_analyzer.py --baseline ruta/baseline.json`

Salida (por defecto): `Documentacion/Refactorizacion_Completa/Fase_12C/ui_dto_boundary_report.json`
`Documentacion/Refactorizacion_Completa/Fase_12C/ui_dto_boundary_report.md`

Clase Finding

Hallazgo de acceso tipo diccionario dentro de UI.

-  `should_report_name_dict_access`: False: el acceso se considera estado interno UI (receptor en `INTERNAL_UI_DICT_VARS` y clave no es de dominio forzado). True: incluir en el informe.
 -  `_iter_ui_files`: Devuelve todos los `.py` bajo `ui/` (sin `venv/git`).
 -  `_is_os_environ_access`: True si la expresión es `os.environ` (no es frontera UI/DTO).
 -  `analyze_file`: Analiza un archivo UI y devuelve hallazgos de acceso dict-like. - Subscript: `x["campo"]` o `x['campo'] - get_call: x.get("campo", ...)`
 -  `build_report`: Construye el reporte completo de hallazgos en `ui/`.
 -  `generate_md`: Genera un informe legible en Markdown para Fase 12C.
 -  `_gate_exit_code`: Devuelve (código_salida, mensaje_vacío_o_error). Prioridad: `--enforce-zero > --baseline > --max-findings`.
 -  `main`: Punto de entrada CLI.
-

 **scripts/ui_dto_boundary_decision_report.py**

Nombre del Módulo: scripts.ui_dto_boundary_decision_report

Descripción: Funciones y datos de apoyo del paquete; conviene enlazar qué controlador o servicio las consume y qué estructuras devuelven (ver firmas al inicio del archivo). Integración típica con: `json`, `pathlib`.

`scripts/ui_dto_findings_catalog.py`

Nombre del Módulo: `ui_dto_findings_catalog` Descripción: Inventario de hallazgos UI/DTO (subscript y `.get` con clave literal) con metadatos de conexión: receptor AST, agrupación por archivo/receptor, imports del módulo y enlaces entre hallazgos del mismo grupo.

Uso: `python3 scripts/ui_dto_findings_catalog.py` `python3 scripts/ui_dto_findings_catalog.py --no-production-flow` `python3 scripts/ui_dto_findings_catalog.py --json-only`

Salida (por defecto): `Documentacion/Refactorizacion_Completa/Fase_12C/ui_dto_findings_catalog.json`
`Documentacion/Refactorizacion_Completa/Fase_12C/ui_dto_findings_catalog.md` `Documentacion/Refactorizacion_Completa/Fase_12C/ui_dto_findings_checklist.md`

-  `_item_signature`: Clave estable al cambiar numeración F0001 tras regenerar.
-

 `scripts/update_readme_metrics.py`

Nombre del Módulo: `scripts.update_readme_metrics`

Descripción: Actualiza el bloque de métricas del README entre los marcadores `HIPATIA_METRICS`.

 **scripts/update_test_imports.py**

Nombre del Módulo: scripts.update_test_imports

Descripción: Script ejecutable (`update_test_imports`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

 `scripts/verify_migration.py`

Nombre del Módulo: `scripts.verify_migration`

Descripción: Migration Verification Script ===== Detects orphan code patterns that may have been left behind during the SQLAlchemy/DTO migration.

Clase `CodeIssue`

Represents a potential code issue found during verification.

-  `find_python_files`: Find all Python files in the project, excluding certain directories.
 -  `check_orphan_method_calls`: Check for direct calls to methods that should go through repositories.
 -  `check_tuple_access_on_dtos`: Check for tuple access patterns that might indicate unconverted code.
 -  `check_removed_methods`: Check for usage of methods that have been removed from `DatabaseManager`.
 -  `check_file`: Run all checks on a single file.
 -  `print_issues`: Print issues in a formatted way and return counts.
 -  `main`: Main entry point.
-

 `scripts/verify_qr_optimization.py`

Nombre del Módulo: `scripts.verify_qr_optimization`

Descripción: Script ejecutable (`verify_qr_optimization`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

 **scripts/verify_structure.py**

Nombre del Módulo: scripts.verify_structure

Descripción: Script ejecutable (verify_structure): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

 `scripts/windows_path_audit.py`

Nombre del Módulo: windows_path_audit

Descripción: Recorre código de producto y lista patrones de rutas que pueden fallar en Windows (concatenación con /, /tmp sin guard Darwin, etc.). Escribe `reports/windows_path_audit.md`.

 `scripts/analysis/analyze_codebase.py`

Nombre del Módulo: `scripts.analysis.analyze_codebase`

Descripción: Script ejecutable (`analyze_codebase`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

 **Clase `FileStats`**

Métricas por fichero (derivadas del AST) para análisis de tipado.

 **Clase `DirectorySummary`**

Resumen agregado por directorio.

 **scripts/analysis/analyze_controller.py**

Nombre del Módulo: scripts.analysis.analyze_controller

Descripción: Funciones puras de apoyo (sin estado de proceso): `analyze_file`. Integración típica con: `ast`, `os`, `sys`.

 **scripts/analysis/analyze_coverage_risks.py**

Nombre del Módulo: scripts.analysis.analyze_coverage_risks

Descripción: Script ejecutable (`analyze_coverage_risks`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

 **scripts/analysis/analyze_db_usage.py**

Nombre del Módulo: scripts.analysis.analyze_db_usage

Descripción: Script ejecutable (`analyze_db_usage`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

-  `analyze_file`: Analyzes a python file for DB related keywords and AST nodes.
-

 **`scripts/analysis/analyze_dependencies.py`**

Nombre del Módulo: `scripts.analysis.analyze_dependencies`

Descripción: Funciones puras de apoyo (sin estado de proceso): `get_imports`, `analyze_dependencies`.

Integración típica con: `ast`, `os`, `sys`.

 **scripts/analysis/analyze_fabrication_dialogs.py**

Nombre del Módulo: scripts.analysis.analyze_fabrication_dialogs

Descripción: Script ejecutable (analyze_fabrication_dialogs): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

 `scripts/analysis/analyze_loose_mock.py`

Nombre del Módulo: `scripts.analysis.analyze_loose_mock`

Descripción: Script ejecutable (`analyze_loose_mock`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

 `scripts/analysis/analyze_refactoring_impact.py`

Nombre del Módulo: `scripts.analysis.analyze_refactoring_impact`

Descripción: Script ejecutable (`analyze_refactoring_impact`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

 **scripts/analysis/analyze_repository_connections.py**

Nombre del Módulo: scripts.analysis.analyze_repository_connections

Descripción: Define protocolos o tipos principales: FileAnalysisResult, RepoUsageData, ProjectAnalysisResult, RepositoryConnectionAnalyzer. **Analiza conexiones y dependencias de repositorios.** Integración típica con: os, re, ast, sys, pathlib.

Clase RepositoryConnectionAnalyzer

Analiza conexiones y dependencias de repositorios.

Métodos Principales:

- `analyze_file`: Analiza un archivo Python buscando usos de repositorios.
 - `analyze_project`: Analiza todo el proyecto.
 - `generate_report`: Genera un reporte legible.
 - `export_to_markdown`: Exporta los resultados a un archivo Markdown.
 -  `main`: Función principal.
-

 `scripts/analysis/analyze_root_files.py`

Nombre del Módulo: `scripts.analysis.analyze_root_files`

Descripción: Funciones y datos de apoyo del paquete; conviene enlazar qué controlador o servicio las consume y qué estructuras devuelven (ver firmas al inicio del archivo). Integración típica con: `ast`, `os`, `re`, `pathlib`, `json`.

-  `get_definitions_and_imports`: Extrae clases, funciones e imports de un archivo.
 -  `find_usages`: Busca ocurrencias del nombre del módulo o sus definiciones en el proyecto.
-

 **scripts/analysis/analyze_structure.py**

Nombre del Módulo: scripts.analysis.analyze_structure

Descripción: Script ejecutable (`analyze_structure`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

 `scripts/analysis/analyze_tracking_impact.py`

Nombre del Módulo: `scripts.analysis.analyze_tracking_impact`

Descripción: Script ejecutable (`analyze_tracking_impact`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

 **scripts/analysis/analyze_typing_deep.py**

Nombre del Módulo: scripts.analysis.analyze_typing_deep

Descripción: Funciones puras de apoyo (sin estado de proceso): `analyze_file`, `analyze_directory`, `print_summary`. Integración típica con: `ast`, `os`, `sys`.

 **scripts/analysis/analyze_ui_structure.py**

Nombre del Módulo: scripts.analysis.analyze_ui_structure

Descripción: Script ejecutable (`analyze_ui_structure`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

-  `analyze_file`: Analyzes a Python file and produces a markdown report.
 -  `analyze_class`: Analyzes a class node.
 -  `analyze_method_row`: Analyzes a method and adds a table row.
 -  `analyze_function`: Analyzes a standalone function.
-

 `scripts/analysis/detect_obsolete_code.py`

Nombre del Módulo: `scripts.analysis.detect_obsolete_code`

Descripción: Script ejecutable (`detect_obsolete_code`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

-  `get_python_files`: Recursively find all Python files in the directory.
 -  `extract_definitions`: Extract class and function names defined in a file.
 -  `search_usages`: Count usages of target names in all files.
 -  `check_deprecated`: Busca decoradores de deprecación y comentarios TODO: Remove.
-

 `scripts/analysis/verify_naming_conventions.py`

Nombre del Módulo: `scripts.analysis.verify_naming_conventions`

Descripción: Script ejecutable (`verify_naming_conventions`): automatización, informes o mantenimiento del proyecto (no forma parte del runtime de la app).

-  `get_python_files`: Recursively find all Python files in the directory.
 -  `is_snake_case`: Check if string is snake_case.
 -  `is_camel_case`: Check if string is CamelCase (PascalCase).
 -  `check_file_conventions`: Check naming conventions in a single file.
-

 `scripts/maintenance/backup_database.py`

Nombre del Módulo: `scripts.maintenance.backup_database`

Descripción:

===== SCRIPT DE
BACKUP - BASE DE DATOS

===== Este script
crea una copia de seguridad de tu base de datos ANTES de realizar cualquier modificación al esquema.

-  `create_backup`: Crea una copia de seguridad de la base de datos. Args: `db_path`: Ruta a la base de datos (por defecto usa la config de entorno)
 -  `backup_all_databases`: Crea backup de la base de datos principal configurada.
-

`scripts/maintenance/reset_admin.py`

Nombre del Módulo: `scripts.maintenance.reset_admin`

Descripción: Concentra datos de configuración o catálogos estáticos: `_PROJECT_ROOT`, consumidos por la UI y controladores. Integración típica con: `__future__`, `sys`, `pathlib`, `sqlalchemy`, `database`, `core`.

-  `_resolve_sqlite_db_path`: Devuelve la ruta del fichero SQLite según `DatabaseConfig`, o `None` si no aplica.
 -  `reset_admin_password`: Pone la contraseña del usuario `admin` a `admin` (solo SQLite configurado).
-

Capítulo: `tools/`

Métrica	Valor
Archivos <code>.py</code> en <code>tools/</code>	3
Incluidos en el cuerpo	3
Omitidos (docstrings/reglas)	0
Clases detectadas (AST)	2

```
graph TD
  TOOLS[Herramientas auxiliares] --> QA[Calidad/Docs/Test]
```


 `tools/__init__.py`

Nombre del Módulo: tools

Descripción: Herramienta de consola (`__init__`): análisis estático o asistencia al desarrollo.

 `tools/analyze_app_controller.py`

Nombre del Módulo: `tools.analyze_app_controller`

Descripción: Herramienta de análisis AST: compara métodos y atributos de `AppController` con la documentación de nomenclatura en Markdown (informe de cobertura doc vs código).

 `tools/hardware/detect_cameras.py`

Nombre del Módulo: `tools.hardware.detect_cameras`

Descripción: Script para detectar TODAS las cámaras disponibles en el sistema, incluyendo las que están en índices no continuos.

Capítulo: migrations/

Métrica	Valor
Archivos .py en migrations/	3
Incluidos en el cuerpo	3
Omitidos (docstrings/reglas)	0
Clases detectadas (AST)	0

```
graph TD
  MIG[Schema de BD (Alembic)] --> DB[Database]
```


migrations/env.py

Nombre del Módulo: migrations.env

Descripción: Concentra datos de configuración o catálogos estáticos: `config`, `target_metadata`, consumidos por la UI y controladores. Integración típica con: `sqlalchemy`, `alembic`, `os`, `sys`, `database`.

-  `run_migrations_offline`: Run migrations in 'offline' mode. This configures the context with just a URL and not an Engine, though an Engine is acceptable here as well. By skipping the Engine creation we don't even need a DBAPI to be available. Calls to `context.execute()` here emit the given string to the script output.
 -  `run_migrations_online`: Run migrations in 'online' mode. In this scenario we need to create an Engine and associate a connection with the context.
-

 `migrations/versions/a195b5f170d2_add_security_tables.py`

Nombre del Módulo: `migrations.versions.a195b5f170d2_add_security_tables`

Descripción: Funciones puras de apoyo (sin estado de proceso): `upgrade`, `downgrade`. Integración típica con: `alembic`, `sqlalchemy`.

-  `upgrade`: Upgrade schema.
 -  `downgrade`: Downgrade schema.
-

 `migrations/versions/c1444b2546d3_initial_clean_migration.py`

Nombre del Módulo: `migrations.versions.c1444b2546d3_initial_clean_migration`

Descripción: Funciones puras de apoyo (sin estado de proceso): `upgrade`, `downgrade`. Integración típica con: `alembic`, `sqlalchemy`.

-  `upgrade`: Upgrade schema.
 -  `downgrade`: Downgrade schema.
-